

The WCC Compiler Intermediate Representation (WIR)

Build & Installation Guide

User's Guide

Retargeting Guide

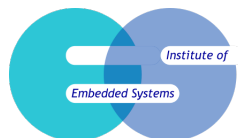
Developer's Guide

Heiko Falk

August 25, 2026

<https://github.com/TUHH-IES/WIR>

[DOI 10.5281/zenodo.22102094](https://doi.org/10.5281/zenodo.22102094)



Institute of Embedded Systems

Hamburg University of Technology (TUHH)

Contributors:

- Dania Alghazal
- Ben Bahe
- Lennard Boysen
- Alireza Chahardori
- Heiko Falk
- Tobias Heinicke
- Maurice Hoffmann
- Shashank Jadhav
- Ailin Jörg
- Timon Kelter
- Simon Kopischke
- Til Mauersberger
- Rasmus Mecklenburg
- Abhirami Meera Sreekumar
- Jonas Oltmanns
- Maik Schulze
- Christian Sühl
- Yom Ismail Tan
- Sasha Tosta
- Muthappan Viswanathan

Contents

Contents	i
List of Figures	v
List of Tables	vii
Listings	ix
Acronyms	xv
1 Introduction	1
1.1 Overview	1
1.1.1 Software Model	2
1.1.2 Hardware Model	3
1.2 Key Features	6
1.3 License	11
1.4 Outline	11
 I Build & Installation Guide	 13
2 Prerequisites	15
2.1 Mandatory Dependencies	15
2.2 Optional Dependencies	15
3 Directory Structure	17
4 Bootstrapping and Configuration	21
4.1 Bootstrapping of the GNU Build System	21
4.2 Configuration of a WIR Build	22
5 Building, Installing and Using WIR	25
 II User's Guide	 29
6 General Handling of List Structures Within WIR	31
6.1 Insertion	31
6.2 Modification	32
6.3 Modification Prevention	33
6.4 Lookup	33
6.5 Destruction	34

7	Construction and Traversal of WIR Object Hierarchies	37
7.1	Construction Using List Interfaces	37
7.2	Variadic Construction	38
7.3	Traversal and Navigation	39
8	The WIR Software Model	41
8.1	Compilation Units	41
8.2	Functions	42
8.3	Basic Blocks	45
8.4	Instructions	47
8.5	Operations	48
8.6	Parameters	52
8.6.1	Register Parameters	53
8.6.2	Immediates	54
8.6.3	Labels	57
8.6.4	Addressing Modes	58
8.6.5	Condition Fields	59
8.6.6	String Parameters	59
8.7	Data Objects and Their Initializers	60
9	The WIR Hardware Model	65
9.1	Processors	65
9.1.1	Generic Processor Model	66
9.1.1.1	Register Types	67
9.1.1.2	Registers	69
9.1.1.3	Immediates	75
9.1.1.4	Opcodes	75
9.1.1.5	Operation Formats	78
9.1.1.6	Addressing Modes	82
9.1.1.7	Condition Fields	83
9.1.1.8	Logical Sections	83
9.1.2	MIPS SPIM Processor Model	87
9.1.3	TriCore Processor Models	88
9.1.4	RISC-V RV32 Processor Models	96
9.1.5	ARM Processor Models	100
9.2	Physical Memory Regions	105
9.3	Caches	108
9.4	Buses	112
9.4.1	Fixed-Priority Bus Arbitration	114
9.4.2	Round-Robin Bus Arbitration	115
9.4.3	Time-Division Multiple Access Bus Arbitration	115
9.4.4	Priority-Division Bus Arbitration	116
10	The WIR System Model	119
10.1	System Configuration Files	119
10.2	Integration of Software and Hardware Models	124
10.3	Symbols	125
10.4	Memory Layout and Allocation	129

11 Basic Set and Map Data Structures	133
11.1 Unique Numerical Identifiers	133
11.2 Deterministic WIR Data Structures	134
12 Code Generation	137
12.1 Generic Stream Output	137
12.2 Stream Manipulators	140
12.3 Processor-Specific Assembly Code Generation	144
12.4 Linker Script Generation	146
13 Code Analysis	147
13.1 Generic Analysis Framework	147
13.2 Basic Intraprocedural Control Flow Analysis	151
13.3 Reachability Analysis	153
13.4 Domination and Immediate Domination Analysis	154
13.5 Structural Control Flow Analysis	155
13.6 Flow Facts for Execution Time Analysis	161
13.7 Reaching Definitions Analysis	168
13.8 Available Definitions Analysis	169
13.9 Def-Use/Use-Def Chains Analysis	170
13.10 Liveness Analysis	172
13.11 Bit-True Data Flow Analysis	173
14 Code Optimization	185
14.1 Generic Optimization Framework	185
14.2 Unused Virtual Register Elimination	186
14.3 Empty Basic Block Elimination	187
14.4 Redundant Basic Block Elimination	187
14.5 Unreachable Basic Block Elimination	188
14.6 Loop-Invariant Code Motion	188
14.7 Web Generation	188
14.8 Graph Coloring-Based Register Allocation	189
14.9 Dead Code Elimination	194
14.10 Constant Folding	194
14.11 Constant Propagation	196
14.12 Redundant Code Elimination	196
14.13 Jump Correction	197
14.14 Operation Bit Width Reduction	198
14.15 Predicated Execution	199
14.16 Peephole Optimization	200
14.17 Local Instruction Scheduling	202
15 Introspection of WIR	205
15.1 Debug Macro Framework	205
15.2 WIR Debug Macros	207

III	Retargeting Guide	211
16	Retargeting of Processor Models	213
16.1	Setup of a New Processor Model's Class	213
16.2	Integration of a New Processor Model Into the WIR Library	215
16.3	Addition of Processor-Specific ISA Features	218
16.3.1	Register Types	218
16.3.2	Registers	219
16.3.3	Immediates	225
16.3.4	Addressing Modes	230
16.3.5	Condition Fields	231
16.3.6	Opcodes	231
16.3.7	Operation Formats	232
16.4	Realization of Processor-Specific Code Generation Mechanisms	235
16.5	Setup of New System Configurations	237
16.6	Implementation of Unit Tests	238
17	Retargeting of Existing Code Analyses	243
17.1	Bit-True Data Flow Analysis	243
18	Retargeting of Existing Code Optimizations	257
18.1	Graph Coloring-Based Register Allocation	257
18.1.1	Initialization	260
18.1.2	Interference Graph Creation	261
18.1.3	Coalescing	263
18.1.4	Spill Cost Computation	264
18.1.5	Color Selection	266
18.1.6	Spill Code Generation	267
18.1.7	Rematerialization	270
18.1.8	Cross-Cutting Issues	271
18.1.8.1	Calling Conventions and Registers Alive During Function Calls	271
18.1.8.2	Stack Frame Management	274
18.1.8.3	Miscellaneous	275
18.2	Dead Code Elimination	277
18.3	Constant Folding	278
18.4	Constant Propagation	280
18.5	Redundant Code Elimination	281
18.6	Jump Correction	282
18.7	Operation Bit Width Reduction	286
18.8	Predicated Execution	288
18.9	Peephole Optimization	289
18.10	Local Instruction Scheduling	291
IV	Developer's Guide	295
	Bibliography	298

List of Figures

1.1	Conceptual Structure of a WIR System	1
1.2	Class Diagram of WIR Software Model	2
1.3	Class Diagram of WIR Hardware Model	4
1.4	Derivation of Processor ISAs from WIR Hardware Model	6
3.1	Directory Structure of WIR Software Package	18
3.2	Detailed Directory Structure of ISA-Specific WIR Parts	19
5.1	Directory Structure of Installed WIR Library	27
8.1	Typical Control Flow Graphs	46
8.2	Inheritance Diagram of Class <code>WIR_Parameter</code>	52
8.3	Simplified Inheritance Diagram of Classes Modelling Immediate Parameters	55
9.1	Simplified Inheritance Diagram of WIR Processor Models	66
9.2	Simplified Inheritance Diagram of TriCore Processor Models	89
9.3	Simplified Inheritance Diagram of RISC-V RV32 Processor Models	96
9.4	Simplified Inheritance Diagram of ARM Processor Models	101
9.5	Inheritance Diagram of WIR Bus Arbitration Policies	114
10.1	System-Level WIR Memory Layout	129
13.1	Inheritance Diagram of Class <code>WIR_Container_API</code>	149
13.2	Control Flow Graph Visualized using Class <code>WIR_ControlFlowAnalysis</code>	152
13.3	Inheritance Diagram of Control Regions	156
13.4	Acyclic, Structured Control Regions	158
13.5	Cyclic, Structured Control Regions	159
13.6	Unstructured Control Regions	160
13.7	Inheritance Diagram of Flow Facts	161
13.8	Partial Order L4	174
13.9	Example of Bit-True Data Flow Graph	176
14.1	Inheritance Diagram of Scheduling Priority Classes	204
18.1	Example of Processor-Specific Interferences for Virtual TriCore Data Registers	262
18.2	Example of Processor-Specific Interferences for Virtual TriCore Extended Data Registers	263

List of Tables

1.1	Availability of WIR Analyses and Optimizations for Supported ISAs	9
4.1	Most Relevant configure Options	23
9.1	Register Types, Classes and Methods of MIPS Processor Model	87
9.2	Classes for Immediates of MIPS Processor Model	87
9.3	Opcode Types of MIPS Processor Model	88
9.4	Naming Scheme for Operation Formats of MIPS Processor Model	88
9.5	Register Types, Classes and Methods of TC13 Processor Model	89
9.6	Classes for Immediates of TC13 Processor Model	91
9.7	Opcode Types of TC13 Processor Model	92
9.8	Opcode Types of TC131 Processor Model	92
9.9	Naming Scheme for Operation Formats of TC13 Processor Model	93
9.10	Standard Extensions of the RISC-V RV32 Instruction Set	96
9.11	Register Types, Classes and Methods of RV32I, RV32IF and RV32IFD Processor Models	97
9.12	Classes for Immediates of RV32I and RV32IC Processor Models	98
9.13	Opcode Types of RV32I Processor Model	98
9.14	Opcode Types of RV32IC, RV32IF and RV32IFD Processor Models	99
9.15	Naming Scheme for Operation Formats of RV32I, RV32IC and RV32IF Processor Models	100
9.16	Register Types, Classes and Methods of ARMv4 Processor Model	101
9.17	Classes for Immediates of ARMv4, ARMv4T, ARMv5T and ARMv6 Processor Models	102
9.18	Opcode Types of ARMv4 Processor Model	102
9.19	Opcode Types of ARMv4T, ARMv5T, ARMv5TE, ARMv5TEJ and ARMv6 Processor Models	103
9.20	Naming Scheme for Operation Formats of ARMv4, ARMv4T, ARMv5T, ARMv5TE and ARMv6 Processor Models	105
10.1	System Configuration Directives for Systems, Cores and Memory Regions	120
10.2	System Configuration Directives for Caches, Buses and Logical Sections	121
12.1	Generic Stream Manipulators from file <code>wirio.h</code>	141
12.2	Processor-Specific Stream Manipulators	145
13.1	Analysis Containers and their Fixing Points Within WIR	148
14.1	Examples of TriCore Peephole Transformations	201

Listings

1.1	Excerpt of a System Configuration File	5
4.1	Directory Listing After Repository Checkout	21
4.2	Execution of <code>Makefile.bootstrap</code>	21
4.3	Directory Listing After <code>automake</code> , <code>autoconf</code> and Friends	22
4.4	Execution of <code>configure</code> Script	22
4.5	Directory Listing After <code>configure</code>	23
5.1	WIR Build using <code>make</code>	25
5.2	WIR Unit Tests	26
5.3	Simple WIR Test Program <code>wirtest.cc</code>	27
6.1	Insertion of WIR Objects at List Heads and Tails	31
6.2	Insertion of WIR Objects at Arbitrary List Positions	32
6.3	Replacement of WIR Objects in Lists	33
6.4	Modification Prevention of WIR Objects	33
6.5	Lookup of WIR Objects in Lists	34
6.6	Removal of WIR Objects from Lists	34
7.1	Construction of Object Hierarchies using List Interface	38
7.2	Variadic Construction of Object Hierarchies	39
7.3	Traversal of Object Hierarchies using Range-Based Loops	40
8.1	Constructors of Class <code>WIR_CompilationUnit</code>	41
8.2	Constructors and Methods for Name Handling of Class <code>WIR_Function</code>	42
8.3	Size Computation by Class <code>WIR_Function</code>	42
8.4	Precoloring and Interference Handling of Class <code>WIR_Function</code>	43
8.5	Handling of Physical Registers Used for Function Argument Passing	44
8.6	Stack Frame Handling of Class <code>WIR_Function</code>	44
8.7	Moving of Instructions by Class <code>WIR_BasicBlock</code>	45
8.8	Constructors and Methods for Name Handling of Class <code>WIR_BasicBlock</code>	45
8.9	Size Computation by Class <code>WIR_BasicBlock</code>	46
8.10	Control Flow Properties of Class <code>WIR_BasicBlock</code>	47
8.11	Moving of Operations by Class <code>WIR_Instruction</code>	47
8.12	Constructors of Class <code>WIR_Instruction</code>	48
8.13	Size Computation by Class <code>WIR_Instruction</code>	48
8.14	Constructors of Class <code>WIR_Operation</code>	48
8.15	Opcode and Format Handling of Class <code>WIR_Operation</code>	49
8.16	Parameter Checking of Class <code>WIR_Operation</code>	49
8.17	Explicit and Implicit Parameter Handling of Class <code>WIR_Operation</code>	50
8.18	Operation Classification	50
8.19	Jump Target Handling of Class <code>WIR_Operation</code>	51

8.20	Size Computation by Class WIR_Operation	52
8.21	Generic Type Handling of Class WIR_Parameter	52
8.22	Handling of Explicit and Implicit WIR_Parameters	53
8.23	Constructors of Class WIR_RegisterParameter	53
8.24	Usage Handling of Class WIR_RegisterParameter	54
8.25	Register Handling of Class WIR_RegisterParameter	54
8.26	Constructors of Classes for Signed and Unsigned Immediates	55
8.27	Value Handling of Signed and Unsigned Immediates	56
8.28	Constructors of Class WIR_LabelParameter	57
8.29	Label Type Handling of Class WIR_LabelParameter	57
8.30	Reference Handling of Class WIR_LabelParameter	58
8.31	Name Handling of Class WIR_LabelParameter	58
8.32	Constructors of Class WIR_AddressingModeParameter	58
8.33	Mode Handling of Class WIR_AddressingModeParameter	59
8.34	Public Interface of Class WIR_ConditionFieldParameter	59
8.35	Constructors of Class WIR_StringParameter	60
8.36	String Handling of Class WIR_StringParameter	60
8.37	Constructors and Methods for Name Handling of Class WIR_Data	60
8.38	Size Computation by Class WIR_Data	61
8.39	Initializer Handling of Class WIR_Data	61
8.40	Constructors of Class WIR_DataInit	62
8.41	Initializer Handling and String Processing of Class WIR_DataInit	62
9.1	Generic Type Handling of Class WIR_SystemComponent	65
9.2	Constructors and Methods for General Attribute Handling of Class WIR_BaseProcessor	66
9.3	Methods for Type Handling of Class WIR_Processor	67
9.4	Generic Register Type Handling of Class WIR_BaseProcessor	67
9.5	Concrete Register Type Handling (here for Class TC13)	68
9.6	Constructors of Class WIR_BaseRegister	69
9.7	Type and Name Handling of Class WIR_BaseRegister	69
9.8	Register Hierarchy Handling of Class WIR_BaseRegister	70
9.9	Distinction Between Physical and Virtual Registers in Class WIR_BaseRegister	70
9.10	Characteristic Features of Class WIR_PhysicalRegister	71
9.11	Characteristic Features of Class WIR_VirtualRegister	72
9.12	Concrete Modelling of Virtual Data Registers (here for TriCore architecture)	72
9.13	Concrete Modelling of Virtual Extended Data Registers (here for TriCore architecture)	73
9.14	Concrete Modelling of Physical Data Registers (here for TriCore architecture)	73
9.15	Physical Register Handling (here for Class TC13)	74
9.16	Physical Register Access in Class WIR_BaseProcessor	74
9.17	Derivation of Processor-Specific Immediates (here for TriCore architecture)	75
9.18	Generic Opcode Handling of Class WIR_BaseProcessor	76
9.19	Classification of Opcodes by Type WIR_OpCodeType	77
9.20	Concrete Opcode Handling (here for Class TC13)	78
9.21	Generic Operation Format Handling of Class WIR_BaseProcessor	79
9.22	Concrete Operation Format Handling (here for Class TC13)	80
9.23	Operation Format Registration (here for Class TC13)	81
9.24	Generic Addressing Mode Handling of Class WIR_BaseProcessor	82
9.25	Concrete Addressing Mode Handling (here for Class TC13)	83
9.26	Constructors and Methods for Name Handling of Class WIR_Section	84

9.27	Characteristic Attributes of Class WIR_Section	85
9.28	Methods for Section Handling of Class WIR_BaseProcessor	86
9.29	Testing of Physical Register Properties of Class TC13	90
9.30	Testing of Pipeline Properties of Class TC13	95
9.31	Testing of Physical Register Properties of Class RV32I	97
9.32	Specific Opcode Handling of Class ARMv4	104
9.33	Name, Address and Size Handling of Class WIR_MemoryRegion	106
9.34	Section Handling of Class WIR_MemoryRegion	106
9.35	Characteristic Attributes of Class WIR_MemoryRegion	107
9.36	Architectural Attributes of Class WIR_Cache	109
9.37	Geometry-Related Attributes of Class WIR_Cache	110
9.38	Address and Address Range Handling of Class WIR_Cache	111
9.39	Event Counting of Class WIR_Cache	112
9.40	Characteristic Attributes of Class WIR_Bus	113
9.41	Characteristic Attributes of Class WIR_FixedPriorityBusArbitration	114
9.42	Characteristic Attributes of Class WIR_TDMABusArbitration	115
9.43	Characteristic Attributes of Class WIR_TDMABusArbitration::SlotInfo	116
9.44	Characteristic Attributes of Class WIR_PriorityDivisionBusArbitration	117
10.1	Example for Shared Memory Hierarchy Configuration	122
10.2	Example for Cached and Uncached Memory Hierarchy Configuration	122
10.3	Constructors and Methods for Name Handling of Class WIR_System	123
10.4	System Component Handling of Class WIR_System	125
10.5	Constructors of Class WIR_Symbol	126
10.6	Type, Reference and Name Handling of Class WIR_Symbol	126
10.7	Property Handling of Class WIR_Symbol	127
10.8	Symbol Handling of Class WIR_System	128
10.9	Section Handling of Class WIR_Symbol	130
10.10	Data Layout Handling of Classes WIR_System and WIR_CompilationUnit	131
11.1	Handling of Unique Object Identifiers by Class WIR_ID_API	134
11.2	Declaration of Deterministic Data Structures in file wirtypes.h	134
12.1	Generic Stream Output of Class WIR_BasicBlock	137
12.2	Simple WIR Program Showing Stream Output	138
12.3	Declaration of defuse and nodefuse Stream Manipulators in file wirio.h	140
12.4	Constructors and Methods for Comment Text Handling of Class WIR_Comment	142
12.5	Constructors and Methods for File Location Handling of Class WIR_FileInfo	142
12.6	Declaration of Basic Block Dumper Type in file wirio.h	145
12.7	Declaration of RISC-V-Specific Basic Block Dumper	145
12.8	Dumper Function Registration (here for Class RV32I)	146
12.9	Declaration of ldscript and noldscript Stream Manipulators in file wirio.h	146
13.1	Code Analysis API of Class WIR_Analysis	147
13.2	Insertion and Removal of WIR Containers	149
13.3	Lookup of WIR Containers	150
13.4	Type Handling of Class WIR_BaseContainer	150
13.5	Public Interface of Class WIR_ControlFlowAnalysis	151
13.6	Declaration of Basic Block Pairs in file wirtypes.h	152
13.7	Reachability Handling of Container Class WIR_Reachability	153

13.8	Public Interface of Class WIR_DominationAnalysis	154
13.9	Domination Handling of Container Classes WIR_Domination and WIR_ImmediateDomination	155
13.10	Public Interface of Class WIR_StructuralAnalysis	155
13.11	Public Interface of Class WIR_ControlTreeNode	157
13.12	Public Interface of Class WIR_FlowFact	162
13.13	Flow Fact Handling of Class WIR_System	162
13.14	Public Interface of Class WIR_EntryPoint	163
13.15	Public Interface of Class WIR_LoopBound	164
13.16	Public Interface of Class WIR_FlowRestriction	166
13.17	Flow Fact Handling of Container Class WIR_FlowFactRef	167
13.18	Public Interface of Class WIR_ReachingDefinitionsAnalysis	169
13.19	Reaching Definitions and Inputs Handling of Container Class WIR_ReachingDefinitions	169
13.20	Available Definitions and Inputs Handling of Container Class WIR_AvailableDefinitions	170
13.21	Def-Use/Use-Def Chain and Inputs Handling of Container Class WIR_DUUDChain	171
13.22	Liveness Handling of Container Class WIR_LiveOut	173
13.23	Location Handling by Class WIR_Location	175
13.24	Constructors of Class WIR_UpDownValue	177
13.25	Value Handling of Class WIR_UpDownValue	178
13.26	Value Interpretation by Class WIR_UpDownValue	179
13.27	Comparison Operators of Class WIR_UpDownValue	180
13.28	Bitwise Operators of Class WIR_UpDownValue	181
13.29	Arithmetical and Miscellaneous Operators of Class WIR_UpDownValue	182
13.30	Handling of Incoming Up/Down Values by Container Class WIR_BitValues	183
14.1	Code Optimization API of Class WIR_Optimization	185
14.2	Bit-Level Optimization API of Class WIR_BitOpt	186
14.3	Public Interface of Class WIR_Webs	189
14.4	Constructors of Class WIR_GraphColoring	190
14.5	Feature Activation of Class WIR_GraphColoring	192
14.6	Public Interface of RISC-V Register Allocator from Class RV_GraphColoring	193
14.7	Public Interface of TriCore Register Allocator from Class TC_GraphColoring	193
14.8	Public Interface of Class WIR_JumpCorrection	198
14.9	Public Interface of Class WIR_Peephole	200
14.10	Constructor of Class WIR_SiliconBugs	202
14.11	Public Interface of Class WIR_LocalScheduling	202
14.12	Relevant Interface of Class WIR_SchedulingPriority	203
15.1	Most Relevant Debug Macros from WIR/LIBUSEFUL/libuseful/debugmacros.h	206
15.2	Activation Names for the In-Depth Introspection of WIR Analyses and Optimizations	209
16.1	Rudimentary Header File for a New Processor Model's Class	215
16.2	Rudimentary Implementation of a New Processor Model's Class	216
16.3	General Structure of a New Processor Model's init Method	217
16.4	Extension of function WIR_Init	217
16.5	Declaration of a New Processor Model's Register Types	218
16.6	Implementation of a New Processor Model's Register Types	219
16.7	Header File for a New Virtual Register's Class	220
16.8	Implementation of a New Virtual Register's Class	221
16.9	Header File for a New Physical Register's Class	222

16.10 Implementation of a New Physical Register's Class	223
16.11 Construction of Physical Registers by a New Processor Model's Class	224
16.12 Declaration of Access Methods for Physical Registers	224
16.13 Implementation of Access Methods for Physical Registers	225
16.14 Header File for a New Signed Immediate Parameter's Class	226
16.15 Implementation of a New Signed Immediate Parameter's Class	227
16.16 Header File for a New Unsigned Immediate Parameter's Class	228
16.17 Implementation of a New Unsigned Immediate Parameter's Class	229
16.18 Declaration of a New Processor Model's Condition Fields	230
16.19 Implementation of a New Processor Model's Condition Fields	231
16.20 Declaration of a New Processor Model's OpCodes	232
16.21 Implementation of a New Processor Model's Opcodes	233
16.22 Declaration of a New Processor Model's Operation Formats	233
16.23 Implementation of a New Processor Model's Operation Formats	234
16.24 Operation Format Registration in a New Processor Model's <code>init</code> Method	234
16.25 Rudimentary Header File for a New Processor Model's Stream Output	235
16.26 Rudimentary Implementation of a New Processor Model's Stream Output	236
16.27 Rudimentary System Configuration File	237
16.28 Rudimentary Makefile for Unit Tests	239
16.29 Makefile for Unit Tests Extended by Assembler Tests	241
17.1 Interface of Class <code>WIR_BitDFA</code> to be Retargeted	243
17.2 Header File for a New Processor Model's Bit-True Data Flow Analysis	244
17.3 Rudimentary Implementation of a New Processor Model's Bit-True Data Flow Analysis	245
17.4 Definition of Lambdas for Top-Down Analysis	246
17.5 Definition of Lambdas for Top-Down Analysis (ctd.)	247
17.6 Definition of Lambdas for Bottom-Up Analysis	252
17.7 Definition of Lambdas for Bottom-Up Analysis (ctd.)	253
18.1 Header File for a New Processor Model's Graph Coloring-Based Register Allocator	258
18.2 Rudimentary Implementation of a New Processor Model's Graph Coloring-Based Register Allocator	259
18.3 Header File for a New Processor Model's Dead Code Elimination	278
18.4 Rudimentary Implementation of a New Processor Model's Dead Code Elimination	279
18.5 Header File for a New Processor Model's Jump Correction	282
18.6 Rudimentary Implementation of a New Processor Model's Jump Correction	283
18.7 Header File for a New Processor Model's Operation Bit Width Reduction	286
18.8 Rudimentary Implementation of a New Processor Model's Operation Bit Width Reduction	287
18.9 Header File for a New Processor Model's Predicated Execution Pass	288
18.10 Header File for a New Processor Model's Peephole Optimization	289
18.11 Header File for a New Processor Model's Local Instruction Scheduler	290
18.12 Header File for a New Processor Model's Block Scheduling Region	292
18.13 Header File for a New Processor Model's Maximum Delay Scheduling Priority	293

Acronyms

ABI Application Binary Interface.

ALAP As Late As Possible.

API Application Programming Interface.

ASAP As Soon As Possible.

BGL Boost Graph Library.

CFG Control Flow Graph.

CISC Complex Instruction Set Computing.

CPU Central Processing Unit.

CSR Control and Status Register.

DFA Data Flow Analysis.

DFG Data Flow Graph.

DSP Digital Signal Processing.

ELF Executable and Linkable Format.

EPIC Explicitly Parallel Instruction Computing.

FPU Floating-Point Unit.

GCC GNU Compiler Collection.

IG Interference Graph.

ILP Instruction-Level Parallelism.

IR Intermediate Representation.

ISA Instruction Set Architecture.

LMA Loaded Memory Address.

LTO Link-Time Optimization.

MMU Memory Management Unit.

PSW Program Status Word.

RAM Random-Access Memory.

RAW Read-After-Write.

RISC Reduced Instruction Set Computing.

ROM Read-Only Memory.

SIMD Single Instruction, Multiple Data.

SoC System-on-Chip.

SRAM Static Random-Access Memory.

STL Standard Template Library.

TDMA Time-Division Multiple Access.

VLIW Very Long Instruction Word.

VMA Virtual Memory Address.

WAR Write-After-Read.

WAW Write-After-Write.

WCC WCET-Aware C Compiler.

WCET Worst-Case Execution Time.

WIR WCC Intermediate Representation.

1 Introduction

WCC Intermediate Representation (WIR) is an open source, low-level **Intermediate Representation (IR)** for compilers that intends to model software and architectural hardware characteristics at the level of and from the perspective of assembly code. Its design has been driven by the needs of and experiences from the work on back-ends and optimizations within the **WCET-Aware C Compiler (WCC)** [10, 33], which also explains its name.

In contrast to classical compiler **IRs**, **WIR** not only models software code in the form of assembly operations, but it aims to model all aspects of an entire hardware/software system that are relevant for the purpose of code generation and optimization. This way, **WIR** also provides means to specify, e.g., memory hierarchies and their properties, multi-core architectures, interconnects etc. **WIR** is not bound to some fixed **Instruction Set Architecture (ISA)**, but it can be retargeted to a broad variety of different instruction sets, typically from the domain of embedded processor architectures. Last but not least, **WIR** comes with a large set of static code analyses and of code optimizations. These analyses and optimizations are kept generic and processor-independent as far as possible, and specializations for specific processor architectures exist.

Section 1.1 gives an overview about the key concepts influencing the design of **WIR**. Section 1.2 shortly describes the key features of the **WIR** library. After that, Section 1.3 specifies the license conditions that apply in order to use, modify and distribute **WIR**. Finally, Section 1.4 describes the overall structure of this document.

1.1 Overview

WIR is a C++ library containing classes that model the various facets of a combined hardware/software system. General properties of such systems which are independent of some actual processor architecture are realized as (abstract) base classes. Object-orientation and inheritance are used to derive specialized classes for dedicated processor architectures.

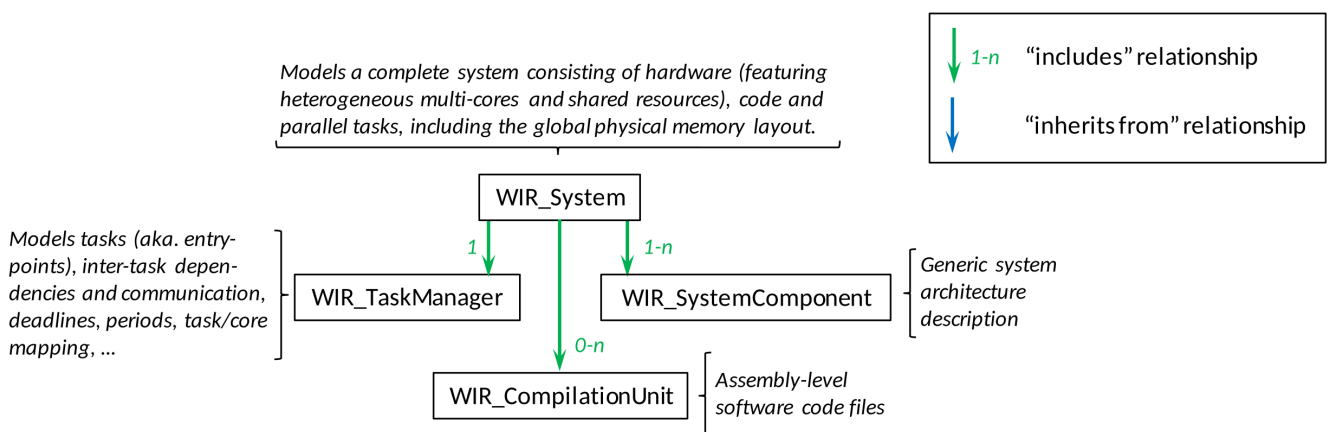


Figure 1.1: Conceptual Structure of a WIR System

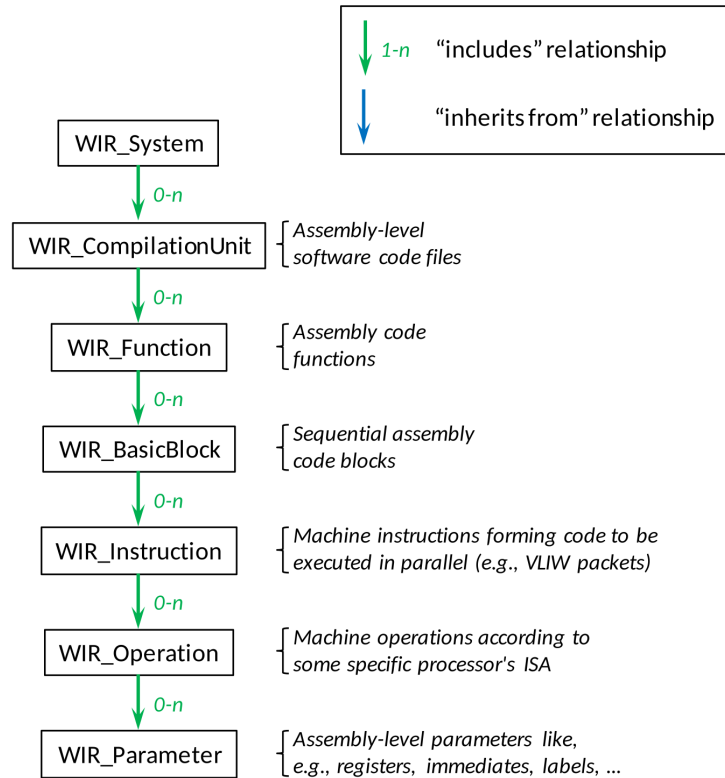


Figure 1.2: Class Diagram of WIR Software Model

The top level of abstraction within **WIR** is modeled by a **WIR_System**. This class represents a complete heterogeneous multi-core hardware and software system (cf. Fig. 1.1). Such a system consists of hardware components (**WIR_SystemComponent**) which stand for, e.g., processor cores, memories, shared buses, etc. The software side of a complete system can stem from various assembly code files which are all represented by class **WIR_CompilationUnit**. Since in real-life systems, software can also be structured into several tasks that are supposed to be executed concurrently under a certain task scheduling and preemption scheme, class **WIR_TaskManager** is intended to model such task-level aspects.

Future Work

At the time of writing of this document, the class **WIR_TaskManager** is not yet designed and implemented in detail. Currently, it is just a dummy class that is fully unaware of any task-specific properties like, e.g., periods, deadlines, scheduling policies etc. However, in order to create a **WIR_System**, one instance of such a dummy task manager has to be passed. It is one of the areas of future work to appropriately realize all required task-level aspects inside **WIR**. For this reason, any more specific description of tasks within **WIR** is omitted in this document. Instead, the emphasis will lie on the software code and on the hardware aspects in the following.

1.1.1 Software Model

The software model of **WIR** is structured into a hierarchy of classes ranging from **WIR_CompilationUnit** down to **WIR_Parameter** as depicted in Fig. 1.2.

As mentioned above, an object of class **WIR_CompilationUnit** represents one single assembly code file. Such

a file can consist of zero to several `WIR_Functions` which call themselves, manage their stack frames, and realize argument and return value passing. Internally, functions themselves are grouped into `WIR_BasicBlocks` where a basic block is defined as follows [22]:

Definition 1.1 (Basic Block)

A basic block is a maximal sequence of instructions that can be entered only at the first of them and exited only from the last of them. Thus, the first instruction in a basic block may be (1) the entry point of the function, (2) a target of a branch, or (3) an instruction immediately following a branch or a return.

By definition, basic blocks model sequences of straight-line code where the execution of a basic block can only start with its very first instruction and must always end with its very last one.

Basic blocks are formed of objects of class `WIR_Instruction`. Conceptually, all the contents of one instruction is supposed to be executed in parallel, at the same point in time. `WIR` instructions are thus the key concept to model [Instruction-Level Parallelism \(ILP\)](#). The exploitation of [ILP](#) and thus the use of `WIR` instructions can generally be driven by two factors: (1) a processor's [ISA](#) mandates to group several operations into one large instruction (e.g., [Very Long Instruction Word \(VLIW\)](#) architectures), or (2) an optimization like, e.g., instruction scheduling, clusters operations into instructions in order to parallelize as many operations as possible.

Objects of class `WIR_Operation` next represent classical machine operations at the level of assembly code. Thus, they basically consist of an opcode that specifies the operator to be computed by an operation. Besides all supported opcodes, processor [ISAs](#) furthermore specify which format each machine operation must have, i.e., how many arguments of which type an operation expects, which arguments are used or defined by an operation, which bit-sizes such arguments can have, etc. In order to support the modelling of only [ISA](#)-compliant machine operations, `WIR` features structured operation formats which capture exactly the above properties of all possible kinds of machine operations.

The arguments of machine operations are represented by objects of class `WIR_Parameter`, and such parameters can finally be registers, immediate values, labels etc.

All the classes presented in this section are generic in that sense that they do not depend on any specific processor architecture. This generic assembly-level software model of `WIR` thus becomes processor-specific by connecting it to the hardware model introduced in the next section.

1.1.2 Hardware Model

Fig. 1.3 depicts the overall structure of the `WIR` hardware model. Class `WIR_SystemComponent` is a base class modelling various kinds of building blocks from which `WIR` systems can be composed. Using inheritance, specialized classes modelling processor cores, physical memory regions, caches and buses are derived.

The central class of the `WIR` hardware model is `WIR_Processor` which serves as abstraction of a processor core with a specific [ISA](#). Objects derived from class `WIR_Processor` provide structured information about a processor core's actual register file. For that purpose, `WIR` processors support the specification of various register types (e.g., general-purpose registers or specific address or floating-point registers). For each register type, its bit width is provided and `WIR_Registers` can be composed to form hierarchical registers so that, e.g., a pair of neighboring 32-bit registers forms a 64-bit extended register. Using these concepts of class `WIR_Register`, a `WIR` processor provides access to all physical registers that an [ISA](#) actually foresees in real hardware.

In addition, `WIR` also inherently supports the concept of virtual registers. Here, the severely limited availability of physical hardware registers is neglected and an infinite amount of such virtual registers is assumed. Using

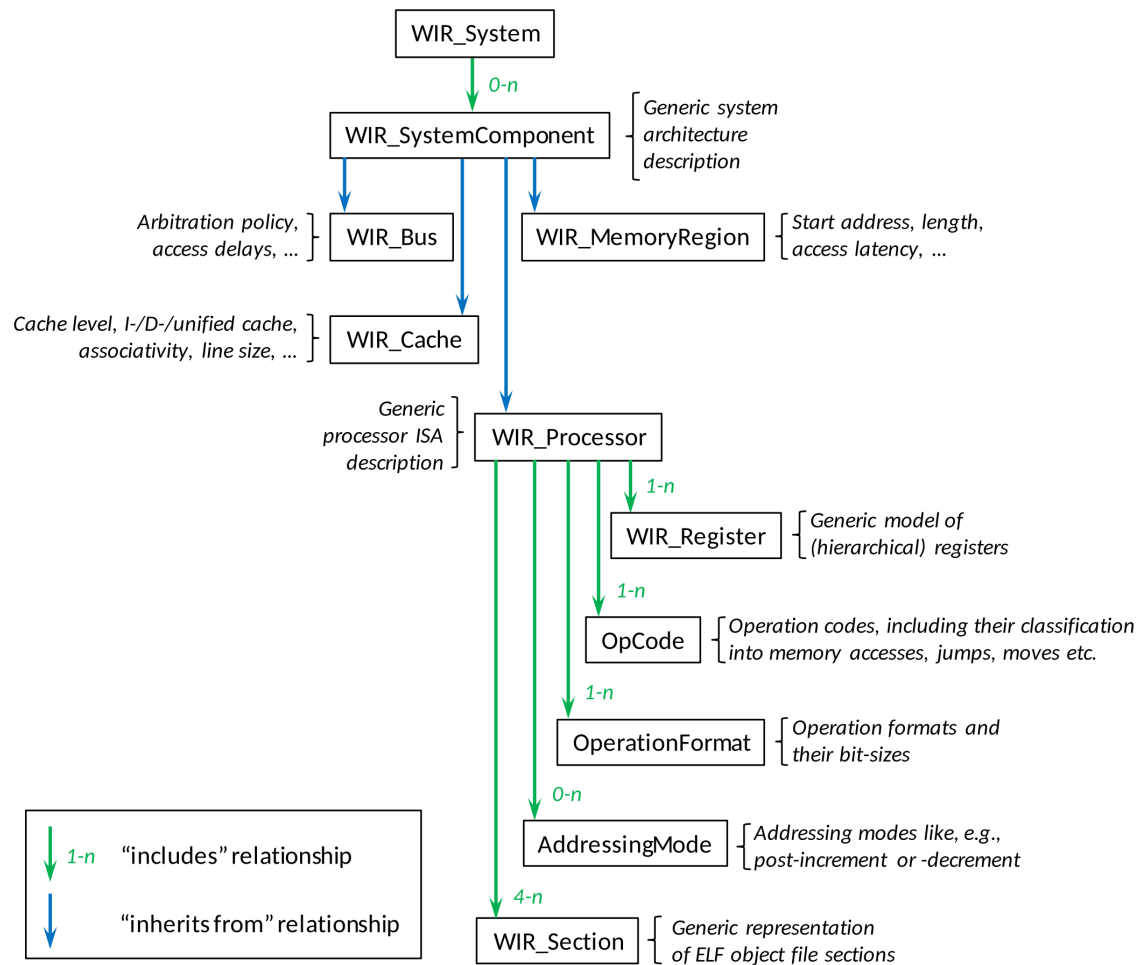


Figure 1.3: Class Diagram of WIR Hardware Model

virtual registers is crucial when implementing **WIR**-based code selectors or optimizations within a compiler, since this allows to not having to worry about the availability of physical registers—whenever **WIR** code shall store something new in a register, a fresh virtual register can be created just for this purpose. It is then the duty of a subsequent register allocator to assign all existing virtual registers to exactly those few physical registers that **WIR_Processor** actually features.

As already introduced in Section 1.1.1, **WIR** operations rely on **ISA**-specific operation codes and formats. They are also part of the hardware model and come in the form of the classes **OpCode** and **OperationFormat**. Similarly, class **AddressingMode** serves to specify all addressing modes supported by a processor's **ISA**.

WIR processors are closely linked to objects of class **WIR_Section** which model object file sections like, e.g., `.text`, `.data`, `.bss` or `.rodata`. For each such section, attributes for, e.g., alignments or start addresses can be specified. **WIR_Sections** are integral part of the memory layout model of **WIR**, which is complemented by objects of class **WIR_MemoryRegion**. A memory region models a physical memory region of a system architecture together with key characteristics like, e.g., base addresses and sizes, access delays or read/write/execute attributes.

This memory layout model allows for a very simple and flexible modelling of memory-related aspects of **WIR** systems during code analyses or optimizations. **WIR_Sections** model the logical memory structure of a **WIR** system, while **WIR_MemoryRegion** models physical hardware characteristics of memories. Sections can now simply be mapped to regions, and objects of the **WIR** software model (cf. Section 1.1.1) like, e.g., whole functions,

Listing 1.1: Excerpt of a System Configuration File

```

1 #
2 # Infineon TriCore TC1796
3 #
4 [tc1796]
5 type          = system
6
7 #
8 # Processor core (ISA TC1.3)
9 #
10 [CORE0]
11 type          = core
12 isa           = TC1.3
13 clockfreq     = 150000000
14 voltage       = 1.5
15
16 #
17 # Program Flash with cached access (PFLASH; PMU; L2)
18 #
19 [PFLASH-C]
20 type          = memory
21 origin        = 0x80100000
22 length        = 0x100000 # 1024K
23 dynamic_allocation = 0
24 attributes    = RXAC      # read/execute/allocate/cached
25 cycles        = 6         # waitstates; possible values are 1-7
26 burstcycles   = 1
27 memory_type   = PFLASH
28 sections      = .text_cached
29 hierarchy     = CORE0 L1-I

```

basic blocks or global data, can be mapped to sections. This model allows for a simple realization of [Link-Time Optimizations \(LTOs\)](#) where knowledge about physical memory addresses of objects is key. Optimizations can exploit different memories by simply assigning [WIR](#) objects to a section mapped onto a fast or energy-efficient physical memory.

Modern system architectures typically form a complete [System-on-Chip \(SoC\)](#) with memory hierarchies consisting of several physical memory regions as well as caches. For this reason, class `WIR_Cache` is also part of the hardware model. Objects of this class allow to model Level-1 and Level-2 instruction, data or unified caches. Key architectural parameters like, e.g., associativity, line sizes, write policies, amounts of offset, index and tag bits, access delays etc. are supported.

Finally, interconnects are required in the hardware model to connect the processor cores to the various components of the memory hierarchy. This is the purpose of objects of class `WIR_Bus` that allows to model different arbitration policies (fixed priorities, round robin, [Time-Division Multiple Access \(TDMA\)](#)), bus access delays etc.

All these components of the [WIR](#) hardware model can be easily instantiated and composed to form complete systems using simple system-level configuration files. Listing 1.1 shows an excerpt of such a configuration file for an Infineon TriCore TC1796 system. As can be seen, one processor core based on the TC1.3 [ISA](#) is instantiated (Lines 10 to 14). It comes with a memory region for program code being 1,024 K large and having an access delay of 6 clock cycles (Lines 19 to 29). The logical memory section `.text_cached` is mapped to this region (Line 28), and between the processor core and this region, an L1 instruction cache is placed (Line 28).

Since all classes shown in Fig. 1.3 are still generic and independent of some actual processor architecture,

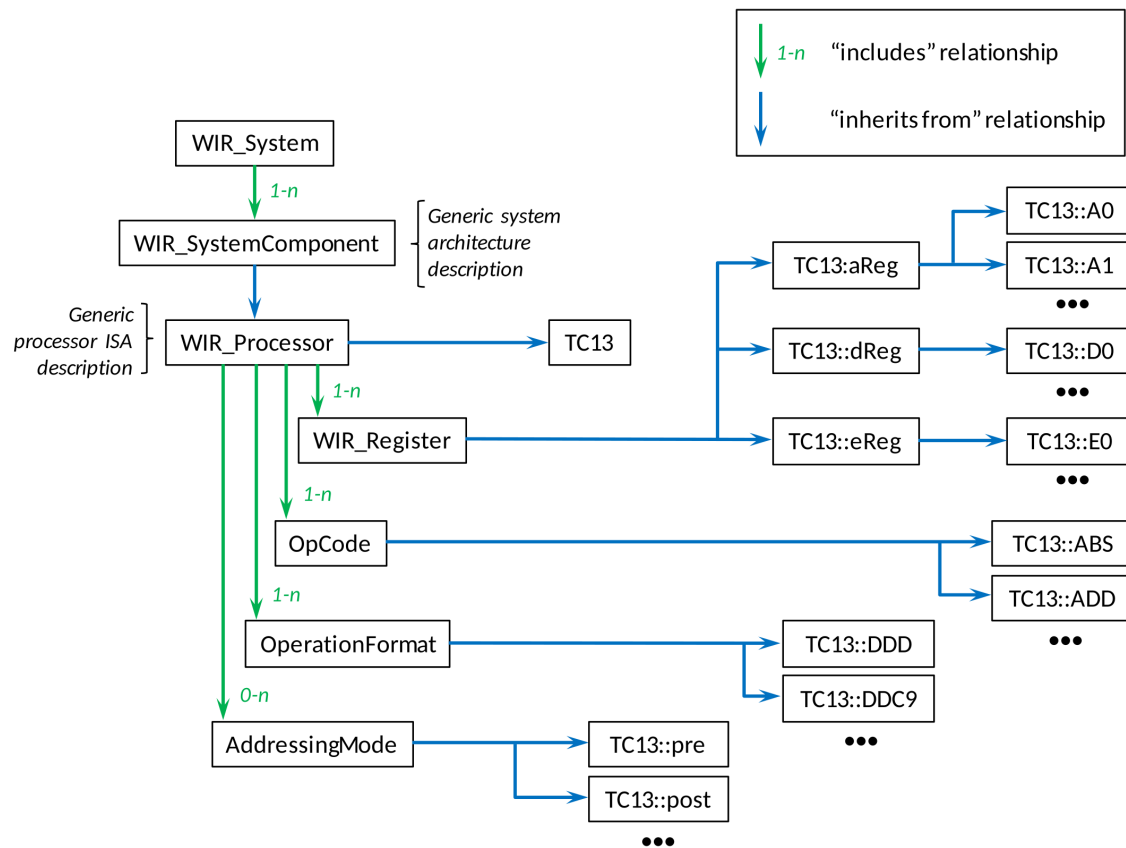


Figure 1.4: Derivation of Processor ISAs from WIR Hardware Model

inheritance is used within **WIR** to derive specific processor **ISAs** (cf. Fig. 1.4). This example inheritance diagram refers to the TC13 **ISA** already mentioned above, which is inherited from class `WIR_Processor`. As register types, classes `aReg`, `dReg` and `eReg` are derived which represent the processor's address, data and extended data registers. Consequently, physical processor registers of these types ranging from A0, ..., A15, D0, ..., E0, ..., E14 are provided. Class `OpCode` is specialized such that it actually enumerates all available TC13 operation codes ranging from ABS to XOR. T. **ISA**-specific operation formats are derived by inheriting from class `OperationFormat`. In Fig. 1.4, DDD denotes a 32-bit operation expecting 3 data registers as parameters where the first one is defined and the last two ones are used. Similarly, operation format DDC9 represents 32-bit operations where, in contrast to above, the third parameter is a signed 9-bit immediate constant. Finally, TC13-specific addressing modes are provided like, e.g., `pre` and `post` standing for pre- and post-increment/decrement, resp.

Based on the above example, generic **WIR** operations as introduced in Section 1.1.1 can now use TC13-specific opcodes, operation formats, registers etc., thus leading to highly specific **WIR** hardware/software systems based on generic concepts as far as at all possible.

1.2 Key Features

As already outlined in the previous section, the key conceptual features of **WIR** are

- the holistic modelling of *combined hardware and software systems*;
- the entirely *generic* concepts to realize the *hardware, the software as well as the system model*;

- its *ISA-safeness* ensuring that only machine operations of well-defined formats can be created so that compliance with a processor manufacturer's ISA specifications can be achieved;
- a *container infrastructure* that allows to add arbitrary kinds of meta-data like, e.g., analysis results or profiling data, to all parts of the WIR software model;
- the *integrated memory layout* providing an easy mapping of software components onto memory regions and/or processor cores;
- its *composability* allowing to model systems featuring heterogeneous multi-cores and complex memory hierarchies;
- the *retargetability of the hardware model* to almost any relevant architecture nowadays, including (but not limited to) Reduced Instruction Set Computing (RISC), Digital Signal Processing (DSP), VLIW, Explicitly Parallel Instruction Computing (EPIC), Single Instruction, Multiple Data (SIMD), Network Processors.

WIR is not only a simple representation of intermediate code for optimizing compilers, but it is a very powerful library coming with a large number of static code analyses as they are described in standard textbooks on compiler design [1, 22]. Currently, the following code analyses are integral part of WIR:

- *Control Flow Analysis* provides a generic representation and visualization of Control Flow Graphs (CFGs), classification of CFG edges as, e.g., regular or back edges, CFG traversal in depth-first or reverse-topological order etc.
- *Domination Analysis* determines dominators and immediate dominators according to [1, chapter 18.1] or [22, chapter 7.3].
- *Reachability Analysis* computes sets of basic blocks that might be reachable from a given position in the CFG.
- *Structural Analysis* [22, chapter 7.7] creates a hierarchical control tree formed out of nodes modelling, e.g., if-then- or if-then-else constructs, pre- or post-checked loops, switch/case constructs, proper or improper control structures etc.
- *Reaching Definitions Analysis* [1, chapter 17.2][22, chapter 8.1] determines which assignments to each register *may*, by *some* control flow path, reach any particular point in a WIR function.
- *Available Definitions Analysis* is very similar to reaching definitions, but it is a *must* analysis that determines whether a definition reaches a use along *all* control flow paths.
- *Def-Use and Use-Def Chain Analysis* [1, chapter 17.4][22, chapter 8.10] bases on reaching definitions and provides a sparse representation of data flow information. A Def-Use chain for a variable connects a definition of that variable to all the uses it may flow to, while a Use-Def chain connects a use to all the definitions that may flow to it.
- *Liveness Analysis* [1, chapter 10] determines which registers may be “in use” at the same time within some WIR function.
- *Bit-true Data Flow Analysis* [32] computes the flow of definitions and uses not with the granularity of individual registers, but it instead analyzes data flow for single bits of some register. Furthermore, this analysis provides detailed information about potential values of bits, i.e., whether they are 0 or 1, whether they don't care, or whether the original location of some bit is known.

Based on the aforementioned analyses, WIR comes with the following code optimizations:

- *Unused Virtual Register Elimination* removes all unused virtual registers from WIR functions.

-
- *Empty Block Elimination* removes all empty basic blocks from [WIR](#) functions.
 - *Redundant Block Elimination* merges two succeeding basic blocks into one whenever the latter can only be reached from the former.
 - *Unreachable Block Elimination* removes all unreachable basic blocks.
 - *Loop-Invariant Code Motion* [22, chapter 13.2] recognizes computations in loops that produce the same value on every iteration of the loop, and moves them out of the loop.
 - *Web Generation* determines, for each virtual register, all maximal unions of intersecting def-use chains, which are called webs [22, chapter 8.10]. For each identified web of some virtual register, a new virtual register is created, replacing the original one. This way, live range splitting is effectively realized since the newly created virtual registers become subject to register allocation.
 - *Graph Coloring-based Register Allocation* assigns all virtual registers present in [WIR](#) functions to physical registers. Its implementation bases on [22, chapter 16.3][1, chapter 11][5, 3, 6]. The [WIR](#) register allocator inherently supports hierarchical registers; precolored virtual registers; live range splitting based on webs; safe coalescing using both Briggs' and George's heuristics; Chaitin's spilling heuristic taking individual positions of defs/uses, their loop nesting depths, the graph node's degree and whether source/target of a move instruction are spilled into account; Bernstein's best-of-three spilling; stack frame coalescing.
 - *Dead Code Elimination* [1, chapter 17.3][22, chapter 18.10] removes all [WIR](#) operations whose results are not used on any control flow path leading from the operation.
 - *Constant Folding* [22, chapter 12.1] replaces [WIR](#) operations that are known to produce only constant results by compile-time evaluated constants.
 - *Constant Propagation* [22, chapter 12.6] typically is a transformation that, given an assignment of a constant to a register, replaces later uses of the register by the constant. In the context of [WIR](#) being a low-level [IR](#), this regularly boils down to replacing register parameters of operations by immediates in order to make constant operands explicit.
 - *Redundant Code Elimination* builds upon the aforementioned bit-true data flow analysis and determines whether two positions in a [WIR](#) function produce equivalent register values in a bit-true sense. If so, one code position is eliminated and only the remaining one is kept. This transformation can be seen as a kind of copy propagation [1, chapter 17.3][22, chapter 12.5], generalized to the level of bit values inside registers.
 - *Jump Correction* uses the memory layout infrastructure of [WIR](#) and determines the addresses of jumps and of their jump targets. Should the displacement between a jump and its target be too large for the jump's operation format, e.g., due to some memory allocation optimization, this pass generates specific code for long jumps.
 - *Operation Bit Width Reduction* replaces operations of large bit widths by more compact ones in order to reduce code size.
 - *Predicated Execution* replaces conditionally executed operations by variants featuring a predicate in order to avoid branching.
 - *Peephole Optimization* supports very local optimizations where a small window or peephole is moved across the [WIR](#) code to search for specific code patterns subject to optimization.

Table 1.1: Availability of WIR Analyses and Optimizations for Supported ISAs

	ARMv4	ARMv4T	ARMv5T	ARMv5TE	ARMv5TEJ	ARMv6	MIPS SPIM	RV32IMC	RV32IFD	TC1.3	TC1.3.1
Analyses	Control Flow	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Domination	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Reachability	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Structural Control Flow	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Reaching Definitions	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Available Definitions	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Def-Use/Use-Def Chains	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Liveness	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Bit-true Data Flow							✓		✓	✓
Optimizations	Unused Virtual Registers	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Empty Blocks	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Redundant Blocks	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Unreachable Blocks	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Loop-Invariant Code Motion	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Webs	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Register Allocation							✓	(✓)	✓	✓
	Dead Code							✓	✓	✓	✓
	Constant Folding							✓	✓	✓	✓
	Constant Propagation							✓	✓	✓	✓
	Redundant Code							✓	✓	✓	✓
	Jump Correction									✓	✓
	Operation Bit Widths							✓		✓	✓
	Predicated Execution									✓	✓
	Peephole									✓	✓
	Instruction Scheduling									✓	✓

- *Local Instruction Scheduling* [22, chapter 13.2] reorders operations locally inside basic blocks in order to overlap their execution whenever possible. **As Soon As Possible (ASAP)** and **As Late As Possible (ALAP)** can be used for scheduling, and various priority heuristics using, e.g., maximum delays or mobility of operations can be applied.

All aforementioned analyses and optimizations are designed such that they base on the generic and processor-independent concepts of **WIR** as far as possible. Processor-specific adaptations are again realized using inheritance from the analyses' and optimizations' base classes, enabling the support of different architectures with only moderate effort.

At the time of writing of this document, the following **ISAs** are available as **WIR** hardware models:

- *ARM* instruction sets ARMv4, ARMv4T, ARMv5T, ARMv5TE, ARMv5TEJ, ARMv6 [2]
- *MIPS* SPIM instruction set [21]

- *RISC-V* base integer instruction set RV32I including [ISA](#) standard extensions “M” (integer multiplication and division), “F” (single-precision floating-point), “D” (double-precision floating-point) and “C” (compressed instructions) [24], and all possible combinations of these
- *Infineon TriCore* instruction sets V1.3, V1.3.1 [31]

Based on these [ISAs](#), the following actual hardware implementations are modeled as [WIR](#) systems:

- NXP LPC2880 (ARM7TDMI)
- STMicroelectronics STM32F051xx (ARM Cortex-M0)
- Espressif ESP32-C3 (RV32IMC)
- RISC-V RudolV (RV32IMC)
- Infineon TriCore TC1796 (TriCore v1.3)
- Infineon TriCore TC1797 (TriCore v1.3.1)

Table 1.1 shows in how far all the analyses and optimizations described above are available for the various [ISAs](#) supported by [WIR](#). Due to their maximally generic design, many of them are available for all [ISAs](#) out of the box. The Infineon TriCore [ISAs](#) are supported by all analyses and optimizations, followed by RISC-V which currently is under continuous development. The MIPS hardware model is not under active development, it basically serves as a demonstrator for the various design concepts of [WIR](#).

Future Work

Fully supporting the ARM and RISC-V models by the analyses and optimizations is part of the future work.

The code base of [WIR](#) comes with the following key features:

- entirely written in *modern C++17*, with a *clean and straightforward [Application Programming Interface \(API\)](#)*;
- *public interfaces* of [WIR](#) classes are almost completely *free of pointers*; instead, [WIR](#) maximally exploits references in order to avoid memory mismanagement;
- all [WIR](#) classes come with a *clear concept of ownership* featuring *automatic destruction of objects*, this way eliminating the need for explicit object deletion in application code;
- *regular code quality checks* via nightly builds using the static analyzers `cppcheck` and `clang's scan-build` *demonstrate the absence of any issues* in the [WIR](#) code base;
- *dynamic code quality checks* based on tens of thousands of compiler runs using [WIR](#) built with active `AddressSanitizer` and memory protection perform *without any observed issue* like, e.g., memory leaks, use-after-free errors etc.;
- [WIR](#) comes with a series of unit tests covering both its generic as well as [ISA](#)-specific parts; these *unit tests help to avoid regressions* in the code base, and also *serve as simple code examples* how to create and manipulate the various [WIR](#) classes.

1.3 License

The code of the [WIR](#) library is © Heiko Falk, 2015 – 2026, and it is made available under the MIT license enclosed with the software.

Over and above the legal restrictions imposed by this license, if you use this software for an academic publication then you are obliged to provide proper attribution. This shall come in the form of a citation in your publication such as **TODO: DOIs**

TODO

H. Falk. The WCC Intermediate Representation (WIR) Framework. J. Awes. Softw. 12, 37654, DOI ??? (2026).

or in the form of attributions directly in this code such as

H. Falk. WIR: The WCC Intermediate Representation, release 3.0 (2026-08-25). <https://github.com/TUHH-IES/WIR>, DOI [10.5281/zenodo.22102094](https://doi.org/10.5281/zenodo.22102094).

or (ideally) both. The full license text as enclosed with the software is:

MIT License

Copyright 2015 - 2026, Heiko Falk.

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

1.4 Outline

The typical use cases in the context of [WIR](#) are:

- How to build and install the software?
- How to use the software as is in my own compiler project?
- How to extend [WIR](#) by support for new target architectures and [ISAs](#)?
- How to extend [WIR](#) by new features and concepts, analyses and optimizations?

In order to address these topics, the rest of this document is structured into the following parts which aim to provide more details about the different scenarios how to use and work with [WIR](#):

- Part [I](#): Build & Installation Guide (Page [15](#) ff.)
- Part [II](#): User's Guide (Page [31](#) ff.)
- Part [III](#): Retargeting Guide (Page [213](#) ff.)
- Part [IV](#): Developer's Guide (Page [297](#) ff.)

This manual does not contain any form of a [WIR](#) Reference Guide, since this would exceed the limits of a reasonably manageable document by far. Instead, the reader is referred to the browsable HTML documentation of [WIR](#) that can be generated using doxygen as described in Section [4.2](#).

Part I

Build & Installation Guide

Outline

Purpose of this guide is to explain with some actual examples how to compile and use [WIR](#) when it has just freshly been checked out of its source code repository.

Chapter [2](#) (Page [15](#) ff.) specifies both the mandatory and optional dependencies that must be fulfilled in order to build and use [WIR](#). After that, Chapter [3](#) (Page [17](#) ff.) gives a quick overview of the [WIR](#) directory structure. The procedures required to bootstrap and configure the [WIR](#) software package are described in Chapter [4](#) (Page [21](#) ff.). How to finally build and install [WIR](#) as well as first steps to use the built libraries are presented in Chapter [5](#) (Page [25](#) ff.).

2 Prerequisites

This chapter describes the dependencies that are required to build and use [WIR](#). Section [2.1](#) first discusses the mandatory dependencies that must always be satisfied, and Section [2.2](#) summarizes optional prerequisites.

2.1 Mandatory Dependencies

[WIR](#) is currently known to work under all recent Linux distributions. Active development and testing of [WIR](#) is carried out on Ubuntu systems, Ubuntu versions 22.04 LTS or younger fulfill all prerequisites out of the box. [WIR](#) builds and works smoothly both for 32- and 64-bit Linux systems. Any other operating system like, e.g., Microsoft Windows, is currently completely unsupported. Even though the code base of [WIR](#) consists of clean and host system-independent C++ code that might easily be ported to other platforms, porting the build system might require some effort.

As host compilers, a recent installation of a C++ compiler with full support of the C++17 language standard is required. Currently, both the C++ compiler g++ from the [GNU Compiler Collection \(GCC\)](#) [\[11\]](#) as well as clang++ from the LLVM project [\[20\]](#) are supported.

The build process is entirely driven by Makefiles. For this reason, the existence of GNU make [\[16\]](#) is mandatory. Furthermore, the maintenance of all Makefiles as well as the configuration of a [WIR](#) build is done using the GNU Build System [\[14\]](#). Due to this, proper installations of autoconf, autoheader, automake and libtool on the host system are required.

All internal data structures used by [WIR](#) build on the C++ Standard Library, also known as [Standard Template Library \(STL\)](#). Of high relevance for [WIR](#) are containers like, e.g., `std::list`, `std::map` or `std::set`, as well as algorithms to efficiently find or manipulate items in such containers. Also, the internal memory management of [WIR](#) relies on this standard library due to the consequent use of `std::unique_ptr`.

The C++ Standard Library, however, does not come with efficient data structures and algorithms for graphs, which are especially critical for an [IR](#) such as [WIR](#). For this reason, [WIR](#) has a strong mandatory dependence on the Boost C++ Libraries [\[4\]](#), especially on the [Boost Graph Library \(BGL\)](#).

Finally, the Infineon TriCore hardware model of [WIR](#) comes with an assembly parser, allowing to create operations directly from assembly code files. As usual, such a parser is realized using standard scanner and parser generators. Thus, the corresponding GNU tools `flex` [\[15\]](#) and `bison` [\[13\]](#) are required by [WIR](#), too.

2.2 Optional Dependencies

If the optional generation of browsable HTML reference documentation for all [WIR](#) classes is configured (cf. Section [4.2](#)), the documentation generator tool `doxygen` [\[8\]](#) is needed.

doxygen not only generates readable text for all [WIR](#) classes. In addition, it also produces graphs visualizing inheritance, use and include relationships. For this purpose, the graph visualization software Graphviz [17] and its dot language are furthermore required. The tool xdot is invoked by [WIR](#) whenever internal graph data structures like, e.g., [CFGs](#), [Data Flow Graphs \(DFGs\)](#) or interference graphs shall be inspected.

This present manual has been typeset using $\text{\LaTeX}$. Should it have to be re-generated from its source files, a full installation of pdf $\text{\LaTeX}$ is required, including all standard packages (e.g., KOMA), the bibliography infrastructure bib $\text{\LaTeX}$ and biber, as well as the glossary generator makeglossaries.

[WIR](#) comes with a collection of unit tests covering both the generic parts of [WIR](#) as well as its [ISA](#)-specific modules. For the target architectures currently modeled in [WIR](#), it is important to verify that an assembly dump of all [WIR](#) operations of an [ISA](#) is syntactically correct and well accepted by a subsequent assembler tool. Thus, these unit tests internally call assemblers for all relevant [ISAs](#), making, e.g., arm-elf-as, riscv32-unknown-elf-as or tricore-as additional, optional dependencies of [WIR](#).

3 Directory Structure

Fig. 3.1 depicts the overall directory structure of the [WIR](#) software package. The top-level directory `WIR` contains a couple of files specifying authors having contributed to the software, the license from Section 1.3, a brief readme file, and the documentation about the major release versions of [WIR](#). The most important file in this top-level directory is `Makefile.bootstrap` which serves to bootstrap the software package after it has been checked out of its repository (cf. Section 4.1).

The entire C++ code base of [WIR](#) is structured into the various sub-directories shown in Fig. 3.1. Here, sub-directory `wir` plays a central role as it contains the header files and implementations of all generic and processor-independent classes of [WIR](#). This in particular includes all the components introduced in Section 1.1 and depicted in Figs. 1.1 to 1.3 like, e.g., `WIR_CompilationUnit`, `WIR_Function` or `WIR_Processor`.

All [ISA](#)-specific parts of [WIR](#) (cf. Fig. 1.4) reside in the `arch` sub-directory. All currently supported [WIR](#) hardware models as given in Table 1.1 can be found here, grouped into sub-directories `arch/arm` (ARMv4 – ARMv6), `arch/generic` (MIPS SPIM), `arch/riscv` (RV32IMFDC), and `tricore` (TriCore v1.3 and v1.3.1).

The generic and [ISA](#)-independent implementations of the various code analyses and optimizations introduced in Section 1.2 can be found in the sub-directories `analyses` and `optimizations`, resp. Again, these sub-directories contain a dedicated folder for the implementation of each individual analysis or optimization like, e.g., `analyses/availabledefinitions` or `optimizations/constfold`.

Sub-directory `containers` comprises a large part of the various container classes for meta-data that are used to attach, e.g., analysis results to [WIR](#) objects (cf. Section 1.2). For example, the classes storing the results of available definitions, domination, or liveness analyses can be found here. A special category of meta-data are so-called flow facts which reside in sub-directory `flowfacts`. Flow facts are meta-information which provide hints about the set of possible control flow paths of a program [18]. Such flow facts are especially needed for static timing analysis of code, i.e., for [Worst-Case Execution Time \(WCET\)](#) analysis. [WIR](#) provides flow facts allowing to annotate, e.g., minimum and maximum loop iteration bounds or inequations over execution frequencies of blocks.

All unit tests targeting the validation of the generic, processor-independent features of [WIR](#) reside in sub-directory `tests`. This covers, e.g., the creation and traversal of objects forming the [WIR](#) software model (cf. Fig. 1.2), or the inspection of control trees produced by structural analysis. Since even such generic tests at some stage must refer to some actual processor's machine operations, all these unit tests build on the generic MIPS hardware model due to its simplicity.

Sub-directory `doc` is the place where the entire documentation of [WIR](#) can be found, esp. the optionally generated browsable HTML documentation (cf. Section 4.2) as well as this manual and its $\text{\LaTeX}$ sources.

Finally, sub-directory `DEBUGMACRO_CONF` contains configuration files that can be used to control the activation or deactivation of introspection output produced by [WIR](#) (cf. Section 15.2). This debug infrastructure bases on a small but useful support library which resides in sub-directory `LIBUSEFUL` [19].

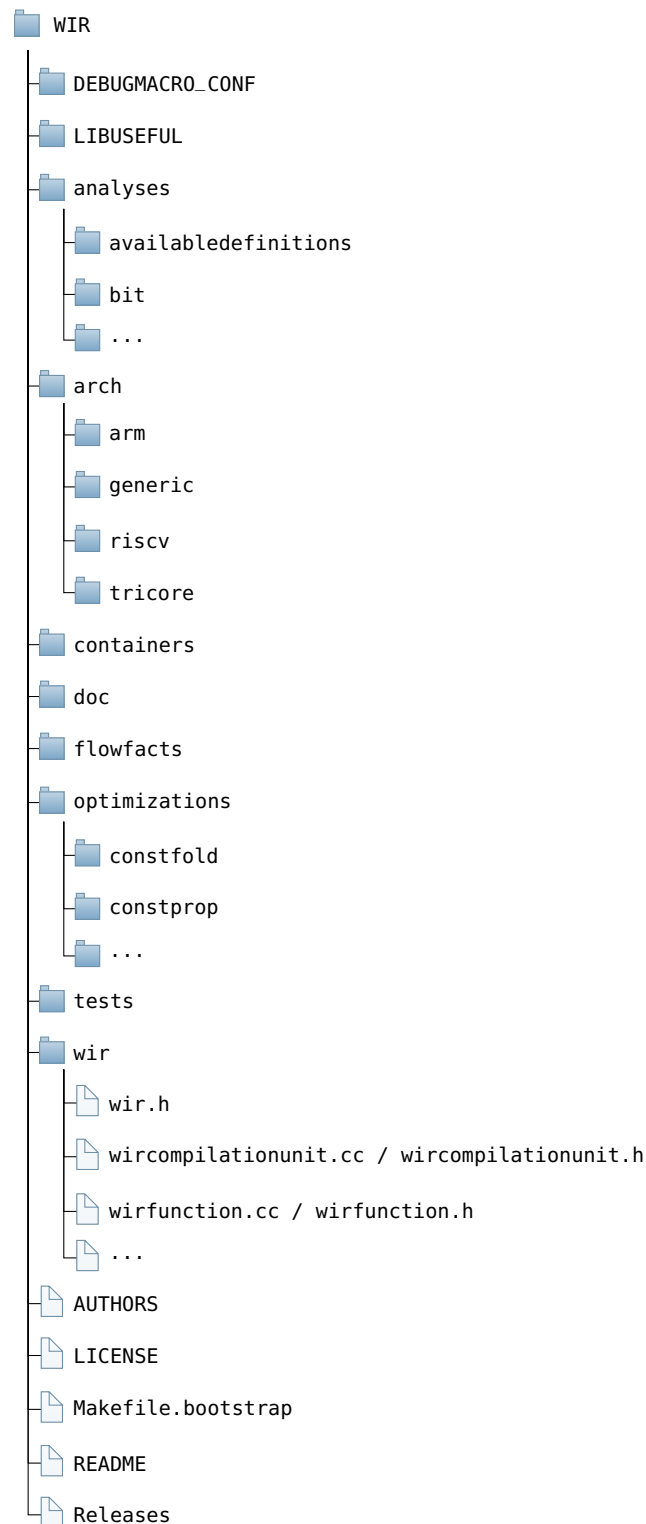


Figure 3.1: Directory Structure of WIR Software Package

Due to its importance, Fig. 3.2 shows the structure of the arch sub-directory in more detail. Since the Infineon TriCore ISA is the most complete one within WIR, this diagram focuses on the arch/tricore directory tree, but the structure for the other ISAs available within WIR is comparable.

Each ISA is implemented via C++ specializations of generic WIR classes, comprising, e.g., the inherited classes

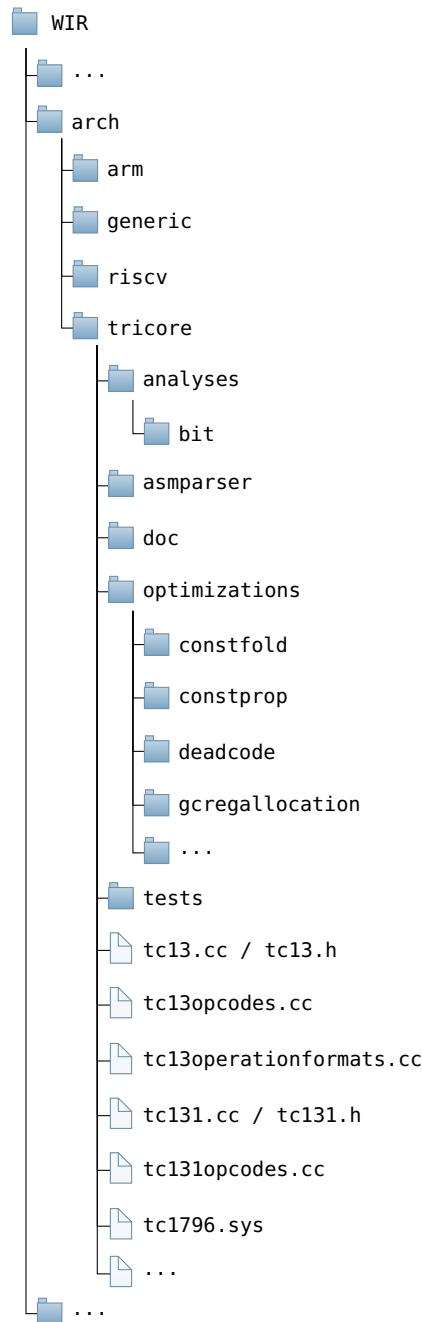


Figure 3.2: Detailed Directory Structure of ISA-Specific WIR Parts

TC13 and TC131 in Fig. 3.2. The [WIR](#) hardware model requires implementations of the various opcodes and operation formats which reside in files like, e.g., `tc13opcodes.cc` or `tc13operationformats.cc`. There exist lots more of [ISA](#)-specific C++ code in order to model, e.g., processor registers or immediate parameters, which are omitted in Fig. 3.2 for the sake of simplicity. System-level configuration files as depicted in Listing 1.1 like, e.g., `tc1796.sys` also reside at this place in the [WIR](#) directory structure.

As already described in Section 1.2, all implementations in the directories `analyses` and `optimizations` from Fig. 3.1 are generic and processor-independent. Thus, the directories with the very same names, but located underneath the `arch` tree contain all their processor-specific adaptations.

Directory `asmparser` might contain a `flex/bison` based assembly code parser that creates [WIR](#) code from a

given stream or string.

Processor-specific unit tests can be found within the `tests` directories inside the `arch` tree, while the `doc` directories therein collect manufacturer documents like, e.g., [ISA](#) manuals or [Application Binary Interface \(ABI\)](#) specifications.

4 Bootstrapping and Configuration

This chapter describes the procedures that are required to bootstrap and to configure [WIR](#). Section [4.1](#) first discusses the steps needed to set up the GNU Build System, and Section [4.2](#) describes configuration options.

4.1 Bootstrapping of the GNU Build System

After a checkout of the software package from its repository, [WIR](#) does not yet contain any useful Makefiles in order to build the library from its C++ sources. As can be seen from Listing [4.1](#), only some drivers for the GNU Build System are provided in the form of files `Makefile.am` and `configure.ac`.

Listing 4.1: Directory Listing After Repository Checkout

```
1 /tmp/WIR> ls -F
2 analyses/  configure.ac  flowfacts/  Makefile.am  README
3 arch/      containers/  LIBUSEFUL/  Makefile.bootstrap  Releases
4 AUTHORS    DEBUGMACRO_CONF/  LICENSE     NEWS          tests/
5 ChangeLog  doc/         m4/         optimizations/  wir/
```

These files need to be processed by the GNU tools `automake` and `autoconf` in order to generate a configuration script which is then used to produce tailored Makefiles (cf. Section [4.2](#)). The invocation of these GNU tools after a virgin repository checkout is handled by `Makefile.bootstrap` which needs to be run first (cf. Listing [4.2](#)).

Listing 4.2: Execution of `Makefile.bootstrap`

```
1 /tmp/WIR> make -f Makefile.bootstrap
2 make[1]: Entering directory '/tmp/WIR/LIBUSEFUL'
3 libtoolize -q -c -f --subproject
4 aclocal
5 autoheader
6 automake -a -c
7 autoconf
8 make[1]: Leaving directory '/tmp/WIR/LIBUSEFUL'
9 libtoolize -q -c -f --subproject
10 aclocal
11 autoheader
12 automake -a -c
13 parallel-tests: installing './test-driver'
14 autoconf
```

As can be seen, the various tools of the GNU Build System are properly executed for the entire software package. As a result of this, a couple of new files and scripts can now be found in the top-level directory of [WIR](#) (cf. Listing [4.3](#)). The interested reader is referred to the documentation of the GNU Build System [[14](#)] for further information about all these new contents, this guide solely focuses on the newly generated script `configure` in the following.

Listing 4.3: Directory Listing After automake, autoconf and Friends

```

1 /tmp/WIR> ls -F
2 aclocal.m4      configure*      LIBUSEFUL/      optimizations/
3 analyses/       configure.ac    LICENSE         README
4 arch/           config_wir.h.in ltmain.sh*      Releases
5 AUTHORS        containers/    m4/             test-driver*
6 autom4te.cache/ DEBUGMACRO_CONF/ Makefile.am     tests/
7 ChangeLog       depcomp*       Makefile.bootstrap wir/
8 compile*        doc/           Makefile.in
9 config.guess*   flowfacts/     missing*
10 config.sub*     install-sh*    NEWS

```

4.2 Configuration of a WIR Build

In order to keep the main source directories of [WIR](#) containing all C++ header and implementation files uncluttered, the following procedures will confine all derived files like, e.g., object files, libraries and programs to a single BUILD directory. For this purpose, this directory needs to be created and the configure script from the previous step needs to be invoked from this place (cf. Listing 4.4).

Listing 4.4: Execution of configure Script

```

1 /tmp/WIR> mkdir BUILD
2
3 /tmp/WIR> cd BUILD
4
5 /tmp/WIR/BUILD> ../configure
6 checking build system type... x86_64-pc-linux-gnu
7 checking host system type... x86_64-pc-linux-gnu
8 checking target system type... x86_64-pc-linux-gnu
9 checking for a BSD-compatible install... /usr/bin/install -c
10 checking whether build environment is sane... yes
11 [...]
12 WIR Configuration:
13   Install path:                /usr/local
14   C++ Compiler flags:          -Og -g -Wall -Wextra -Wno-parentheses -std=c++17 -m64
15   C Compiler flags:            -Og -g -Wall -Wextra -m64 -g -O2
16   C Preprocessor flags:        -I/tmp/WIR/LIBUSEFUL -I/tmp/WIR
17   Linker flags:
18   Build shared libraries:      yes
19   Build static libraries:      no
20
21   Enabled processor models:     generic arm riscv tricore
22   Enabling failsafe mode:      no
23   Enabling debug macros:       no
24   Build DOXYGEN documentation: yes
25   Use LIBUSEFUL in:            /tmp/WIR/BUILD/LIBUSEFUL

```

configure executes a series of checks of the host system (mostly omitted in the above Listing), creates all required Makefiles within the BUILD directory and prints a short configuration summary. The contents of the BUILD directory after configuration is depicted in Listing 4.5. It contains a directory structure that is equivalent to that of the main source directories of [WIR](#) and that will contain all files derived while building. Furthermore, the final Makefile driving the entire build procedure can now be found in WIR/BUILD/Makefile.

While it is fully sufficient to call configure without any additional arguments for simple default builds as shown in

Listing 4.5: Directory Listing After configure

```

1 /tmp/WIR/BUILD> ls -F
2 analyses/    config.status*  doc/           LIBUSEFUL/     stamp-h1
3 arch/        config_wir.h    flowfacts/     Makefile        tests/
4 config.log   containers/     libtool*       optimizations/  wir/

```

Line 5 of Listing 4.4, it regularly becomes necessary to configure a WIR build to specific needs. Table 4.1 provides a short overview over the most relevant options for the configure script. They serve

- to tailor the specifics of the built binary WIR libraries on a certain host system, e.g., their installation location (`--prefix`), whether highly optimized binary libraries are produced (`--with-release-quality`), whether 64- or 32-bit libraries are built (`--enable-x64`), or whether shared or static libraries are built (`--enable-shared` and/or `--enable-static`); or
- to activate or deactivate certain features of the WIR software package like, e.g., the browsable HTML reference documentation (`--with-doxygen-doc`), the debug macro based infrastructure (`--enable-wir-debugmacros`), or the full memory debugging features based on AddressSanitizer and memory protection (`--enable-memory-debugging`).

Besides such explicit command line options, the configure script is also sensitive to shell environment variables like, e.g., `CXX` or `CXXFLAGS`. They can be used to select a non-default host compiler for the translation of the WIR C++ code base into binary libraries. By default, configure uses the GNU g++ compiler [11] for WIR builds. In order to switch to LLVM's clang++ [20], these shall variables need to be set as follows (example refers to a Bourne-compatible shell):

```

1 /tmp/WIR/BUILD> export CXX=clang++
2
3 /tmp/WIR/BUILD> export CXXFLAGS="-ftemplate-depth=1024 -D_GLIBCXX_USE_CXX11_ABI=0"

```

Table 4.1: Most Relevant configure Options

Option (Pair)	Description
<code>--prefix=<INSTALLDIR></code>	Specify <code><INSTALLDIR></code> as installation location
<code>--with-release-quality</code>	Build with highest optimization level of host compiler (default: no)
<code>--with-doxygen-doc</code> or <code>--without-doxygen-doc</code>	Build browsable HTML documentation using doxygen (default: yes)
<code>--enable-wir-debugmacros</code> or <code>--disable-wir-debugmacros</code>	Enable use of debug macros infrastructure (cf. Section 15.1) (default: no)
<code>-C</code>	Cache results of checks in order to speed up configuration
<code>--with-debugmacro-configuration=<PATH></code>	Specify <code><PATH></code> as location of debug macro configuration files (default: source directory WIR/DEBUGMACRO_CONF)
<code>--enable-memory-debugging</code> or <code>--disable-memory-debugging</code>	Build with activated stack protection, address sanitizer and libstdc++ debugging (default: no)
<code>--enable-x64</code> or <code>--disable-x64</code>	Build 64-bit or 32-bit libraries (default: 64)
<code>--enable-shared</code> or <code>--disable-shared</code>	Build shared libraries (default: yes)
<code>--enable-static</code> or <code>--disable-static</code>	Build static libraries (default: no)

Note

Setting these environment variables as shown above has to be done *before* executing the configure script.

For a typical development build of [WIR](#), an appropriate configuration could be:

```
1 /tmp/WIR/BUILD> ../configure --prefix=<INSTALLDIR> --enable-wir-debugmacros
```

while a highly optimized production build could be generated using:

```
1 /tmp/WIR/BUILD> ../configure --prefix=<INSTALLDIR> --with-release-quality --without-doxygen-doc
```

5 Building, Installing and Using WIR

Once the build system has been set up as described in Chapter 4 before, building [WIR](#) is as easy as depicted in Line 1 of Listing 5.1 – a simple call of make in the BUILD directory is sufficient.

Listing 5.1: WIR Build using make

```

1 /tmp/WIR/BUILD> make -j 8
2 make[1]: Entering directory '/tmp/WIR/BUILD'
3 Making all in LIBUSEFUL
4 make[2]: Entering directory '/tmp/WIR/BUILD/LIBUSEFUL'
5 make all-recursive
6 make[3]: Entering directory '/tmp/WIR/BUILD/LIBUSEFUL'
7 Making all in libuseful
8 make[4]: Entering directory '/tmp/WIR/BUILD/LIBUSEFUL/libuseful'
9 libtool: compile: g++ -DHAVE_CONFIG_H -I. -I../.../LIBUSEFUL/libuseful -I.. -Og -g -Wall -Wextra
   -std=c++17 -m64 -MT auxfuncs.lo -MD -MP -MF .deps/auxfuncs.Tpo -c
   ../.../LIBUSEFUL/libuseful/auxfuncs.cc -fPIC -DPIC -o .libs/auxfuncs.o
10 libtool: compile: g++ -DHAVE_CONFIG_H -I. -I../.../LIBUSEFUL/libuseful -I.. -Og -g -Wall -Wextra
   -std=c++17 -m64 -MT debugmacros.lo -MD -MP -MF .deps/debugmacros.Tpo -c
   ../.../LIBUSEFUL/libuseful/debugmacros.cc -fPIC -DPIC -o .libs/debugmacros.o
11 libtool: compile: g++ -DHAVE_CONFIG_H -I. -I../.../LIBUSEFUL/libuseful -I.. -Og -g -Wall -Wextra
   -std=c++17 -m64 -MT exceptions.lo -MD -MP -MF .deps/exceptions.Tpo -c
   ../.../LIBUSEFUL/libuseful/exceptions.cc -fPIC -DPIC -o .libs/exceptions.o
12 [...]
13 libtool: link: ( cd ".libs" && rm -f "libwir.la" && ln -s "../libwir.la" "libwir.la" )
14 make[2]: Leaving directory '/tmp/WIR/BUILD'
15 Making all in doc
16 make[2]: Entering directory '/tmp/WIR/BUILD/doc'
17 make[2]: Leaving directory '/tmp/WIR/BUILD/doc'
18 make[1]: Leaving directory '/tmp/WIR/BUILD'

```

The example from Line 1 of Listing 5.1 uses 8 processor cores of the host system in parallel, which significantly speeds up the build procedure. After [WIR](#) has been built, its unit tests can be executed (cf. Line 1 of Listing 5.2). These tests are designed to cover true-positive and true-negative scenarios which are represented by PASS and XFAIL in Listing 5.2. When running the unit tests, obviously no false-positives (XPASS) or false-negatives (FAIL) should occur. If configured accordingly (cf. Table 4.1), the HTML reference documentation generated by doxygen can be inspected by opening file `WIR/BUILD/doc/html/index.html` in a browser.

After the successful build of [WIR](#), the complete software package can be installed in the directory configured as installation location with the `--prefix` option of `configure` (cf. Table 4.1):

```

1 /tmp/WIR/BUILD> make install

```

This creates a directory hierarchy as depicted in Fig. 5.1. The `etc` directory contains sub-directories for each [ISA](#), and the system-level configuration files are installed therein. The `include` directory tree contains all C++ header files of [WIR](#), following the same structure as shown in Fig. 3.1. The built binary libraries `libuseful.so` and

Listing 5.2: WIR Unit Tests

```

1 /tmp/WIR/BUILD> make -j 8 check
2 Making check in LIBUSEFUL
3 [...]
4 Making check in tests
5 make[1]: Entering directory '/tmp/WIR/BUILD/tests'
6 [...]
7 PASS: tc1796MemoryLayout2.sh
8 PASS: tcFunction3.sh
9 PASS: tcCompilationUnit1.sh
10 PASS: tcOperations2.sh
11 PASS: tcBasicBlock1.sh
12 =====
13 Testsuite summary for wir 2.0
14 =====
15 # TOTAL: 126
16 # PASS: 89
17 # SKIP: 0
18 # XFAIL: 37
19 # FAIL: 0
20 # XPASS: 0
21 # ERROR: 0
22 =====
23 make[5]: Leaving directory '/tmp/WIR/BUILD/arch/tricore/tests'
24 make[2]: Leaving directory '/tmp/WIR/BUILD/arch/tricore'
25 make[1]: Leaving directory '/tmp/WIR/BUILD/arch'
26 make[1]: Leaving directory '/tmp/WIR/BUILD'

```

`libwir.so` (and/or their static counterparts if configured accordingly) reside in sub-directory `lib` after installation. Finally, sub-directory `share` contains the authors, license and version information of the software package.

This installation can now be used by WIR-based applications. Listing 5.3 shows a very simple test program that includes the generic and ISA-independent headers in Line 2. All C++ classes and types of the WIR library are placed into a dedicated C++ namespace called WIR. Line 4 enables the global usage of this namespace so that no explicit qualifiers need to be given in the subsequent code.¹ Before any use of WIR data structures in application code, the library must be initialized (cf. Line 9). After that, the test program creates an instance of a `WIR_System` based on the MIPS SPIM generic ISA model installed in file `<INSTALLDIR>/etc/generic/genericmips.sys` (cf. Line 13).

The test program can finally be compiled and linked against the installed WIR library as follows:

```

1 /tmp> export LD_LIBRARY_PATH=<INSTALLDIR>/lib:$LD_LIBRARY_PATH
2
3 /tmp> g++ wirtest.cc -o wirtest.bin -I<INSTALLDIR>/include -L<INSTALLDIR>/lib -luseful -lwir

```

Setting the variable `LD_LIBRARY_PATH` as shown in Line 1 above (again assuming a Bourne-compatible shell) is required so that the host system's dynamic linker is able to find the installed binary libraries. Line 3 afterwards invokes the `g++` compiler with the proper include and library arguments.

¹In the rest of this entire document, we from now on assume in all code listings that this namespace WIR is always used. All following listings will thus not contain any qualifier `WIR::` in order to improve code legibility.

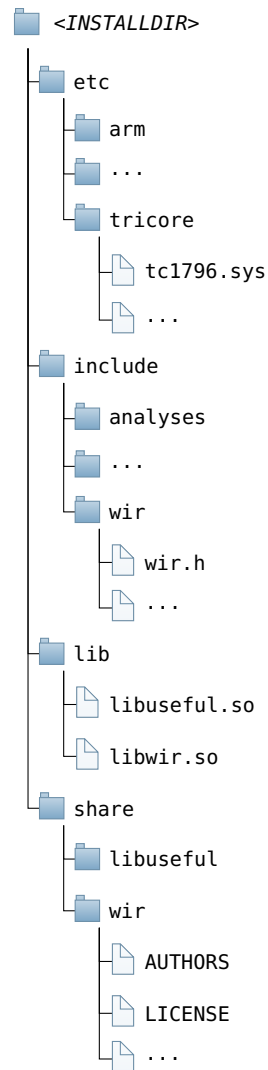


Figure 5.1: Directory Structure of Installed WIR Library

Listing 5.3: Simple WIR Test Program `wirtest.cc`

```
1 // Include WIR headers.
2 #include <wir/wir.h>
3
4 using namespace WIR;
5
6 int main( void )
7 {
8     // Initialize WIR.
9     WIR_Init();
10
11     // Create test WIR system.
12     WIR_TaskManager t;
13     WIR_System sys( "genericmips.sys", t );
14
15     sys.setName( "Test WIR system" );
16
17     return( 0 );
18 }
```


Part II

User's Guide

Outline

Purpose of this guide is to explain how to use [WIR](#) in some own compiler project. For this reason, this guide describes the most important concepts driving the design of the public interfaces of central [WIR](#) classes.

Chapter [6](#) (Page [31](#) ff.) presents the general design of list-based data structures within [WIR](#). Techniques to construct and traverse complex hierarchies of [WIR](#) objects are described in Chapter [7](#) (Page [37](#) ff.). The properties and features of all classes constituting the [WIR](#) software model are subject of Chapter [8](#) (Page [41](#) ff.). Chapter [9](#) (Page [65](#) ff.) discusses the structures and properties of all classes forming the [WIR](#) hardware model. How both the software and the hardware model are combined to represent complete hardware/software systems is presented in Chapter [10](#) (Page [119](#) ff.). Chapter [11](#) (Page [133](#) ff.) describes elementary deterministic data structures that base on unique numerical identifiers and that regularly appear in the public [API](#) of the [WIR](#) library. How to generate actual assembly code from the [WIR](#) library is discussed in Chapter [12](#) (Page [137](#) ff.). The code analyses that are integral part of [WIR](#) are introduced in Chapter [13](#) (Page [147](#) ff.), and Chapter [14](#) is devoted to [WIR](#)'s code optimizations (Page [185](#) ff.). Finally, Chapter [15](#) (Page [205](#) ff.) briefly introduces [WIR](#)'s introspection facilities.

6 General Handling of List Structures Within WIR

At many places within [WIR](#), lists are a central data structure. This chapter thus describes the general [API](#) of all list-based structures as they regularly appear in the public interfaces of [WIR](#) classes.

When revisiting [Fig. 1.2](#), it is easy to see that a `WIR_System` can contain 0 to many `WIR_CompilationUnits` which are given in some specific order. A compilation unit in turn can contain 0 to many `WIR_Functions`, which again can consist of 0 to many `WIR_BasicBlocks` in some given order, etc. This scheme applies to the entire hierarchy from [Fig. 1.2](#) from `WIR_System` down to `WIR_Parameter`, and to many other places inside [WIR](#). Since the order matters, in which objects of some lower parts of this class hierarchy are stored by their respective parent objects, standard C++ lists (`std::list`) come into play here.

As one design goal of the [WIR](#) class interfaces is to keep the [API](#) free of pointers as far as possible, all [WIR](#) lists exposed to application code make maximal use of references instead. In the following, this chapter thus describes the general [API](#) design of such lists of references to [WIR](#) objects. For the sake of simplicity, the examples given in the following sections only refer to `WIR_Instructions` that are organized into a list within class `WIR_BasicBlock`. By replacing the names of the contained class (here: `Instruction`) and of the containing class (here: `BasicBlock`), it is straightforward to generalize the following examples to all other places inside [WIR](#).

[Section 6.1](#) describes how to insert new elements into lists. Hereafter, [Section 6.2](#) introduces the [API](#) for list modification. In certain situations, it might be desired to prevent some objects from being modified. [Section 6.3](#) thus describes a simple [API](#) for modification prevention. Finally, [Sections 6.4](#) and [6.5](#) introduce the [API](#) for list lookup and destruction of list elements, resp.

6.1 Insertion

Inserting at the head and the tail of a list are the most frequent scenarios to insert a new element. This is realized within [WIR](#) by the group of `pushBack` and `pushFront` methods shown in [Listing 6.1](#). They all receive an object `o` to be inserted as argument, insert it at the list tail or head, resp., and return a reference to the newly inserted

Listing 6.1: Insertion of WIR Objects at List Heads and Tails

```

1 // Copy-add a new element at the end of a list, after its current last element.
2 WIR_Instruction & WIR_BasicBlock::pushBackInstruction( const WIR_Instruction &o );
3
4 // Move-add a new element at the end of a list, after its current last element.
5 WIR_Instruction & WIR_BasicBlock::pushBackInstruction( WIR_Instruction &&o );
6
7 // Copy-add a new element at the beginning of a list, before its current first element.
8 WIR_Instruction & WIR_BasicBlock::pushFrontInstruction( const WIR_Instruction &o );
9
10 // Move-add a new element at the beginning of a list, before its current first element.
11 WIR_Instruction & WIR_BasicBlock::pushFrontInstruction( WIR_Instruction &&o );

```

Listing 6.2: Insertion of WIR Objects at Arbitrary List Positions

```

1 // Copy-insert a new element before the element given by iterator pos.
2 std::list<std::reference_wrapper<WIR_Instruction>>::iterator WIR_BasicBlock::insertInstruction(
   std::list<std::reference_wrapper<WIR_Instruction>>::const_iterator pos, const WIR_Instruction &o );
3
4 // Move-insert a new element before the element given by iterator pos.
5 std::list<std::reference_wrapper<WIR_Instruction>>::iterator WIR_BasicBlock::insertInstruction(
   std::list<std::reference_wrapper<WIR_Instruction>>::const_iterator pos, WIR_Instruction &&o );

```

list element. Those variants receiving a const reference as argument (cf. Lines 2 and 8 above) implicitly create a copy of `o`, insert this copy into the list, and return a reference to that copy of `o`. In contrast, the variants from Lines 5 and 11 above expect an R-value reference of `o` and directly move that object `o` into the list. Due to the move semantics of C++, these move-based methods are much more efficient than those ones implicitly copying objects.

In order to insert an element at some arbitrary list position, the insert methods shown in Listing 6.2 are provided. Due to the fact that C++ standard containers like, e.g., `std::list` cannot store references to objects, the `WIR` list interfaces heavily use class `std::reference_wrapper` in order to wrap references into lists. Thus, the list of instructions managed by some `WIR` basic block as shown in all the previous code examples technically is a `std::list` of objects of class `std::reference_wrapper`, which in turn keep the references to `WIR_Instructions`.

The insert methods from Listing 6.2 thus expect an iterator `pos` specifying the position in the list *before* which the new element `o` shall be inserted. Both methods return an iterator referring to the position of the newly inserted list element. Again, the `WIR` interface provides both a copy- and a move-based variant of the insert methods.

Warning

After the insertion of some element `o` into a list using either of the methods from Listings 6.1 and 6.2, it is considered wrong if application code assumes that `o` itself is part of some list afterwards. Instead, application code is recommended to use the references or iterators returned by these methods in order to obtain a safe handle to the newly inserted list item.

Warning

Whenever some object `o` is inserted into some `WIR` list using the above `pushBack`, `pushFront` or `insert` methods, the list takes over the exclusive ownership of the inserted list element. This means that the `WIR` lists take care of the proper deletion of list elements. Application code thus shall never take pointers to `WIR` list elements and must never manually delete such pointers.

6.2 Modification

Modification of already existing entries in some `WIR` list can be achieved using the `replace` methods from Listing 6.3. They expect an iterator `pos` specifying the position in the list that shall be replaced by the new element `o`. Both methods return an iterator referring to the position of the newly inserted list element, and again, both a copy- and a move-based variant of the `replace` methods is provided. The list element that gets replaced is immediately destroyed.

Listing 6.3: Replacement of WIR Objects in Lists

```

1 // Replace the list element given by iterator pos by copying o to the specified position.
2 std::list<std::reference_wrapper<WIR_Instruction>>::iterator WIR_BasicBlock::replaceInstruction(
   std::list<std::reference_wrapper<WIR_Instruction>>::const_iterator pos, const WIR_Instruction &o );
3
4 // Replace the list element given by iterator pos by moving o to the specified position.
5 std::list<std::reference_wrapper<WIR_Instruction>>::iterator WIR_BasicBlock::replaceInstruction(
   std::list<std::reference_wrapper<WIR_Instruction>>::const_iterator pos, WIR_Instruction &&o );

```

6.3 Modification Prevention

Sometimes, a system designer might want certain parts of the code base not to be modified, e.g., by some automatically applied optimization. A typical example is hand-written assembly code that is embedded into, e.g., C/C++ application code using inline assembly directives. For the purpose of obfuscation and improved software security, such human-written inline assembly shall remain untouched and shall not be optimized or even completely eliminated by subsequent optimizations.

WIR supports this throughout its complete software model from Fig. 1.2 by providing the methods shown in Listing 6.4. The set method (cf. Line 2) serves to lock or unlock some WIR object, while the get method (cf. Line 5) returns its current status. getDontOptimize always works in a hierarchical manner. This means that as soon as some basic block has been locked using `WIR_BasicBlock::setDontOptimize()`, automatically all instructions, operations and parameters contained by this basic block are also prevented from modification. Thus, `WIR_Instruction::getDontOptimize()` will return true for some instruction contained by a locked basic block, even if the instruction itself has not been locked explicitly.

Listing 6.4: Modification Prevention of WIR Objects

```

1 // Set whether some object can be modified or must not be changed by some optimization.
2 void WIR_BasicBlock::setDontOptimize( bool f = true );
3
4 // Determine whether some object can be modified or must not be changed by some optimization.
5 bool WIR_BasicBlock::getDontOptimize( void ) const;

```

Any attempt to modify a WIR object for which `getDontOptimize` returns true will result in a runtime error.

6.4 Lookup

The lookup of list structures within WIR is realized using the methods depicted in Listing 6.5. Here, the get method (cf. Line 2) returns a reference to the full STL list which can then be used for traversal or check of emptiness etc. using the standard lookup operators of STL class `std::list`.

Warning

Application code shall never attempt to directly manipulate a list returned by the get method from Listing 6.5. Instead, only the methods described in this chapter must be used for insertion, manipulation or deletion of list elements. A direct manipulation of such a list using standard STL list operations like, e.g., `push_back`, `emplace` or `splice` inevitably leads to corrupt internal data structures within WIR.

Listing 6.5: Lookup of WIR Objects in Lists

```

1 // Get the full list of references to list elements.
2 std::list<std::reference_wrapper<WIR_Instruction>> &WIR_BasicBlock::getInstructions( void ) const;
3
4 // Determine whether the list contains an item with the given ID.
5 bool WIR_BasicBlock::containsInstruction( WIR_id_t id ) const;
6
7 // Determine whether the list contains the given item.
8 bool WIR_BasicBlock::containsInstruction( const WIR_Instruction &o ) const;
9
10 // Find the list element with the given ID.
11 std::list<std::reference_wrapper<WIR_Instruction>>::const_iterator WIR_BasicBlock::findInstruction(
    WIR_id_t id ) const;
12
13 // Find the given list element.
14 std::list<std::reference_wrapper<WIR_Instruction>>::const_iterator WIR_BasicBlock::findInstruction(
    const WIR_Instruction &o ) const;

```

The contains methods test whether a list contains some specified item and provide the corresponding Boolean result. Internally, all WIR objects carry a unique numerical identifier, represented by type WIR_id_t (cf. Section 11.1 for more information about the rationale behind WIR IDs). The contains variant from Line 5 thus checks whether a list contains an element having the given ID. In contrast, the variant from Line 8 checks whether some actual object is contained in a list.

The find methods (cf. Lines 11 to 14) locate a given item in a list and provide an iterator to the found position. If the list does not contain the item to be found, the end() iterator is returned. As above, both variants to find an item with a given ID (cf. Line 11) and to find some actual object (cf. Line 14) exist.

6.5 Destruction

In order to remove some element, WIR lists provide the methods depicted in Listing 6.6. To remove the list's tail or head element, the popBack (cf. Line 2) and popFront (cf. Line 5) methods exist. Using erase from Line 8, a list element at some arbitrary position specified by an iterator can be erased. erase returns an iterator referring to the list element following the erased element. Removing all elements from a list can be achieved using clear (cf. Line 11).

Listing 6.6: Removal of WIR Objects from Lists

```

1 // Remove the last list element.
2 void WIR_BasicBlock::popBackInstruction( void );
3
4 // Remove the first list element.
5 void WIR_BasicBlock::popFrontInstruction( void );
6
7 // Remove the specified list element.
8 std::list<std::reference_wrapper<WIR_Instruction>>::iterator WIR_BasicBlock::eraseInstruction(
    std::list<std::reference_wrapper<WIR_Instruction>>::const_iterator pos );
9
10 // Remove all list elements.
11 void WIR_BasicBlock::clearInstructions( void );

```

Whenever some element is removed from a [WIR](#) list using the methods from Listing 6.6, the removed element is automatically destroyed and its memory released.

Warning

Keeping references or pointers to [WIR](#) objects even though they have been removed from their respective parent objects is considered wrong. Using such dangling references or dereferencing such pointers inevitably introduces use-after-free errors.

7 Construction and Traversal of WIR Object Hierarchies

Referring once more back to Fig. 1.2, it is evident that [WIR](#) based applications regularly have to create object hierarchies where, e.g., some functions are contained by a `WIR_CompilationUnit`, or where various parameters are used to build up a `WIR_Operation`. This chapter thus describes the mechanisms how to set up such object hierarchies, and how to traverse and to navigate within these [WIR](#) hierarchies.

Section 7.1 first describes the straightforward approach to set up complex object hierarchies based on the list interfaces from Chapter 6. A simpler and more efficient approach for the very same purpose exploits C++ variadic templates and is introduced in Section 7.2. Finally, the traversal and navigation inside hierarchies of [WIR](#) objects is subject of Section 7.3.

7.1 Construction Using List Interfaces

Using the interfaces described in Chapter 6, it is simple to create objects and plug them together into a hierarchy as shown in Fig. 1.2. Listing 7.1 shows a code snippet demonstrating this by means of [WIR](#) classes `WIR_System` down to `WIR_Parameter`, and by referring to the generic MIPS hardware model.

In Line 17, a compilation unit is inserted into a [WIR](#) system, and this compilation unit to be inserted is created such that it directly contains a `WIR_Function` named `foo` from scratch. A reference to the newly inserted compilation unit returned by `pushBackCompilationUnit` is kept in `c`.

A reference `f` to the implicitly created [WIR](#) function `foo` is retrieved in Line 18 by getting the list of functions from `c` and picking its head. In lines Lines 19 and 20, two virtual MIPS registers `r1` and `r2` are created using method `pushBackVirtualRegister` of [WIR](#) function `f`. Furthermore, a basic block `b` is declared (cf. Line 21) as well as an instruction `i` (cf. Line 22).

Lines 25 to 28 next create a [WIR](#) MIPS assembly operation `o`. For this purpose, the parameters of this intended operation are created first, where `p1` is a register parameter where the previously created register `r1` gets defined/written by the operation (cf. Line 25). Similarly, parameter `p2` represents register `r2` that is used/read by the operation (cf. Line 26). In the given example, parameter `p3` represents a signed 16-bit immediate value in the MIPS [ISA](#), which is set to the value 42 (cf. Line 27). These three parameters are then used to construct the MIPS `ADDI` operation `o` in Line 28. The constructor of class `WIR_Operation` expects the operation's opcode and its operation format as first arguments, followed by the operation's parameters. In the given example, the MIPS operation format `RRI` denotes an operation taking a Register as first parameter, another Register as second parameter, and an Immediate as third parameter.

Finally, all these various objects are used to set up a proper [WIR](#) object hierarchy by again using the list interfaces from Chapter 6 (cf. Lines 31 to 33). First, operation `o` is pushed-back into instruction `i`. Next, the same happens with `i` and basic block `b`. Finally, `b` is plugged into function `f`, which already was part of compilation unit `c`, which in turn is owned by [WIR](#) system `sys`.

Listing 7.1: Construction of Object Hierarchies using List Interface

```

1 // Include WIR headers.
2 #include <wir/wir.h>
3 #include <arch/generic/mips.h>
4
5 using namespace WIR;
6
7 int main( void )
8 {
9     // Initialize WIR.
10    WIR_Init();
11
12    // Create WIR system.
13    WIR_TaskManager t;
14    WIR_System sys( "genericmips.sys", t );
15
16    // Create various WIR objects.
17    WIR_CompilationUnit &c = sys.pushBackCompilationUnit( WIR_Function( "foo" ) );
18    WIR_Function &f = c.getFunctions().front().get();
19    WIR_VirtualRegister &r1 = f.pushBackVirtualRegister( WIR_VirtualRegister( MIPS::RegisterType::reg ) );
20    WIR_VirtualRegister &r2 = f.pushBackVirtualRegister( WIR_VirtualRegister( MIPS::RegisterType::reg ) );
21    WIR_BasicBlock b;
22    WIR_Instruction i;
23
24    // Create an operation.
25    WIR_RegisterParameter p1( r1, WIR_Usage::def );
26    WIR_RegisterParameter p2( r2, WIR_Usage::use );
27    MIPS_Immediate16_Signed p3( 42 );
28    WIR_Operation o( MIPS::OpCode::ADDI, MIPS::OperationFormat::RRI, p1, p2, p3 );
29
30    // Create object hierarchy using list API.
31    i.pushBackOperation( o );
32    b.pushBackInstruction( i );
33    f.pushBackBasicBlock( b );
34
35    return( 0 );
36 }

```

7.2 Variadic Construction

While the construction and creation of object hierarchies as illustrated in the previous section is simple and straightforward, the separate declarations, creations, and insertions of objects as shown in lines Lines 21 to 33 of Listing 7.1 can be quite cumbersome in many situations. Due to this, many WIR classes come with constructors using C++ variadic templates. They allow to create a new object that directly contains an arbitrary number of contained other objects right from the beginning.

An example of this concept already showed up in Line 28 of Listing 7.1, where an operation with three parameters was constructed. However, the constructor of class WIR_Operation is not bound to exactly three parameters. Instead, it can create operations with 0, or 1, or, e.g., 13 parameters if desired.

This genericity allows to simplify the example from Listing 7.1 heavily. Listing 7.2 shows a code example where directly from scratch, a hierarchy consisting of a function with one basic block, two instructions, and each of them featuring one operation is constructed and plugged together.

Line 8 pushes a new basic block into WIR function f. Lines 9 to 16 next specify the structure of this new basic

Listing 7.2: Variadic Construction of Object Hierarchies

```

1 // Create various WIR objects.
2 auto &c = sys.pushBackCompilationUnit( WIR_Function( "foo" ) );
3 auto &f = c.getFunctions().front().get();
4 auto &r1 = f.pushBackVirtualRegister( WIR_VirtualRegister( MIPS::RegisterType::reg ) );
5 auto &r2 = f.pushBackVirtualRegister( WIR_VirtualRegister( MIPS::RegisterType::reg ) );
6
7 // Create two instructions, directly embedded into object hierarchy.
8 f.pushBackBasicBlock(
9     { WIR_Instruction(
10         { MIPS::OpCode::ADDI, MIPS::OperationFormat::RRI,
11           WIR_RegisterParameter( r1, WIR_Usage::def ),
12           WIR_RegisterParameter( r2, WIR_Usage::use ),
13           MIPS_Immediate16_Signed( 42 ) } },
14     WIR_Instruction(
15         { MIPS::OpCode::JR, MIPS::OperationFormat::R_2,
16           WIR_RegisterParameter( r1, WIR_Usage::use ) } } ) );

```

block in a variadic manner. As can be seen from Lines 9 and 14, the basic block will feature two instructions. In general, an arbitrary amount of instructions can be specified, separated by commas during construction. The first instruction in turn consists of the MIPS ADDI operation already used as example in Listing 7.1. But in contrast to that previous example, Lines 11 to 13 now construct the required number of parameters directly on the fly in variadic style. Similarly, the second instruction of the newly created basic block is constructed. This instruction contains one MIPS JR operation which requires one register as used parameter (cf. Lines 15 to 16).

This concept of variadic creation can of course also be used to construct compilation units with an arbitrary number of functions, and functions with an arbitrary number of basic blocks right upfront. Using these mechanisms is highly efficient, since the already previously mentioned move semantics of C++ is involved here, avoiding unnecessary copies of WIR objects.

7.3 Traversal and Navigation

Once complex, hierarchical structures of WIR objects have been constructed using the techniques described in Sections 7.1 and 7.2, these hierarchies regularly need to be traversed. Given the hierarchy resulting from Listing 7.2, its complete traversal is as simple as shown in Listing 7.3. Range-based for loops of C++ can be used to directly traverse almost all object hierarchies within WIR, and to obtain proper references to all objects contained by some parent object. Compiling and executing this example produces the output

```

1 Visiting compilation unit.
2   Visiting function.
3     Visiting basic block.
4       Visiting instruction.
5         Visiting operation.
6           Visiting parameter.
7           Visiting parameter.
8           Visiting parameter.
9       Visiting instruction.
10        Visiting operation.
11         Visiting parameter.

```

which shows the expected structure of the traversed WIR object hierarchy created by Listing 7.2.

Listing 7.3: Traversal of Object Hierarchies using Range-Based Loops

```

1 // Traverse object hierarchy.
2 for ( WIR_CompilationUnit &cu : sys ) {
3     std::cout << "Visiting compilation unit." << std::endl;
4     for ( WIR_Function &fct : cu ) {
5         std::cout << "    Visiting function." << std::endl;
6         for ( WIR_BasicBlock &bb : fct ) {
7             std::cout << "        Visiting basic block." << std::endl;
8             for ( WIR_Instruction &ins : bb ) {
9                 std::cout << "            Visiting instruction." << std::endl;
10                for ( WIR_Operation &op : ins ) {
11                    std::cout << "                Visiting operation." << std::endl;
12                    for ( WIR_Parameter &param : op )
13                        std::cout << "                    Visiting parameter." << std::endl;
14                }
15            }
16        }
17    }
18 }

```

Since **WIR** fully bases on standard C++ **STL** data structures, of course all standard algorithms to find, test and to process **WIR** objects can directly be used. This includes, amongst others, `std::find_if`, `std::any_of` or `std::accumulate`. The below example builds on Listing 7.2 and demonstrates how to find a function with a specific name inside a compilation unit `c`.

```

1 // Find a WIR function having a given name.
2 auto it =
3     std::find_if(
4         c.begin(), c.end(),
5         [&]( const WIR_Function &f ) { return( f.getName() == "foo" ); } );

```

WIR objects can be inserted into some object hierarchy, or they can exist outside such hierarchies. Lines 21 and 22 of Listing 7.1 show the declaration and default-construction of a basic block `b` and an instruction `i`. By default, both objects are not part of the object hierarchy that finally is constructed in Listing 7.1. Only after instruction `i` has been pushed into basic block `b` in Line 32, `i` then gets inserted.

In order to determine the current status of some **WIR** object, the `isInserted` method (cf. Line 2 below) is provided. In analogy to Chapter 6, the code fragment below again refers just to the relationship between objects of class `WIR_Instruction` and `WIR_BasicBlock`. Of course, this can be generalized for all other parts of the **WIR** object hierarchy.

```

1 // Determine whether an object is inserted into some basic block.
2 bool WIR_Instruction::isInserted( void ) const;
3
4 // Get the basic block containing an instruction.
5 WIR_BasicBlock &WIR_Instruction::getBasicBlock( void ) const;

```

The call `i.inserted()` immediately after Line 22 of Listing 7.1 would thus yield `false` as result, but `true` if it were placed after Line 32 instead. As soon as objects are inserted into some **WIR** hierarchy, the `get` family of methods can be used to obtain a reference of the containing object in the hierarchy (cf. Line 5 above). This way, a bottom-up traversal of the hierarchy depicted in Fig. 1.2 is supported. These `get` methods fail with a runtime error if they are applied to a non-inserted object.

8 The WIR Software Model

This chapter describes and specifies the [API](#) of all classes that serve to model software in some [WIR](#) system. Thus, the subsequent Sections [8.1](#) to [8.6](#) focus in detail on the complete hierarchy from compilation units down to parameters as depicted in [Fig. 1.2](#). Furthermore, Section [8.7](#) introduces the key concepts of data objects within [WIR](#).

8.1 Compilation Units

Class `WIR_CompilationUnit` is the generic representation of code files. As such, its primary purpose is to manage a list of [WIR](#) functions that are part of a compilation unit. All list operations from Chapter [6](#) like, e.g., `WIR_CompilationUnit::pushBackFunction` are provided by class `WIR_CompilationUnit`.

Construction of compilation units can be achieved using the constructors shown in [Listing 8.1](#). The default constructor (cf. [Line 2](#)) creates an empty compilation unit without any functions, while the variadic constructor (cf. [Line 5](#)) can be used to create an object with an arbitrary amount of functions (see [Section 7.2](#)). Copy and move constructors are provided as well (cf. [Lines 8 and 11](#)).

Listing 8.1: Constructors of Class `WIR_CompilationUnit`

```

1 // Default constructor creating an empty compilation unit.
2 WIR_CompilationUnit::WIR_CompilationUnit( void );
3
4 // Variadic constructor creating a compilation unit containing the given functions.
5 template<typename... Args> WIR_CompilationUnit::WIR_CompilationUnit( WIR_Function &&f, Args&&... __args );
6
7 // Copy constructor.
8 WIR_CompilationUnit::WIR_CompilationUnit( const WIR_CompilationUnit &c );
9
10 // Move constructor.
11 WIR_CompilationUnit::WIR_CompilationUnit( WIR_CompilationUnit &&c );

```

While functions, instructions or operations represent code within the [WIR](#) software model, data regularly is managed either in a system's stack at a low abstraction level, or resides in some dedicated memory areas. Usually, local variables that are visible only within individual functions or in even smaller scopes are managed via the stack. All other kinds of data with a scope larger than local variables like, e.g., global variables, reside in dedicated memory areas and are modeled as so-called data objects within [WIR](#). An in-depth discussion of data objects and of the associated class `WIR_Data` follows later in [Section 8.7](#).

However, as data can be declared to be visible only within a code file's scope in many programming languages, class `WIR_CompilationUnit` also includes the management of a list of data objects. In analogy to [Chapter 6](#), methods ranging from `WIR_CompilationUnit::pushBackData` to `WIR_CompilationUnit::findData` are provided. Compilation units finally provide a method `getDataLayout` that groups data objects to sections. We defer the discussion of this method to [Section 10.4](#) introducing the details of the [WIR](#) memory layout.

8.2 Functions

According to the software model from Fig. 1.2, class `WIR_Function` represents code functions where functions in turn act as container for a list of basic blocks. Thus, `WIR_Function` again contains all list operations from Chapter 6 for the management of basic blocks.

Since functions usually have a unique and human-readable name, all regular constructors of class `WIR_Function` require a string containing the function name as argument, as can be seen in Listing 8.2, Lines 2 to 11. Such strings can either be copied into the constructed `WIR` function, or they can be moved for improved efficiency. Functions can also be copy- or move-constructed (cf. Lines 14 to 17 below). Furthermore, dedicated methods to set and get a function's name are also provided (cf. Lines 20 to 26).

Listing 8.2: Constructors and Methods for Name Handling of Class `WIR_Function`

```

1 // Default constructor creating an empty function with a copied string as function name.
2 WIR_Function::WIR_Function( const std::string &s );
3
4 // Default constructor creating an empty function with a moved string as function name.
5 WIR_Function::WIR_Function( std::string &&s );
6
7 // Variadic constructor creating a function containing the given basic blocks, copied function name.
8 template<typename... Args> WIR_Function::WIR_Function( const std::string &s, WIR_BasicBlock &&b,
9                                                         Args&&... __args );
10
11 // Variadic constructor creating a function containing the given basic blocks, moved function name.
12 template<typename... Args> WIR_Function::WIR_Function( std::string &&s, WIR_BasicBlock &&b,
13                                                         Args&&... __args );
14
15 // Copy constructor.
16 WIR_Function::WIR_Function( const WIR_Function &f );
17
18 // Move constructor.
19 WIR_Function::WIR_Function( WIR_Function &&f );
20
21 // Set function name to a copied string.
22 void WIR_Function::setName( const std::string &s );
23
24 // Set function name to a moved string.
25 void WIR_Function::setName( std::string &&s );
26
27 // Get the function name.
28 std::string WIR_Function::getName( void ) const;

```

A function's code size in bytes can be retrieved using method `getSize` from Listing 8.3. The returned value is the accumulation of the byte sizes of all basic blocks contained by a function.

Listing 8.3: Size Computation by Class `WIR_Function`

```

1 // Get a function's byte size.
2 unsigned long long WIR_Function::getSize( void );

```

As already introduced in Section 1.1.2, `WIR` supports the concept of virtual registers. Virtual registers serve as an abstraction of actual physical registers that are available in some processor architecture. For each group of physical registers that an architecture might feature like, e.g., general-purpose registers or floating-point registers, `WIR` provides a corresponding group of virtual registers. These virtual registers have the same properties as their

Listing 8.4: Precoloring and Interference Handling of Class WIR_Function

```

1 // Precolor the given virtual register with a physical register for register allocation.
2 void WIR_Function::insertPrecolor( const WIR_VirtualRegister &v, const WIR_PhysicalRegister &p );
3
4 // Remove any precoloring information related to some virtual register.
5 void WIR_Function::erasePrecolor( const WIR_VirtualRegister &v );
6
7 // Remove all precoloring information from a WIR function.
8 void WIR_Function::clearPrecolors( void );
9
10 // Determine whether a virtual register is precolored or not.
11 bool WIR_Function::containsPrecolor( const WIR_VirtualRegister &v ) const;
12
13 // Find the physical register associated with some virtual register.
14 WIR_PhysicalRegister &WIR_Function::findPrecolor( const WIR_VirtualRegister &v ) const;
15
16 // Mark the two given registers as interfering for register allocation.
17 void WIR_Function::insertInterference( const WIR_VirtualRegister &v, const WIR_PhysicalRegister &p );
18
19 // Remove any interference between the two specified registers.
20 void WIR_Function::eraseInterference( const WIR_VirtualRegister &v );
21
22 // Remove all interference information from a WIR function.
23 void WIR_Function::clearInterferences( void );
24
25 // Determine whether the two given registers interfere.
26 bool WIR_Function::interfere( const WIR_VirtualRegister &v, const WIR_PhysicalRegister &p ) const;
27
28 // Find the set of all physical registers that a virtual register interferes with.
29 WIR_PhysicalRegisterSet WIR_Function::findInterferences( const WIR_VirtualRegister &v ) const;

```

physical counterparts, esp. their bit widths are identical. The abstraction from physical to virtual registers simply comes by assuming that there is an infinite amount of virtual registers available, while the number of physical registers of course is finite and limited.

This abstraction is useful to simplify the design of code optimizations, since they not necessarily need to bother with the availability of some actual, physical registers. Instead, an optimization can simply create a new virtual register on the fly and generate optimized code using such virtual registers. WIR thus generally supports so-called virtual code on the one hand, i.e., code containing at least one such virtual register, and physical code on the other hand, i.e., code containing no virtual registers at all and only relying on physical registers.

According to various processor manufacturer's ABI specifications, the management of registers is usually done at the level of code functions. Thus, WIR functions also provide the complete list API from Chapter 6 for the creation, insertion and management of objects of class WIR_VirtualRegister.

Obviously, virtual WIR code is not executable by some real processor architecture due to the incompatible numbers of used registers. It is thus necessary to transform virtual to physical WIR code by mapping all virtual registers to the few actual physical registers of some real processor architecture. This is the main purpose of the WIR register allocators described later in Section 14.8. Such mappings of virtual to physical registers regularly have to follow certain rules. E.g., a processor's calling conventions can mandate that certain physical registers must be used, especially for passing function arguments or return values. Or some physical register must not be used in certain situations, e.g., when a machine operation shall not touch the stack pointer.

It is thus necessary to express such constraints within WIR. For this purpose, class WIR_Function offers the

Listing 8.5: Handling of Physical Registers Used for Function Argument Passing

```

1 // Add a physical register as external function argument.
2 void WIR_Function::addFunctionInput( const WIR_PhysicalRegister &p );
3
4 // Get set of physical registers providing external function inputs.
5 const WIR_PhysicalRegisterSet &WIR_Function::getFunctionInputs( void ) const;

```

precoloring and interference methods shown in Listing 8.4.

The term precoloring comes from the fact that nowadays, graph coloring is the de-facto standard technique for register allocation (cf. Section 14.8). Adding precoloring information thus constrains the register allocator to use a certain physical register when allocating some virtual register. **WIR** functions now allow to insert and erase such precoloring information between a virtual and a physical register (cf. Lines 2 to 8 of Listing 8.4).

Adding a precolor using `insertPreColor` requires that the specified virtual register is actually managed by the used **WIR** function. Furthermore, both the virtual and the physical register to be used during precoloring have to belong to the same group of processor registers. Like this, it is prevented to precolor, e.g., a general-purpose virtual register with some physical floating-point register. Finally, the two register to be used for precoloring must not have been marked as interfering before (see below). If any of these rules is violated, `insertPreColor` fails with a runtime error.

The lookup of precoloring information can simply be done using `containsPreColor` (cf. Line 11 of Listing 8.4) and `findPreColor` (cf. Line 14). Both methods expect a reference to a virtual register as argument and return whether it is precolored, or its actual precolor, resp. `findPreColor` again fails with a runtime error if it is applied to a virtual register that is either not managed by the used function, or that is not precolored at all.

An interference denotes a constraint where a virtual register must not be mapped to some physical register. Interferences thus represent the opposite kind of register allocation constraints than precolors. For this simple reason, **WIR** ensures that no contradicting precoloring and interference constraints can be applied to the very same virtual and physical registers.

The insertion and deletion of interferences can be done with the corresponding `insert`, `erase` and `clear` methods from Listing 8.4 (cf. Lines 17 to 23). Looking up interferences is simple using methods `interfere` (cf. Line 26) and `findInterferences` (cf. Line 29). `findInterferences` returns a set of references to physical registers that interfere with some given virtual register. The returned set of type `WIR_PhysicalRegisterSet` is basically declared as `std::set<std::reference_wrapper<WIR_PhysicalRegister>>` within **WIR**. More details about the nature of such sets or maps declared as basic data types within **WIR** can be found in Chapter 11.

Listing 8.6: Stack Frame Handling of Class **WIR_Function**

```

1 // Set a function's stack frame size to s bytes.
2 void WIR_Function::setFrameSize( int s );
3
4 // Determine a function's stack frame size in bytes.
5 int WIR_Function::getFrameSize( void ) const;

```

Processor-specific calling conventions regularly also specify physical registers that have to be used for function argument passing. Such physical registers are thus defined outside the called function, and are used inside a function. For a correct data flow analysis – esp. during Available Definitions, Reaching Definitions, or Def-Use/Use-Def Chain analyses (cf. Chapter 13) – it is thus important to properly consider such definitions of physical

Listing 8.7: Moving of Instructions by Class WIR_BasicBlock

```

1 // Move a given instruction from its current basic block to this one.
2 WIR_Instruction &WIR_BasicBlock::moveInstruction( WIR_Instruction &i );

```

registers serving as external function input. For this purpose, [WIR](#) functions allow to specify such physical registers using the methods `addFunctionInput` and `getFunctionInputs` shown in [Listing 8.5](#).

As motivated previously, local variables are usually managed via the stack. Thus, it is crucial to properly model stack frames of functions within low-level code.

Future Work

At the time of writing of this document, class `WIR_Function` does not yet include any well-structured concepts for the generic modelling of stack frames. Especially, [WIR](#) functions currently do not support individual sections inside a stack frame for, e.g., backups of callee-saved registers, local variables, or overflow arguments. Likewise, no means currently exist to model different symbols in a stack frame along with their byte sizes, alignments, or stack or frame pointer-related stack offsets. Such a generic model of stack frames inside [WIR](#) functions is currently under development.

For the abovementioned reasons, class `WIR_Function` currently provides only rudimentary support for stack frames, in the sense that only the overall byte size of a function's stack frame can be specified or queried (cf. [Listing 8.6](#)). The inner organization of a stack frame and the generation of code properly accessing stack items with their correct offsets thus remains in the sole responsibility of application code making use of the [WIR](#) library.

8.3 Basic Blocks

Basic blocks as specified in [Definition 1.1](#) primarily represent instruction sequences so that class `WIR_BasicBlock` once more features the complete list [API](#) (cf. [Chapter 6](#)) for the handling of instructions. In addition to that, a convenience method `moveInstruction` exists (cf. [Listing 8.7](#)) that allows to move a specified instruction `i` between different basic blocks. Both the source basic block currently holding instruction `i` and the target basic block to which `i` shall be moved have to belong to the very same function. Otherwise, [WIR](#) issues a runtime error. `moveInstruction` then moves `i` to the last position of the instruction list of the target basic block.

Listing 8.8: Constructors and Methods for Name Handling of Class WIR_BasicBlock

```

1 // Default constructor creating an empty basic block.
2 WIR_BasicBlock::WIR_BasicBlock( void );
3
4 // Variadic constructor creating a basic block containing the given instructions.
5 template<typename... Args> WIR_BasicBlock::WIR_BasicBlock( WIR_Instruction &&i, Args&&... __args );
6
7 // Copy constructor.
8 WIR_BasicBlock::WIR_BasicBlock( const WIR_BasicBlock &b );
9
10 // Move constructor.
11 WIR_BasicBlock::WIR_BasicBlock( WIR_BasicBlock &&b );
12
13 // Get the basic block name.
14 std::string WIR_BasicBlock::getName( void ) const;

```

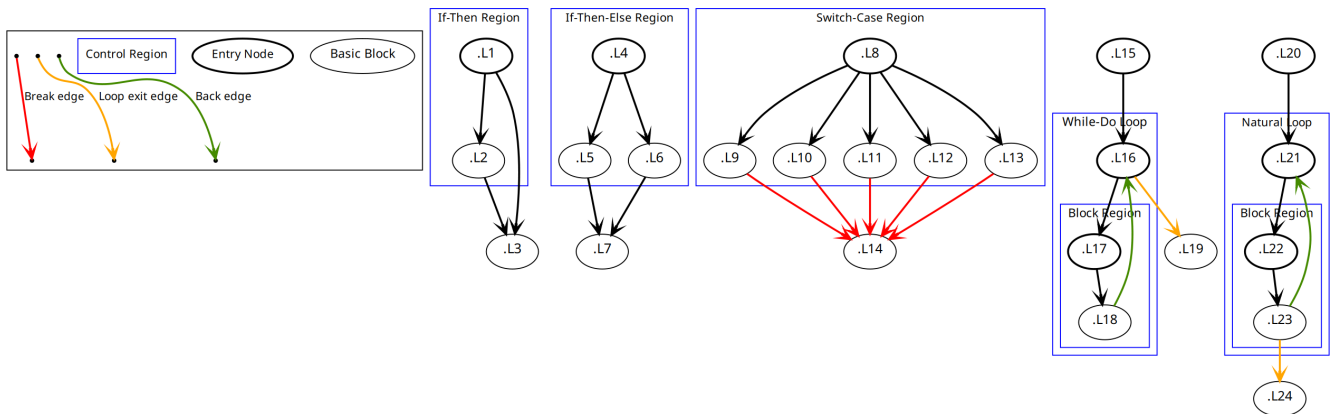



Figure 8.1: Typical Control Flow Graphs

The construction of basic blocks is straightforward using the standard constructors from Listing 8.8. Basic blocks carry a unique name that usually represents some label within assembly code. WIR creates these basic block names automatically. By default, basic block names start with “.L” followed by a unique numerical ID. If a basic block is the very first one within its owning function, its name is equal to the function’s name (this default naming convention is configurable, of course). This basic block name can be retrieved using method `getName` (cf. Line 14 from Listing 8.8).

For the computation of a basic block’s code size in bytes, method `getSize` (cf. Listing 8.9) accumulates the byte size of all instructions within a basic block.

Listing 8.9: Size Computation by Class WIR_BasicBlock

```
1 // Get a basic block's byte size.
2 unsigned long long WIR_BasicBlock::getSize( void );
```

Due to the fact that the execution of basic blocks according to Definition 1.1 must always end with its very last instruction, only this very last instruction can be a branch that directs a program’s control flow to some different location in the code. Thus, a **Control Flow Graph (CFG)** is defined as follows:

Definition 8.1 (Control Flow Graph)

A control flow graph $CFG = (V, E, s)$ is a directed graph with

- $V = \{ b_1, \dots, b_n \}$ being the set of all basic blocks,
- $E = \{ (b_i \rightarrow b_j) \mid \text{basic block } b_j \text{ can be executed immediately after } b_i \}$,
- $s \in V$ being the unique start node of the CFG.

Fig. 8.1 shows some examples of typical CFGs representing if-then, if-else and switch-case constructs, as well as typical pre-checked (for or while-do) and post-checked (do-while) loops. Obviously, such CFGs imply a successor and predecessor relation between basic blocks where, e.g., .L6 is a successor of .L4 in Fig. 8.1, while .L18 is predecessor of .L16. WIR basic blocks automatically maintain these relations and support their query using the methods shown in Listing 8.10. `getSuccessors` and `getPredecessors` return the sets of basic blocks that could be executed immediately after or before another block, resp. These sets of type `WIR_BasicBlockSet` are basically declared as `std::set<std::reference_wrapper<WIR_BasicBlock>>` within WIR (cf. Chapter 11).

Listing 8.10: Control Flow Properties of Class WIR_BasicBlock

```

1 // Get the set of successor basic blocks.
2 WIR_BasicBlockSet WIR_BasicBlock::getSuccessors( void );
3
4 // Get the set of predecessor basic blocks.
5 WIR_BasicBlockSet WIR_BasicBlock::getPredecessors( void );

```

In order to fully automatically determine these successor and predecessor relations, [WIR](#) inspects a basic block's very last instruction, if existing. If this last instruction contains a (conditional, unconditional, or indirect) jump operation, all potential jump targets are collected as successors. Otherwise, the very next basic block immediately following in the list of basic blocks maintained by the enclosing [WIR](#) function is considered as so-called implicit successor. Once the successors are identified, [WIR](#) determines the predecessors by exploiting the inverse of the successors. Methods `getSuccessors` and `getPredecessors` do not consider interprocedural control flow stemming from function calls or returns.

Future Work

At the time of writing of this document, processor architectures coming with delay slots are not yet supported. Such architectures have the effect that it is not necessarily the very last instruction of a basic block that may contain a jump operation leading to control flow changes. The control flow analyses built into [WIR](#) will be extended by this feature in the future.

8.4 Instructions

Class `WIR_Instruction` serves to model [ILP](#) in that sense that an instruction forms a group of assembly operations which are all supposed to be executed in parallel by the underlying processor hardware. As such, [WIR](#) instructions simply boil down to a container of operations so that the complete list [API](#) from Chapter 6 for the management of operations exists. In analogy to class `WIR_BasicBlock`, a convenience method `moveOperation` exists (cf. Listing 8.11) in order to move a specified operation `o` between different instructions. Again, the source and the target instructions of such a move have to belong to the very same function, and `moveOperation` appends `o` at the tail of the target instruction.

Listing 8.11: Moving of Operations by Class WIR_Instruction

```

1 // Move a given operation from its current instruction to this one.
2 WIR_Operation &WIR_Instruction::moveOperation( WIR_Operation &o );

```

Class `WIR_Instruction` comes with the standard set of constructors shown in Listing 8.12.

In order to determine the size of an instruction, the methods from Listing 8.13 can be used. `getBitWidth` accumulates the bit widths of all operations inside an instruction and returns this value. `getSize` in turn uses this bit width and divides it by 8 in order to get an instruction's byte size. In the unlikely case that the bit width of an instruction is not a multiple of 8, `getSize` rounds up so that the only partially occupied byte is fully counted.

Listing 8.12: Constructors of Class WIR_Instruction

```

1 // Default constructor creating an empty instruction.
2 WIR_Instruction::WIR_Instruction( void );
3
4 // Variadic constructor creating an instruction containing the given operations.
5 template<typename... Args> WIR_Instruction::WIR_Instruction( WIR_Operation &&o, Args&&... __args );
6
7 // Copy constructor.
8 WIR_Instruction::WIR_Instruction( const WIR_Instruction &i );
9
10 // Move constructor.
11 WIR_Instruction::WIR_Instruction( WIR_Instruction &&i );

```

Listing 8.13: Size Computation by Class WIR_Instruction

```

1 // Get an instruction's bit width.
2 unsigned int WIR_Instruction::getBitWidth( void );
3
4 // Get an instruction's byte size.
5 unsigned long long WIR_Instruction::getSize( void );

```

8.5 Operations

Machine operations are modelled by class `WIR_Operation` within `WIR`. They can be created using the constructors from Listing 8.14.

Listing 8.14: Constructors of Class WIR_Operation

```

1 // Variadic constructor creating an operation containing the given parameters.
2 template<typename... Args> WIR_Operation::WIR_Operation( const WIR_BaseProcessor::OpCode &oc,
3                                                         const WIR_BaseProcessor::OperationFormat &of,
4                                                         Args&&... __args );
5
6 // Copy constructor.
7 WIR_Operation::WIR_Operation( const WIR_Operation &o );
8
9 // Move constructor.
10 WIR_Operation::WIR_Operation( WIR_Operation &&o );

```

As can be seen, there is no default constructor creating a completely empty operation, because each machine operation must at least have some opcode and some operation format. Both are processor-specific properties of machine operations that specify the operator computed by an operation, and the number and types of arguments required by an operation according to a processor manufacturer's [ISA](#) specification. They are generically represented by classes `WIR_BaseProcessor::OpCode` and `WIR_BaseProcessor::OperationFormat`, and Chapter 9 explains their concepts in more detail. The general rationale behind this design of `WIR` operations is to achieve [ISA](#)-safeness, i.e., to prevent the creation of operations that are not compliant with some [ISA](#) specifications. If, e.g., a processor manufacturer specifies that an `ADD` machine operation must have three register parameters, then the according operation format must be provided to the constructor from Line 2. And if only two register parameters are actually passed to the constructor, it will fail with a runtime error indicating that the required operation format is violated.

An operation's opcode and format can be retrieved using methods `getOpCode` and `getOperationFormat`, resp.

Listing 8.15: Opcode and Format Handling of Class WIR_Operation

```

1 // Get an operation's opcode.
2 const WIR_BaseProcessor::OpCode &WIR_Operation::getOpCode( void );
3
4 // Get an operation's format.
5 const WIR_BaseProcessor::OperationFormat &WIR_Operation::getOperationFormat( void );

```

(cf. Listing 8.15).

Any kind of operand that an operation requires is modeled by parameters within **WIR** (cf. Section 8.6). As a consequence, class **WIR_Operation** thus acts as container of parameters so that the entire list **API** from Chapter 6 is available for parameter management. **WIR** parameters generally can be classified into either explicit or implicit parameters. An explicit parameter is one that is explicitly required by some machine operation, since an architecture's **ISA** mandates this. Explicit parameters thus ensure the syntactical correctness of **WIR** operations according to some processor manufacturer's specifications. They are thus mandatory part of an operation of some given format, and explicit parameters are usually always printed out as operation argument whenever a file dump of **WIR** is performed – e.g., during assembly output creation.

In contrast, implicit parameters are optional. An arbitrary amount of implicit parameters can freely be added to a **WIR** operation, fully independent of the operation's actual format. Implicit parameters are printed out in a file dump only upon explicit request. The rationale of implicit parameters is to ensure the semantical correctness of some operation. There exist situations where some operation implicitly reads or writes a certain register, where this register is, according to some **ISA** specification, not explicitly given as operand. Prime examples for such situations are machine operations for function calls or returns. According to an architecture's calling conventions, such operations implicitly deal with well-specified registers for argument passing, or for passing a function's return value, or for the proper handling of the return address register. The **WIR** data flow analyses – esp. its Available Definitions, Reaching Definitions, or Def-Use/Use-Def Chain analyses (cf. Chapter 13) – must be aware of such “hidden” register accesses so that subsequent code optimizations do not generate invalid code. An operation's implicit parameters are automatically considered by all **WIR** analyses so that proper analysis results can be ensured this way.

The property whether some parameter is explicit or implicit is specified during parameter construction (cf. Section 8.6), and any kind of parameters can be added to an operation using the list **API** from Chapter 6. Adding, modifying or erasing explicit parameters of an operation, however, bears the risk to end up with operations that violate their specified operation format, i.e., that are no longer **ISA**-safe. By default, **WIR** assumes that the user knows what she/he does so that no sanity checks regarding the operation format are carried out during the manipulation of an operation's parameters. This default behavior significantly speeds up the parameter handling of class **WIR_Operation**. If, however, such parameter sanity checks are desired, the static methods from Listing 8.16 come into play. After the use of `enforceParameterChecking`, **WIR** fails with a runtime error whenever parameter manipulations lead to an operation whose structure violates its given operation format. `resetParameterChecking` basically reverts this behavior.

Listing 8.16: Parameter Checking of Class WIR_Operation

```

1 // Activate explicit checks of parameters and operation formats whenever operation parameters are modified.
2 static void WIR_Operation::enforceParameterChecking( void );
3
4 // Deactivate explicit checks of parameters and operation formats.
5 static void WIR_Operation::resetParameterChecking( void );

```

Listing 8.17: Explicit and Implicit Parameter Handling of Class WIR_Operation

```

1 // Get the list of all explicit parameters.
2 const std::list<std::reference_wrapper<WIR_Parameter>> &WIR_Operation::getExplicitParameters( void ) const;
3
4 // Get the nth explicit parameter, starting with n = 1.
5 WIR_Parameter &WIR_Operation::getExplicitParameter( unsigned int n ) const;
6
7 // Get the total number of explicit parameters.
8 unsigned int WIR_Operation::getNumberOfExplicitParameters( void ) const;
9
10 // Get the total number of implicit parameters.
11 unsigned int WIR_Operation::getNumberOfImplicitParameters( void ) const;

```

For the handling of explicit and implicit parameters, class WIR_Operation provides the methods shown in Listing 8.17. A list of references to all explicit parameters is provided by getExplicitParameters. A reference to the n^{th} explicit parameter is provided by getExplicitParameter, where parameter numbers begin with the value 1. Generally, WIR does not perform a sanity check if a parameter number n is provided that does

Listing 8.18: Operation Classification

```

1 // Determine whether an operation reads from memory.
2 bool WIR_Operation::isMemoryLoad( void ) const;
3
4 // Determine whether an operation writes to memory.
5 bool WIR_Operation::isMemoryStore( void ) const;
6
7 // Determine whether an operation implicitly accesses memory, without being an explicit load or store.
8 bool WIR_Operation::isImplicitMemoryAccess( void ) const;
9
10 // Determine whether an operation is a register-register move.
11 bool WIR_Operation::isMove( void ) const;
12
13 // Determine whether an operation calls a function.
14 bool WIR_Operation::isCall( void ) const;
15
16 // Determine whether an operation indirectly calls a function.
17 bool WIR_Operation::isIndirectCall( void ) const;
18
19 // Determine whether an operation returns from a function call.
20 bool WIR_Operation::isReturn( void ) const;
21
22 // Determine whether an operation performs a conditional jump.
23 bool WIR_Operation::isConditionalJump( void ) const;
24
25 // Determine whether an operation performs an unconditional jump.
26 bool WIR_Operation::isUnconditionalJump( void ) const;
27
28 // Determine whether an operation indirectly performs a jump.
29 bool WIR_Operation::isIndirectJump( void ) const;
30
31 // Determine whether an operation performs a jump.
32 bool WIR_Operation::isJump( void ) const;
33
34 // Determine whether an operation has side effects.
35 bool WIR_Operation::hasSideEffects( void ) const;

```

Listing 8.19: Jump Target Handling of Class WIR_Operation

```

1 // Add a given basic block to a jump operation's set of explicit jump targets.
2 void WIR_Operation::addJumpTarget( const WIR_BasicBlock &b );
3
4 // Remove a given basic block from a jump operation's set of explicit jump targets.
5 void WIR_Operation::eraseJumpTarget( const WIR_BasicBlock &b );
6
7 // Determine whether explicit jump targets have been specified.
8 bool WIR_Operation::jumpTargetsAdded( void ) const;
9
10 // Get the set of basic blocks being jump targets of an operation.
11 WIR_BasicBlockSet WIR_Operation::getJumpTargets( void ) const;

```

not exist. The overall number of both explicit and implicit parameters of an operation can be retrieved using `getNumberOfExplicitParameters` or `getNumberOfImplicitParameters`, resp.

Depending on their opcodes and the nature of their functional behavior, operations can generally be classified into different categories. In the context of an assembly-level IR, it is especially relevant to determine whether operations access memory, or whether they change a program's control flow. Such information is provided by class `WIR_Operation` via the methods shown in Listing 8.18. These properties can only be retrieved, but they cannot actively be set for some operation, since this classification is hard-wired with the architecture-specific mnemonics (cf. Section 9.1.1.4). Method `isMove` helps to identify operations copying a value from one register to another. Information of this kind is of particular interest for register allocation. Operations realizing function calls and where the called function is directly provided as explicit parameter can be identified via `isCall`, register-indirect calls can be handled using `isIndirectCall`. Regarding jump operations, `WIR` operations allow to distinguish between conditional and unconditional direct jumps where the jump target is given as explicit parameter. Register-indirect jumps are addressed by method `isIndirectJump`. If an operation is any kind of abovementioned jump, `isJump` returns `true`. Finally, method `hasSideEffects` helps to identify operations that lead to non-obvious changes of a processor's internal state such as, e.g., disabling interrupts, changing into supervisor mode, or invalidating cache lines, etc.

As already discussed in Section 8.3, a basic block's very last operation determines the overall control flow of a program. Using the methods shown in Listing 8.18, it is easy to automatically identify control flow branches. But for the proper generation of CFGs, it is in addition important to identify all potential targets of some jump operation. This is supported by the methods of class `WIR_Operation` shown in Listing 8.19. `WIR` follows a semi-automatic approach here, since it is in general impossible to determine all potential jump targets fully automatically. Especially in the presence of register-indirect jumps or far jumps using so-called literal pools, `WIR` relies on additional annotations. Such annotations can be added to a jump operation via method `addJumpTarget` so that the specified basic block is stored as jump target. Likewise, `eraseJumpTarget` can be used to remove such annotated jump targets. `jumpTargetsAdded` returns whether additional jump target annotations have been specified for an operation or not. Finally, method `getJumpTargets` provides the set of all basic blocks to which an operation actually jumps. This set of jump targets is determined as follows:

- If the operation is not a jump at all, the empty set is returned.
- If jump targets were explicitly specified using `addJumpTarget`, the set of all these annotated jump targets is returned.
- If no jump targets were annotated using `addJumpTarget`, `getJumpTargets` assumes that the operation's jump target is provided as explicit parameter. It thus checks all operation parameters, determines whether they represent some label, and finally returns the set of all basic blocks that are referred by these labels.

Listing 8.20: Size Computation by Class WIR_Operation

```

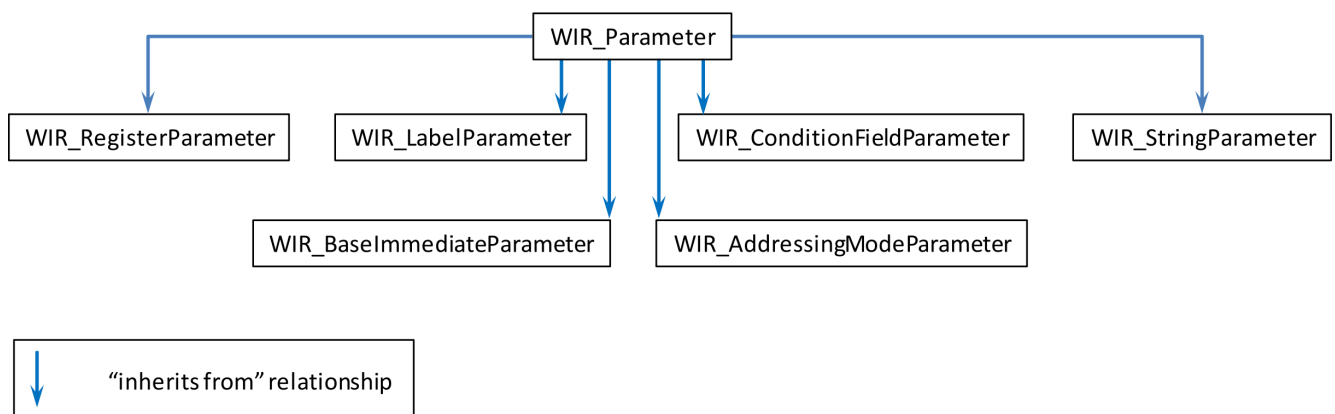
1 // Get an operation's bit width.
2 unsigned int WIR_Operation::getBitWidth( void );
3
4 // Get an operation's byte size.
5 unsigned long long WIR_Operation::getSize( void );

```

Since indirect jumps are the only class of jumps that need a manual annotation of jump targets, and since they occur rather infrequent in real-life code, the presented approach of jump target handling within **WIR** requires minimal effort for a proper control flow analysis.

In analogy to instructions, class **WIR_Operation** features methods for bit width and byte size computation (cf. Listing 8.20). Both methods `getBitWidth` and `getSize` behave as described earlier in Section 8.4.

8.6 Parameters

Figure 8.2: Inheritance Diagram of Class **WIR_Parameter**

Class **WIR_Parameter** is the generic representation of arguments of machine operations. Due to the large variety of different kinds of parameters, this class is an abstract base class from which the specialized **WIR** parameters

Listing 8.21: Generic Type Handling of Class **WIR_Parameter**

```

1 // Get a parameter's type.
2 virtual WIR_ParameterType WIR_Parameter::getType( void ) const = 0;
3
4 // Enum WIR_ParameterType represents different types of WIR parameters.
5 enum class WIR_ParameterType : char
6 {
7     addr,    // An addressing mode parameter.
8     cond,    // A condition field parameter.
9     imm,     // An immediate parameter.
10    label,   // A label parameter.
11    reg,     // A register parameter.
12    str,     // A string parameter.
13 };

```

Listing 8.22: Handling of Explicit and Implicit WIR_Parameters

```

1 // Set whether a parameter is implicit or not.
2 void WIR_Parameter::setImplicit( bool i = true );
3
4 // Determine whether a parameter is explicit or not.
5 bool WIR_Parameter::isExplicit( void ) const;
6
7 // Determine whether a parameter is implicit or not.
8 bool WIR_Parameter::isImplicit( void ) const;

```

depicted in Fig. 8.2 inherit. For this reason, the constructors of class `WIR_Parameter` do not require any further discussion here. References and pointers to actual objects derived from `WIR_Parameter` can, as usual for C++, dynamically be casted back and forth using `std::dynamic_cast`. For a convenient check whether some actual object is, e.g., a register or a label parameter, the purely virtual method `getType` (cf. Line 2 of Listing 8.21) can be used. `getType` returns the appropriate element of the enumeration type `WIR_ParameterType`, cf. Line 5.

The notion of explicit and implicit parameters has already been introduced earlier in Section 8.5. The status whether some parameter is explicit or implicit can be set or retrieved using the methods from Listing 8.22. Here, `setImplicit` (cf. Line 2) can be used to mark a parameter as implicit or not, depending on its Boolean argument. `isExplicit` (cf. Line 5) returns whether a parameter is explicit, while `isImplicit` (cf. Line 8) is its negation.

The following sections present the properties of the derived parameter classes in more detail, starting with the most relevant classes for register parameters (Section 8.6.1), immediates (Section 8.6.2) and labels (Section 8.6.3), followed by addressing mode parameters (Section 8.6.4), condition fields (Section 8.6.5) and finally string parameters (Section 8.6.6).

8.6.1 Register Parameters

Class `WIR_RegisterParameter` models any kind of register that acts as operand of an operation. The constructors of this class are shown in Listing 8.23. Obviously, the construction of register parameters requires a reference to some register (cf. Line 2). This can be any kind of physical or virtual register, of any kind of processor architecture and ISA modeled in WIR. Furthermore, the parameter's def/use characteristics must be given, i.e.,

Listing 8.23: Constructors of Class `WIR_RegisterParameter`

```

1 // Default constructor creating a register parameter with the given register.
2 WIR_RegisterParameter::WIR_RegisterParameter( const WIR_BaseRegister &r, WIR_Usage u, bool i = false );
3
4 // Copy constructor.
5 WIR_RegisterParameter::WIR_RegisterParameter( const WIR_RegisterParameter &p );
6
7 // Move constructor.
8 WIR_RegisterParameter::WIR_RegisterParameter( WIR_RegisterParameter &&p );
9
10 // Enum WIR_Usage represents def/use properties of WIR parameters.
11 enum class WIR_Usage : char
12 {
13     def,          // A parameter that is defined/written.
14     defuse,       // A parameter that is both defined/written and used/read.
15     use           // A parameter that is used/read.
16 };

```


Listing 8.24: Usage Handling of Class WIR_RegisterParameter

```

1 // Set a register parameter's usage characteristics.
2 void WIR_RegisterParameter::setUsage( WIR_Usage u );
3
4 // Get a register parameter's usage characteristics.
5 WIR_Usage WIR_RegisterParameter::getUsage( void ) const;
6
7 // Determine whether a register parameter is defined.
8 bool WIR_RegisterParameter::isDefined( void ) const;
9
10 // Determine whether a register parameter is used.
11 bool WIR_RegisterParameter::isUsed( void ) const;
12
13 // Determine whether a register parameter is both defined and used.
14 bool WIR_RegisterParameter::isDefUsed( void ) const;

```

the information whether the specified register will be written, read, or both in the context of its associated **WIR** operation. Information of this kind is handled by the enumeration type **WIR_Usage** (cf. Line 11), and it is essential for a proper automatic data flow analysis. An example showing the construction of a register parameter like this can be found in Line 25 of Listing 7.1 (see Page 38).

After its construction, the def/use characteristics of a register parameter can be managed with the methods from Listing 8.24. The usage can explicitly be set via `setUsage` (Line 2), and its current usage can be retrieved with `getUsage` (Line 5). For convenience, class **WIR_RegisterParameter** also provides the dedicated Boolean predicates `isDefined` (Line 8), `isUsed` (Line 11) and `isDefUsed` (Line 14) for the simple query of a specific usage status.

Finally, a reference to the register associated with a register parameter can be obtained using `getRegister` (cf. Listing 8.25).

Listing 8.25: Register Handling of Class WIR_RegisterParameter

```

1 // Get the register associated with a parameter.
2 WIR_BaseRegister &WIR_RegisterParameter::getRegister( void ) const;

```

8.6.2 Immediates

Immediate parameters model constant numbers that can be used as argument of machine operations. By nature, such immediates are typically characterized by architecture-specific properties like, e.g., their bit widths and whether they are considered to be signed or unsigned. For these reasons, **WIR** again makes use of a C++ class hierarchy modelling generic signed and unsigned immediates, and from which finally processor-specific immediates of actual bit widths are inherited. A simplified inheritance diagram of this class hierarchy is depicted in Fig. 8.3. For the sake of simplicity, internal details of this class hierarchy like, e.g., some C++ templates, are omitted from the following discussion, since application code will not come into touch with them. Instead, only the relevant concepts that are finally available in classes **WIR_SignedImmediateParameter** and **WIR_UnsignedImmediateParameter** will be presented. These two classes provide the generic interfaces for signed and unsigned immediate parameters of any bit width, resp. And processor-specific immediates are then derived by inheriting from these two classes like, e.g., the two RISC-V classes **RV_Const12_Signed** and **RV_Const20_Unsigned** shown in Fig. 8.3.

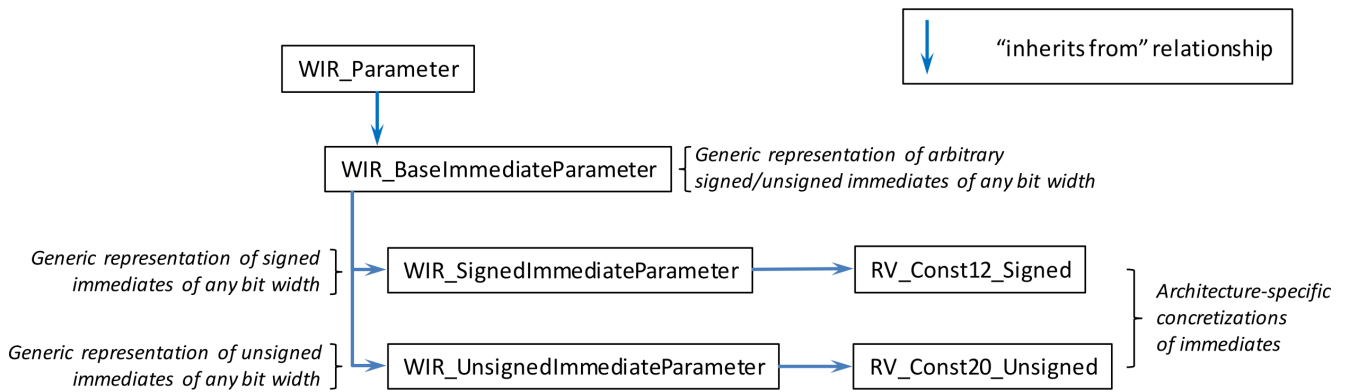


Figure 8.3: Simplified Inheritance Diagram of Classes Modelling Immediate Parameters

Both signed and unsigned immediates have in common that the `WIR` library does not come with any support for arbitrary bit widths of immediates. In particular, no third-party libraries for arbitrary precision arithmetics like, e.g., GMP [12], are used. Instead, `WIR` relies on the C++ builtin types `signed long long` and `unsigned long long` and their actual bit widths on the host system on which the `WIR` library has been built (cf. Chapter 5).

Warning

Due to this dependence of `WIR` immediates on the host system's bit widths, the construction of any immediate parameter with a bit width exceeding that of the host system will fail with a runtime error. Likewise, an attempt to assign some actual value exceeding the host's `long long` bit widths to an immediate will also fail. Furthermore, this behavior of `WIR` immediates also depends on whether the whole software library has been built using 64 or 32 bits of the host system, cf. Chapter 4, Table 4.1.

This dependence of `WIR` immediates on the standard data types `signed long long` and `unsigned long long` is also visible from the two default constructors shown in Lines 2 and 12 of Listing 8.26. They take the immediate's actual value `v` as argument, which either is a `signed long long` (class `WIR_SignedImmediateParameter`) or an `unsigned long long` (class `WIR_UnsignedImmediateParameter`). Furthermore, the immediate's bit width

Listing 8.26: Constructors of Classes for Signed and Unsigned Immediates

```

1 // Default constructor creating a signed immediate parameter with the given value and bit width.
2 WIR_SignedImmediateParameter::WIR_SignedImmediateParameter( signed long long v, unsigned int w );
3
4 // Copy constructor.
5 WIR_SignedImmediateParameter::WIR_SignedImmediateParameter( const WIR_SignedImmediateParameter &i );
6
7 // Move constructor.
8 WIR_SignedImmediateParameter::WIR_SignedImmediateParameter( WIR_SignedImmediateParameter &&i );
9
10
11 // Default constructor creating an unsigned immediate parameter with the given value and bit width.
12 WIR_UnsignedImmediateParameter::WIR_UnsignedImmediateParameter( unsigned long long v, unsigned int w );
13
14 // Copy constructor.
15 WIR_UnsignedImmediateParameter::WIR_UnsignedImmediateParameter( const WIR_UnsignedImmediateParameter &i );
16
17 // Move constructor.
18 WIR_UnsignedImmediateParameter::WIR_UnsignedImmediateParameter( WIR_UnsignedImmediateParameter &&i );

```

Listing 8.27: Value Handling of Signed and Unsigned Immediates

```

1 // Set a signed immediate's actual value.
2 virtual void WIR_SignedImmediateParameter::setValue( signed long long v );
3
4 // Get a signed immediate's actual value.
5 signed long long WIR_SignedImmediateParameter::getValue( void ) const;
6
7 // Get a signed immediate's bit width.
8 unsigned int WIR_SignedImmediateParameter::getBitWidth( void ) const;
9
10 // Get the minimal signed value that can be represented in two's-complement using the given number of bits.
11 static signed long long WIR_SignedImmediateParameter::getMinValue( unsigned int w );
12
13 // Get the maximal signed value that can be represented in two's-complement using the given number of bits.
14 static signed long long WIR_SignedImmediateParameter::getMaxValue( unsigned int w );
15
16 // Determine whether some immediate parameter is signed.
17 virtual bool WIR_SignedImmediateParameter::isSigned( void ) const;
18
19 // Determine whether some immediate parameter is unsigned.
20 virtual bool WIR_SignedImmediateParameter::isUnsigned( void ) const;
21
22
23 // Set an unsigned immediate's actual value.
24 virtual void WIR_UnsignedImmediateParameter::setValue( unsigned long long v );
25
26 // Get an unsigned immediate's actual value.
27 unsigned long long WIR_UnsignedImmediateParameter::getValue( void ) const;
28
29 // Get an unsigned immediate's bit width.
30 unsigned int WIR_UnsignedImmediateParameter::getBitWidth( void ) const;
31
32 // Get the maximal unsigned value that can be represented using the given number of bits.
33 static unsigned long long WIR_UnsignedImmediateParameter::getMaxValue( unsigned int w );
34
35 // Determine whether some immediate parameter is signed.
36 virtual bool WIR_UnsignedImmediateParameter::isSigned( void ) const;
37
38 // Determine whether some immediate parameter is unsigned.
39 virtual bool WIR_UnsignedImmediateParameter::isUnsigned( void ) const;

```

w is also required for construction. Class `WIR_SignedImmediateParameter` always assumes that immediates are represented as binary number in two's-complement format, while class `WIR_UnsignedImmediateParameter` assumes the natural unsigned binary representation. Thus, the range of values v that a signed immediate of w bits can have, is $-2^{w-1} \leq v \leq +2^{w-1} - 1$, while the value range of unsigned immediates is restricted to $0 \leq v \leq +2^w - 1$. Both constructors ensure that the immediate's value v is always within these abovementioned ranges. Otherwise, they produce a runtime error. As usual, both immediate classes come with proper copy and move constructors.

Listing 8.27 shows the methods of classes `WIR_SignedImmediateParameter` and `WIR_UnsignedImmediateParameter` that can be used to handle an immediate's actual value. The methods `setValue` (Lines 2 and 24) set a signed or unsigned immediate's value, resp. Again, both methods ensure that the new value must fall into the abovementioned value ranges for the immediate's bit width. `getValue` (Lines 5 and 27) and `getBitWidth` (Lines 8 and 30) return an immediate's value and bit width, resp. The static methods `getMinValue` and `getMaxValue` of class `WIR_SignedImmediateParameter` (Lines 11 and 14) provide the minimal and maximal signed values that can be represented using w bits, resp. Thus, these methods determine the bounds of the correspond-

ing two's-complement value range. Likewise, method `WIR_UnsignedImmediateParameter::getMaxValue` (Line 33) returns the maximal possible unsigned value using `w` bits. A `getMinValue` method does not exist for unsigned immediates, since this minimal value is always 0. Finally, the methods `isSigned` (Lines 17 and 36) and `isUnsigned` (Lines 20 and 39) return whether an immediate is signed or unsigned, resp.

8.6.3 Labels

Class `WIR_LabelParameter` models parameters that act as a label, i.e., that refer to a symbolic memory address where some piece of code or data resides. Prime examples where labels come into play are branch or function call operations where the jump target or the called function is explicitly given as a label. Many architectures also feature memory load or store operations where the address to be read or written is also specified using the label of some (global) data object. These facets of label parameters lead to the constructors shown in Listing 8.28. The default constructors from Lines 2, 5 and 8 receive a reference to a basic block, data object (cf. Section 8.7), or a function as argument, resp. As usual, copy and move constructors also exist.

Listing 8.28: Constructors of Class `WIR_LabelParameter`

```

1 // Default constructor creating a label parameter referring to the given basic block.
2 explicit WIR_LabelParameter( const WIR_BasicBlock &b );
3
4 // Default constructor creating a label parameter referring to the given data object.
5 explicit WIR_LabelParameter( const WIR_Data &d );
6
7 // Default constructor creating a label parameter referring to the given function.
8 explicit WIR_LabelParameter( const WIR_Function &f );
9
10 // Copy constructor.
11 WIR_LabelParameter( const WIR_LabelParameter &l );
12
13 // Move constructor.
14 WIR_LabelParameter( WIR_LabelParameter &&l );

```

After its construction, the characteristics of a label parameter, i.e., whether it refers to a data object or a function etc. can be queried using method `getLabelType` (cf. Line 2 of Listing 8.29). This method returns the appropriate element of the enumeration type `WIR_SymbolType` as defined in Line 5.

The actual object to which a label parameter refers can then be retrieved with the methods from Listing 8.30. The methods `getBasicBlock`, `getData` and `getFunction` return a reference to the corresponding piece of code or data, resp. However, all these methods fail with a runtime error if the label's actual type as determined by

Listing 8.29: Label Type Handling of Class `WIR_LabelParameter`

```

1 // Get a label's type.
2 WIR_SymbolType WIR_LabelParameter::getLabelType( void ) const;
3
4 // Enum WIR_SymbolType represents different types of symbolic WIR addresses.
5 enum class WIR_SymbolType : char
6 {
7     block,    // A symbol referring to a basic block.
8     data,     // A symbol referring to a data object.
9     function  // A symbol referring to a function.
10 };

```

Listing 8.30: Reference Handling of Class WIR_LabelParameter

```

1 // Get the basic block to which a label refers, if any.
2 WIR_BasicBlock &WIR_LabelParameter::getBasicBlock( void ) const;
3
4 // Get the data object to which a label refers, if any.
5 WIR_Data &WIR_LabelParameter::getData( void ) const;
6
7 // Get the function to which a label refers, if any.
8 WIR_Function &WIR_LabelParameter::getFunction( void ) const;

```

getLabelType above is incompatible, e.g., if getFunction is called for a label parameter that actually refers to a basic block.

The name of a label parameter can be obtained with method getName (cf. Listing 8.31), which simply boils down to the strings returned by the getName methods of classes WIR_Function (cf. Listing 8.2), WIR_BasicBlock (cf. Listing 8.8) or WIR_Data (cf. Listing 8.37), resp.

Listing 8.31: Name Handling of Class WIR_LabelParameter

```

1 // Get the label name.
2 std::string WIR_LabelParameter::getName( void ) const;

```

8.6.4 Addressing Modes

For some processor *ISAs*, memory load or store operations could require the specification of some addressing mode, i.e., of the way how the machine computes the effective address to be loaded or stored from the parameters provided by the operation. Prominent examples of such addressing modes are *Base + Offset* where the effective address is the sum of a base address residing in a register, and a constant offset, or *Post-Increment Addressing* where the base address in some register is automatically incremented by a constant after the memory load or store. Such addressing modes are obviously highly *ISA*-specific and are generally modeled by class WIR_BaseProcessor::AddressingMode (cf. Section 9.1.1.6). However, in order to be able to specify such an addressing mode as parameter of a *WIR* operation, class WIR_AddressinModeParameter gets involved. Its construction requires a reference to the actual architecture-specific addressing mode (cf. Line 2 of Listing 8.32).

Listing 8.32: Constructors of Class WIR_AddressinModeParameter

```

1 // Default constructor creating a parameter using the given addressing mode.
2 explicit WIR_AddressinModeParameter( const WIR_BaseProcessor::AddressingMode
3                                     &m );
4
5 // Copy constructor.
6 WIR_AddressinModeParameter( const WIR_AddressinModeParameter &a );
7
8 // Move constructor.
9 WIR_AddressinModeParameter( WIR_AddressinModeParameter &&a );

```

Setting and getting a parameter's addressing mode can easily be done with the methods setAddressinMode and getAddressinMode from Listing 8.33.

Listing 8.33: Mode Handling of Class WIR_AddressModeParameter

```

1 // Set the addressing mode.
2 void WIR_AddressModeParameter::setAddressingMode( const WIR_BaseProcessor::AddressingMode &m );
3
4 // Get the actual addressing mode.
5 WIR_BaseProcessor::AddressingMode &WIR_AddressModeParameter::getAddressingMode( void );

```

8.6.5 Condition Fields

Conditional Execution (also known as *Predicated Execution*) is a concept supported by some ISAs where machine operations are effectively evaluated only if some Boolean condition is satisfied. Such conditions then become integral part of machine operations and thus may also be specified as parameter. Very similar to addressing modes (cf. Section 8.6.4), such ISA-specific conditions are generically represented by class `WIR_BaseProcessor::Condition`, and class `WIR_ConditionFieldParameter` then only serves as specialized parameter referring to such a condition.

As a consequence, the public API of `WIR_ConditionFieldParameter` is equivalent to that of addressing mode parameters (cf. Listing 8.34) so that we omit its detailed discussion here for the sake of brevity.

Listing 8.34: Public Interface of Class WIR_ConditionFieldParameter

```

1 // Default constructor creating a parameter using the given condition.
2 explicit WIR_ConditionFieldParameter( const WIR_BaseProcessor::Condition &c );
3
4 // Copy constructor.
5 WIR_ConditionFieldParameter( const WIR_ConditionFieldParameter &cf );
6
7 // Move constructor.
8 WIR_ConditionFieldParameter( WIR_ConditionFieldParameter &&cf );
9
10 // Set the condition.
11 void WIR_ConditionFieldParameter::setCondition( const WIR_BaseProcessor::Condition &c );
12
13 // Get the actual condition.
14 WIR_BaseProcessor::Condition &WIR_ConditionFieldParameter::getCondition( void );

```

8.6.6 String Parameters

Finally, some ISAs might – in extremely rare cases – feature operations with parameters that do not fit to any of the specialized parameter classes like, e.g., register or label parameters. Such situations can happen if a standardized processor architecture is extensible by co-processors or special-purpose registers which are not part of the architecture's ISA itself. In such a scenario, machine operations need to pass non-standard opcodes and register arguments to such a co-processor, while these opcodes or registers might not be captured by the well-structured WIR classes introduced up to this point.

In order to also model such non-standard parameters, WIR provides the class `WIR_StringParameter` which allows to use arbitrary unstructured C++ strings as operation parameters. Not surprisingly, the constructors of this class expect the unstructured string as argument, cf. Lines 2 and 5 of Listing 8.35.

After its construction, a parameter's string can be set and retrieved using methods `setString` and `getString`, cf. Listing 8.36.

Listing 8.35: Constructors of Class WIR_StringParameter

```

1 // Default constructor creating a parameter with a copied string.
2 explicit WIR_StringParameter( const std::string &s );
3
4 // Default constructor creating a parameter with a moved string.
5 explicit WIR_StringParameter( std::string &&s );
6
7 // Copy constructor.
8 WIR_StringParameter( const WIR_StringParameter &sp );
9
10 // Move constructor.
11 WIR_StringParameter( WIR_StringParameter &&sp );

```

Listing 8.36: String Handling of Class WIR_StringParameter

```

1 // Set parameter to a copied string.
2 void WIR_StringParameter::setString( const std::string &s );
3
4 // Set parameter to a moved string.
5 void WIR_StringParameter::setString( std::string &&s );
6
7 // Get the parameter's string.
8 std::string WIR_StringParameter::getString( void ) const;

```

8.7 Data Objects and Their Initializers

While all the classes presented in the previous Sections 8.1 to 8.6 represent code within the WIR software model, data regularly is managed either in a system's stack at a low abstraction level, or resides in some dedicated memory areas. Usually, local variables that are visible only within individual functions or in even smaller scopes is managed via the stack. All other kinds of data with a scope larger than local variables like, e.g., global variables,

Listing 8.37: Constructors and Methods for Name Handling of Class WIR_Data

```

1 // Default constructor creating a data object with a copied string as name.
2 explicit WIR_Data::WIR_Data( const std::string &s );
3
4 // Default constructor creating a data object with a moved string as name.
5 explicit WIR_Data::WIR_Data( std::string &&s );
6
7 // Copy constructor.
8 WIR_Data::WIR_Data( const WIR_Data &d );
9
10 // Move constructor.
11 WIR_Data::WIR_Data( WIR_Data &&d );
12
13 // Set data object name to a copied string.
14 void WIR_Data::setName( const std::string &s );
15
16 // Set data object name to a moved string.
17 void WIR_Data::setName( std::string &&s );
18
19 // Get the data object's name.
20 std::string WIR_Data::getName( void ) const;

```

Listing 8.38: Size Computation by Class WIR_Data

```

1 // Set a data object's byte size.
2 void WIR_Data::setSize( std::size_t s );
3
4 // Get a data object's byte size.
5 std::size_t WIR_Data::getSize( void ) const;

```

reside in dedicated memory areas and are modeled as so-called data objects within [WIR](#). More generally, class `WIR_Data` is the generic representation of any kind of data objects that are not allocated on a system's stack.

Class `WIR_Data` comes with the standard set of constructors and the usual methods to set and get its name as shown in [Listing 8.37](#). A data object's byte size in memory can be set and retrieved using the methods `setSize` and `getSize` shown in [Listing 8.38](#).

The main purpose of class `WIR_Data` – besides simply occupying a certain amount of bytes in some memory – is to manage all the values that are used to initialize a data object, if any. Such initializers are represented by class `WIR_DataInit`, and a data object provides the list [API](#) shown in [Listing 8.39](#) for the handling of such initializers.

Listing 8.39: Initializer Handling of Class WIR_Data

```

1 // Determine whether a data object is initialized.
2 bool WIR_Data::isInitialized( void ) const;
3
4 // Copy-add a new initializer at the end of a list, after its current last element.
5 WIR_DataInit WIR_Data::pushBackInitData( const WIR_DataInit &i );
6
7 // Move-add a new initializer at the end of a list, after its current last element.
8 WIR_DataInit WIR_Data::pushBackInitData( WIR_DataInit &&i );
9
10 // Get the full list of references to all initializers.
11 std::list<std::reference_wrapper<WIR_DataInit>> &WIR_Data::getInitData( void ) const;
12
13 // Remove all initializers.
14 void WIR_Data::clearInitData( void );
15
16 // Fold subsequent space entries in the initializer list into one combined space entry.
17 void WIR_Data::foldSpaces( void );

```

Method `isInitialized` (cf. [Line 2](#)) can be used to check whether a data object is initialized at all. The following methods to add, get and clear initializers ([Lines 5 to 14](#)) behave identical to what has been described earlier in [Chapter 6](#). In case that only some parts of a data object are initialized, so-called space initializers need to be added for the uninitialized parts. If several of these space initializers directly follow each other within a data object, they can be turned into a single one by applying method `foldSpaces` from [Line 17](#).

Within [WIR](#), initializers carry a certain type that basically specifies how much memory gets initialized, and of course the actual initialization value(s). This is reflected by the variadic default constructor of class `WIR_DataInit` shown in [Line 2](#) of [Listing 8.40](#). It expects a specifier of an initializer's type as first argument and one or several references to strings holding the initialization values next. Initializer types are represented by the enumeration type `WIR_DataInitType` (cf. [Lines 14 to 27](#)), and they represent, e.g., string literals, byte, half-word or word literals, or floating-point literals. Technically, such initializers refer to specific assembly code directives, and the elements of `WIR_DataInitType` directly match with the directives supported by GNU-compatible assemblers.

Listing 8.40: Constructors of Class WIR_DataInit

```

1 // Variadic default constructor creating an initializer of a certain type.
2 template<typename... Args> WIR_DataInit::WIR_DataInit( WIR_DataInitType t, const std::string &s,
3                                                     const Args&... __args );
4
5 // Default constructor creating a space initializer.
6 explicit WIR_DataInit::WIR_DataInit( std::size_t s );
7
8 // Copy constructor.
9 WIR_DataInit::WIR_DataInit( const WIR_DataInit &i );
10
11 // Move constructor.
12 WIR_DataInit::WIR_DataInit( WIR_DataInit &&i );
13
14 // Enum WIR_DataInitType represents different types of WIR initializers.
15 enum class WIR_DataInitType : char
16 {
17     iascii,    // A string literal initializer.
18     iasciz,    // A zero-terminated string literal initializer.
19     ibyte,     // A byte literal initializer.
20     idouble,   // A double-precision float literal initializer.
21     idword,    // A double-word literal initializer.
22     ifloat,    // A single-precision float literal initializer.
23     ihword,    // A half-word literal initializer.
24     ishort,    // A short-word literal initializer.
25     ispace,    // No initializer literal, only some skipped space.
26     isymbol,   // A symbolic pointer literal initializer.
27     iword     // A word literal initializer.
28 };

```

The variadic constructor from Line 2 should not be used for uninitialized spaces inside some data object, i.e., it should not be applied for initialization type `ispace`. Instead, the constructor from Line 5 should be chosen for this purpose. This constructor expects the number of bytes to be left uninitialized as only argument. As usual, copy and move constructors are also provided (Lines 8 and 11).

The attributes assigned to an initializer during its construction can be retrieved using the methods shown in Listing 8.41. Method `getType` (cf. Line 2) returns an initializer's type as encoded by the enumeration `WIR_DataInitType`. All actual initialization values can be accessed using method `getValues` (Line 5). This method returns a

Listing 8.41: Initializer Handling and String Processing of Class WIR_DataInit

```

1 // Get an initializer's type.
2 WIR_DataInitType WIR_DataInit::getType( void ) const;
3
4 // Get all initialization literals.
5 const std::list<std::string> &WIR_DataInit::getValues( void ) const;
6
7 // Get the number of bytes skipped in initialization.
8 std::size_t WIR_DataInit::getSpace( void ) const;
9
10 // Replace special characters in a string by escaped versions.
11 static std::string WIR_DataInit::escapeString( const std::string &s );
12
13 // Replace escaped character sequences in a string by special characters.
14 static std::string WIR_DataInit::unescapeString( const std::string &s );

```


reference to a list of strings containing all initializer literals that have been passed to the variadic default constructor above. For initializers representing some uninitialized space, method `getSpace` (Line 8) can be used to obtain the number of bytes left uninitialized.

As class `WIR_DataInit` solely uses string literals to represent initialization values for the sake of simplicity, two static convenience methods for string handling are furthermore provided. Depending on the nature of assembly code that shall be created using the [WIR](#) library, string literals may contain non-printable special characters like, e.g., line feeds or tabs, or not. If such special characters shall be avoided, method `escapeString` (Line 11) can be used which replaces non-printable characters by escape sequences such as `"\n"` or `"\t"`. The reverse conversion from strings containing escape sequences to strings with non-printable special characters is realized by method `unescapeString` (Line 14).

9 The WIR Hardware Model

This chapter describes and specifies the structure and [API](#) of all classes that serve to model hardware in some [WIR](#) system. Thus, the subsequent Sections [9.1](#) to [9.4](#) focus in detail on the complete hierarchy of system components from processors via memory regions to system buses as depicted in [Fig. 1.3](#).

Class `WIR_SystemComponent` depicted in [Fig. 1.3](#) serves as abstract base class for all the hardware components described in the following. In analogy to parameters, the method `getType` (cf. [Listing 9.1](#)) can be used to quickly check whether some actual object within the [WIR](#) hardware model is, e.g., a processor core or a cache.

Listing 9.1: Generic Type Handling of Class `WIR_SystemComponent`

```

1 // Get a system component's type.
2 virtual WIR_SystemComponentType WIR_SystemComponent::getType( void ) const = 0;
3
4 // Enum WIR_SystemComponentType represents different types of WIR parameters.
5 enum class WIR_SystemComponentType : char
6 {
7     bus,          // A system bus.
8     cache,        // A cache.
9     core,          // A processor core.
10    memory,        // A physical memory region.
11    section,       // An ELF executable section.
12    system         // An entire system.
13 };

```

9.1 Processors

[WIR](#) processors model both the software and the hardware characteristics of actual processor cores, i.e., their [ISAs](#), register files, clock frequencies, logical memory sections etc. The [WIR](#) library is designed to support an arbitrary number of different processor models at the same time, without the need to rebuild the library. The rationale behind this design is to enable the modelling of heterogeneous multi-core systems consisting of various processor cores of different type. As already described in [Section 1.2](#), [WIR](#) features models for a basic ARMv4 core, MIPS SPIM, RISC-V RV32I, and TriCore TC1.3, plus all the derived models listed in [Table 1.1](#) that inherit from these base models. The simplified inheritance diagram of this basic class hierarchy is depicted in [Fig. 9.1](#).

The following [Section 9.1.1](#) presents the general concepts of the base classes `WIR_BaseProcessor` and `WIR_Processor` in more detail. Hereafter, [Sections 9.1.5](#) to [9.1.3](#) provide a short overview about the actual models for the MIPS, TriCore, RISC-V and ARM architectures.

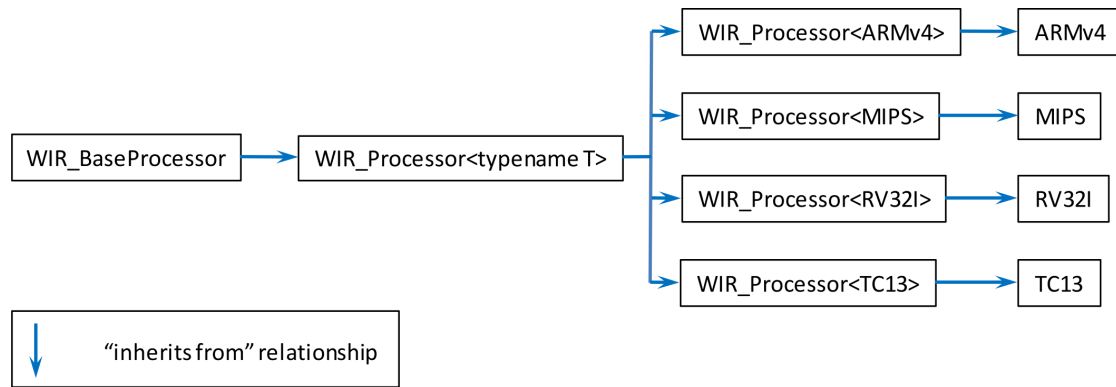


Figure 9.1: Simplified Inheritance Diagram of WIR Processor Models

9.1.1 Generic Processor Model

Class `WIR_BaseProcessor` comes with the usual set of constructors as shown in Lines 2 to 8 of Listing 9.2. Furthermore, processor models are characterized by a few descriptive attributes storing its names, clock frequency and supply voltage. Methods `getProcessorName` (cf. Line 11) provides a string containing a processor model's general name (e.g., "RISC-V"), while the string returned by `getISAName` (cf. Line 14) describes its specific ISA version (e.g., "RV32I V2.0"). A processor's clock frequency can be retrieved using `getClockFrequency`, and `getVoltage` provides its supply voltage. Both values can be relevant for WIR based timing or energy analyses.

Listing 9.2: Constructors and Methods for General Attribute Handling of Class `WIR_BaseProcessor`

```

1 // Default constructor creating an empty processor.
2 WIR_BaseProcessor::WIR_BaseProcessor( void );
3
4 // Copy constructor.
5 WIR_BaseProcessor::WIR_BaseProcessor( const WIR_BaseProcessor &p );
6
7 // Move constructor.
8 WIR_BaseProcessor::WIR_BaseProcessor( WIR_BaseProcessor &&p );
9
10 // Get the processor's descriptive name.
11 std::string WIR_BaseProcessor::getProcessorName( void ) const;
12
13 // Get the ISA's descriptive name.
14 std::string WIR_BaseProcessor::getISAName( void ) const;
15
16 // Get the processor's clock frequency in Hertz.
17 unsigned long long WIR_BaseProcessor::getClockFrequency( void ) const;
18
19 // Get the processor's supply voltage.
20 float WIR_BaseProcessor::getVoltage( void ) const;

```

As motivated earlier, WIR is designed to cope with various processor types simultaneously. For this sole purpose, the template class `WIR_Processor<typename T>` has been introduced. It inherits from class `WIR_BaseProcessor`, features exactly the same constructors, and extends this base class by type handling mechanisms which allow to distinguish, e.g., an ARM processor within WIR from a RISC-V one. In order to do so, the type names of the derived, actual processors are made explicit using the class' template argument `T` (cf. Fig. 9.1). Then, the methods `getProcessorTypeID` or `getProcessorType` (cf. Listing 9.3) can be used to obtain the unique, numerical identifier of the processor model. Both methods behave absolutely identically, they only differ in the

Listing 9.3: Methods for Type Handling of Class WIR_Processor

```

1 // Get the unique ID of a processor type.
2 static WIR_id_t WIR_Processor<T>::getProcessorTypeID( void );
3
4 // Get the unique ID of a processor type.
5 virtual WIR_id_t WIR_Processor<T>::getProcessorType( void ) const;

```

sense that `getProcessorTypeID` is static (cf. Line 2), while `getProcessorType` (cf. Line 5) can only be used for actual instances of processor models.

A WIR processor model consists of the specification of a core's register file(s), its available constant immediates, the supported opcodes and operation formats, addressing modes and condition fields, plus the handling of logical sections (e.g., for code, data or constants) in memory. All these aspects are discussed in detail in the following.

9.1.1.1 Register Types

A processor core features a certain amount of physical hardware registers which might be grouped into several register files and might serve for dedicated purposes. Fig. 1.4 already showed the example of the TC13 model including different types of address, data and extended data registers. Since registers of such different types typically cannot be used interchangeably, the WIR hardware model must be able to distinguish between different register types. This is realized by the generic base class `RegisterType` which is a sub-class of `WIR_BaseProcessor` as shown in Listing 9.4.

Listing 9.4: Generic Register Type Handling of Class WIR_BaseProcessor

```

1 class WIR_BaseProcessor : public WIR_SystemComponent
2 {
3     public:
4
5         [...]
6
7         class RegisterType : public WIR_InheritableEnum
8         {
9             public:
10
11                 // Get a register type's bit width.
12                 unsigned int getBitWidth( void ) const;
13
14                 // Get a register type's prefix strings used to create a register's name.
15                 const std::string ( & getPrefixes( void ) const )[ 2 ];
16
17                 // Get a register type's suffix strings used to create a register's name.
18                 const std::string ( & getSuffixes( void ) const )[ 2 ];
19
20                 [...]
21
22             protected:
23
24                 // Default constructor creating a register type with given prefix and suffix strings and bit width.
25                 RegisterType( const std::string &pp, const std::string &pv,
26                             const std::string &sp, const std::string &sv, unsigned int w );
27 };

```

Listing 9.5: Concrete Register Type Handling (here for Class TC13)

```

1 class TC13 : public WIR_Processor<TC13>
2 {
3     public:
4
5     [...]
6
7     class RegisterType : public WIR_Processor::RegisterType
8     {
9         public:
10
11         // Declaration of address registers.
12         static const RegisterType aReg;
13
14         // Declaration of data registers.
15         static const RegisterType dReg;
16
17         // Declaration of extended data registers.
18         static const RegisterType eReg;
19
20         [...]
21     };
22
23     [...]
24 };
25
26 // Definition of register types.
27 const TC13::RegisterType TC13::RegisterType::aReg { "a", "a_", "", "", 32 };
28 const TC13::RegisterType TC13::RegisterType::dReg { "d", "d_", "", "", 32 };
29 const TC13::RegisterType TC13::RegisterType::eReg { "e", "e_", "", "", 64 };

```

One crucial aspect of registers of different types are their potentially differing bit widths. For the TC13 example from Fig. 1.4, address and data registers are 32 bits wide each, while extended data registers have a width of 64 bits. The bit width of a certain register type can thus be retrieved using method `getBitWidth` (cf. Line 12).

Regularly, all the registers of a certain type follow a given naming scheme where some characters or strings are used to specify the register type, together with the actual unique number of a register. Coming back to the running TC13 example, the names of TriCore address registers always begin with “a”, the prefix “d” characterizes data registers, and “e” is used for extended data registers. For the proper and ISA-compliant output of WIR assembly code, these naming schemes of register types also need to be supported. For this purpose, method `getPrefixes` (cf. Line 15) returns a reference to an array of two strings. These strings are both prefixes that are put before an actual register’s number. The first string inside the returned array denotes the prefix to be used for physical registers of some register type, while the second string is used as prefix for virtual registers. This way, physical and virtual registers can clearly be recognized in WIR assembly output. Similarly, method `getSuffixes` (cf. Line 18) provides two strings used as suffixes for physical or virtual register names, resp.

This characteristic information about register types is provided during their construction. As can be seen from Line 25 of Listing 9.4, the constructor expects all the strings for prefixes and suffixes of both physical and virtual registers as arguments (e.g., argument `sp` stands for the suffix string of physical registers), as well as a register type’s bit width.

The class `RegisterType` itself inherits from a simple WIR-internal support class named `WIR_InheritableEnum` (cf. Line 7). The sole purpose of this support class is to enable the simple enumeration of all available register types in a WIR processor model, while simultaneously being able to inherit and extend such an enumeration in

derived classes (which is impossible with standard C/C++ enum types). This procedure is illustrated in Listing 9.5, again for the TC13 model as example. As can be seen from Line 7, the TriCore register types are derived from the base class described earlier. Class `TC13::RegisterType` then contains the static enumeration and declaration of actual register types (cf. Lines 12 to 18). These declarations are finally defined and implemented in Lines 27 to 29 with appropriate calls of the inherited constructor of class `WIR_BaseProcessor::RegisterType`. In the given example, physical data registers use prefix “d”, while the names of virtual data registers begin with “d_”.

9.1.1.2 Registers

Any register within **WIR** – irrespective of some actual processor model, or be it a virtual or physical register – is generically modeled by class `WIR_BaseRegister`. This class comes with the constructors depicted in Listing 9.6. As can be seen from Line 2, the construction of a register always requires a reference to its register type as introduced in Section 9.1.1.1.

Listing 9.6: Constructors of Class `WIR_BaseRegister`

```

1 // Default constructor creating a register of a given type.
2 explicit WIR_BaseRegister( const WIR_BaseProcessor::RegisterType &t );
3
4 // Copy constructor.
5 WIR_BaseRegister( const WIR_BaseRegister &r );
6
7 // Move constructor.
8 WIR_BaseRegister( WIR_BaseRegister &&r );
```

The attributes of the given register type are then used by the methods from Listing 9.7. `getType` (cf. Line 2) provides a reference to a register’s type, while `getBitWidth` (Line 5) and `getName` (Line 8) return the bit width and name as discussed earlier.

Listing 9.7: Type and Name Handling of Class `WIR_BaseRegister`

```

1 // Get a register's actual type.
2 WIR_BaseProcessor::RegisterType &WIR_BaseRegister::getType( void );
3
4 // Get a register's bit width.
5 unsigned int WIR_BaseRegister::getBitWidth( void );
6
7 // Get a register's name.
8 std::string WIR_BaseRegister::getName( void );
```

Many processor architectures feature so-called hierarchical registers. Here, two (or more) registers of a type with n bits each are grouped together so that they form a large register of, say, $2 * n$ bits. The two basic registers thus become part of a hierarchy where the larger parent register spans over its child registers. In principle, several of these parent registers could also be grouped together to form an even larger hierarchy of registers, even though this is not common in real architectures. Anyways, **WIR** registers support the modelling of arbitrary such hierarchies of registers.

Listing 9.8 shows the basic methods for the handling of such register hierarchies provided by class `WIR_BaseRegister`. `isChild` (cf. Line 2) returns whether a register is child of some register at all, while `isChildOf` (Line 5) can be used to check the child relation between two specific registers. Methods `hasChilds` (Line 8) and `containsChild` (Line 11) can be used to obtain the reverse information, i.e., whether a register contains

Listing 9.8: Register Hierarchy Handling of Class WIR_BaseRegister

```

1 // Determine whether a register is child of some register, i.e., if it is part of a register hierarchy.
2 bool WIR_BaseRegister::isChild( void ) const;
3
4 // Determine whether this register is a child of a given register in a register hierarchy.
5 bool WIR_BaseRegister::isChildOf( const WIR_BaseRegister &r ) const;
6
7 // Determine whether a register has child registers.
8 virtual bool WIR_BaseRegister::hasChilds( void ) const;
9
10 // Determine whether a register contains a child register with the given ID.
11 virtual bool WIR_BaseRegister::containsChild( WIR_id_t id ) const;
12
13 // Determine whether a register is part of a register hierarchy, i.e., if it has some child or is a child.
14 bool WIR_BaseRegister::isHierarchical( void ) const;
15
16 // Get all leafs of the hierarchy underneath this register.
17 const std::vector<std::reference_wrapper<WIR_BaseRegister>> WIR_BaseRegister::getLeafs( void ) const;

```

child registers at all, or whether one register contains a specific other register given by its unique numerical ID as child. Furthermore, `isHierarchical` (Line 14) returns whether a register is part of some hierarchy, i.e., if it is a child register or if it has child. Finally, `getLeafs` (Line 17) can be used to inspect all leaf registers underneath a given register. It returns a vector with references to all lower-level registers in the hierarchy that do not have child registers in turn.

As already introduced earlier, **WIR** generally differentiates between physical and virtual registers. For this purpose, class `WIR_BaseRegister` furthermore provides methods to distinguish between physical and virtual registers (cf. Listing 9.9). As their names suggest, method `isPhysical` (Line 2) returns true only for physical registers, while the opposite holds for `isVirtual` (Line 5).

Listing 9.9: Distinction Between Physical and Virtual Registers in Class WIR_BaseRegister

```

1 // Determine whether a register is physical.
2 bool WIR_BaseRegister::isPhysical( void ) const;
3
4 // Determine whether a register is virtual.
5 virtual bool WIR_BaseRegister::isVirtual( void ) const;

```

In addition, **WIR** models physical and virtual registers as specialized classes `WIR_PhysicalRegister` and `WIR_VirtualRegister`, which both inherit from `WIR_BaseRegister`. Everything that has been discussed about registers up to this point holds for both of these derived classes. In their specific details, however, both physical and virtual registers slightly differ from each other.

The characteristic features of physical registers are represented by the methods shown in Listing 9.10. The most relevant aspect of physical registers is that they are tightly connected to some processor model within **WIR**. Thus, method `getProcessor` (Line 2) can be used to retrieve a reference to the base processor containing that physical register. One physical register that plays an exceptional role for almost all processor architectures is the stack pointer. In order to quickly determine whether some physical register is an architecture's stack pointer, method `isStackPointer` (Line 5) can be used.

Register hierarchies as introduced earlier in this section can only consist of either physical or virtual registers. Thus, the generic **API** for register hierarchy handling from Listing 9.8 requires some extensions specifically for

Listing 9.10: Characteristic Features of Class WIR_PhysicalRegister

```

1 // Get the processor model to which a physical register belongs.
2 WIR_BaseProcessor &WIR_PhysicalRegister::getProcessor( void ) const;
3
4 // Determine whether a register is a processor's stack pointer.
5 bool WIR_PhysicalRegister::isStackPointer( void ) const;
6
7 // Get the list of all child registers, in little-endian order.
8 const std::list<std::reference_wrapper<WIR_PhysicalRegister>> &WIR_PhysicalRegister::getChilDs( void )
   const;
9
10 // Determine whether a register contains a given physical register as child.
11 bool WIR_PhysicalRegister::containsChild( const WIR_PhysicalRegister &r ) const;
12
13 // Find the child register with the given ID.
14 std::list<std::reference_wrapper<WIR_PhysicalRegister>>::const_iterator WIR_PhysicalRegister::findChild(
   WIR_id_t id ) const;
15
16 // Find the given child register.
17 std::list<std::reference_wrapper<WIR_PhysicalRegister>>::const_iterator WIR_PhysicalRegister::findChild(
   const WIR_PhysicalRegister &r ) const;
18
19 // Get a reference to a register's immediate parent in the hierarchy, or a reference to itself.
20 WIR_PhysicalRegister &WIR_PhysicalRegister::getParent( void ) const;
21
22 // Get a reference to a register's top-level root in the hierarchy, or a reference to itself.
23 WIR_PhysicalRegister &WIR_PhysicalRegister::getRoot( void ) const;

```

physical registers. Method `getChilDs` (cf. Line 8 in Listing 9.10) returns the list of references to all physical child registers that a physical register has. This list comes in little-endian order, i.e., the least-significant child register is the first element in the returned list. The `containsChild` variant from Line 11 can be used to check whether a given physical register is child of another one. In analogy to the WIR list API described in Section 6.4, the two `find` methods from Lines 14 and 17 can be used to find some physical register in the list of child registers of some other physical register. In order to navigate upwards within a hierarchy of physical registers, class `WIR_PhysicalRegister` provides the methods `getParent` (Line 20) and `getRoot` (Line 23). The former returns a reference to a register's immediate parent, i.e., to that physical register that contains this register as direct child. If `getParent` is used for a register that has no such parent, it returns a reference to the very same register. The latter provides a reference to the overall root of a register hierarchy, i.e., to that physical register that has no more parent.

Section 8.2 has already discussed the connection between virtual registers and WIR functions, as well as the concept of precoloring virtual with physical registers. Thus, virtual registers are conceptually owned by functions, and not by processor models, unlike physical registers. Listing 9.11 finally shows the characteristic features of virtual registers, and method `getFunction` (Line 2) can be used to retrieve the function to which a virtual register belongs. Methods `isPrecolored` (Line 5) and `getPreColor` (Line 8) help to check if a virtual register is precolored and, if so, to obtain the physical register serving as precolor of a virtual register. If a virtual register is not precolored, `getPreColor` fails with a runtime error. The methods to handle hierarchies of virtual registers from Lines 11 to 26 are equivalent to those of class `WIR_PhysicalRegister`.

After having presented the generic concepts of both physical and virtual WIR registers up to this point, only the connection between these generic base classes – `WIR_PhysicalRegister` and `WIR_VirtualRegister` – and actual WIR processor models remains to be discussed. For each register type of a WIR processor model, dedicated classes exist that model both physical and virtual registers of that processor-specific register type.

Listing 9.11: Characteristic Features of Class WIR_VirtualRegister

```

1 // Get the function to which a virtual register belongs.
2 WIR_Function &WIR_VirtualRegister::getFunction( void ) const;
3
4 // Determine whether a virtual register is precolored.
5 bool WIR_VirtualRegister::isPrecolored( void ) const;
6
7 // Get the physical register associated with a virtual register.
8 WIR_PhysicalRegister &WIR_VirtualRegister::getPrecolor( void ) const;
9
10 // Get the list of all child registers, in little-endian order.
11 const std::list<std::reference_wrapper<WIR_VirtualRegister>> &WIR_VirtualRegister::getChilds( void ) const;
12
13 // Determine whether a register contains a given virtual register as child.
14 bool WIR_VirtualRegister::containsChild( const WIR_VirtualRegister &r ) const;
15
16 // Find the child register with the given ID.
17 std::list<std::reference_wrapper<WIR_VirtualRegister>>::const_iterator WIR_VirtualRegister::findChild(
18     WIR_id_t id ) const;
19
20 // Find the given child register.
21 std::list<std::reference_wrapper<WIR_VirtualRegister>>::const_iterator WIR_VirtualRegister::findChild(
22     const WIR_VirtualRegister &r ) const;
23
24 // Get a reference to a register's immediate parent in the hierarchy, or a reference to itself.
25 WIR_VirtualRegister &WIR_VirtualRegister::getParent( void ) const;
26
27 // Get a reference to a register's top-level root in the hierarchy, or a reference to itself.
28 WIR_VirtualRegister &WIR_VirtualRegister::getRoot( void ) const;

```

Naturally, these processor-specific register classes are inherited from the base classes `WIR_PhysicalRegister` and `WIR_VirtualRegister`, resp.

We again fall back to the TC13 example already used earlier in this section. Within the TriCore architecture, two 32-bit data registers are tied together and form a 64-bit extended data register. Thus, `WIR` on the one hand comes with the class `TC_DRegV` modelling virtual data registers of the TriCore architecture, cf. Listing 9.12. The specification of such an actual virtual register class is extremely simple and basically boils down to the constructor from Line 6 that requires no arguments at all and simply invokes the constructor of `WIR_VirtualRegister` with the TriCore-specific register type `dReg`.

Listing 9.12: Concrete Modelling of Virtual Data Registers (here for TriCore architecture)

```

1 class TC_DRegV final : public WIR_VirtualRegister
2 {
3     public:
4
5     // Default constructor for virtual TriCore data registers.
6     TC_DRegV( void ) : WIR_VirtualRegister { TC13::RegisterType::dReg }
7     {};
8
9     [...]
10 };

```

On the other hand, the TriCore model requires a class `TC_ERegV` modelling virtual extended data registers, which is conceptually shown in Listing 9.13. Its structure is as simple as before, with the key difference that its constructor

Listing 9.13: Concrete Modelling of Virtual Extended Data Registers (here for TriCore architecture)

```

1 class TC_ERegV final : public WIR_VirtualRegister
2 {
3     public:
4
5         // Default constructor for virtual TriCore extended data registers.
6         TC_ERegV( void ) : WIR_VirtualRegister { TC13::RegisterType::eReg }
7         {
8             // Create two virtual child data registers.
9             pushBackChild( TC_DRegV() );
10            pushBackChild( TC_DRegV() );
11        };
12
13        [...]
14 };

```

on the fly generates two virtual data registers that serve as childs of the extended data register to be created (cf. Lines 9 and 10).

Very similarly, class `TC_DRegP` modelling physical data registers is derived as depicted in Listing 9.14. Its constructor (cf. Line 8) gets a string characterizing the TriCore's register name as argument, as well as a Boolean flag denoting whether this will be the stack pointer register. This information is then simply delegated to class `WIR_PhysicalRegister`, together with the TriCore register type `dReg`. The only difference between processor-specific classes for virtual and physical registers is that the constructor is declared private in the latter case. This has the effect that application code using the `WIR` library is unable to create physical registers itself, while this of course is possible for virtual registers. The rationale behind this design is that only processor models within `WIR` shall be able to create processor-specific physical registers. For this reason, the `TC13` model is declared as friend in Line 5 of Listing 9.14.

Listing 9.14: Concrete Modelling of Physical Data Registers (here for TriCore architecture)

```

1 class TC_DRegP final : public WIR_PhysicalRegister
2 {
3     private:
4
5         friend class TC13;
6
7         // Default constructor for physical TriCore data registers.
8         explicit TC_DRegP( const std::string &n, bool sp = false ) :
9             WIR_PhysicalRegister { TC13::RegisterType::dReg, n, sp }
10        {};
11
12        [...]
13 };

```

Listing 9.15 finally shows the overall connection between a `WIR` processor model and its various classes modelling physical registers. Whenever a new instance of a processor model is constructed, this constructor is responsible to also create all the physical registers that the modeled architecture actually features. Line 6 of Listing 9.15 shows the declaration of the constructor of class `TC13` which simply receives no arguments at all and arranges all aspects of the TriCore model silently under the hood. Thus, exactly this constructor `TC13::TC13` repetitively calls the constructor of class `TC_DRegP` from Line 8 of Listing 9.14 in order to set up all physical data registers `D0`, ..., `D15` of the TriCore architecture. Likewise, `TC13::TC13` constructs the TriCore's physical extended data registers

Listing 9.15: Physical Register Handling (here for Class TC13)

```

1 class TC13 : public WIR_Processor<TC13>
2 {
3     public:
4
5         // Default constructor for TC13 processor architectures.
6         TC13( void );
7
8         [...]
9
10        // Access to physical data register D0.
11        const TC_DRegP &D0( void ) const;
12
13        // Access to physical data register D1.
14        const TC_DRegP &D1( void ) const;
15
16        [...]
17
18        // Access to physical extended data register E0.
19        const TC_ERegP &E0( void ) const;
20
21        // Access to physical extended data register E2.
22        const TC_ERegP &E2( void ) const;
23
24        [...]
25 };

```

E0, ..., E14. It furthermore ensures that the correct pairs of physical data registers are added as child registers so that all register hierarchies are properly represented.

For the convenient access to its various physical registers, a [WIR](#) processor model provides specific methods shown in Lines 11 to 22 of Listing 9.15. Here, TC13::D1 (Line 14) returns a reference to the physical data register D1 that has been created by the constructor of class TC13 as described above. Similarly, a reference to extended data register E0 can be obtained using method TC13::E0 (Line 19). The register hierarchy underneath this register E0 can finally be explored using the methods already presented in Listing 9.10. In particular, method getChilds from Line 8 returns a list consisting of those physical registers D0 and D1 that are also provided by methods TC13::D0 and TC13::D1 (cf. Lines 11 and 14 of Listing 9.15). This way, all physical registers and their hierarchies are properly modeled within [WIR](#) processor models.

Structured access to all physical registers of a processor model is provided by the first variant of getPhRegs of the generic base class WIR_BaseProcessor shown in Line 2 of Listing 9.16. The second variant shown in Line 5 can be used to conveniently obtain all physical registers of a given type.

Listing 9.16: Physical Register Access in Class WIR_BaseProcessor

```

1 // Get a vector containing all physical registers of a processor architecture.
2 const std::vector<std::reference_wrapper<const WIR_PhysicalRegister>> WIR_BaseProcessor::getPhRegs( void )
   const;
3
4 // Get a vector containing all physical registers of a processor architecture of the specified type.
5 const std::vector<std::reference_wrapper<const WIR_PhysicalRegister>> WIR_BaseProcessor::getPhRegs(
   const RegisterType &t ) const;

```

9.1.1.3 Immediates

Fig. 8.3 from Section 8.6.2 (see Page 55) already revealed that processor-specific classes for immediate parameters are derived from the generic base classes `WIR_SignedImmediateParameter` and `WIR_UnsignedImmediateParameter`, resp. This process is as simple as depicted in Listing 9.17 for signed 9-bit immediates of the TriCore processor model. As can be seen from Line 7, an actual value `v` is simply forwarded to the constructor of the base class, together with the hard-coded width of 9 bits. This design enables `WIR` to enforce that actual values `v` are always within the ranges that can be represented with the immediate's bit width.

Listing 9.17: Derivation of Processor-Specific Immediates (here for TriCore architecture)

```

1 class TC_Const9_Signed final : public WIR_SignedImmediateParameter<TC_Const9_Signed>
2 {
3     public:
4
5     // Default constructor for TriCore signed const9 immediates.
6     explicit TC_Const9_Signed( signed long long v ) :
7         WIR_SignedImmediateParameter<TC_Const9_Signed> { v, 9 }
8     {};
9
10    [...]
11 };

```

Furthermore, it allows to add specific checks to an immediate class' constructor constraining the valid value range even further in a processor-specific way. For example, some TriCore operations require 10-bit unsigned immediates that are multiples of 4. Thus, the constructor of class `TC_Const10_Unsigned` asserts that the division of value `v` by 4 produces no remainder.

9.1.1.4 Opcodes

As already discussed earlier in Section 9.1.1.1, register types are modeled using `WIR` classes resembling enumeration types which can be inherited into actual processor models. The very same concept is exploited to represent a processor model's opcodes (cf. Listing 9.18).

As can be seen, generic opcodes are realized by class `OpCode` which again is a sub-class of `WIR_BaseProcessor`. And in analogy to register types, this class `OpCode` is derived from `WIR_InheritableEnum` (Line 7). Objects of this class `WIR_BaseProcessor::OpCode` are the very first mandatory argument when `WIR` operations are created, cf. Listing 8.14 in Section 8.5. An opcode is tightly coupled with a mnemonic string where such strings are used to produce ISA-compliant assembly output of `WIR` code. The mnemonic string associated with an opcode can be retrieved via method `getName` (Line 12).

Section 8.5 furthermore already discussed that operations can be classified into different categories, depending on their opcodes (cf. Listing 8.18). Thus, an operation's opcode in general determines whether an operation is, e.g., a function call or something else. As a consequence, the very same Boolean properties as already presented in Listing 8.18 in the context of `WIR` operations can directly be checked for opcodes, cf. Lines 15 to 48 of Listing 9.18. In rare cases, there could be situations where the classification might not only depend on an opcode itself, but maybe on the specific context where and how an opcode occurs in some `WIR` operation. For this reason, all these methods from Listing 9.18 take a reference to an operation as argument so that the particular operation could be inspected by these classification methods. In the methods' default implementation within class `WIR_BaseProcessor::OpCode` as shown in Listing 9.18, the operations `o` provided as arguments are simply ignored. But since all these methods are virtual methods, they can be overloaded in inherited processor models so that a precise classification of some opcode in the context of its operation can be realized.

Listing 9.18: Generic Opcode Handling of Class WIR_BaseProcessor

```

1  class WIR_BaseProcessor : public WIR_SystemComponent
2  {
3      public:
4
5          [...]
6
7      class OpCode : public WIR_InheritableEnum
8      {
9          public:
10
11             // Get an opcode's specific mnemonic.
12             std::string getName( void ) const;
13
14             // Determine whether an opcode reads from memory.
15             virtual bool isMemoryLoad( const WIR_Operation &o ) const;
16
17             // Determine whether an opcode writes to memory.
18             virtual bool isMemoryStore( const WIR_Operation &o ) const;
19
20             // Determine whether an opcode implicitly accesses memory, without being an explicit load or store.
21             virtual bool isImplicitMemoryAccess( const WIR_Operation &o ) const;
22
23             // Determine whether an opcode is a register-register move.
24             virtual bool isMove( const WIR_Operation &o ) const;
25
26             // Determine whether an opcode calls a function.
27             virtual bool isCall( const WIR_Operation &o ) const;
28
29             // Determine whether an opcode indirectly calls a function.
30             virtual bool isIndirectCall( const WIR_Operation &o ) const;
31
32             // Determine whether an opcode returns from a function call.
33             virtual bool isReturn( const WIR_Operation &o ) const;
34
35             // Determine whether an opcode performs a conditional jump.
36             virtual bool isConditionalJump( const WIR_Operation &o ) const;
37
38             // Determine whether an opcode performs an unconditional jump.
39             virtual bool isUnconditionalJump( const WIR_Operation &o ) const;
40
41             // Determine whether an opcode indirectly performs a jump.
42             virtual bool isIndirectJump( const WIR_Operation &o ) const;
43
44             // Determine whether an opcode is an assembly data directive.
45             virtual bool isAsmDataDirective( const WIR_Operation &o ) const;
46
47             // Determine whether an opcode has side effects.
48             virtual bool hasSideEffects( const WIR_Operation &o ) const;
49
50             [...]
51
52         protected:
53
54             // Default constructor creating an opcode with given mnemonic string.
55             explicit OpCode( const std::string &s );
56
57             // Variadic constructor creating an opcode with given mnemonic string and properties.
58             template<typename... Args> OpCode( const std::string &s, WIR_OpCodeType t, Args&&... __args );
59     };
60 };

```

Listing 9.19: Classification of Opcodes by Type WIR_OpCodeType

```

1 // Enum WIR_OpCodeType represents different types of WIR opcodes.
2 enum class WIR_OpCodeType : char
3 {
4     load,                // An opcode that reads from memory.
5     store,               // An opcode that writes to memory.
6     implicitMemoryAccess, // An opcode that implicitly accesses memory.
7     move,                // An opcode that is a register-register move.
8     call,                // An opcode that calls a function.
9     indirectCall,        // An opcode that indirectly calls a function.
10    ret,                  // An opcode that returns from a function call.
11    condJump,             // An opcode that conditionally branches.
12    uncondJump,           // An opcode that unconditionally branches.
13    indirectJump,         // An opcode that indirectly branches.
14    asmDirective,         // An opcode standing for an assembly data directive.
15    sideEffect             // An opcode that has side effects.
16 };

```

All these characteristics of opcodes, their mnemonics and their classification, are set during construction. Lines 55 and 58 show the declaration of the constructors of class `OpCode` which both expect the mnemonic string as mandatory argument. The constructor from Line 58 furthermore accepts a variadic number of opcode types so that all necessary properties of opcodes can be specified conveniently. The enumeration type `WIR_OpCodeType` is defined in Listing 9.19.

The concretization of opcodes for some real processor architecture is as easy as already introduced earlier in the context of register types. An excerpt of the opcode modelling for the running TC13 example is shown in Listing 9.20. All opcodes simply need to be enumerated in the class declaration (cf. Lines 12 to 18). The opcodes' definitions in Lines 27 to 29 then construct all the required objects and associate all attributes to them.

This way, rather simple opcodes that do not belong to any specialized category captured in Listing 9.19 can be specified in a very lightweight manner. See, e.g., the `ABS` and `ADD` opcodes which only require their respective mnemonic strings in Lines 27 and 28, resp. Using opcode types, the TC13 opcode `CALL` is classified as both a function call and a memory access (cf. Line 29).

All the mechanisms described in this section allow the very simple but yet flexible and powerful specification and classification of opcodes for real-life processor architectures. This flexibility is demonstrated in the following example that – in contrast to the running example that bases on the TriCore architecture – now refers to RISC-V.

Example

The RISC-V RV32 ISA [24] does not feature a dedicated opcode for register-register moves. However, recognizing such operations that copy a value from one register to another is of particular interest during register allocation (cf. Section 14.8). Such moves are realized in RISC-V assembly code using operations of the style `addi tgtReg, srcReg, 0` where the constant 0 is added to a given source register, and the result of this (dummy) addition is put in the target register.

In order to identify such variants of `addi` as moves, it would be incorrect to simply classify the opcode `RV32I::OpCode::ADDI` as move during its construction, since this would hold for all kinds of `addi`'s, even for those where non-zero constants are added.

Instead, an overloaded method

```
virtual bool RV32I::OpCode::isMove( const WIR_Operation &o )
```

should be provided which returns true only if the mnemonic of `o` is `RV32I::OpCode::ADDI` and if its

Listing 9.20: Concrete Opcode Handling (here for Class TC13)

```

1  class TC13 : public WIR_Processor<TC13>
2  {
3      public:
4
5          [...]
6
7      class OpCode : public WIR_Processor::OpCode
8      {
9          public:
10
11             // Declaration of Absolute Value opcode.
12             static const OpCode ABS;
13
14             // Declaration of Addition opcode.
15             static const OpCode ADD;
16
17             // Declaration of Function Call opcode.
18             static const OpCode CALL;
19
20             [...]
21         };
22
23         [...]
24     };
25
26     // Definition of opcodes.
27     const TC13::OpCode TC13::OpCode::ABS { "abs" };
28     const TC13::OpCode TC13::OpCode::ADD { "add" };
29     const TC13::OpCode TC13::OpCode::CALL { "call",
30                                             WIR_OpCodeType::call, WIR_OpCodeType::implicitMemoryAccess };
31     [...]

```

immediate value is 0.

Alternatively, a pseudo-opcode that explicitly models a register-register move could be added to the processor model. A pseudo-opcode is an opcode that is not directly part of the [ISA](#) specification, but that is added to the processor model for convenience reasons:

```
const RV32I::OpCode RV32I::OpCode::MOV { "addi", WIR_OpCodeType::move };
```

This MOV pseudo-opcode is clearly classified as move, and it uses the same mnemonic string “addi” as the regular RISC-V opcode `RV32I::OpCode::ADDI` does. If application code ensures that each operation using this MOV opcode always has a target register, a source register, and the immediate constant 0 as parameters, then the [WIR](#) library will always produce [ISA](#)-compliant RISC-V assembly code and will properly recognize such moves.

9.1.1.5 Operation Formats

As discussed in Section [8.5](#), operation formats are – besides opcodes – the second critical attribute that is required during the creation of [WIR](#) operations in order to ensure [ISA](#)-safeness. Operation formats specify the number and the types of an operation’s explicit parameters, as well as their order. For the enumeration of a processor model’s operation format, exactly the same mechanisms as already introduced for opcodes are used (cf. Listing [9.21](#)). Here, sub-class `WIR_BaseProcessor::OperationFormat` represents such operation formats. Their most rel-

Listing 9.21: Generic Operation Format Handling of Class WIR_BaseProcessor

```

1  class WIR_BaseProcessor : public WIR_SystemComponent
2  {
3      public:
4
5          [...]
6
7      class OperationFormat : public WIR_InheritableEnum
8      {
9          public:
10
11              // Get an operation format's bit width.
12              unsigned int getBitWidth( void ) const;
13
14              // Get an operation format's byte size.
15              unsigned int getSize( void ) const;
16
17              // Get an operation format's specific name.
18              std::string getName( void ) const;
19
20              [...]
21
22          protected:
23
24              // Default constructor creating an operation format with given bit width and name.
25              OperationFormat( unsigned int w, const std::string &n );
26      };
27 };

```

evant attribute is their bit width, e.g., whether an operation of a specific format is 16 or 32 bits wide, and this information can be retrieved using method `getBitWidth` (cf. Line 12). `getSize` (Line 15) uses this bit width, divides it by 8 and return this as a format's byte size. Should a bit width not be a multiple of 8, `getSize` rounds up accordingly. For the purpose of WIR code inspection, the human-readable name of an operation format can be retrieved via `getName` (Line 18). The constructor of class `OperationFormat` takes a format's bit width and its name as argument (Line 25).

Declaration and definition of actual operation formats for some concrete processor model – here again for the TC13 example – follow the same lines as described in earlier sections, cf. Listing 9.22. The naming scheme for the operation formats used in this code example mostly uses capital letters to denote which kind of parameter is used where by an operation format. For the TC13 example, “A” represents any address register parameter, while “D” stands for an arbitrary data register parameter. Thus, operation format “AAA” (cf. Line 12) requires three registers as explicit parameters, all of them need to be address registers. The comment attached to this format furthermore indicates that the first address register is defined while the following ones are supposed to be used. Likewise, format “DDC9_1” (Line 15) expects two data registers as first two parameters. The “C9” here denotes a 9-bit constant immediate as third parameter. Again, the format's comment furthermore reveals that the first data register is assumed to be written while the second one is read. The constant immediate of DDC9_1 is assumed to be signed. Since the TriCore ISA features several operations using two data registers and 9-bit constants, but which slightly differ from each other, operation formats might have a suffix like, e.g., “_1”. In Line 18, the prefix “S” denotes a small operation format, i.e., one that is 16 bits wide instead of 32 bits. Furthermore, this particular operation format requires exactly one physical register as read and written parameter, which has to be the stack pointer. This is indicated by “SP” in Line 18, and “C8” here denotes an unsigned 8-bit constant.

From the definition of these exemplary operation formats in Lines 27 to 29 of Listing 9.22, it becomes clear

Listing 9.22: Concrete Operation Format Handling (here for Class TC13)

```

1  class TC13 : public WIR_Processor<TC13>
2  {
3      public:
4
5          [...]
6
7      class OperationFormat : public WIR_Processor::OperationFormat
8      {
9          public:
10
11             // Declaration of 32-bit operation format aReg (def), aReg (use), aReg (use).
12             static const OperationFormat AAA;
13
14             // Declaration of 32-bit operation format dReg (def), dReg (use), signed const9.
15             static const OperationFormat DDC9_1;
16
17             // Declaration of 16-bit operation format SP (defuse), unsigned const8.
18             static const OperationFormat SSPC8;
19
20             [...]
21         };
22
23         [...]
24     };
25
26     // Definition of operation formats.
27     const TC13::OperationFormat TC13::OperationFormat::AAA { 32, "AAA" };
28     const TC13::OperationFormat TC13::OperationFormat::DDC9_1 { 32, "DDC9_1" };
29     const TC13::OperationFormat TC13::OperationFormat::SSPC8 { 16, "SSPC8" };
30     [...]

```

that their bit widths and their names get properly assigned. What remains to be discussed is how the [WIR](#) library is able to enforce that, e.g., an operation of format DDC9_1 is always guaranteed to have one defined data register parameter first, a used data register next, finally followed by a signed 9-bit immediate. Documenting such properties in a comment is nice, but not sufficient at all.

Information of this kind is registered within [WIR](#) during its initialization. Chapter 5 already mentioned – without going into any details – that the [WIR](#) library must be initialized before any of its data structures can be used in application code. This initialization is shown in Line 9 of Listing 5.3 (see Page 27) where the global function `WIR_Init` is called. Main purpose of `WIR_Init` is to recursively trigger the initialization of all the [WIR](#) processor models. Thus, all processor models come with a static `init` method that gets invoked by `WIR_Init`, below again for the running TC13 example:

```

1  class TC13 : public WIR_Processor<TC13>
2  {
3      public:
4
5          [...]
6
7          // Perform global initialization tasks for the TC13 processor model.
8          static void init( void );
9
10         [...]
11     };

```

Listing 9.23: Operation Format Registration (here for Class TC13)

```

1 void TC13::init( void )
2 {
3     [...]
4
5     // TC13 operation formats.
6     const TC_ARegV *aV = new TC_ARegV;
7     const TC_DRegV *dV = new TC_DRegV;
8     const TC_ARegP *SP = new TC_ARegP( "10" );
9
10    registerOperationFormat(
11        OperationFormat::AAA,
12        { new WIR_RegisterParameter( *aV, WIR_Usage::def ), new WIR_RegisterParameter( *aV, WIR_Usage::use ),
13          new WIR_RegisterParameter( *aV, WIR_Usage::use ) } );
14
15    registerOperationFormat(
16        OperationFormat::DDC9_1,
17        { new WIR_RegisterParameter( *dV, WIR_Usage::def ), new WIR_RegisterParameter( *dV, WIR_Usage::use ),
18          new TC_Const9_Signed( 0 ) } );
19
20    registerOperationFormat(
21        OperationFormat::SSPC8,
22        { new WIR_RegisterParameter( *SP, WIR_Usage::defuse ), new TC_Const8_Unsigned( 0 ) } );
23
24    [...]
25
26    // TC13 opcode to operation format mapping.
27    registerOpCode( OpCode::ADD_A, OperationFormat::AAA );
28    registerOpCode( OpCode::ADD, OperationFormat::DDC9_1 );
29    registerOpCode( OpCode::SUB_A, OperationFormat::SSPC8 );
30
31    [...]
32 };

```

An excerpt of this method `TC13::init` showing the registration of the exact shape of the operation formats from Listing 9.22 is shown in Listing 9.23. In Lines 6 and 7, virtual TriCore address and data registers are created first. Line 8 creates a physical address register “A10” for the purpose of initialization, which is the TriCore’s stack pointer. These registers are afterwards used to specify the exact layout of each operation format. For example, Lines 10 to 13 show the registration of TriCore operation format AAA, which gets associated with three register parameters, each of them referring to the virtual address register from Line 6. Furthermore, these register parameters clearly indicate whether they are defined or used (cf. Section 8.6.1). This registration now allows the WIR library to reject any operation of format AAA that, e.g., has a data register as first parameter, or where an address register at the second position is defined instead of being used, or where only two address registers are provided.

Similarly, lines Lines 15 to 18 show the precise specification of the TriCore operation format DDC9_1. It requires the virtual data register from Line 7. The fact that the TriCore ISA makes use of immediates of exactly 9 bits width which are supposed to be signed is represented by the `TC_Const9_Signed` immediate in Line 18 (see also Listing 9.17 on Page 75). This way, any attempt to create an operation of format DDC9_1 featuring, e.g., a constant that does not fit into the value range of 9 bits in two’s complement format leads to a runtime error.

Using virtual registers during registration has the effect that an operation format will accept any register as parameter that has the very same type as the provided virtual register. However, there might be cases where an operation of a certain format requires one specific physical register as parameter and does not accept the whole class of, e.g., address registers. This can be achieved by registering an operation format with exactly the needed

Listing 9.24: Generic Addressing Mode Handling of Class WIR_BaseProcessor

```

1  class WIR_BaseProcessor : public WIR_SystemComponent
2  {
3      public:
4
5          [...]
6
7      class AddressingMode : public WIR_InheritableEnum
8      {
9          public:
10
11             // Get an addressing mode's specific name.
12             std::string getName( void ) const;
13
14             [...]
15
16         protected:
17
18             // Default constructor creating an addressing mode with given name.
19             explicit AddressingMode( const std::string &n );
20     };
21 };

```

physical register as parameter, as can be seen for SSPC8 in Lines 20 to 22. Here, the TriCore's physical stack pointer register is used as parameter that is both defined and used. This way, WIR permits to create operations of this format only if the provided register parameter either is a physical register that is identical to the stack pointer, or if it is a virtual address register that is precolored with the stack pointer.

Now that the registration of operation formats has been introduced, the last issue that remains to achieve ISA-safeness is the specification of which opcodes can have which operation formats. Similar to the registration of operation formats, this mapping of opcodes to formats is realized during the initialization of the WIR library, again within the processor-specific init methods. This is shown in Lines 27 to 29 of Listing 9.23 for the running TC13 example. As can be seen, the init method simply registers each opcode with all supported formats as specified by an architecture's ISA. This way, any attempt to create an operation with an opcode and some non-registered operation format will be rejected by the WIR library with a runtime error.

9.1.1.6 Addressing Modes

As already motivated in Section 8.6.4, memory load or store operations of some ISAs might require the specification of some addressing mode, i.e., of the way how the machine computes the effective address to be loaded or stored, given the parameters provided by the operation. Once more, the specification of such addressing modes is realized within WIR using the enumeration class `WIR_BaseProcessor::AddressingMode` as shown in Listing 9.24. Besides their human-readable name, addressing modes do not carry any further specific attributes. The sole purpose of this class is to be able to use them explicitly as parameters of an operation using class `WIR_AddressModeParameter` (cf. Section 8.6.4), and to be able to distinguish between them during code generation, analysis, or optimization.

For example, the TC13 processor model features the so-called post-increment and pre-increment addressing modes, where a pointer residing in some address register is automatically incremented by some bytes either before or after a memory access. This is reflected within WIR in the straightforward way as depicted in Listing 9.25, where the two addressing modes `pre` and `post` are declared and defined.

Listing 9.25: Concrete Addressing Mode Handling (here for Class TC13)

```

1  class TC13 : public WIR_Processor<TC13>
2  {
3      public:
4
5          [...]
6
7      class AddressingMode : public WIR_Processor::AddressingMode
8      {
9          public:
10
11              // Declaration of pre-increment addressing.
12              static const AddressingMode pre;
13
14              // Declaration of post-increment addressing.
15              static const AddressingMode post;
16
17              [...]
18          };
19
20          [...]
21      };
22
23      // Definition of addressing modes.
24      const TC13::AddressingMode TC13::AddressingMode::pre { "preincr" };
25      const TC13::AddressingMode TC13::AddressingMode::post { "postincr" };
26      [...]

```

9.1.1.7 Condition Fields

The analogy between condition field parameters (cf. Section 8.6.5) and sub-class `WIR_BaseProcessor::Condition` is exactly of the same nature as described for addressing modes in the previous section. Condition fields in the `WIR` hardware model also only carry their name string as their only attribute. For the sake of brevity, we thus omit any code listings here, because they are identical to Listings 9.24 and 9.25 and would not provide any additional insights.

9.1.1.8 Logical Sections

The `WIR` library assumes that binary code executed by some processor model has a structure resembling that of [Executable and Linkable Format \(ELF\)](#) files [9]. Thus, it is logically structured into individual sections that hold, e.g., program code, data, uninitialized data, or constant read-only data. This section structure is orthogonal to that introduced in the `WIR` software model (cf. Chapter 8), the relationship between the software model and the section structure of the `WIR` hardware model will be explained in detail in Chapter 10.

Such sections are represented by class `WIR_Section`. The construction of `WIR` sections requires a non-empty string holding the section's name, as well as a reference to a physical memory region (cf. Section 9.2) to which the section will be attached, see Lines 2 and 5 of Listing 9.26. As usual, method `getName` (Line 14) can be used to retrieve the section name.

A `WIR` section is characterized by various attributes which again resemble those of `ELF` file sections. First, there is a section's external alignment that determines the shape of a section's start and end addresses in memory. The alignment can be set and retrieved using methods `setAlignment` and `getAlignment`, resp. (cf. Lines 2

Listing 9.26: Constructors and Methods for Name Handling of Class WIR_Section

```

1 // Default constructor creating a named section attached to some physical memory region.
2 WIR_Section::WIR_Section( const std::string &s, const WIR_MemoryRegion &r );
3
4 // Default constructor creating a named section attached to some physical memory region.
5 WIR_Section::WIR_Section( std::string &&s, const WIR_MemoryRegion &r );
6
7 // Copy constructor.
8 WIR_Section::WIR_Section( const WIR_Section &s );
9
10 // Move constructor.
11 WIR_Section::WIR_Section( WIR_Section &&s );
12
13 // Get the section name.
14 std::string WIR_Section::getName( void ) const;

```

and 5 of Listing 9.27). Such alignments use the standard data type `std::size_t`, and they denote how many least-significant bits of the section's start and end address must be 0. E.g., setting a section's alignment to 3 means that the least-significant 3 bits of the start and end addresses are 0. In other words, the start and end addresses must be multiples of 8. A default external alignment of 10 bits is applied by the constructors of class `WIR_Section`.

In a similar fashion, the internal alignment of a section can be specified using methods `setBlock` and `getBlock` (Lines 8 and 11). The internal alignment influences which start and end addresses in memory individual symbols get that are packed into one section. The constructors of class `WIR_Section` apply a default internal alignment of 3 bits.

Next, `getLength` (Line 14) returns the byte size of a section which includes the byte sizes of all symbols grouped into that section, plus maybe required additional padding bytes resulting from the section's internal alignment.

As already mentioned above, sections are attached to physical memory regions during construction. Thus, various sections can be associated with one and the same memory region. This has the consequence that sections are kind of floating around within their respective region so that the start address of one section usually depends on the sizes of preceding sections within their memory region. In certain cases, it might, however, be necessary to fix a section at some explicit memory address within its region. The fact whether such an explicit start address has been provided can be checked using method `isStartSet` (cf. Line 17). Method `getStart` (Line 20) then provides this explicitly set start address, or 0 if no explicit start address has been defined.

To be more precise, the `WIR` hardware model distinguishes between *execution time* and *load time* at this point. Execution time refers to that moment when a processor model actively starts executing some binary code. In contrast, load time takes place before and denotes that moment when a processor's memories are loaded with the contents from the binary executable code. A section's explicit start address as returned by method `getStart` now refers to execution time. For most sections, this start address at execution time is identical to that at load time.

An example where this is not the case are data sections mapped to some [Read-Only Memory \(ROM\)](#). Since data is usually both read and written, such particular sections need to be copied from [ROM](#) to some [Random-Access Memory \(RAM\)](#) prior to execution time. Such sections thus have different start addresses at load and at execution time. For this reason, methods `isAtSet` (Line 23) and `getAt` (Line 26) can be used to determine whether a start address at load time has been specified and which address that is, resp. Referring once more to the [ELF](#) standard, the start address at execution time provided by `getStart` denotes the so-called [Virtual Memory Address \(VMA\)](#), while the address returned by `getAt` stands for the [Loaded Memory Address \(LMA\)](#).

Listing 9.27: Characteristic Attributes of Class WIR_Section

```

1 // Set a section's external alignment, i.e., the alignment of the section's start/end boundaries.
2 void WIR_Section::setAlignment( std::size_t a );
3
4 // Get a section's external alignment, i.e., the alignment of the section's start/end boundaries.
5 std::size_t WIR_Section::getAlignment( void ) const;
6
7 // Set a section's internal alignment, i.e., the alignment of contents within a section.
8 void WIR_Section::setBlock( std::size_t a );
9
10 // Get a section's internal alignment, i.e., the alignment of contents within a section.
11 std::size_t WIR_Section::getBlock( void ) const;
12
13 // Get a section's length in bytes.
14 std::size_t WIR_Section::getLength( void );
15
16 // Determine whether an explicit start address of a section has been set.
17 bool isStartSet( void ) const;
18
19 // Get a section's explicit start address during execution time, i.e., its Virtual Memory Address (VMA).
20 WIR_MemoryAddress WIR_Section::getStart( void ) const;
21
22 // Determine whether the load address of a section has been set previously.
23 bool isAtSet( void ) const;
24
25 // Get a section's load address, i.e., its Loaded Memory Address (LMA).
26 WIR_MemoryAddress WIR_Section::getAt( void ) const;
27
28 // Get the memory region to which a section is attached.
29 WIR_MemoryRegion &WIR_Section::getRegion( void ) const;
30
31 // Get a section's load memory region.
32 WIR_MemoryRegion &WIR_Section::getLoadRegion( void ) const;
33
34 // Determine whether a load memory region has been set previously.
35 WIR_MemoryAddress WIR_Section::isLoadRegionSet( void ) const;

```

Memory addresses are represented by type `WIR_MemoryAddress` within `WIR`, which in turn simply is defined as `WIR_SaturatingInt<std::uint32_t>`. The `WIR`-internal template class `WIR_SaturatingInt` is a useful abstraction for memory addresses in that sense that overflows in address computations shall be avoided. Instead, all computations applied to `WIR` addresses provide the maximum or minimum value of a saturating integer's value range in the case of over- or underflows, resp. Class `WIR_SaturatingInt` basically provides all standard arithmetical, shift, logical and comparison operators with inherent saturation, its implementation can be found in file `analyses/generic/wirsaturatingint.h`.

Finally, method `getRegion` (cf. Line 29) can be used to get the physical memory region to which a section is attached. If a section gets loaded from one position to another during boot time as described above, method `getLoadRegion` (Line 32) can be used to retrieve that memory region from which the section is loaded. However, `getLoadRegion` fails with a runtime error if no such load section has been set before. In order to determine whether a load region was specified or not, method `isLoadRegionSet` (Line 35) should be used.

Sections are finally owned and managed by `WIR` processors. `WIR` mandates and enforces that each processor model always includes at least the following 4 default sections reserved for the indicated contents:

.text: Executable code

Listing 9.28: Methods for Section Handling of Class WIR_BaseProcessor

```

1 // Get the set of all sections of a processor model.
2 const WIR_SectionSet &WIR_BaseProcessor::getSections( void ) const;
3
4 // Get a processor's default ".text" section.
5 WIR_Section &WIR_BaseProcessor::getTextSection( void ) const;
6
7 // Get a processor's default ".data" section.
8 WIR_Section &WIR_BaseProcessor::getDataSection( void ) const;
9
10 // Get a processor's default ".bss" section.
11 WIR_Section &WIR_BaseProcessor::getBssSection( void ) const;
12
13 // Get a processor's default ".rodata" section.
14 WIR_Section &WIR_BaseProcessor::getRODataSection( void ) const;
15
16 // Determine whether a processor contains a section with the given ID.
17 bool WIR_BaseProcessor::containsSection( WIR_id_t id ) const;
18
19 // Determine whether a processor contains a section with the given name.
20 bool WIR_BaseProcessor::containsSection( const std::string &n ) const;
21
22 // Determine whether a processor contains a section with the given VMA.
23 bool WIR_BaseProcessor::containsSectionStart( const WIR_MemoryAddress &a ) const;
24
25 // Determine whether a processor contains a section with the given LMA.
26 bool WIR_BaseProcessor::containsSectionAt( const WIR_MemoryAddress &a ) const;
27
28 // Find the section with the given ID.
29 WIR_SectionSet::const_iterator WIR_BaseProcessor::findSection( WIR_id_t id ) const;
30
31 // Find the section with the given name.
32 WIR_SectionSet::const_iterator WIR_BaseProcessor::findSection( std::string &n ) const;
33
34 // Find the section with the given VMA.
35 WIR_SectionSet::const_iterator WIR_BaseProcessor::findSectionByStart( const WIR_MemoryAddress &a ) const;
36
37 // Find the section with the given LMA.
38 WIR_SectionSet::const_iterator WIR_BaseProcessor::findSectionByAt( const WIR_MemoryAddress &a ) const;

```

.data: Initialized data with read/write access rights

.bss: Uninitialized data with read/write access rights

.rodata: Constant initialized data with read access rights only

In addition to the [API](#) already shown in [Listing 9.2](#) (cf. [Page 66](#)), class `WIR_BaseProcessor` provides the methods depicted in [Listing 9.28](#) for the handling of `WIR` sections. A reference to the set of all sections managed by a processor model is returned by `getSections` (cf. [Line 2](#)), where the return type `WIR_SectionSet` is basically declared as `std::set<std::reference_wrapper<WIR_Section>>` within `WIR` (cf. [Chapter 11](#)). The abovementioned 4 default sections can conveniently be accessed using methods `getTextSection` to `getRODataSection` from [Lines 5](#) to [14](#).

Whether a certain section exists within a processor model can be queried via the various `contains` methods. They allow to check the presence of a section with a given numerical ID ([Line 17](#)), with a given name string ([Line 20](#)), or with given `VMAs` or `LMAs`, resp. ([Lines 23](#) and [26](#)). The equivalent find methods (cf. [Lines 29](#) to [38](#))

locate a section within a processor's section set and provide an iterator to the found position. If the set does not contain the section to be found, the `end()` iterator is returned.

9.1.2 MIPS SPIM Processor Model

Within the [WIR](#) library, the MIPS SPIM processor model just serves as a demonstrator for its various design concepts, and not for actual code generation or optimization. For the sake of simplicity, this processor model thus only focuses on the MIPS Core Instruction Set [21] and the integer arithmetic instructions. Class `MIPS` from file `WIR/arch/generic/mips.h` is the central element of this processor model, and – as depicted in Fig. 9.1 – is inherited from base class `WIR_Processor`.

The modeled MIPS architecture features a total of 32 general-purpose integer registers `$0 – $31`, each of them being 32 bits wide. Thus, this processor model declares only one register type `MIPS::RegisterType::reg`, defined in file `WIR/arch/generic/mipsregistertypes.cc`. The key characteristics of this register type and of the resulting physical and virtual MIPS registers are summarized in Table 9.1. As can be seen, the modeled registers are not hierarchical. The physical registers are represented by class `MIPS_RegP` (file `mipsregphysical.h`), and class `MIPS` provides the methods `r0()` to `r31()` to access them. The physical register returned by `r29()` is marked as the architecture's stack pointer. The model's virtual registers are defined by class `MIPS_RegV` (file `mipsregvirtual.h`), and their names always begin with the prefix `"$r_"` in order to clearly distinguish them from the physical MIPS registers.

Table 9.1: Register Types, Classes and Methods of MIPS Processor Model

Register Type	Bit Width	Has Childs	Physical Registers		Virtual Registers	
			Class	Prefix	Class	Prefix
reg	32		<code>MIPS_RegP</code> <code>r0() – r31()</code>	<code>"\$"</code>	<code>MIPS_RegV</code>	<code>"\$r_"</code>

Code examples how to create virtual MIPS registers and how to check their various properties can be found in the unit tests provided in directory `WIR/arch/generic/tests`, more specifically in test programs `mipsVirtualRegister1.cc` and `genericmips.cc`.

The classes of immediate parameters of the MIPS model are summarized in Table 9.2. Due to the architecture's simplicity, basically only 16-bit immediates are used by MIPS operations in their signed and unsigned variants. The MIPS' shift operations allow to shift a register's contents by up to 31 bits so that they use class `MIPS_Immediate5_Shamt` as immediates specifying such shift amounts.

Table 9.2: Classes for Immediates of MIPS Processor Model

Immediate Type	Bit Width	Class	Value Range	Comment
Signed 16-bit	16	<code>MIPS_Immediate16_Signed</code>	$[-2^{15}, +2^{15} - 1]$	
Unsigned 16-bit	16	<code>MIPS_Immediate16_Unsigned</code>	$[0, +2^{16} - 1]$	
Unsigned 5-bit	5	<code>MIPS_Immediate5_Shamt</code>	$[0, +31]$	Shift amounts for 32-bit architecture

All MIPS unit tests in `WIR/arch/generic/tests/mipsImmediate*` feature concrete code examples how to create and use immediates of the classes shown in Table 9.2. These unit tests also cover scenarios where values

Table 9.3: Opcode Types of MIPS Processor Model

Opcode Type	Opcodes	Comment
load	LB, LBU, LH, LHU, LW	
store	SB, SH, SW	
call	JAL, JALR	
indirectCall	JALR	
ret	JR	
condJump	BEQ, BNE	
uncondJump	J, JR	
indirectJump	JR	

that are beyond an immediate's valid value range are used. These tests demonstrate the behavior of the [WIR](#) library in the presence of ill-formed immediates.

Since the MIPS processor model excludes all floating-point operations, the opcodes modeled by class `MIPS::OpCode` consist of the 45 remaining ones according to [21], beginning with `ADD` and ending with `XORI` (cf. file `mipsopcodes.cc`). None of the modeled opcodes is a pseudo-opcode, and their classification into opcode types is shown in Table 9.3. Opcodes not occurring in the table are regular integer arithmetic operations not carrying any specific type within [WIR](#).

Operation formats are defined in file `mipsoperationformats.cc`, and they are all 32 bits wide. The naming scheme used for the MIPS operation formats distinguishes between signed and unsigned immediates, labels, registers, and shift amounts, see also Table 9.4. Using these specifiers, a total of 14 operation formats is defined, ranging from `MIPS::OperationFormat::L` (one used label only) to `MIPS::OperationFormat::RRS` (one defined and one used register, one shift amount).

Table 9.4: Naming Scheme for Operation Formats of MIPS Processor Model

Operation Format Specifier	Parameter Type
"I"	Signed immediate <code>MIPS_Immediate16_Signed</code>
"IU"	Unsigned immediate <code>MIPS_Immediate16_Unsigned</code>
"L"	Label parameter
"R"	Register parameter of type <code>MIPS::RegisterType::reg</code>
"S"	Shift amount

As already described in Section 9.1.1.5, the static method `MIPS::init` serves to register the exact layout of all operation formats and maps all MIPS opcodes to their respective operation formats. Code examples showing the creation and use of all modeled MIPS opcodes in all supported operation formats can be found in unit test `WIR/arch/generic/tests/mipsOperations1.cc`.

9.1.3 TriCore Processor Models

Many of the fundamental concepts of [WIR](#) processor models have been introduced in Section 9.1.1 with the TriCore architecture as running example. We thus continue next with the description of this family of processor



Figure 9.2: Simplified Inheritance Diagram of TriCore Processor Models

models. More precisely, [WIR](#) contains two TriCore models, one for [ISA](#) version 1.3, and one for version 1.3.1 [31]. As shown in Fig. 9.2, the model TC13 is inherited from base class `WIR_Processor`, while TC131 in turn inherits from TC13. This shows how the generic concepts introduced in Section 9.1.1 enable the elegant and simple modelling of entire families of processors that all build upon each other.

As already mentioned in Section 9.1.1.1, the TriCore architecture distinguishes between address registers which are reserved for pointer/address arithmetics and memory accesses on the one hand, and data registers that hold general integer or floating-point data on the other hand. Thus, the TC13 model of [WIR](#) from file `WIR/arch/tri-core/tc13.h` defines the register types `aReg` and `dReg` that are both 32 bits wide (cf. Table 9.5). The classes realizing physical and virtual registers of these types are `TC_ARegP` and `TC_ARegV` (files `tcaregphysical.h` and `tcaregvirtual.h`) and `TC_DRegP` and `TC_DRegV` (files `tcdregphysical.h` and `tcdregvirtual.h`), resp. For the simple access to these physical address and data registers, class TC13 comes with methods `A0()` – `A15()` and `D0()` – `D15()`. Physical address register `A10()` is the model's stack pointer.

Table 9.5: Register Types, Classes and Methods of TC13 Processor Model

Register Type	Bit Width	Has Childs	Physical Registers		Virtual Registers	
			Class	Prefix	Class	Prefix
aReg	32		TC_ARegP A0() – A15()	“a”	TC_ARegV “a_”	
dReg	32		TC_DRegP D0() – D15()	“d”	TC_DRegV “d_”	
eReg	64	✓ dReg	TC_ERegP E0(), E2(), ..., E14()	“e”	TC_ERegV “e_”	
pReg	64	✓ aReg	TC_PRegP P0(), P2(), ..., P14()	“p”	TC_PRegV “p_”	
pswBit	1		TC_PSWBit PSW_C()	“PSW.”		

According to the TriCore [ISA](#), neighboring pairs of such 32-bit address or data registers are hierarchically grouped to form so-called 64-bit extended registers. For example, data registers D4 and D5 form the extended data register E4. These extended registers serve to store 64-bit integer or double-precision floating-point data, or double addresses. For these reasons, the TC13 model introduces the hierarchical register types `eReg` for extended data registers and `pReg` for extended address registers, resp. They all have exactly two child data or address registers. Their implementations come with classes `TC_ERegP` and `TC_ERegV` for extended data registers (files `tceregphysical.h` and `tceregvirtual.h`), and with classes `TC_PRegP` and `TC_PRegV` for extended address registers (files `tcpregphysical.h` and `tcpregvirtual.h`), resp. Again, convenient access methods `E0()` – `E14()` and `P0()` – `P14()` exist. Data flow through such hierarchies of registers is fully automatically considered by the [WIR](#) library. This means that any modification of, e.g., data register D5 automatically also leads to a re-definition of extended register E4. In the other sense of direction, writing to E4 is equivalent to a definition of both D4 and D5.

The last TC13 register type models the Carry bit of the TriCore's [Program Status Word \(PSW\)](#) register. Since

Listing 9.29: Testing of Physical Register Properties of Class TC13

```

1 // Determine whether a register is the TriCore's stack pointer A10.
2 static bool TC13::isSP( const WIR_BaseRegister &r );
3
4 // Determine whether a register is the TriCore's return address register A11.
5 static bool TC13::isRA( const WIR_BaseRegister &r );
6
7 // Determine whether a register is the TriCore's implicit address register A15.
8 static bool TC13::isA15( const WIR_BaseRegister &r );
9
10 // Determine whether a register is the TriCore's implicit data register D15.
11 static bool TC13::isD15( const WIR_BaseRegister &r );
12
13 // Determine whether a register is the TriCore's PSW carry bit.
14 static bool TC13::isPSW_C( const WIR_BaseRegister &r );
15
16 // Determine whether a register is part of the TriCore's lower register context.
17 static bool TC13::isLCReg( const WIR_BaseRegister &r );

```

the TriCore [ISA](#) contains a few operations that implicitly define or use this internal Carry bit, this very special register type has been introduced for the proper modelling of data flow via this status bit. There only exists class `TC_PSWBit` as implementation of the physical [PSW](#) bit, but no virtual register counterpart. Class `TC13` offers the method `PSW_C()` to access this physical register.

Within this structure of the TriCore register file, certain registers play special roles. The stack pointer `A10` has already been mentioned above, and address register `A11` is the TriCore's return address register. Furthermore, registers `A15` and `D15` are the machine's implicit address and data registers, resp. In order to easily determine whether some arbitrary register `r` is one of these special registers, class `TC13` offers the methods shown in Lines 2 to 14 of Listing 9.29. They all return `true` if the given register `r` either is exactly the physical special-purpose TriCore register, or if it is virtual but precolored with the according physical register. Last but not least, the TriCore's register file is structured into an upper and a lower context, where the upper context registers automatically get saved during function calls. Lower context registers¹ require special handling during function calls so that method `TC13::isLCReg` (cf. Line 17) can be used to determine whether some register `r` is from the lower context.

Since TriCore [ISA](#) version 1.3.1 does not add more registers to the architecture, [WIR](#) model `TC131` does not need to declare and implement additional register types and classes. Instead, it simply inherits all the above concepts from `TC13` without any modification or extension.

The unit tests in files `WIR/arch/tricore/tests/tc*Reg*` contain various examples showing how to create and use both physical and virtual registers of the various TriCore register types.

In sharp contrast to the extremely simple [ISA](#) design of the MIPS SPIM architecture, the TriCore designers seemed to enjoy the motto “the more, the better”. The TriCore [ISA](#) is characterized by features stemming from various different domains of external computer architecture, amongst others [DSP](#), [SIMD](#) computing, or network processing. Due to this feature richness of the instruction set, a quite large number of different immediate parameters needed to be modeled within [WIR](#), cf. Table 9.6. Most of them are the straightforward realizations of signed or unsigned immediates of their given bit widths so that we omit their discussion here for the sake of brevity. Some others are, however, rather special and need a few words of explanation.

Most notably, class `TC_Const18_Unsigned` represents absolute addresses in the context of memory access operations. According to the TriCore [ISA](#), such absolute addresses are encoded in machine operations using 18

¹ TriCore Lower Context: `A2`, `A3`, `A4`, `A5`, `A6`, `A7`; `D0`, `D1`, `D2`, `D3`, `D4`, `D5`, `D6`, `D7`; `E0`, `E2`, `E4`, `E6`; `P2`, `P4`, `P6`.

Table 9.6: Classes for immediates of TC13 Processor Model

Immediate Type	Bit Width	Class	Value Range	Comment
Unsigned 18-bit	32	TC_Const18_Unsigned	$[0, +0xF0003FFF]$	Absolute addresses
Signed 16-bit	16	TC_Const16_Signed	$[-2^{15}, +2^{15} - 1]$	
Unsigned 16-bit	16	TC_Const16_Unsigned	$[0, +2^{16} - 1]$	
Signed 10-bit	10	TC_Const10_Signed	$[-512, +511]$	
Unsigned 10-bit	10	TC_Const10_Unsigned	$[0, +1023]$	Stack pointer-relative offsets, must be multiples of 4
Signed 9-bit	9	TC_Const9_Signed	$[-256, +255]$	
Unsigned 9-bit	9	TC_Const9_Unsigned	$[0, +511]$	
Unsigned 8-bit	8	TC_Const8_Unsigned	$[0, +255]$	
Unsigned 5-bit	5	TC_Const5_Unsigned	$[0, +31]$	Word-level bit indices and shift amounts
Signed 4-bit	4	TC_Const4_Signed	$[-8, +7]$	
Unsigned 4-bit	4	TC_Const4_Unsigned	$[0, +15]$	
Unsigned 3-bit	3	TC_Const3_Unsigned	$[0, +7]$	Byte-level bit indices
Unsigned 2-bit	2	TC_Const2_Unsigned	$[0, +3]$	Address factor of 1, 2, 4 or 8, encoded as logarithm of base 2
Unsigned 1-bit	1	TC_Const1_Unsigned	$[0, +1]$	Handling of corner cases by Q-format arithmetic operations

bits. However, the machine's interpretation of these 18 bits differs fundamentally from that of classical unsigned binary numbers, requiring special provisions in class `TC_Const18_Unsigned`. The TriCore architecture expands such 18-bit absolute addresses to full 32-bit addresses by splitting its 4 most-significant bits from the 14 least-significant bits, and putting a series of 14 zero bits in between:

$$addr_{32} := \text{const18}[17 : 14] \& \text{"00 0000 0000 0000"} \& \text{const18}[13 : 0]$$

where the `&` operator stands for the concatenation of bit strings. The value range that can be represented with such 18-bit absolute addresses is thus completely different from that of unsigned numbers of 18 bits width (see Table 9.6). For this reason, class `TC_Const18_Unsigned` internally uses the full width of 32 bits in order to properly represent absolute addresses having the above scheme. Any arbitrary unsigned 32-bit values can be used to construct immediates of type `TC_Const18_Unsigned`, while the class' constructor rejects those values whose inner bits at positions 14 – 27 are non-zero.

`TC_Const10_Unsigned` immediates are used as offsets in special variants of load or store operations exploiting stack pointer-relative addressing. Since the stack pointer has to be aligned to 4 bytes, constant offsets to be added to the stack pointer have to adhere to the same rule. According to the TriCore [ISA](#), these special load and store operations use an unsigned 8-bit immediate that is internally multiplied by 4. The TC13 processor model of [WIR](#) now uses 10-bit unsigned immediates for this purpose, such that the constructor of class `TC_Const10_Unsigned` issues a runtime error whenever actual values are used that are not even multiples of 4.

Class `TC_Const5_Unsigned` comes into play for word-level bit manipulation and shift operations where bit positions or shift amounts between 0 or 31 bits need to be specified. 3-bit unsigned immediates (class `TC_Const3_Unsigned`) are used only by TriCore opcode `ST_T` that stores one single bit somewhere inside a byte. For the specification of the bit's position within its byte, 3 bits are required. Similarly, 2-bit unsigned immediates of type `TC_Const2_Unsigned` are only used by the special-purpose `ADDSC_A` opcode for address computation. Here,

Table 9.7: Opcode Types of TC13 Processor Model

Opcode Type	Opcodes	Comment
load	LD_A, LD_B, LD_BU, LD_D, LD_DA, LD_H, LD_HU, LD_Q, LD_W, LDLCX, LDMST, LDU CX, RSLCX, ST_T, SWAP_W	Standard load operations, context handling, load-modify-store operations
store	LDMST, ST_A, ST_B, ST_D, ST_DA, ST_H, ST_Q, ST_T, ST_W, STLCX, STUCX, SVLCX, SWAP_W	Standard store operations, context handling, load-modify-store operations
implicitMemoryAccess	BISR, CACHEA_I, CACHEA_W, CACHEA_WI, CALL, CALLA, CALLI, RET, RFE, RFM	
move	MOV_RR, MOV_AA	Pseudo-opcode MOV_RR
call	CALL, CALLA, JL, JLA	
indirectCall	CALLI, JLI	
ret	RET, RFE, RFM	Returns, incl. from exceptions or monitors
condJump	JEQ, JEQ_A, JGE, JGE_U, JGEZ, JGTZ, JLEZ, JLT, JLT_U, JLTZ, JNE, JNE_A, JNED, JNEI, JNZ, JNZ_A, JNZ_T, JZ, JZ_A, JZ_T, LOOP	
uncondJump	J, JA, JI, LOOPU	
indirectJump	JI	
sideEffect	BISR, CACHEA_I, CACHEA_W, CACHEA_WI, DEBUG, DISABLE, DSYNC, ENABLE, ISYNC, LDLCX, LDU CX, MTCR, RSLCX, RSTV, SVLCX, SYSCALL, TLBDEMAP, TLBFLUSH_A, TLBFLUSH_B, TLBMAP, TLBPROBE_A, TLBPROBE_I, TRAPSV, TRAPV, UPDFL	Interrupt, cache, Translation Look-Aside Buffer (TLB) handling, debugging, context and core register handling, system call operations

some address is multiplied by 1, 2, 4 or 8 bytes, and this small factor is encoded as its respective logarithm. One bit immediates of class TC_Const1_Unsigned are widely used by the TriCore's operations for fractional arithmetics to handle certain arithmetic corner cases in the Q15 format.

For all abovementioned classes of TriCore immediates, dedicated unit tests exist showing how to use these immediates and demonstrating that invalid values are properly rejected. These unit tests can be found in files WIR/arch/tricore/tests/tcConst*.

The entire TriCore 1.3 instruction set [31] is carefully modeled by WIR class TC13, including all its [Central Processing Unit \(CPU\)](#), [Floating-Point Unit \(FPU\)](#) and [Memory Management Unit \(MMU\)](#) instructions. Due to the sheer complexity of this [ISA](#), the TC13 model features a total of 332 opcodes, beginning with ABS and ending with XOR_T

Table 9.8: Opcode Types of TC131 Processor Model

Opcode Type	Opcodes	Comment
implicitMemoryAccess	CACHEI_W, CACHEI_WI	Data cache index handling
sideEffect	CACHEI_W, CACHEI_WI	

Table 9.9: Naming Scheme for Operation Formats of TC13 Processor Model

Operation Format Specifier	Parameter Type
"A"	Address register parameter of type TC13::RegisterType::aReg
"C1"	Unsigned 1-bit immediate constant
"C2"	Unsigned 2-bit immediate constant
"C3"	Unsigned 3-bit immediate constant
"C4"	Signed or unsigned 4-bit immediate constant
"C5"	Unsigned 5-bit immediate constant
"C8"	Unsigned 8-bit immediate constant
"C9"	Signed or unsigned 9-bit immediate constant
"C10"	Signed or unsigned 10-bit immediate constant
"C16"	Signed or unsigned 16-bit immediate constant
"C18"	Unsigned 18-bit immediate constant
"D"	Data register parameter of type TC13::RegisterType::dReg
"E"	Extended data register parameter of type TC13::RegisterType::eReg
"I"	Physical implicit data or address register parameter A15 or D15
"L"	Label parameter
"P"	Extended address register parameter of type TC13::RegisterType::pReg
"PSW"	PSW register parameter of type TC13::RegisterType::pswBit
"S . . ."	Prefix for short 16-bit operation formats
"SP"	Physical stack pointer register parameter A10
"SYS"	No parameters at all
"ABSA"	Absolute addressing mode
"BOA"	Base + offset addressing mode
"BRA"	Bit-reverse addressing mode
"CA"	Circular addressing mode
"PIA"	Post- or pre-increment addressing mode

(see file `tc13opcodes.cc`). Their classification into opcode types is shown in Table 9.7. It is worth mentioning that there are some operations that are classified as both memory loads and stores. These operations atomically load some data from memory, potentially modify it, and write it back to memory. Various operations perform implicit memory accesses, especially those realizing function calls and returns from functions. This comes from the TriCore's automatic saving and restoring of upper context registers in memory during function calls/returns. Finally, quite many operations cause side effects to the processor's internal state and should thus be taken with care during [WIR](#)-based code analyses and optimizations. One pseudo-opcode named `MOV_RR` has been added to the TC13 processor model in order to clearly characterize register-register moves. This opcode `MOV_RR` uses the TriCore's regular assembly mnemonic "mov". But since the [ISA](#) knows various flavors of this mov operation, and only one of them realizes a register-register-move, a pseudo-opcode was added.

The TriCore 1.3.1 version extends the instruction set by 5 additional machine operations. By exploiting inheritance again (cf. Fig. 9.2), processor model TC131 (file `tc131.h`) simply inherits all abovementioned opcodes from TC13 and extends them by the novel opcodes `CACHEI_W`, `CACHEI_WI`, `FT0IZ`, `FT0Q31Z` and `FT0UZ`. The opcode classification of these TC131 opcodes is summarized in Table 9.8, see also file `tc131opcodes.cc`.

The TriCore operation formats are defined in file `tc13operationformats.cc`. As the TC131 processor model

does not require any additional formats, there is no such file for this architecture. A total of 176 32 bits wide formats are defined, while 54 operation formats are 16 bits wide. According to the used naming scheme (cf. Table 9.9), the prefix “S” indicates such short operation formats. This naming scheme uses different characters or strings to represent the various register parameter types, immediate types, or label parameters. Furthermore, some operation formats require the physical TriCore registers A15, D15 (implicit address or data registers) or the stack pointer A10. The presence of these physical registers is indicated by the specifiers “I” (implicit register) or “SP” (stack pointer). Operations of format “SYS” are 32 bits wide and possess no parameters at all – this is the case for, e.g., the NOP (no operation) opcode. Based on this naming scheme, it is easy to see that an operation of, e.g., format `TC13::OperationFormat::SSPC8` is a short operation of 16 bits width, requiring the physical stack pointer and an unsigned 8-bits immediate as parameters.

In order to see how to construct and use all TriCore opcodes in all supported operation formats, unit tests `WIR/arch/tricore/tests/tc0operations1.cc` and `tc0operations2.cc` can be inspected. The former creates all operations in all flavors for both the TC13 and TC131 ISAs and checks all specific operation properties. The latter also creates all these operations, but afterwards prints them in TriCore-compliant assembly syntax and feeds this printout into the GNU TriCore assembler. This way, unit test `tc0operations2.cc` ensures that `WIR` definitely produces valid TriCore assembly output.

The last 5 rows of Table 9.9 already draw the bow towards the modelling of the TriCore's various addressing modes within `WIR`. TriCore supports absolute, base + offset, bit-reverse, circular, and post- or pre-increment addressing. In principle, all these addressing modes could be modeled explicitly by inheriting from class `WIR_BaseProcessor::AddressingMode` and by exploiting the concepts sketched in Section 9.1.1.6. When, however, having a closer look at the ISA design, it turns out that the structure of these addressing modes is so different that most of them require their very own, special combinations of parameters.

Example

Absolute addressing makes use of 18-bit immediates of type `TC_Const18_Unsigned`, cf. Table 9.6. In contrast, base + offset addressing needs one address register parameter for the base address, plus either a 10-bit or a 16-bit immediate. Bit-reverse addressing bases on the provision of an extended address register. Circular addressing also requires an extended address register, but furthermore needs a 10-bit immediate parameter. Finally, post- or pre-increment addressing needs an address register parameter and a 10-bit immediate.

These examples show that the modelling of each addressing mode inherently leads to dedicated, specialized operation formats that have to be modeled, too. For the sake of simplicity, and to avoid modelling the same aspect twice within `WIR`, only the post- and pre-increment addressing modes are modeled using class `AddressingMode`. In contrast, the special structure of all other addressing modes is made explicit already in the underlying operation formats by introducing the specifiers “ABSA” to “PIA” from the bottom of Table 9.9. The TC13 modes for post- and pre-increment addressing have already been introduced earlier, see Listing 9.25 on Page 83.

As can be seen from the following example, only the structure of the two post- and pre-increment addressing modes is identical, justifying their modelling using class `WIR_BaseProcessor::AddressingMode`. The structures of all other addressing modes all differ from each other so that the remaining addressing modes are properly and uniquely specified by their underlying operation formats.

Example

The `LD_W` operation loading a word from memory to a data register supports all the abovementioned addressing modes. Assuming that `b` is a `WIR_BasicBlock` and that `a`, `d` and `p` are address, data and extended address registers, resp., the following lines of code add `LD_W` operations with the various ad-

Listing 9.30: Testing of Pipeline Properties of Class TC13

```

1 // Determine whether an operation gets executed on the TriCore Integer Pipeline.
2 static bool TC13::isIP( const WIR_Operation &o );
3
4 // Determine whether an operation gets executed on the TriCore Loop Pipeline.
5 static bool TC13::isLP( const WIR_Operation &o );
6
7 // Determine whether an operation gets executed on the TriCore Load/Store Pipeline.
8 static bool TC13::isLS( const WIR_Operation &o );

```

dressing modes:

```

b.pushBackInstruction( // Absolute Addressing
    { { TC13::OpCode::LD_W, TC13::OperationFormat::DC18ABSA,
        WIR_RegisterParameter( d, WIR_Usage::def ), TC_Const18_Unsigned( 16383 ) } } );
b.pushBackInstruction( // Base + Long Offset Addressing
    { { TC13::OpCode::LD_W, TC13::OperationFormat::DAC16BOA,
        WIR_RegisterParameter( d, WIR_Usage::def ),
        WIR_RegisterParameter( a, WIR_Usage::use ), TC_Const16_Signed( -32768 ) } } );
b.pushBackInstruction( // Bit-Reverse Addressing
    { { TC13::OpCode::LD_W, TC13::OperationFormat::DPBRA,
        WIR_RegisterParameter( d, WIR_Usage::def ),
        WIR_RegisterParameter( p, WIR_Usage::defuse ) } } );
b.pushBackInstruction( // Circular Addressing
    { { TC13::OpCode::LD_W, TC13::OperationFormat::DPC10CA,
        WIR_RegisterParameter( d, WIR_Usage::def ),
        WIR_RegisterParameter( p, WIR_Usage::defuse ), TC_Const10_Signed( -512 ) } } );
b.pushBackInstruction( // Post-Increment Addressing
    { { TC13::OpCode::LD_W, TC13::OperationFormat::DAC10PIA,
        WIR_RegisterParameter( d, WIR_Usage::def ),
        WIR_AddressModeParameter( TC13::AddressingMode::post ),
        WIR_RegisterParameter( a, WIR_Usage::defuse ), TC_Const10_Signed( -512 ) } } );
b.pushBackInstruction( // Pre-Increment Addressing
    { { TC13::OpCode::LD_W, TC13::OperationFormat::DAC10PIA,
        WIR_RegisterParameter( d, WIR_Usage::def ),
        WIR_AddressModeParameter( TC13::AddressingMode::pre ),
        WIR_RegisterParameter( a, WIR_Usage::defuse ), TC_Const10_Signed( -512 ) } } );

```

WIR provides the unit test `WIR/arch/tricore/tests/tcAddressingModeParameter1.cc` that illustrates how to use and work with these TriCore-specific addressing modes.

The TriCore architecture has three different internal pipelines – the standard integer pipeline, a load/store pipeline reserved for memory accesses and address arithmetics, and a loop pipeline realizing zero-overhead loops. Under certain conditions [28], these pipelines can execute in parallel and the TriCore is able to issue two operations in parallel. In order to be able to exploit this parallelism, a compiler needs to know on which of these pipelines an operation gets executed. This is supported by the TC13 processor model using the methods shown in Listing 9.30. They return `true` if a given operation is executed on the integer, load/store, or loop pipeline, resp.

9.1.4 RISC-V RV32 Processor Models

Key feature of the RISC-V RV32 [ISA](#) [24] is the simplicity of the instruction set on the one hand, that resembles that of the MIPS model from Section 9.1.2. On the other hand, the RISC-V operations are organized in a highly modular fashion, allowing to include or not to include a couple of optional standard [ISA](#) extensions. This design has the effect that each and every RV32 implementation must include the mandatory base integer instruction set RV32I. On top of this base [ISA](#), the extensions listed in Table 9.10 can be added. Concrete RISC-V RV32 implementations are thus characterized by the prefix letters shown in Table 9.10 of the base [ISA](#) plus all included extensions.

Table 9.10: Standard Extensions of the RISC-V RV32 Instruction Set

Prefix	Extension
I	Mandatory Base Integer Instruction Set
M	Integer Multiplication and Division
A	Atomic Operations
F	Single-Precision Floating-Point
D	Double-Precision Floating-Point
C	Compressed Operations

Within the [WIR](#) library, the base instruction set RV32I is modeled, as well as the standard extensions C, F, D, and M. The extensibility of the [ISA](#) is perfectly supported by the high modularity of the general [WIR](#) hardware model. This allows to provide a base class RV32I that models the base integer instruction set. Using inheritance, standard extensions like, e.g., C or M are modeled leading to derived classes such as RV32IC or RV32IM. Multiple inheritance is then used to simply derive all possible combinations of the base [ISA](#) and potential standard extensions. This is illustrated in Fig. 9.3 showing the complete inheritance relationships between RISC-V related classes within [WIR](#).

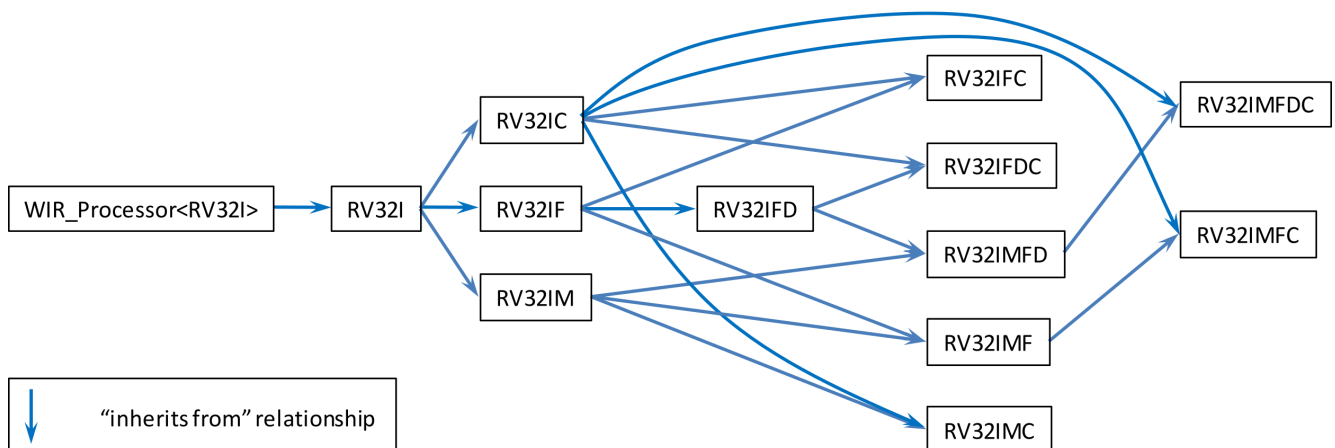


Figure 9.3: Simplified Inheritance Diagram of RISC-V RV32 Processor Models

The RV32I base integer architecture defines a total of 32 general-purpose integer registers $x0 - x31$ that all have a width of 32 bits. Thus, the RV32I processor model declares one register type `RV32I::RegisterType::reg` which is defined in file `WIR/arch/riscv/rv32iregistertypes.cc`. If the [ISA](#) is extended by some floating-point operations, i.e., by extensions F, D or Q from Table 9.10, another set of 32 dedicated floating-point registers $f0 - f31$ is added to the architecture. Thus, [WIR](#) class RV32IF introduces the second register type `fReg` (cf. file

Table 9.11: Register Types, Classes and Methods of RV32I, RV32IF and RV32IFD Processor Models

Register Type	Bit Width	Has Childs	Physical Registers		Virtual Registers	
			Class	Prefix	Class	Prefix
reg	32		RV_RegP x0() – x31()	“x”	RV_RegV	“x_”
fReg	32 or 64		RV_FRegP f0() – f31()	“f”	RV_FRegV	“f_”

rv32ifregistertypes.cc) with a default width of 32 bits. The RV32IFD model simply inherits these registers from its respective base class RV32IF and adjusts the width of the floating-point registers to 64 bits.

The key characteristics of these registers and their types are again summarized in Table 9.11. As can be seen, none of these register types are hierarchical. **TODO: Correct this!** The classes implementing physical and virtual registers of these types are RV_RegP and RV_RegV (files rvregphysical.h and rvregvirtual.h) and RV_FRegP and RV_FRegV (files rvfregphysical.h and rvfregvirtual.h), resp. As usual, the processor models RV32I and RV32IF provide methods x0() – x31() and f0() – f31() for the convenient access to the models' physical registers.

TODO

Listing 9.31: Testing of Physical Register Properties of Class RV32I

```

1 // Determine whether a register is the RISC-V's null register x0.
2 static bool RV32I::isX0( const WIR_BaseRegister &r );
3
4 // Determine whether a register is the RISC-V's link register x1.
5 static bool RV32I::isX1( const WIR_BaseRegister &r );
6
7 // Determine whether a register is the RISC-V's stack pointer x2.
8 static bool RV32I::isSP( const WIR_BaseRegister &r );
9
10 // Determine whether a register is the RISC-V's frame pointer x8.
11 static bool RV32I::isFP( const WIR_BaseRegister &r );

```

Physical register x0 is the null register that is always hardwired to the constant 0. Register x1 is the link register of the RV32 architecture, x2 the stack pointer, and register x8 serves as frame pointer. In order to check whether a given register r is one of these particular RISC-V registers, class RV32I contains the methods shown in Listing 9.31. They return true if r is exactly the physical register to be identified, or if r is virtual and precolored accordingly.

As was the case for the previous WIR processor models, also the RISC-V model comes with dedicated unit tests showing how to use the physical and virtual registers shown in Table 9.11. These tests can be found in files WIR/arch/riscv/tests/rv*Reg*.

The simplicity of the RISC-V's ISA is reflected by the few different types of immediate parameters that had to be modeled within WIR, see Table 9.12. For the base class RV32I, only three different kinds of immediates are required. 20-bit immediates of type RV_Const20_Unsigned are used to set the most-significant 20 bits of some register. For the remaining least-significant bits, constants with a width of 12 bits are sufficient which are modeled by class RV_Const12_Signed. With only these two kinds of immediates, any 32-bit value can be loaded into a register and all arithmetic operations of the RV32I ISA exploiting immediates can be expressed. As was the case already before for the MIPS and the TriCore ISAs, specialized 5-bit immediates are required to encode shift amounts.

Table 9.12: Classes for immediates of RV32I and RV32IC Processor Models

Immediate Type	Bit Width	Class	Value Range	Comment
Unsigned 20-bit	20	RV_Const20_Unsigned	$[0, +2^{20} - 1]$	
Signed 12-bit	12	RV_Const12_Signed	$[-2^{11}, +2^{11} - 1]$	
Unsigned 5-bit	5	RV_Const5_Unsigned	$[0, +31]$	Shift amounts for 32-bit architecture
Signed 10-bit	10	RV_Const10_Signed	$[-512, +496]$	Non-zero stack pointer adjustment, must be multiples of 16
Unsigned 10-bit	10	RV_Const10_Unsigned	$[4, +1020]$	Non-zero stack pointer-relative offsets, must be multiples of 4
Unsigned 8-bit	8	RV_Const8_Unsigned	$[0, +255]$	
Unsigned 7-bit	7	RV_Const7_Unsigned	$[-64, +60]$	Pointer-relative offsets, must be multiples of 4
Signed 6-bit	6	RV_Const6_Signed	$[-32, +31]$	
Unsigned 6-bit	6	RV_Const6_Unsigned	$[0, +63]$	

Due to the fact that the C ISA extension defines compressed operations of only 16 bits width, the RV32IC model requires additional immediate types with shorter bit widths. These immediate types are summarized in the lower part of Table 9.12. The 10-bit immediates dedicated to stack pointer handling have to be non-zero and must be multiples of either 16 or 4. The 7-bit unsigned immediates for compact memory accesses have to be multiples of 4. For all other immediate types, there exist no special provisions to constrain these value ranges any further.

The unit tests from files WIR/arch/riscv/tests/rvConst* provide code examples for all modeled RISC-V immediate classes.

Out of the overall 47 operations defined by the base integer instruction set, all of them are modeled within class RV32I except the FENCE and FENCEI operations (see file rv32iopcodes.cc). Their classification into opcode types is shown in Table 9.13. The following three pseudo-opcodes have been added for convenience reasons and should consequently be used for moves, returns and unconditional jumps:

Table 9.13: Opcode Types of RV32I Processor Model

Opcode Type	Opcodes	Comment
load	LB, LBU, LH, LHU, LW	
store	SB, SH, SW	
move	MOV	Pseudo-opcode
call	JAL	
ret	RET	Pseudo-opcode
condJump	BEQ, BGE, BGEU, BLT, BLTU, BNE	
uncondJump	J	Pseudo-opcode
indirectJump	JALR	
sideEffect	CSRRC, CSRRCI, CSRRS, CSRRSI, CSRRW, CSRRWI, EBREAK, ECALL	Control and Status Register (CSR) handling, environment calls/breakpoints

Table 9.14: Opcode Types of RV32IC, RV32IF and RV32IFD Processor Models

Opcode Type	Opcodes			Comment
	RV32IC	RV32IF	RV32IFD	
load	CLW, CLWSP	FLW	FLD	
store	CSW, CSWSP	FSW	FSD	
move	CMV	FMV_S	FMV_D	Pseudo-opcodes FMV_S and FMV_D
call	CJAL			
indirectCall	CJALR			
condJump	CBEQZ, CBNEZ			
uncondJump	CJ			
indirectJump	CJR			
sideEffect	CEBREAK	FEQ_S, FLE_S, FLT_S	FEQ_D, FLE_D, FLT_D	

- J represents an unconditional branch and gets substituted by `jal x0, label`
- MOV denotes a register-register move that is replaced by `addi tgt, src, 0`
- RET realizes a return from a function call that boils down to `jalr x0, x1, 0`

Using the class RV32IC or its derivatives extends the processor model by 27 additional operations ranging from CADD to CXOR (cf. file `rv32icopcodes.cc`). Yet another 27 operations (FADD_S to FSW) for single-precision floating point computations are added by class RV32IF (file `rv32ifopcodes.cc`). Likewise for file `rv32ifdopcodes.cc` defining the 27 double-precision floating-point operations FADD_D to FSUB_D. Finally, all 8 operations DIV to REMU for integer division and multiplication (cf. file `rv32imopcodes.cc`) can be added by using class RV32IM or derivatives. The classification of all these opcodes implementing the RISC-V standard extensions is summarized in Table 9.14. Each of the two floating-point extensions RV32IF and RV32IFD specifies one pseudo-opcode representing moves of floating-point registers: FMV_S and FMV_D, resp. As usual, operations that are not listed in Table 9.13 or Table 9.14 carry no particular attributes within WIR and thus usually represent standard arithmetical operations.

Base class RV32I makes use of a total of 17 operation formats that are all 32 bits wide (cf. file `rv32ioperation-formats`). The naming scheme used in the context of the RV32 processor model within WIR is shown in the upper part of Table 9.15. It is worth mentioning that two operation formats (RV32I::OperationFormat::RSC5_1 and RSR_1) make use of unstructured string parameters (cf. Section 8.6.6). These formats are exclusively used by the RISC-V's operations manipulating the CSRs. As the ISA specification [24] lacks a proper definition of available CSRs and how they shall be named in assembly code, these CSR names have to be provided as plain strings to the corresponding RV32I operations.

Warning

Due to the fully unstructured nature of string parameters, the WIR library has no means to distinguish between valid and invalid strings used to identify CSRs. Thus, it is easy to generate invalid WIR operations here, it lies in the sole responsibility of application code to ensure that only valid CSR names are used as string parameters.

Table 9.15: Naming Scheme for Operation Formats of RV32I, RV32IC and RV32IF Processor Models

Operation Format Specifier	Parameter Type
"C12"	Signed 12-bit immediate constant
"C20"	Unsigned 20-bit immediate constant
"C5"	Unsigned 5-bit immediate constant
"L"	Label parameter
"NULL"	No parameters at all
"R"	Register parameter of type <code>RV32I::RegisterType::reg</code>
"S"	Unstructured string parameter
"C10"	Signed or unsigned 10-bit immediate constant
"C8"	Unsigned 8-bit immediate constant
"C7"	Unsigned 7-bit immediate constant
"C6"	Signed or unsigned 6-bit immediate constant
"S . . ."	Prefix for short 16-bit operation formats
"F"	Floating-point register parameter of type <code>RV32I::RegisterType::fReg</code>

The 16 additional operation formats added by class RV32IC (cf. file `rv32icoperationformats.cc`) are all 16 bits wide, and their names – as usual within [WIR](#) – all start with the prefix "S" (cf. middle part of Table 9.15). Additionally, the naming scheme is extended to also capture the 8- and 6-bits immediates required by the C ISA extension. Finally, extension RV32IF introduces 7 more 32-bit operation formats (file `rv32ifoperationformats.cc`), and the specifier "F" here denotes a floating-point register parameter (cf. bottom part of Table 9.15).

Examples how to deal with all modeled RISC-V opcodes and operation formats can be found in unit tests `WIR/arch/riscv/tests/rv32*0operations*`. In analogy to the TriCore model, the unit tests ending with the number 1 (e.g., `rv32i0operations1.cc`) instantiate and verify all opcodes in all formats and test their individual attributes and properties. Unit tests ending with the number 2 (e.g., `rv32imfdc0operations2.cc`) print out all modeled operations and pass them through the GNU RISC-V assembler.

Since the RISC-V ISA only knows one addressing mode, i.e., base + offset addressing, no addressing modes are explicitly modeled within [WIR](#).

9.1.5 ARM Processor Models

The ARM-based ISAs [2] are organized in a hierarchical manner with an ARMv4 instruction set as base (older ISA versions v1, v2 or v3 are considered obsolete). On top of this base, various more recent instruction set versions have been defined over the years. This structure is properly reflected by the [WIR](#) processor models which, again, exploit inheritance to realize a class hierarchy ranging from the base ARMv4 model to the ARMv6 processor model. This class hierarchy is depicted in Fig. 9.4.

The ARM architecture has a total of 16 general-purpose registers `r0 – r15` that are all 32 bits wide. These registers are represented by register type `ARMv4::RegisterType::reg` that is defined in file `WIR/arch/arm/arm-registertypes.cc`, see also Table 9.16. As usual, there exist implementations of classes `ARM_RegP` and `ARM_RegV` modelling both physical and virtual registers of this type (files `armregphysical.h` and `armregvirtual.h`). However, the ARM base class `ARMv4` does not provide any access methods for the quick access to the architecture's physical registers of type `reg`. This is motivated by the fact that the ARM ISA regularly partitions

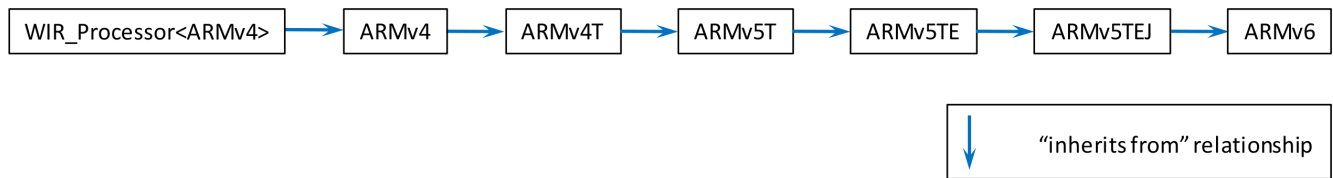


Figure 9.4: Simplified Inheritance Diagram of ARM Processor Models

this general-purpose register file into high and low registers. 16-bit ARM THUMB operations can only access low registers, while the 32-bit ARM operations can access both low and high registers, i.e., the entire register file. Due to this, the ARMv4 processor model within [WIR](#) furthermore defines the register types `hi` and `lo` as can be seen from Table 9.16. As their names suggest, these types stand for registers `r8 – r15` and `r0 – r7`, resp. Class `ARMv4` now includes convenient access methods `R8()` – `R15()` that return references to the physical `hi` registers, while methods `R0()` – `R7()` provide access to the physical `lo` registers. Physical and virtual `hi` registers are realized by classes `ARM_HiRegP` and `ARM_HiRegV` (files `armhiregphysical.h` and `armhiregvirtual.h`), resp. Analogously, classes `ARM_LoRegP` and `ARM_LoRegV` from files `armloregphysical.h` and `armloregvirtual.h` model the ARM’s `lo` registers, resp. None of the register types defined for the ARMv4 model is hierarchical, register `R13` is the ARM’s stack pointer.

Hierarchical registers are introduced beginning with the ARMv5TE model. These extended ARM registers group neighboring pairs of 32-bit general-purpose registers together in order to form 64-bit registers. For example, standard registers `R4` and `R5` form the extended 64-bit register `P4`. Thus, the ARMv5TE model introduces the hierarchical register type `pReg`. Physical registers of this type are implemented by class `ARMv5TE_PRegP` (file `armv5tepreghysical.h`), while class `ARMv5TE_PRegV` realizes virtual extended registers (file `armv5tepregvirtual.h`). Consequently, class `ARMv5TE` provides access methods `P0()` – `P14()` for the easy access to these physical extended registers.

Unit tests `WIR/arch/arm/tests/armPhRegs1.cc` and `armVirtualRegister1.cc` provide code examples for the usage of all [WIR](#) classes modeling both physical and virtual ARM registers.

The ARM [ISA](#) is characterized by a rather small and compact base of opcodes provided by class `ARMv4`, resembling a classical [RISC](#) design. Similar to the modular approach of the RISC-V architecture (cf. Section 9.1.4), this base [ISA](#) can be extended by additional operations. The main difference of ARM operations compared to previous architectures, however, lies in the enormous complexity of the operations’ encodings themselves. The ARM [ISA](#) allows to generate a huge number of different variants of one and the same machine operation, which resembles

Table 9.16: Register Types, Classes and Methods of ARMv4 Processor Model

Register Type	Bit Width	Has Childs	Physical Registers		Virtual Registers	
			Class	Prefix	Class	Prefix
<code>reg</code>	32		<code>ARM_RegP</code>	“r”	<code>ARM_RegV</code>	“r_”
<code>hi</code>	32		<code>ARM_HiRegP</code> <code>R8()</code> – <code>R15()</code>	“r”	<code>ARM_HiRegV</code>	“h_”
<code>lo</code>	32		<code>ARM_LoRegP</code> <code>R0()</code> – <code>R7()</code>	“r”	<code>ARM_LoRegV</code>	“l_”
<code>pReg</code>	64	✓ reg	<code>ARMv5TE_PRegP</code> <code>P0()</code> , <code>P2()</code> , ..., <code>P14()</code>	“p”	<code>ARMv5TE_PRegV</code>	“p_”

Table 9.17: Classes for Immediates of ARMv4, ARMv4T, ARMv5T and ARMv6 Processor Models

Immediate Type	Bit Width	Class	Value Range	Comment
Unsigned 24-bit	24	ARM_Const24_Unsigned	$[0, +2^{24} - 1]$	
Unsigned 12-bit	12	ARM_Const12_Unsigned	$[0, +2^{12} - 1]$	
Unsigned 10-bit	10	ARM_Const10_CoprocessorOffset	$[0, +1023]$	Coprocessor offsets, must be multiples of 4
Unsigned 8-bit	8	ARM_Const8_Unsigned	$[0, +255]$	
Unsigned 6-bit	6	ARM_Const6_Unsigned0	$[+1, +32]$	Value 0 not included
Unsigned 5-bit	5	ARM_Const5_Unsigned	$[0, +31]$	
Unsigned 5-bit	5	ARM_Const5_Unsigned0	$[+1, +31]$	Value 0 not included
Unsigned 5-bit	5	ARM_Const5_RotateAmount	$[0, +30]$	Rotate amounts, must be multiples of 2
Unsigned 4-bit	4	ARM_Const4_CoprocessorOpcode	$[0, +15]$	Coprocessor opcodes
Unsigned 3-bit	3	ARM_Const3_CoprocessorOpcode	$[0, +7]$	Coprocessor opcodes
Unsigned 9-bit	9	ARM_Const9_Unsigned	$[0, +511]$	Must be multiples of 4
Unsigned 7-bit	7	ARM_Const7_Unsigned	$[0, +127]$	Must be multiples of 4
Unsigned 6-bit	6	ARM_Const6_Unsigned	$[0, +63]$	Must be multiples of 2
Unsigned 3-bit	3	ARM_Const3_Unsigned	$[0, +7]$	
Unsigned 16-bit	16	ARM_Const16_Unsigned	$[0, +2^{16} - 1]$	
Unsigned 5-bit	5	ARM_Const5_SatPos	$[+1, +16]$	Saturation positions

more a classical [Complex Instruction Set Computing \(CISC\)](#) architecture design. This is mostly due to the various highly complex addressing modes that the ARM ISA offers, and the various operation encodings have the effect that a rather large number of different immediate types are required.

All [WIR](#) immediate classes used in the context of the ARM processor models are summarized in Table 9.17. For the sake of brevity, this user's guide will not discuss all the various classes and their rationale in detail, the interested reader is referred to [2] instead. Instead, only a few comments on very particular immediate classes will be given in the following.

The ARMv4 ISA inherently supports to communicate with a coprocessor, requiring the specification of coprocessor offsets and opcodes. These elements are modeled by classes `ARM_Const10_CoprocessorOffset`,

Table 9.18: Opcode Types of ARMv4 Processor Model

Opcode Type	Opcodes	Comment
load	LDC, LDM, LDR, LDRB, LDRBT, LDRH, LDRSB, LDRSH, LDRT, SWP, SWPB	
store	STC, STM, STR, STRB, STRBT, STRH, STRT, SWP, SWPB	
call	BL	
indirectCall	SWI	
condJump	B	
uncondJump	B	

Table 9.19: Opcode Types of ARMv4T, ARMv5T, ARMv5TE, ARMv5TEJ and ARMv6 Processor Models

Opcode Type	Opcodes					Comment
	ARMv4T	ARMv5T	ARMv5TE	ARMv5TEJ	ARMv6	
load	LDMIA, POP	LDC2	LDRD		LDREX	
store	PUSH, STMIA	STC2	STRD		SRS, STREX	
move					CPY	
call		BLX				
ret					RFE	
indirectJump	BX	BKPT		BXJ		

ARM_Const4_CoprocessorOpcode and ARM_Const3_CoprocessorOffset, where such coprocessor offsets have to be multiples of 4. Class ARM_Const6_Unsigned0 is designed to cover the value range from 1 to 32, excluding the value 0. Similarly, class ARM_Const5_Unsigned0 ranges from 1 to 31, excluding 0. Some addressing modes support the automatic rotation of some argument, which requires a rotate amount as parameter. Such rotate amounts are realized by class ARM_Const5_RotateAmount which covers the value range from 0 to 30, and rotate amounts are enforced to be even numbers.

The ARMv4T model introduces 16-bit thumb operations which again require additional immediate types as can be seen from the middle part of Table 9.17. New immediates with widths of 9, 7, 6 and 3 bits are realized by the according classes, resp. Actual values for ARM_Const9_Unsigned and ARM_Const7_Unsigned must be multiples of 4, and ARM_Const6_Unsigned only accepts even values.

The ARMv5T processor model adds yet another class ARM_Const16_Unsigned, and immediate type ARM_Const5_SatPos is introduced by the ARMv6 processor model and reflects saturation positions ranging from 1 to 16 (cf. bottom of Table 9.17).

Code examples for all ARM immediate classes can be found in unit tests WIR/arch/arm/tests/armConst*.

The ARMv4 model specifies all 26 opcodes of the base ISA, beginning with ADC and ranging to UMULL. As can be seen from Table 9.18, many of these opcodes are characterized within WIR either as memory load or store operations. Three more opcodes are furthermore modeled to be branches or function calls, resp. None of the opcodes modeled within WIR is a pseudo-opcode.

Table 9.19 summarizes the opcode characterizations of all the ARM processor models derived from the base model ARMv4. This table again shows that most of these new opcodes are characterized as loads or stores. A couple of other opcodes being register-register moves, function calls and returns, and indirect branches are introduced by the derived processor models as well.

Due to the complex different encodings of ARM operations, one opcode can play different roles when it comes to its characterization within WIR. For this reason, it might not be appropriate to simply classify, e.g., opcode B as unconditional branch, since it might also be a conditional branch depending on its condition field. For this reason, the ARMv4 model contains dedicated overloaded methods for opcode handling as shown in Listing 9.32. These methods inspect an opcode in its specific context of some WIR operation and return whether the opcode definitely realizes a register-register move, an indirect call, a function return, a conditional, an unconditional or an indirect jump, resp.

The complexity of the ARM ISA can be seen from the fact that a total of 212 different operation formats are required within WIR to capture all possible encodings resulting from the only 26 opcodes of base class ARMv4. The majority of 128 of all these formats are solely required to model the two opcodes LDM and STM. These two opcodes allow to load or store multiple registers in one machine operation, where the list of registers to be loaded or stored can contain 1 up to 16 registers. For the proper data flow and def-use analysis within WIR, all possible lengths of such

Listing 9.32: Specific Opcode Handling of Class ARMv4

```

1  class ARMv4 : public WIR_Processor<ARMv4>
2  {
3      public:
4
5          [...]
6
7      class OpCode : public WIR_Processor::OpCode
8      {
9          public:
10
11             [...]
12
13             // Determine whether an ARM opcode is a register-register move.
14             virtual bool isMove( const WIR_Operation &o ) const;
15
16             // Determine whether an ARM opcode indirectly calls a function.
17             virtual bool isIndirectCall( const WIR_Operation &o ) const;
18
19             // Determine whether an ARM opcode returns from a function call.
20             virtual bool isReturn( const WIR_Operation &o ) const;
21
22             // Determine whether an ARM opcode performs a conditional jump.
23             virtual bool isConditionalJump( const WIR_Operation &o ) const;
24
25             // Determine whether an ARM opcode performs an unconditional jump.
26             virtual bool isUnconditionalJump( const WIR_Operation &o ) const;
27
28             // Determine whether an ARM opcode indirectly performs a jump.
29             virtual bool isIndirectJump( const WIR_Operation &o ) const;
30
31             [...]
32     };
33 };

```

register lists need to be made explicit, causing this observed effect. The [WIR](#) processor models ARMv4T, ARMv5T, ARMv5TE and ARMv6 introduce additional 81, 10, 16 and 24 operation formats, resp. The naming scheme used for all ARM operation formats is summarized in [Table 9.20](#) and follows the usual approach already introduced in the previous sections.

Unit tests `WIR/arch/arm/tests/arm*Operations*` illustrate the construction and use all of modeled ARM opcodes in all existing formats. As for the previous processor models, these unit tests once create and validate all opcodes and their properties, and once print them out and feed them through the GNU ARM assembler.

The [WIR](#) processor models feature all addressing modes defined by the ARM [ISA](#). This covers, e.g., all variants of shifts and rotations, address additions or subtractions, pre- or post-indexed addressing, increment or decrement before and after, full ascending/descending etc., as well as all coprocessor addressing modes. In total, the ARMv4 processor model features 49 different addressing modes whose implementation can be found in file `armaddressingmodes.cc`.

The ARM architecture furthermore defines a total of 15 condition codes that are used for the predicated execution of machine operations. All these condition codes are properly modeled within [WIR](#), ranging from “eq” that stands for a test on equality, up to “al” that denotes to execute an operation always. The realization of all these ARMv4 condition codes can be found in file `armconditions.cc`.

Table 9.20: Naming Scheme for Operation Formats of ARMv4, ARMv4T, ARMv5T, ARMv5TE and ARMv6 Processor Models

Operation Format Specifier	Parameter Type
"A"	Addressing mode parameter
"C"	Condition field parameter
"C24"	Unsigned 24-bit immediate constant
"C12"	Unsigned 12-bit immediate constant
"C10"	Unsigned 10-bit coprocessor offset
"C8"	Unsigned 8-bit immediate constant
"C60"	Unsigned 6-bit immediate constant without 0
"C5"	Unsigned 5-bit immediate constant
"C50"	Unsigned 5-bit immediate constant without 0
"L"	Label parameter
"R"	Register parameter of type <code>ARMv4::RegisterType::reg</code>
"Rn"	A series of <i>n</i> register parameters of type <code>ARMv4::RegisterType::reg</code>
"RA"	Unsigned 5-bit rotate amount
"O"	Unsigned 3-bit or 4-bit coprocessor opcode
"S"	Unstructured string parameter
"C10"	Unsigned 10-bit coprocessor offset
"C9"	Unsigned 9-bit immediate constant
"C7"	Unsigned 7-bit immediate constant
"C3"	Unsigned 3-bit immediate constant
"LR"	Physical link register parameter R14
"PC"	Physical program counter register parameter R15
"SP"	Physical stack pointer register parameter R13
"T . . ."	Prefix for 16-bit thumb operation formats
"C16"	Unsigned 16-bit immediate constant
"P"	Extended register parameter of type <code>ARMv5TE::RegisterType::pReg</code>
"C5SP"	Unsigned 5-bit saturation position

How to deal with the ARM-specific condition field parameters can be seen in unit test `WIR/arch/arm/tests/armConditionFieldParameter1.cc`.

9.2 Physical Memory Regions

While processor models as described in the previous Section 9.1 play a critical role within [WIR](#), they are by far not the only components of modern computing systems that require careful modelling. Due to the fact that physical memories have a direct impact on an entire system's performance, it is crucial to also consider the properties of memory regions in [WIR](#) system models. This is precisely the purpose of the class `WIR_MemoryRegion` described in the following. [WIR](#) uses the notion of memory regions instead of entire memories in order to enable the partitioning of complete memories into different chunks that might carry different attributes. Each such chunk of a full memory is then represented by a dedicated [WIR](#) memory region.

Listing 9.33: Name, Address and Size Handling of Class WIR_MemoryRegion

```

1 // Get the region name.
2 std::string WIR_MemoryRegion::getName( void ) const;
3
4 // Get a region's base address.
5 WIR_MemoryAddress WIR_MemoryRegion::getBaseAddress( void ) const;
6
7 // Get a region's length in bytes.
8 std::size_t WIR_MemoryRegion::getLength( void ) const;

```

The most fundamental attributes of a region are its name, start address and size. They can be retrieved using the methods `getName`, `getBaseAddress` and `getLength` shown in Lines 2 to 8 of Listing 9.33.

As already introduced in Section 9.1.1.8, logical sections of an executable are attached to a physical memory region (cf. Listing 9.26). Regions thus act as containers for sections within WIR. The information which sections are actually associated with a region can be retrieved using method `getSections` shown in Line 2 of Listing 9.34 which returns a reference to a section set. Method `containsSection` (Line 5) can be used to determine whether some logical section is actually associated with a memory region or not. Furthermore, `getFreeSpace` (Line 8) computes how many bytes are still free and unoccupied within a region. This free space depends on the sizes of all the sections associated with a region, their respective contents and alignments, etc.

Within WIR, a physical memory region carries a couple of additional properties which can be queried using the methods shown in Listing 9.35. Firstly, method `isDynamicallyAllocated` (cf. Line 2) can be used to determine whether the amount of used space within a memory region may vary dynamically during runtime. This can, e.g., happen when a system's runtime stack is mapped to some region. Depending on whether a region's contents is read-only or executable etc., it is furthermore characterized by additional attributes that model its "access policy". These attributes can be obtained via method `getAttributes` (Line 5). This method returns an unsigned long value that forms a bit-field whose individual bits represent specific access attributes. These bit-level attributes are modeled by the WIR enumeration type `WIR_MemoryRegionAttributes` shown in Lines 31 to 42. Besides their very own names, regions can also be given a human-readable name standing for the memory type on which a region bases like, e.g., "FLASH" or "ROM". This memory type name can be retrieved via method `getMemoryType` (Line 8).

The next couple of methods is useful to obtain performance figures related to a memory region. First, method `getClockRatio` (Line 11) returns a double-precision floating-point value that characterizes the relationship between the region's and the processor core's clock frequencies. `getMinDelay` and `getMaxDelay` (Lines 14 and 17) return a region's minimum and maximum access delays in clock cycles, resp. Argument `d` of `getMaxDelay` intends to represent the additional delay induced by other components in a system's memory hierarchy. Since a memory region by construction is the very last component in a memory hierarchy, this argument is simply ignored here.

Listing 9.34: Section Handling of Class WIR_MemoryRegion

```

1 // Get the set of all sections attached to a region.
2 const WIR_SectionSet &WIR_MemoryRegion::getSections( void ) const;
3
4 // Determine whether a region contains a given section.
5 bool WIR_MemoryRegion::containsSection( const WIR_Section & ) const;
6
7 // Get a region's free space in bytes.
8 std::size_t WIR_MemoryRegion::getFreeSpace( void );

```

Listing 9.35: Characteristic Attributes of Class WIR_MemoryRegion

```

1 // Determine whether a region is dynamically allocated during runtime.
2 bool WIR_MemoryRegion::isDynamicallyAllocated( void ) const;
3
4 // Get a bit-field encoding memory region attributes.
5 unsigned long WIR_MemoryRegion::getAttributes( void ) const;
6
7 // Get the name of the memory type that a region belongs to.
8 std::string WIR_MemoryRegion::getMemoryType( void ) const;
9
10 // Get the ratio of the memory region's clock frequency compared to the processor's clock frequency.
11 long double WIR_MemoryRegion::getClockRatio( void ) const;
12
13 // Get the minimum number of clock cycles for which a region can delay a memory access.
14 unsigned int WIR_MemoryRegion::getMinDelay( void ) const;
15
16 // Get the maximum number of clock cycles for which a region can delay a memory access.
17 virtual unsigned int WIR_MemoryRegion::getMaxDelay( const unsigned int d = 0 ) const;
18
19 // Get the minimum burst access delay of a region in clock cycles.
20 unsigned long WIR_MemoryRegion::getMinBurstDelay( void ) const;
21
22 // Get the maximum burst access delay of a region in clock cycles.
23 unsigned long WIR_MemoryRegion::getMaxBurstDelay( void ) const;
24
25 // Get the byte width of the memory bus that connects a region.
26 unsigned long WIR_MemoryRegion::getBusWidth( void ) const;
27
28 // Get the hierarchy of all system components residing in front of this memory region.
29 const std::set<std::list<std::reference_wrapper<WIR_SystemComponent>>> &WIR_MemoryRegion::getHierarchy(
    void ) const;
30
31 // Enum WIR_MemoryRegionAttributes represents different attributes of WIR memory regions.
32 enum class WIR_MemoryRegionAttributes : unsigned char
33 {
34     none = 0, // A memory region without any specific attributes.
35     read = 1 << 0, // A memory region that can be read.
36     write = 1 << 1, // A memory region that can be written.
37     execute = 1 << 2, // A memory region that can be executed.
38     allocated = 1 << 3, // A memory region that is allocated.
39     initialized = 1 << 4, // A memory region that is initialized.
40     cached = 1 << 5, // A memory region that is cached.
41     all = ( cached << 1 ) - 1 // Covers all existing attributes.
42 };

```

Memories usually exhibit different access latencies depending on whether single accesses or burst accesses are considered. Methods `getMinDelay` and `getMaxDelay` provide the latencies for one single access. Accordingly, methods `getMinBurstDelay` and `getMaxBurstDelay` (Lines 20 and 23) provide the minimum and maximum access delays for bursts, resp.

WIR allows to model systems with complex memory hierarchies consisting of buses (see Section 9.4), various levels of caches (cf. Section 9.3), and memories of different types. When considering the position of a memory region in such a hierarchy, the bit width of the interface that connects the region to its bus is yet another important attribute. This bit width can be obtained via method `getBusWidth` from Line 26. Finally, method `getHierarchy` (Line 29) can be used to obtain the full hierarchy of system components that reside “in front of” a region. Here, “in front” refers to all components that are closer to a processor core than the considered region itself. `getHierarchy`

returns a set of lists of such system components. Each of these lists only contains buses or caches, and the component at a list's head is closest to the processor core, while the component at a list's tail is the farthest away from the processor core, i.e., the closest to the currently considered memory region. In real-life system architectures, there can be multiple paths from a processor core through the memory hierarchy to one and the same physical memory region. This can happen if, e.g., a certain part of a region's address space is covered by a cache, while other parts of the same region's address space are uncached. For this reason, method `getHierarchy` returns a set potentially containing several of these lists described earlier. Each list within the set returned by `getHierarchy` represents exactly one unique path through a system's memory hierarchy from a processor core to the region itself.

9.3 Caches

Modern processor architectures feature sophisticated memory hierarchies that not only consist of physical memory regions as described in the previous section, but that also use caches. Thus, class `WIR_Cache` is derived from base class `WIR_SystemComponent` and provides the representation of various flavors of caches within the [WIR](#) library.

The interface of class `WIR_Cache` to query all system-level architectural attributes of a cache without considering its internal organization is shown in Listing 9.36. Caches can regularly be enabled or disabled by setting system-specific status bits. The current state of a cache can be retrieved with method `isEnabled` (cf. Line 2). Caches are regularly classified as instruction, data or unified caches. This classification is modeled by the enumeration class `CacheType` (cf. Line 26), and a cache's type can be obtained using method `getCacheType` (Line 5). The position of a cache within a system's memory hierarchy, i.e., the cache's level, is modeled by enumeration `CacheLevel` (Line 34), and it can be determined using method `getLevel` (Line 8). In multi-core systems, a cache can be shared between several processor cores, and this information is accessible via method `isShared` (Line 11).

The best-case delay in clock cycles that a cache can cause for a cache hit can be obtained with method `getHitDelay` (Line 14). Similarly, `getMissDelay` (Line 17) returns the worst-case delay in clock cycles for the case of a cache miss. Note that this miss delay only refers to the extra time required by the cache itself, excluding the delays caused by accesses to parts of the memory hierarchy lying behind the missing cache. The maximal delay of a memory cache that can be caused by a cache including that of subsequent parts of the memory hierarchy can be obtained with the virtual method `getMaxDelay` (Line 20). It expects an unsigned integer argument `d` that represents the access delay caused by the tail of the memory hierarchy. `getMaxDelay` assumes a worst-case scenario where an evicted cache line needs to be written back and a new cache line is loaded. It computes how many bus transfers are required for these two transactions, multiplies this by delay `d`, and finally adds the cache's very own intrinsic miss delay. This default implementation of `getMaxDelay` can be overridden for specialized classes inheriting from `WIR_Cache`. The number of bus transfers initiated by a cache naturally depends on the width of the interface connecting the cache with a system's bus. This bus width in bytes can be retrieved using method `getBusWidth` (Line 23).

All attributes related to the internal geometry of a cache can be retrieved with the methods shown in Listing 9.37. Most relevantly, this includes the sizes of a cache and its numbers of rows and columns. Thus, the overall byte size of a cache can be obtained using method `getSize` (cf. Line 2). The returned size refers solely to the cache's data memory, i.e., that memory storing its payload in the form of cached code or data memory lines. `getSize` does not consider the size of the tag memory that a cache requires for its hit/miss checks. The associativity of a cache can be queried via method `getAssociativity` (Line 5). Typical values might range from 1 (direct-mapped cache) to 8 (set-associative cache). As `getAssociativity` in other words returns the number of internal columns of a cache, method `getNumberOfSets` (Line 8) provides the total number of sets, i.e., rows that a cache features.

Listing 9.36: Architectural Attributes of Class WIR_Cache

```

1 // Determine whether a cache is enabled and active.
2 bool WIR_Cache::isEnabled( void ) const;
3
4 // Get a cache's type.
5 WIR_Cache::CacheType WIR_Cache::getCacheType( void ) const;
6
7 // Get a cache's position in a system's memory hierarchy.
8 WIR_Cache::CacheLevel WIR_Cache::getLevel( void ) const;
9
10 // Determine whether a cache is shared between multiple processor cores.
11 bool WIR_Cache::isShared( void ) const;
12
13 // Get the time in clock cycles that a cache hit needs to complete in the best case.
14 unsigned int WIR_Cache::getHitDelay( void ) const;
15
16 // Get the time in clock cycles that a cache miss needs to complete in the worst case.
17 unsigned int WIR_Cache::getMissDelay( void ) const;
18
19 // Get the maximum number of clock cycles for which a cache can delay a memory access.
20 virtual unsigned int WIR_Cache::getMaxDelay( const unsigned int d ) const;
21
22 // Get the byte width of the memory bus that connects a cache.
23 unsigned long WIR_Cache::getBusWidth( void ) const;
24
25 // Enum CacheType represents the different types a cache can have.
26 enum class WIR_Cache::CacheType : char
27 {
28     I,          // An instruction cache.
29     D,          // A data cache.
30     U           // A unified cache.
31 };
32
33 // Enum CacheLevel represents a cache's position in the memory hierarchy.
34 enum class WIR_Cache::CacheLevel : char
35 {
36     NOTSET = 0, // A cache whose level in the memory hierarchy has not been set.
37     L1,         // A level-1 cache.
38     L2          // A level-2 cache.
39 };

```

Class WIR_Cache enforces that a cache's size, associativity and its number of sets are all powers of 2. Each single element inside a cache in a certain row and column is called a cache line. The byte size of such a cache line can be retrieved by method `getLineSize` (Line 11).

Based on the above geometrical attributes, class WIR_Cache ensures that the following relationship always holds:

$$Size_{Cache} := Associativity * \#Sets * Size_{Line}$$

By multiplying associativity and the number of available sets, the dimension of a cache's internal memory matrix is determined. The final multiplication by a single element's byte size then yields the absolute byte size of the full cache (excluding tag memories).

Besides specifying a cache's size as absolute byte size according to some real processor architecture, it might occasionally be useful to characterize a cache size in a relative fashion, as a percentage of the size of some software task executed on a system. Typical use cases for such relative cache sizes are, e.g., benchmarking

Listing 9.37: Geometry-Related Attributes of Class WIR_Cache

```

1 // Get the overall byte size of the entire data memory of a cache.
2 unsigned int WIR_Cache::getSize( void ) const;
3
4 // Get a cache's associativity.
5 unsigned int WIR_Cache::getAssociativity( void ) const;
6
7 // Get the number of sets (i.e., rows) of a cache.
8 unsigned int WIR_Cache::getNumberOfSets( void ) const;
9
10 // Get a cache's line size in bytes.
11 unsigned int WIR_Cache::getLineSize( void ) const;
12
13 // Determine whether a cache's size is given as percentage, and not as an absolute byte size.
14 bool WIR_Cache::isSizedInPercent( void ) const;
15
16 // Set a cache's size to a percentage of a given task size.
17 void WIR_Cache::setRelativeSize( unsigned int t );
18
19 // Get the number of offset bits that memory addresses require for this cache.
20 int WIR_Cache::getOffsetBits( void ) const;
21
22 // Get the number of index bits that memory addresses require for this cache.
23 int WIR_Cache::getIndexBits( void ) const;
24
25 // Get the number of tag bits that memory addresses require for this cache.
26 int WIR_Cache::getTagBits( void ) const;
27
28 // Determine whether a cache implements the write-through policy. If not, the cache is a write-back cache.
29 bool WIR_Cache::isWriteThrough( void ) const;
30
31 // Determine whether the cache allocates on writes.
32 bool WIR_Cache::isWriteAllocate( void ) const;

```

campaigns that evaluate a cache's performance depending on the sizes of the used benchmarks. Whether or not the size of a **WIR** cache is given as such a percentage can be checked with method `isSizedInPercent` (cf. Line 14 of Listing 9.37). The absolute byte size of such a benchmark task which is considered to be 100% can be specified using method `setRelativeSize` (Line 17).

In order to locate an entry at some given memory address inside a cache, its row and column positions within the cache's internal matrix need to be determined. For this purpose, the bits of a memory address are partitioned into its offset (least-significant part), index (middle part), and tag (most-significant part). How many bits are actually needed for a cache with a given geometry can be determined with the methods `getOffsetBits`, `getIndexBits` and `getTagBits` (cf. Lines 20 to 26). Here, the **WIR** library always adheres to the following relationships:

$$\begin{aligned}
 Size_{Line} &:= 2^{\#OffsetBits} \\
 \#Sets &:= 2^{\#IndexBits} \\
 \#TagBits &:= AddrWidth - \#IndexBits - \#OffsetBits
 \end{aligned}$$

where *AddrWidth* denotes the bit width of a **WIR** memory address.

Besides looking up an entry inside a cache, various policies exist whenever a cache entry is overwritten. The most simple write policy is write-through, where the overwritten cache line is directly transferred to the rear parts of the memory hierarchy coming after the involved cache. The write-back policy in contrast delays such costly transfers as long as possible. Which of these two write policies is implemented by a **WIR** cache can be checked using

Listing 9.38: Address and Address Range Handling of Class WIR_Cache

```

1 // Local type definition to represent cache lines (tag + index).
2 using WIR_Cache::CacheLineID = WIR_MemoryAddress;
3
4 // Local type definition to represent cache index bits.
5 using WIR_Cache::CacheIndex = WIR_MemoryAddress;
6
7 // Local type definition to represent cache tag bits.
8 using WIR_Cache::CacheTag = WIR_MemoryAddress;
9
10 // Get the cache line's ID (tag and index bits) targeted by an access to a given address.
11 WIR_Cache::CacheLineID WIR_Cache::getLineID( const WIR_MemoryAddress &a ) const;
12
13 // Get the index bits from a given memory address.
14 WIR_Cache::CacheIndex WIR_Cache::getIndex( const WIR_MemoryAddress &a ) const;
15
16 // Get the tag bits from a given memory address.
17 WIR_Cache::CacheTag WIR_Cache::getTag( const WIR_MemoryAddress &a ) const;
18
19 // Get the range of cache line IDs that may be affected by an access to a given address range.
20 WIR_Interval<WIR_Cache::CacheLineID> WIR_Cache::getAffectedLineIDs( const WIR_AddressRange &r ) const;
21
22 // Get the range of cache sets that may be affected by an access to a given address range.
23 WIR_Interval<WIR_Cache::CacheIndex> WIR_Cache::getAffectedSets( const WIR_AddressRange &r ) const;
24
25 // Get the range of cache tags that may be affected by an access to a given address range.
26 WIR_Interval<WIR_Cache::CacheTag> WIR_Cache::getAffectedTags( const WIR_AddressRange &r ) const;

```

method `isWriteThrough` (Line 29). This method returns true for write-through caches and false for write-back caches. In the case of a write miss inside a cache, the write allocate or the no-write allocate policies come into play. The former allocates space in the missing cache for the new data to be written, while the latter simply forwards the new data to the tail of the memory hierarchy without changing the cache's internal state. Method `isWriteAllocate` (Line 32) finally returns true if write allocate is active and false if a cache implements no-write allocate.

The functional behavior of caches heavily depends on memory addresses and address ranges that serve as input to a cache. As already described on Page 85, addresses are represented by type `WIR_MemoryAddress`. Based on this type, address ranges are modeled within `WIR` as intervals of addresses having a well-defined lower and upper bound. Thus, type `WIR_AddressRange` is defined as `WIR_Interval<WIR_MemoryAddress>`. The `WIR`-internal template class `WIR_Interval` (cf. file `analyses/generic/wirinterval.h`) provides the simple construction of and access to such intervals with lower and upper bounds, as well as all standard arithmetical and comparison operations for intervals.

In order to support static analyses that aim to estimate how `WIR` code consisting of sequences of operations changes a cache's internal state, it is necessary to relate addresses of memory accesses issued by such operations to the proper locations inside the cache which might be affected. For this purpose, class `WIR_Cache` provides the API depicted in Listing 9.38.

Methods `getLineID`, `getIndex` and `getTag` (cf. Lines 11 to 17) return which cache line, set and tag is affected by a given memory address, resp. As described earlier, the internal geometry of a cache determines which and how many bits of an address represent tags, indices and offsets. The aforementioned methods then extract the tag and index bits from the given address and basically return these bits. For the sake of type safeness, class `WIR_Cache` defines its own data types representing cache line IDs, index and tag bits. These local types finally

Listing 9.39: Event Counting of Class WIR_Cache

```

1 // Get the number of counted cache hits.
2 unsigned int WIR_Cache::getHitCount( void ) const;
3
4 // Increase the number of counted cache hits by one.
5 void WIR_Cache::increaseHitCount( void ) const;
6
7 // Clear the number of counted cache hits.
8 void WIR_Cache::clearHitCount( void ) const;
9
10 // Get the number of counted cache misses.
11 unsigned int WIR_Cache::getMissCount( void ) const;
12
13 // Increase the number of counted cache misses by one.
14 void WIR_Cache::increaseMissCount( void ) const;
15
16 // Clear the number of counted cache misses.
17 void WIR_Cache::clearMissCount( void ) const;
18
19 // Get the number of counted unknown cache hits/misses.
20 unsigned int WIR_Cache::getUnknownCount( void ) const;
21
22 // Increase the number of counted unknown cache hits/misses by one.
23 void WIR_Cache::increaseUnknownCount( void ) const;
24
25 // Clear the number of counted unknown cache hits/misses.
26 void WIR_Cache::clearUnknownCount( void ) const;

```

simply boil down to **WIR** addresses as can be seen from Lines 2 to 8 of Listing 9.38.

During static cache analysis, not only the effect of one single address on a cache's internal state needs to be estimated, but also that of entire address ranges. Consequently, methods `getAffectedLineIDs`, `getAffectedSets` and `getAffectedTags` (cf. Lines 20 to 26) get an interval r of addresses as argument. They then determine which ranges of cache lines, sets or tags, resp., are potentially affected by memory accesses within the given address range r .

A static cache analysis usually classifies each individual memory access as either a definite cache hit, a definite cache miss, or – in the case of uncertainty – as unknown cache hit/miss. To support such counting of events, class **WIR_Cache** provides the methods shown in Listing 9.39. For each of the three event classes, there is a `get` (cf. Lines 2, 11 and 20), an `increase` (Lines 5, 14 and 23) and a `clear` (Lines 8, 17 and 26) method.

9.4 Buses

The very last system component that finally completes the description of the **WIR** hardware model in this chapter are buses. Their purpose is to connect a number of processor cores with various caches and/or physical memory regions.

Future Work

At the time of writing of this document, class **WIR_Bus** is not yet designed and implemented in full detail, since it is not yet actively exploited by actual code analyses or optimizations. The bus model that is

Listing 9.40: Characteristic Attributes of Class WIR_Bus

```

1 // Get the maximum number of clock cycles for which a bus can delay a memory access.
2 virtual unsigned int WIR_Bus::getMaxDelay( const unsigned int d = 0 ) const;
3
4 // Get the maximum access time for a single memory access via this bus.
5 unsigned int WIR_Bus::getMaxAccessTime( void ) const;
6
7 // Get a buses' arbitration strategy.
8 WIR_BusArbitrationType WIR_Bus::getArbitrationType( void ) const;
9
10 // Get a buses' arbitration configuration.
11 const WIR_BusArbitration &WIR_Bus::getBusArbitration( void ) const;
12
13 // Enum BusArbitrationType represents the different arbitration strategies of buses.
14 enum class WIR_BusArbitrationType : char
15 {
16     fp,          // Fixed-Priority arbitration.
17     pd,          // Priority-Division arbitration.
18     rr,          // Round-Robin arbitration.
19     tdma         // Time-Division Multiple Access (TDMA) arbitration.
20 };

```

currently implemented within [WIR](#) actually boils down to the specification of bus arbitration policies and their configuration parameters. Other relevant aspects like, e.g., arbitration and access delays, the distinction between single and burst accesses, or hierarchical buses with bridges in between are currently not modeled. This is left up to future work.

Similar to caches, class `WIR_Bus` comes with a method `getMaxDelay` (see Line 2 of Listing 9.40) that determines the maximal delay of a bus including that of subsequent parts of the memory hierarchy following behind the bus. Argument `d` of this method denotes the maximal delay that can be caused by system components that are connected to the bus and that form the tail of a system's memory hierarchy. Method `getMaxAccessTime` (Line 5) returns the maximum number of clock cycles for a single memory access via a bus. This value does not include the arbitration time potentially required by a bus itself, it only reflects the delay stemming from an access to some memory module connected to the bus. This method can be used to determine whether a pending memory access via a bus still has sufficient time left to complete within the currently running arbitration slot of the bus, or whether the access has to be carried over into some future slot.

[WIR](#) currently supports four different bus arbitration strategies: Fixed-Priority, Priority-Division, Round-Robin, and [Time-Division Multiple Access \(TDMA\)](#). These arbitration policies are modeled by enumeration type `WIR_BusArbitrationType` (cf. Line 14 of Listing 9.40), and a buses' actual arbitration policy can be determined using method `getArbitrationType` (cf. Line 8).

Finally, method `getBusArbitration` (cf. Line 11) provides access to the actual configuration parameters of the arbitration policy of a bus. This method returns a reference to an object of class `WIR_BusArbitration` which is simply an abstract base class from which specialized classes implementing, e.g., fixed-priority arbitration are inherited. Fig. 9.5 depicts the inheritance diagram of all currently supported [WIR](#) bus arbitration policies. These policies are described in more detail in the following Sections 9.4.1 to 9.4.4.

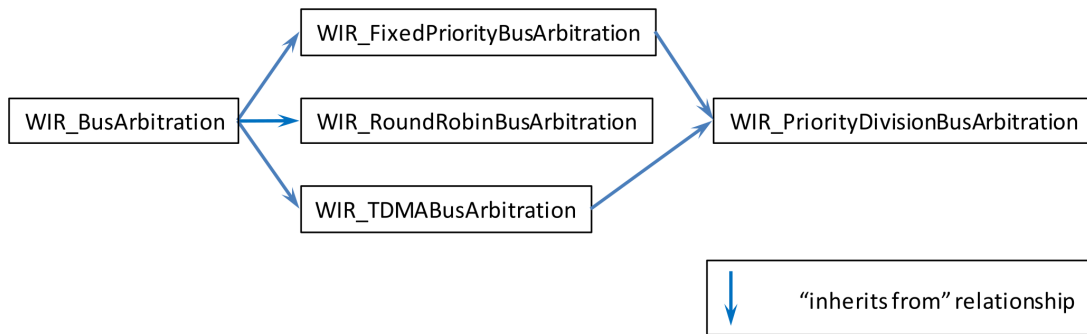


Figure 9.5: Inheritance Diagram of WIR Bus Arbitration Policies

9.4.1 Fixed-Priority Bus Arbitration

Class `WIR_FixedPriorityBusArbitration` realizes fixed-priority bus arbitration where each processor core has a fixed priority assigned. Whenever a conflict occurs where several cores compete for access to the shared bus simultaneously, always that core with the highest fixed priority wins. Currently ongoing bus transactions are, however, not preempted so that a bus transfer by some low-priority core is not interrupted if a request is made by a high-priority core.

Arbitration priorities are modeled using data type `Priority` which in turn goes back to the standard data type `unsigned int` (see Line 2 of Listing 9.41). The higher the numerical value represented by type `Priority` is, the higher a core's arbitration policy is. Assuming that a total of n cores are attached to a fixed-priority bus, the priorities can be from the interval $[0, n]$. A priority of 0 can be used for cores that shall not be considered by the arbiter of a bus, which might be useful for special arbitration policies like, e.g., priority-division (cf. Section 9.4.4). Each processor core that is actively managed by a fixed-priority arbiter must carry its exclusive priority so that one priority value cannot be assigned to several processor cores (except priority 0).

Method `getPriority` (cf. Line 5) provides the priority assigned to some actual processor core c . A map with all priorities over all processor cores can be obtained using method `getPriorities` (Line 8). In the returned map, the unique identifiers of the processor cores (cf. Section 11.1) serve as keys. The total number of cores that are attached to a fixed-priority bus can be retrieved via method `getNumberOfCores` (Line 11). This includes cores

Listing 9.41: Characteristic Attributes of Class `WIR_FixedPriorityBusArbitration`

```

1 // Local type definition to represent fixed priorities of processor cores.
2 using WIR_FixedPriorityBusArbitration::Priority = unsigned int;
3
4 // Get a processor core's fixed arbitration priority.
5 Priority WIR_FixedPriorityBusArbitration::getPriority( const WIR_BaseProcessor &c ) const;
6
7 // Get all processor cores' fixed arbitration priorities for this bus.
8 std::map<WIR_id_t, Priority> &WIR_FixedPriorityBusArbitration::getPriorities( void ) const;
9
10 // Get the total number of processor cores attached to this fixed-priority bus.
11 unsigned int WIR_FixedPriorityBusArbitration::getNumberOfCores( void ) const;
12
13 // Get the total number of processor cores with non-zero arbitration priority.
14 virtual unsigned int WIR_FixedPriorityBusArbitration::getNumberOfArbitratedCores( void ) const;
15
16 // Get the vector of cores managed by this bus arbiter, sorted by priority (highest priority first).
17 const std::vector<WIR_id_t> &WIR_FixedPriorityBusArbitration::getCoreOrder( void ) const;

```

Listing 9.42: Characteristic Attributes of Class WIR_TDMABusArbitration

```

1 // Type definition to represent relative times within cyclic TDMA schedules.
2 using WIR_TDMAOffset = std::uint16_t;
3
4 // Get the total number of cores arbitrated by this TDMA bus.
5 virtual unsigned int WIR_TDMABusArbitration::getNumberOfArbitratedCores( void ) const;
6
7 // Get properties of all TDMA slots of this bus.
8 const std::vector<SlotInfo> &WIR_TDMABusArbitration::getSlots( void ) const;
9
10 // Get properties of all TDMA slots assigned to a given processor core.
11 const std::list<SlotInfo> WIR_TDMABusArbitration::getSlots( const WIR_BaseProcessor &c ) const;
12
13 // Get the total number of TDMA slots of this bus.
14 unsigned int WIR_TDMABusArbitration::getNumberOfSlots( void ) const;
15
16 // Get the total length of the configured cyclic TDMA schedule.
17 WIR_TDMAOffset WIR_TDMABusArbitration::getScheduleLength( void ) const;

```

with arbitration priority 0, i.e., which are not actively managed by this bus. The number of cores that effectively are managed by fixed priorities can be obtained with method `getNumberOfArbitratedCores` (Line 14). Finally, method `getCoreOrder` (Line 17) can be used to obtain a vector with the unique identifiers of all processor cores, where this vector is sorted from highest priority cores at its head to lowest priorities at its tail.

9.4.2 Round-Robin Bus Arbitration

A round-robin arbiter cyclically rotates the bus access among all cores, where each core possesses a unique number from the interval $[0, n - 1]$ and n denotes the total number of cores attached to a bus. Whenever the currently active core n_c releases the bus, n_c is incremented to the next core currently requesting the bus. If n_c is advanced beyond the interval's maximal value $n - 1$, a wrap around takes place and n_c is set to 0.

This arbitration policy obviously does not require any configurable parameters at all so that the API of the corresponding class `WIR_RoundRobinBusArbitration` basically is empty.

9.4.3 Time-Division Multiple Access Bus Arbitration

TDMA arbitration realizes a repetitive cyclic schedule over time that consists of a fixed sequence of n_{TDMA} TDMA slots. Each slot $s \in [0, n_{TDMA} - 1]$ is characterized by its length l_s over time as well as by a processor core $o_s \in [0, n - 1]$ that is the exclusive owner of that TDMA slot. Again, n represents the total number of processor cores connected to a TDMA-arbitrated bus. The total length l_{TDMA} of one complete cycle of the schedule over all slots is $l_{TDMA} = \sum_{s=0}^{n_{TDMA}-1} l_s$. The current position t_{curr} in this cyclic schedule is determined by taking a global clock's time T and computing its modulus to the schedule length: $t_{curr} = T \bmod l_{TDMA}$. The use of the modulo operator here has the effect that once the current time reaches the end of the TDMA schedule, it is reset to 0 so that the entire schedule restarts from the beginning, implementing the cyclic nature of TDMA over time. Based on these concepts, bus access is exclusively granted to that core o_s that owns the TDMA slot s that is active at the current TDMA time t_{curr} . Simultaneous bus requests by other cores are rejected so that these other cores have to wait until the current time t_{curr} has progressed to their very own dedicated TDMA slot.

Listing 9.43: Characteristic Attributes of Class WIR_TDMABusArbitration::SlotInfo

```

1 // Get the position of a slot within the entire cyclic TDMA schedule.
2 unsigned int WIR_TDMABusArbitration::SlotInfo::getIndex( void ) const;
3
4 // Get the processor core owning a TDMA slot.
5 const WIR_BaseProcessor &WIR_TDMABusArbitration::SlotInfo::getOwner( void ) const;
6
7 // Get a TDMA slot's start time, counted from the beginning of the entire TDMA schedule.
8 WIR_TDMAOffset WIR_TDMABusArbitration::SlotInfo::getStart( void ) const;
9
10 // Get a TDMA slot's length in clock cycles.
11 WIR_TDMAOffset WIR_TDMABusArbitration::SlotInfo::getLength( void ) const;

```

WIR class WIR_TDMABusArbitration models this behavior of a TDMA-arbitrated bus, and its key attributes can be inspected using the methods shown in Listing 9.42. Method `getNumberOfArbitratedCores` (cf. Line 5) provides the total number n of processor cores that are connected to and arbitrated by a TDMA bus. The specific information about all slots making up the entire cyclic TDMA schedule can be retrieved via method `getSlots` from Line 8. This variant of `getSlots` returns a reference to a vector containing the information about each TDMA slot. This slot information is represented by class WIR_TDMABusArbitration::SlotInfo described below in more detail. In order to obtain information about all TDMA slots associated with a given processor core, the `getSlots` variant from Line 11 can be used. It expects a reference `c` to the processor core under consideration as argument and returns a list containing only the information of all slots exclusively owned by core `c`. The total number n_{TDMA} of TDMA slots can be obtained via method `getNumberOfSlots` (Line 14). Finally, method `getScheduleLength` (Line 17) provides the overall duration l_{TDMA} of one full TDMA cycle. Such relative times within a TDMA schedule are represented by type WIR_TDMAOffset, which in turn is declared to be `std::uint16_t` (cf. Line 2).

Class SlotInfo is a sub-class of WIR_TDMABusArbitration modelling all relevant properties of a TDMA slot. These properties can be obtained using the methods from Listing 9.43. Method `getIndex` returns the position of a slot within the full TDMA schedule, i.e., its index from the interval $[0, n_{TDMA} - 1]$. A reference to a slot's exclusively owning processor core is returned by method `getOwner` (Line 5). The point in time where a slot starts relative to the beginning of the full TDMA schedule can be obtained via method `getStart` (Line 8). Finally, method `getLength` (Line 11) provides a slot's length l_s .

9.4.4 Priority-Division Bus Arbitration

TDMA is criticized in the literature for its inefficiency, because a slot where its exclusively owning processor core does not request the bus, remains unused, thus leading to idle periods. Priority-division arbitration is a generalization of TDMA, it can be seen as a combination of TDMA and fixed-priority arbitration. Under a priority-division policy, all slots are either exclusively owned by one processor core (as is the case for TDMA, cf. Section 9.4.3), or priorities over various cores are defined (like in fixed-priority arbitration, cf. Section 9.4.1). When it comes to bus requests and arbitration decisions, the arbiter again considers that slot s that is currently active at time t_{curr} relative to the cyclic schedule's very beginning. If this slot is configured to have one exclusive owner o_s and this owner currently requests the bus, bus access is granted to o_s . If owner o_s currently does not request the bus, the current slot s remains idle. However, if the current slot is configured to use priorities, bus access is granted to that processor core having the highest priority among all competing cores.

This arbitration scheme is realized by class WIR_PriorityDivisionBusArbitration within WIR. Multiple inheritance from classes WIR_FixedPriorityBusArbitration and WIR_TDMABusArbitration realizes the combination of fixed-priority and TDMA arbitration mentioned above (cf. Fig. 9.5). Since the APIs of both base

classes are already described earlier in Sections 9.4.1 and 9.4.3, there remain only two details for priority-division arbitration.

Listing 9.44: Characteristic Attributes of Class WIR_PriorityDivisionBusArbitration

```
1 // Get the total number of cores arbitrated by this priority-division bus.
2 virtual unsigned int WIR_PriorityDivisionBusArbitration::getNumberOfArbitratedCores( void ) const override;
3
4 // Get the operation modes per bus slot.
5 const std::vector<WIR_BusArbitrationType> &WIR_PriorityDivisionBusArbitration::getSlotModes( void ) const;
```

First, method `getNumberOfArbitratedCores` (cf. Line 2 of Listing 9.44) already known from both fixed-priority and TDMA arbitration is overridden such that it returns the number of cores that exclusively own some bus slot, or that have non-zero fixed priorities. Second, priority-division introduces the novel method `getSlotModes` (cf. Line 5). This method returns a reference to a vector storing how each slot is actually behaving – either as fixed-priority or as TDMA slot. The elements of the returned vector follow the same slot order as provided by `getSlots` from Line 8 from Listing 9.42. The information which policy is actually used per slot is encoded using the enumeration type `WIR_BusArbitrationType` already introduced in Line 14 of Listing 9.40.

10 The WIR System Model

Within the [WIR](#) library, systems in general model a complete hardware and software system, represented by class `WIR_System`. While the previous two chapters focused on the individual [WIR](#) classes for software components and those ones representing hardware modules, this chapter describes how these software and hardware components are integrated and composed to form complete systems. Section [10.1](#) presents simple, plain-text system configuration files that are used to conveniently construct [WIR](#) systems. The system-level [API](#) to access and iterate both the software and the hardware models is subject of Section [10.2](#). Next, Section [10.3](#) introduces the notion of symbols and describes their management by class `WIR_System`. The relations between the [WIR](#) software and hardware models are finally described in Section [10.4](#), with a particular emphasis on a system's memory layout and the allocation of software components to memories.

10.1 System Configuration Files

The sharp-eyed reader will have noticed that the description of the various classes forming the [WIR](#) hardware model from Chapter [9](#) looks incomplete at first glance. These descriptions mostly present only methods to retrieve and inspect attributes of the various classes, but constructors and methods to set properties of hardware components were omitted. This is motivated by the fact that these constructors and set methods are part of the protected or private [API](#) of their respective classes, while this user's guide solely focuses on the classes' public interfaces. It thus remains to be answered how application code can actually instantiate hardware components with specific properties and compose them to form a complete system.

For the sake of convenience, this process is not driven by [API](#) calls of the [WIR](#) library. Instead, systems and their internal hardware structure based on system components described in Chapter [9](#) are specified using lightweight plain-text system configuration files. An excerpt of such a configuration file was already shown in Listing [1.1](#) on Page [5](#).

A configuration file describes a set of [WIR](#) system components, i.e., the overall system, its processor cores, memory regions, caches, buses and logical memory sections. Each such component is described in one dedicated paragraph within the configuration file. Empty lines and comments (starting with a hash (`'#'`) character and going until the end of a line) are ignored, cf. Lines [6](#) to [9](#) of Listing [1.1](#).

A paragraph always starts with a line specifying the component's unique name, put in brackets. The example from Listing [1.1](#) contains three paragraphs for components named `tc1796` (Line [4](#)), `CORE0` (Line [10](#)) and `PFLASH-C` (Line [19](#)). After a paragraph's name, the properties of a system component are specified using `<key> = <value>` directives, exactly one per line. A mandatory property that must exist in each individual paragraph is the specification of a component's type (cf. Lines [5](#), [11](#) and [20](#)) where this type needs to be one of `system`, `core`, `memory`, `cache`, `bus` or `section`. All other properties that can be specified heavily depend on this component type. A summary of all existing directives for systems, cores and memory regions is provided in Table [10.1](#), while Table [10.2](#) lists all directives for caches, buses and logical sections.

Table 10.1: System Configuration Directives for Systems, Cores and Memory Regions

	Key	Value
Systems	type	system
Cores	type	core
	isa	Name of a processor core's ISA , string <i>Each paragraph of type core must contain an isa directive.</i>
	clockfreq	A core's clock frequency in Hertz, unsigned integer
	voltage	A core's supply voltage in Volts, double
Memory Regions	type	memory
	origin	A region's physical start address, unsigned integer
	length	A region's byte length, unsigned integer <i>For each processor core, the address ranges of all regions accessible by the core must not overlap.</i>
	dynamic_allocation	Whether the used space in a region may vary dynamically at runtime or not, bool ('0' or '1')
	attributes	Region attributes given as combination of case-insensitive letters 'a' (allocatable by the linker), 'c' (cached), 'i' (initialized), 'r' (readable), 'w' (writable), 'x' (executable) <i>Use '!' to invert the very next attribute.</i>
	memory_type	Name of the physical memory holding a region, string
	clockratio	Ratio of a memory region's clock frequency compared to the processor's frequency <i>Use ':' to separate both parts of the ratio.</i>
	cycles	A region's access delay in clock cycles, unsigned integer <i>The delay denotes to number of cycles that elapses until content of the memory is completely read/written.</i>
	burstcycles	A region's burst access delay in clock cycles, unsigned integer <i>Similar to attribute cycles above, but for burst accesses to a memory.</i>
	buswidth	Byte width of the memory bus that connects a region, unsigned integer
	hierarchy	A path through a system's architecture along which some processor core can reach the current memory region, whitespace-separated list of strings <i>A path contains a list of component names, always starts with the name of a processor core, and continues with a sequence of cache and/or bus names. A path implicitly always ends with the current region so that it must not be specified in the path.</i> <i>Use multiple hierarchy directives if several cores in a multi-core setup can access one shared memory region.</i>
	sections	A list of logical section names mapped to a memory region, whitespace-separated list of strings
	load	A list of logical section names that are loaded from the current region (which thus usually is some kind of ROM) to their destination region during a system's boot time, whitespace-separated list of strings

Table 10.2: System Configuration Directives for Caches, Buses and Logical Sections

	Key	Value
Caches	type	cache
	size	Total byte size of a cache, unsigned integer <i>Each paragraph of type cache must contain a size directive.</i>
	associativity	Associativity of a cache, unsigned integer <i>Each paragraph of type cache must contain an associativity directive.</i>
	linesize	Byte size of a cache line, unsigned integer <i>Each paragraph of type cache must contain a linesize directive.</i>
	cache_type	Type of a cache, one case-insensitive letter out of 'd' (data cache), 'i' (instruction cache), 'u' (unified cache)
	enabled	Whether a cache is enabled or not, bool ('0' or '1')
	shared	Whether a cache is shared among multiple cores or not, bool ('0' or '1')
	writethrough	Whether a cache applies the write-through policy or not, bool ('0' or '1')
	writeallocate	Whether a cache applies the write-allocate policy or not, bool ('0' or '1')
	hitdelay	Time in clock cycles for a cache hit in the best case, unsigned integer
	missdelay	Time in clock cycles for a cache miss in the worst case, unsigned integer
	buswidth	Byte width of the memory bus that connects a cache, unsigned integer
	percent_size	A cache's total size as a percentage of a task's size, unsigned integer
Buses	type	bus
	arbitration	Arbitration policy of a bus, one case-sensitive string of out "fp" (fixed-priority), "rr" (round-robin), "tdma" (TDMA), "pd" (priority-division)
	priorities	Priority ranking of cores, whitespace-separated list of core names <i>Cores at the beginning of the list have high priority, cores at the tail have low priority. Any cores that are not listed get priority 0, i.e., they will never get bus access. This attribute is only valid for "fp" or "pd" arbitration.</i>
	slot	For "tdma" arbitration: Specification of a TDMA bus slot, whitespace-separated 2-tuple of string and unsigned int <i>For a TDMA bus slot, the name of the processor core owning the slot and the slot's length in clock cycles need to be provided.</i> For "pd" arbitration: Specification of a priority-division bus slot, whitespace-separated tuple <i>For a priority-division bus slot, its internal arbitration policy has to be specified first, i.e., "fp", "rr" or "tdma". If a slot is managed using "fp" or "rr", the slot length in clock cycles has to be specified next. In case of "tdma", the owning processor core's name comes next, followed by the slot length.</i> <i>Use multiple slot directives to specify all bus slots, one after the other.</i>
	arbitrationdelay	The bus arbiter's arbitration delay in clock cycles, unsigned integer
Logical Sections	type	section
	alignment	A section's external alignment, unsigned integer <i>The external alignment specifies how many least-significant bits of a section's start and end address must be 0.</i>
	block	A section's internal alignment, unsigned integer

Listing 10.1: Example for Shared Memory Hierarchy Configuration

```

1 #
2 # Main Memory shared between two cores
3 #
4 [MAINMEM]
5 type          = memory
6 [...]
7 sections      = .text .data
8 hierarchy     = CORE0 L1-C0 BUS
9 hierarchy     = CORE1 L1-C1 BUS

```

For type = system, only the name of a system has to be specified, no other directives can be given for systems. The directives for type core cover the few very basic properties of processor cores shown in Table 10.1, their meaning is straightforward and requires no additional explanation.

The directives currently supported for type memory correspond with the attributes of class WIR_MemoryRegion shown in Listing 9.35. The hierarchy directive is worth mentioning as this is the key to specifying systems with complex memory hierarchies. Assuming, e.g., a dual-core architecture with private L1 caches per core connected to a shared bus, and this bus also includes a shared main memory region, then this region would come with two hierarchy directives as shown in Listing 10.1. The first one from Line 8 states that the memory region named MAINMEM can be reached by processor core CORE0 via some system component named L1-C0 being the private cache of CORE0, followed by the shared interconnect named BUS. Likewise for Line 9 that refers to CORE1 and its private cache L1-C1.

Listing 10.2 shows another example specifying a system where parts of some [Static Random-Access Memory \(SRAM\)](#) memory are cached while other parts of the same memory are not covered by the cache. For this purpose, two different memory regions are described in Lines 4 and 15. The regions' address spaces are assumed to be disjoint, and the cached region I-SRAM-C explicitly carries the 'C' (cached) attribute (cf. Line 18) while this attribute is missing from Line 7 for region I-SRAM-NC. As can be seen from Lines 10 and 21, the hierarchy

Listing 10.2: Example for Cached and Uncached Memory Hierarchy Configuration

```

1 #
2 # Program RAM with uncached access
3 #
4 [I-SRAM-NC]
5 type          = memory
6 origin        = 0x20000000
7 attributes    = RWXA
8 [...]
9 sections      = .text_uncached .text
10 hierarchy    = CORE0 BUS
11
12 #
13 # Program RAM with cached access
14 #
15 [I-SRAM-C]
16 type          = memory
17 origin        = 0x20800000
18 attributes    = RWXAC
19 [...]
20 sections      = .text_cached
21 hierarchy    = CORE0 L1-C0 BUS

```

Listing 10.3: Constructors and Methods for Name Handling of Class WIR_System

```

1 // Default constructor creating a system from a given system configuration.
2 WIR_System::WIR_System( const std::string &n, const WIR_TaskManager &t );
3
4 // Copy constructor.
5 WIR_System::WIR_System( const WIR_System &s );
6
7 // Move constructor.
8 WIR_System::WIR_System( WIR_System &&s );
9
10 // Get a system's architecture name.
11 std::string WIR_System::getSystemName( void ) const;

```

directives of both regions also differ, such that region I-SRAM-C includes the cache component named L1-C0. Finally, both regions also carry different sections directives (cf. Lines 9 and 20). For the given example, two logical sections named `.text_uncached` and `.text` are mapped to region I-SRAM-NC, while a dedicated section named `.text_cached` is assigned to region I-SRAM-C.

The sections directive of memory regions is important since it connects the WIR software model (cf. Chapter 8) with the hardware model from Chapter 9 regarding a system's memory layout. Software components like, e.g., functions or basic blocks are assigned to logical sections, and these sections in turn are assigned to physical memory regions. This way, it is always well-defined within WIR which physical memory is used to store which software component. Thus, the corresponding memory properties can be considered for, e.g., address computations or execution time analysis. More details on the overall WIR memory layout will follow in Section 10.4. Coming back to the example from Listing 10.2, WIR functions can easily be cached or uncached by assigning them to sections `.text_cached` or `.text_uncached`, resp.

The system configuration files introduced above are finally linked to class WIR_System via its default constructor shown in Line 2 of Listing 10.3. This constructor expects a name of a system configuration file as first argument, followed by a reference to a task manager instance (note the future work on task managers already mentioned earlier on Page 2). Line 13 of Listing 5.3 (cf. Page 27) already showed an example how to use this constructor. The string `n` passed to this constructor is interpreted as follows in order to find the actual configuration file to be used:

- If `n` is already a path to a configuration file, i.e., if `n` contains at least some `'/'` character somewhere, this string is considered to be a proper – either absolute or relative – name where in the file system the desired configuration file resides. In this case, the given string is simply taken and class WIR_System attempts to open the respective file.
- If `n` does not contain some `'/'` character, class WIR_System checks each ISA-specific directory in the full software package underneath directory WIR/arch (cf. Fig. 3.1), as well as each corresponding sub-directory after WIR installation underneath `<INSTALLDIR>/etc` (cf. Fig. 5.1). The first of these directories that contains a configuration file with name `n` is finally used by WIR_System.
- If the previous steps do not lead to some valid configuration file, the construction of a WIR system fails with a runtime error.

Next, class WIR_System processes the specified configuration file paragraph by paragraph and dynamically creates all system components of the appropriate types and with all attributes as specified by the configuration. Class WIR_System also comes with standard copy and move constructors as shown in Lines 5 and 8.

The name that is given to a complete system in a configuration file, e.g., "tc1796" from Line 4 of Listing 1.1 can be retrieved using method `getSystemName`, cf. Line 11 of Listing 10.3.

10.2 Integration of Software and Hardware Models

The main purpose of class `WIR_System` is to tightly integrate both the [WIR](#) software and the hardware models at the system level. Only such a tight integration allows to reason about the mutual interactions between software and hardware aspects, and to systematically consider these interactions during code analyses and optimizations. A first important step towards such a software/hardware integration is to provide convenient access to all software and hardware components by class `WIR_System`. The top-level classes of the [WIR](#) software and hardware models are `WIR_CompilationUnit` (cf. Fig. 1.2) and `WIR_SystemComponent` (cf. Fig. 1.3). This section thus describes the [API](#) provided by `WIR_System` to handle both compilation units and all kinds of system components.

Regarding the integration of the software model, there is not too much to say at this place, except that `WIR_System` provides the complete list [API](#) from Chapter 6 for the management of compilation units.

This, however, is not the case for hardware components, since the full-fledged and generic list [API](#) usually is not necessary, but additional specific methods become relevant. For this purpose, Listing 10.4 shows the [API](#) dedicated to the systematic access of system components. As can be seen from Lines 2 and 5, there are two variants of the `getComponents` method. As usual, the first one (Line 2) returns a reference to the complete set of all components of any type that make up a system. This returned set of type `WIR_SystemComponentSet` is basically declared as `std::set<std::reference_wrapper<WIR_SystemComponent>>` within [WIR](#). The second variant (Line 5) expects a type name derived from `WIR_SystemComponent` as template argument, e.g., `WIR_MemoryRegion` or `RV32IMC`. It returns a set of (wrapped) references to only those hardware components of a system that can dynamically be casted to the specified type name.

The `containsComponent` methods from Lines 8 and 11 return whether components with a given unique ID or with a given name exist within a system, resp. Similarly, the position of such components can be retrieved using the `findComponent` methods from Lines 14 and 17. They both return an iterator referring to the found position within the set of all components provided by `getComponents` (cf. Line 2). If no component with the given ID or name exists, `findComponent` returns the `end()` iterator.

For the convenient lookup specifically of memory regions, class `WIR_System` provides the `findMemoryRegion` method from Line 20. It expects a memory address as argument and returns an iterator referring to that region that includes this address within the region's address space. If no such region is found, the `end()` iterator is returned again. In order to identify all those regions that somehow cover a given address range, method `findMemoryRegions` (Line 23) can be used. It returns a set of system components that includes all those regions whose address space has a non-empty intersection with the given address range.

Finally, method `getComponentHierarchyLevels` (Line 26) provides a structured access to all system components according to their position within a system's full memory hierarchy. Based on the hierarchy directives already introduced in Section 10.1, paths from processor cores to memory regions are specified along which components like, e.g., caches or buses can be arranged. Using these paths, class `WIR_System` determines at which level within the entire memory hierarchy a component actually resides, where level 0 contains all processor cores, level 1 contains components that directly follow a core from level 0, etc. `getComponentHierarchyLevels` now returns a reference to a map that gets an unsigned integer denoting the hierarchy level (i.e., 0 or 1 etc.) as key. With such a key, a set of system components is associated by the provided map that all reside at the given level of the memory hierarchy.

Listing 10.4: System Component Handling of Class WIR_System

```

1 // Get the set of all system components.
2 const WIR_SystemComponentSet &WIR_System::getComponents( void ) const;
3
4 // Get the set of all system components of a specified type.
5 template<typename T> const std::set<std::reference_wrapper<T>> WIR_System::getComponents( void ) const;
6
7 // Determine whether a system contains a component with the given ID.
8 bool WIR_System::containsComponent( WIR_id_t id ) const;
9
10 // Determine whether a system contains a component with the given name.
11 bool WIR_System::containsComponent( const std::string &n ) const;
12
13 // Find the system component with the given ID.
14 WIR_SystemComponentSet::const_iterator WIR_System::findComponent( WIR_id_t id ) const;
15
16 // Find the system component with the given name.
17 WIR_SystemComponentSet::const_iterator WIR_System::findComponent( const std::string &n ) const;
18
19 // Find a memory region that includes a given address.
20 WIR_SystemComponentSet::const_iterator WIR_System::findMemoryRegion( const WIR_MemoryAddress &a ) const;
21
22 // Find all memory regions that intersect with the given address range.
23 WIR_SystemComponentSet WIR_System::findMemoryRegions( const WIR_AddressRange &a ) const;
24
25 // Get all system components, grouped by layers within a system's component hierarchy.
26 const std::map<unsigned int, WIR_SystemComponentSet> &WIR_System::getComponentHierarchyLevels( void ) const;

```

10.3 Symbols

The **WIR** software model is a symbolic representation of code where symbols represent actual memory addresses of pieces of code or data. Despite the fact that the **WIR** hardware model is explicitly designed to reason about a system's memory layout and about physical addresses of code or data fragments (cf. Section 10.4 for an in-depth discussion of the **WIR** memory layout), it is desirable to keep the software code free of memory addresses as long as possible. Using hard-coded memory addresses in **WIR** operations would be difficult to adapt as soon as some new operations were inserted or an optimization eliminates code, which would result in changes of such pre-computed addresses.

A symbol within **WIR** is an abstraction of some real memory address. Symbols typically come into play in the context of:

- function calls where the called function is specified in a symbolic, human-readable way;
- conditional or unconditional jumps where a jump target is given by a basic block's name;
- memory loads or stores where some data object is accessed symbolically.

This notion of symbols has already been introduced earlier in Section 8.6.3 where label parameters and their properties have been discussed. Strictly speaking, there is no conceptual difference at all between labels and symbols within **WIR**. Label parameters, however, are derived from the base class `WIR_Parameter`, and their sole purpose is to represent symbolic addresses as explicit parameters of machine operations. Thus, labels actually enable the proper control and data flow analysis of code within **WIR**. Furthermore, label parameters are not unique in that sense that one and the same symbolic address can be represented by several label parameters, e.g., if the same function is called from different **WIR** operations.

Listing 10.5: Constructors of Class WIR_Symbol

```

1 // Default constructor creating a basic block symbol.
2 explicit WIR_Symbol::WIR_Symbol( const WIR_BasicBlock &b );
3
4 // Default constructor creating a data object symbol.
5 explicit WIR_Symbol::WIR_Symbol( const WIR_Data &d );
6
7 // Default constructor creating a function symbol.
8 explicit WIR_Symbol::WIR_Symbol( const WIR_Function &f );
9
10 // Copy constructor.
11 WIR_Symbol::WIR_Symbol( const WIR_Symbol &s );
12
13 // Move constructor.
14 WIR_Symbol::WIR_Symbol( WIR_Symbol &&s );

```

The purpose of symbols is different in that sense that they aim to link the [WIR](#) software model with the hardware model such that memory properties can be taken into account when accessing a symbol, and the visibility of symbols and other linkage properties can be controlled. In addition, symbols are unique so that every symbolic address is represented as symbol exactly once within [WIR](#). For these reasons, [WIR](#) uses the class `WIR_Symbol` to model symbols, while class `WIR_LabelParameter` represents label parameters.

Due to their conceptual similarity, it is not surprising that the constructors and all methods for type, reference and name handling of class `WIR_Symbol` (cf. Listings 10.5 and 10.6) are identical with the corresponding [API](#) of class `WIR_LabelParameter` already shown earlier in Section 8.6.3. We thus omit the discussion of these methods here for the sake of brevity.

A symbol's main properties are of Boolean nature as can be seen from Listing 10.7. The set methods (cf. Lines 2, 8, 14 and 20) thus expect a Boolean flag as only argument that always defaults to `true`. The current state of a symbol's property can be retrieved with the `is` methods shown in Lines 5, 11, 17 and 23. Symbols can be constant and may only be read, which is covered by methods `setConst` and `isConst` (Lines 2 and 5). External symbols are symbols that might be accessed from some [WIR](#) function, but that are actually declared outside the entire code base modeled using [WIR](#). Methods `setExtern` and `isExtern` (Lines 8 and 11) can be used to control this property. Next, symbols might be global or local, i.e., they are visible globally for all compilation units of a system's code base, or just locally within that one compilation unit where a symbol actually resides. This

Listing 10.6: Type, Reference and Name Handling of Class WIR_Symbol

```

1 // Get a symbol's type.
2 WIR_SymbolType WIR_Symbol::getType( void ) const;
3
4 // Get the basic block to which a symbol refers, if any.
5 WIR_BasicBlock &WIR_Symbol::getBasicBlock( void ) const;
6
7 // Get the data object to which a symbol refers, if any.
8 WIR_Data &WIR_Symbol::getData( void ) const;
9
10 // Get the function to which a label refers, if any.
11 WIR_Function &WIR_Symbol::getFunction( void ) const;
12
13 // Get the symbol's name.
14 std::string WIR_Symbol::getName( void ) const;

```


Listing 10.7: Property Handling of Class WIR_Symbol

```

1 // Set whether a symbol is constant/read-only.
2 void WIR_Symbol::setConst( bool c = true );
3
4 // Determine whether a symbol is constant/read-only.
5 bool WIR_Symbol::isConst( void ) const;
6
7 // Set whether a symbol is extern.
8 void WIR_Symbol::setExtern( bool e = true );
9
10 // Determine whether a symbol is extern.
11 bool WIR_Symbol::isExtern( void ) const;
12
13 // Set whether a symbol is global.
14 void WIR_Symbol::setGlobal( bool g = true );
15
16 // Determine whether a symbol is global.
17 bool WIR_Symbol::isGlobal( void ) const;
18
19 // Set whether a symbol is volatile.
20 void WIR_Symbol::setVolatile( bool v = true );
21
22 // Determine whether a symbol is volatile.
23 bool WIR_Symbol::isVolatile( void ) const;

```

attribute is managed by methods `setGlobal` and `isGlobal` (Lines 14 and 17). Finally, volatile symbols might exist that can be read or modified asynchronously by something other than the current WIR code base. Methods `setVolatile` and `isVolatile` (Lines 20 and 23) can be used to handle such volatile symbols.

Since symbols are unique as motivated above, their management is another key aspect of the WIR system level. Symbols refer to basic blocks, data objects or functions. Whenever new objects of classes `WIR_BasicBlock`, `WIR_Data` or `WIR_Function` are somewhere somehow added to a system, new symbols are added to class `WIR_System` fully automatically. This automatic creation of symbols inherently respects the complete hierarchy of the WIR software model from Chapter 8. This means that if some new function is inserted into a compilation unit that is already part of some system, not only a corresponding function symbol is created, but also symbols for all basic blocks within the newly added function. Similar procedures apply when inserting compilation units into systems. Likewise, when some basic block, data object or function gets removed from a WIR system, its symbol is also automatically removed. For these reasons, the public API of class `WIR_System` does not come with any method to have symbols added or removed explicitly by application code.

Thus, symbol management simply reduces to checking the presence of and finding symbols within a WIR system, as can be seen from Listing 10.8. The API of class `WIR_System` here basically consists of the `getSymbols` method (cf. Line 2), a couple of `containsSymbol` methods (Lines 5 to 20), and several variants of `findSymbol` methods (Lines 23 to 38).

As usual, method `getSymbols` provides access to all symbols that are currently known to a system. The returned set of symbols is basically declared as `std::set<std::reference_wrapper<WIR_Symbol>>`.

The `containsSymbol` methods provide the information whether a certain symbol exists in the context of a WIR system or not. The variant from Line 5 allows to check the presence of a symbol via its unique numerical ID. In order to directly verify the existence of some actual symbol, the method from Line 8 can be used that expects a constant reference to the symbol to be checked as argument. As motivated earlier, symbols refer to either basic blocks, data objects or functions. Thus, it regularly happens that the presence of a symbol for some actual basic

Listing 10.8: Symbol Handling of Class WIR_System

```

1 // Get the set of all symbols of a system.
2 const WIR_SymbolSet &WIR_System::getSymbols( void ) const;
3
4 // Determine whether the system contains a symbol with the given ID.
5 bool WIR_System::containsSymbol( WIR_id_t id ) const;
6
7 // Determine whether the system contains a given symbol.
8 bool WIR_System::containsSymbol( const WIR_Symbol &s ) const;
9
10 // Determine whether the system contains symbol for a given basic block.
11 bool WIR_System::containsSymbol( const WIR_BasicBlock &b ) const;
12
13 // Determine whether the system contains symbol for a given data object.
14 bool WIR_System::containsSymbol( const WIR_Data &d ) const;
15
16 // Determine whether the system contains symbol for a given function.
17 bool WIR_System::containsSymbol( const WIR_Function &f ) const;
18
19 // Determine whether the system contains symbol with a given name.
20 bool WIR_System::containsSymbol( const std::string &n ) const;
21
22 // Find the symbol with the given ID.
23 WIR_Symbol &WIR_System::findSymbol( WIR_id_t id ) const;
24
25 // Find the symbol for the given basic block.
26 WIR_Symbol &WIR_System::findSymbol( const WIR_BasicBlock &b ) const;
27
28 // Find the symbol for the given data object.
29 WIR_Symbol &WIR_System::findSymbol( const WIR_Data &d ) const;
30
31 // Find the symbol for the given function.
32 WIR_Symbol &WIR_System::findSymbol( const WIR_Function &f ) const;
33
34 // Find the symbol with the given name.
35 WIR_Symbol &WIR_System::findSymbol( const std::string &n ) const;
36
37 // Find the symbol with the given address.
38 WIR_Symbol &WIR_System::findSymbol( const WIR_MemoryAddress &a ) const;

```

block, data object or function needs to be checked. For this purpose, class WIR_System provides the methods from Lines 11, 14 and 17, resp. Finally, the containsSymbol method from Line 20 can be used to check for a symbol using a string denoting the symbol's name.

In a very similar manner, the findSymbol methods allow to directly obtain a reference to a found symbol, based on its ID (Line 23), on a given basic block, data object or function (Lines 26, 29 and 32), or based on the symbol name (Line 35). Finally, there is a findSymbol variant (Line 38) that can be used to obtain that symbol that occupies a given memory address. In other words, the given address must lie in the interval between the found symbol's base address and the symbol's size. If an address is not covered by some symbol at all, this findSymbol variant produces a runtime error. Similarly, runtime errors are generated by all findSymbol methods if no actual symbol could be found at all. Thus, it is good practice to precede each use of findSymbol by a corresponding call of containsSymbol.

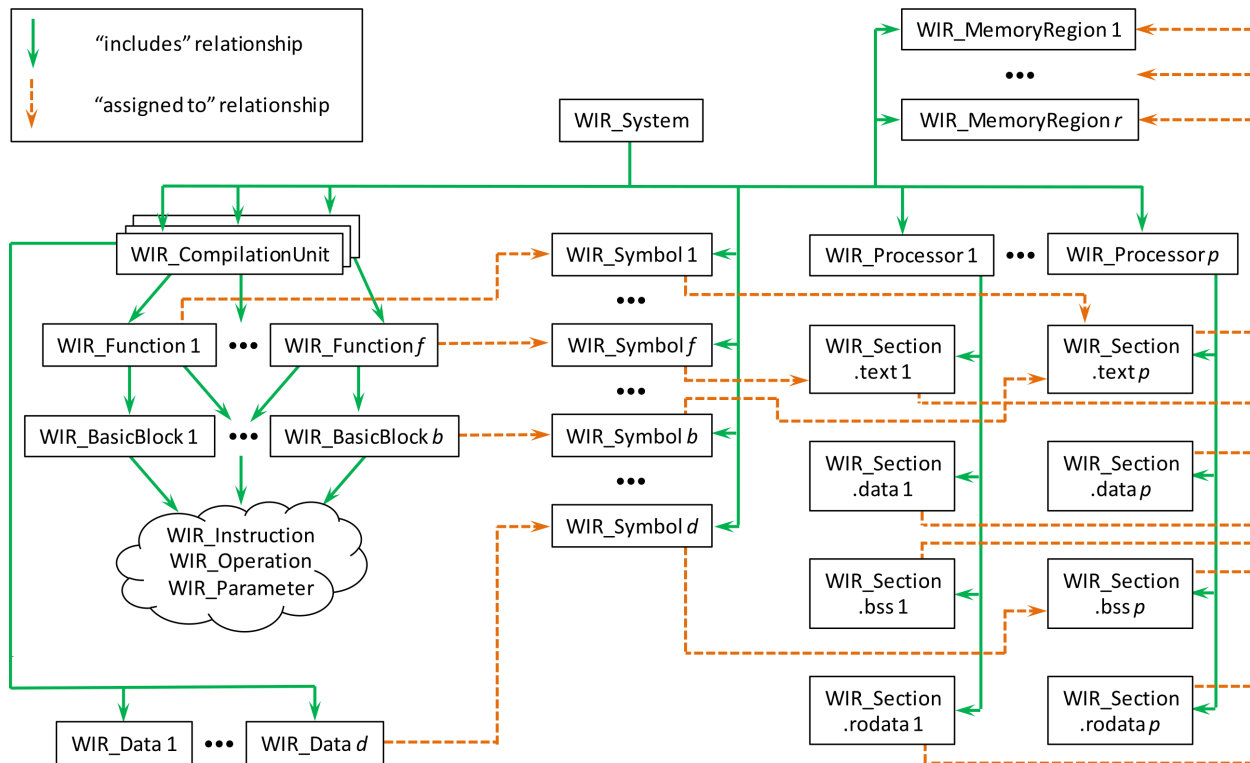


Figure 10.1: System-Level WIR Memory Layout

10.4 Memory Layout and Allocation

This section now puts together all the concepts presented in the previous chapters and discusses the overall **WIR** memory layout. For this purpose, Fig. 10.1 illustrates the system-level relationships between the software model from Chapter 8 and the hardware model from Chapter 9 with a special focus on symbols as introduced in Section 10.3.

The left part of Fig. 10.1 depicts the **WIR** software model according to which a system can contain multiple compilation units, which in turn contain several functions (cf. Section 8.2) consisting of basic blocks (cf. Section 8.3) etc. Furthermore, compilation units can feature various data objects (Section 8.7).

The right part of the above figure shows those components of the hardware model that are directly involved in a system's memory layout. As can be seen, a system features one or multiple processor cores (Section 9.1.1). As introduced in Section 9.1.1.8, each processor core features logical executable sections. By default, at least the sections `.text`, `.data`, `.bss` and `.rodata` must exist. Additional sections can be created, but this is omitted in Fig. 10.1 for the sake of simplicity. Besides processors, a system can include several physical memory regions (cf. Section 9.2).

As discussed in Section 10.1, the setup of the **WIR** hardware model and the creation of all the various objects representing processors, memories, caches etc. is driven by system configuration files. Here, logical sections are assigned to memory regions using the sections directives (cf. Table 10.1). This kind of assignments (and many others) between **WIR** objects at the software, hardware, and system level is shown in Fig. 10.1 using the dashed orange arrows. The effect of the sections directive is represented in the figure by the arrows leading from all the sections to the memory regions.

Listing 10.9: Section Handling of Class WIR_Symbol

```

1 // Set the logical section to which a symbol is assigned.
2 void WIR_Symbol::setSection( const WIR_Section &s );
3
4 // Remove any prior explicit assignment of a symbol to a logical section.
5 void WIR_Symbol::unsetSection( void );
6
7 // Get the logical section to which a symbol is assigned.
8 const WIR_Section &WIR_Symbol::getSection( void );
9
10 // Get a symbol's base address.
11 WIR_MemoryAddress WIR_Symbol::getBaseAddress( void ) const;

```

Section 10.3 has motivated that each function, basic block and data object is represented by a unique symbol at the system level. This unique assignment of the software model's objects to symbols is illustrated by the dashed arrows leading from the software model to the symbols in the left half of Fig. 10.1.

The connection between the software and the hardware models in WIR is now established by the assignment of symbols to logical sections. This kind of assignments has not yet been discussed in detail before, but it is visualized in Fig. 10.1 by the links from each symbol to one of the available logical sections. These assignments are thus the missing links that close the logical chain from software objects via symbols and sections to physical memories. The API of class WIR_Symbol for the assignment of symbols to sections is shown in Listing 10.9.

Method setSection (cf. Line 2) can be used to explicitly assign a symbol to a given section. Such an explicit assignment is performed according to the following rules:

1. If the symbol to be assigned refers to a function, any explicit section assignments of the function's basic blocks are unset.
2. Otherwise, if the symbol to be assigned refers to a basic block, any explicit section assignment of the function to that the basic block belongs is unset. Section assignments of all other basic blocks of that very same function remain unchanged.
3. The current symbol itself is finally assigned to the given section.

These rules ensure for functions and basic blocks that explicit section assignments only exist either for a function or for a function's basic blocks or for none of them, but never for both. If the object represented by a symbol must not be modified (cf. Section 6.3), setSection fails with a runtime error.

Previously performed assignments of symbols to sections can be erased using method unsetSection (Line 5).

The current assignment of a symbol to a section can be retrieved with the help of method getSection (Line 8) that returns a const reference to the assigned WIR section. getSection follows the rules below in order to determine a symbol's section:

1. If the symbol has explicitly been assigned to some section before using setSection, this explicitly specified section is returned.
2. Otherwise, if the symbol refers to a basic block and that block's owning function has been assigned to a section, the function's section is returned.
3. Otherwise, if the symbol refers to a function and the function's basic blocks have been assigned to sections, the section of the very first entry basic block of the function is returned.
4. Otherwise, if no explicit section assignment exists, a default section is determined by getSection as follows:

Listing 10.10: Data Layout Handling of Classes WIR_System and WIR_CompilationUnit

```

1 // Get a map grouping together all data objects of a system to their respective sections.
2 WIR_DataLayoutMap WIR_System::getDataLayout( void );
3
4 // Get a map grouping together all data objects of a compilation unit to their respective sections.
5 WIR_DataLayoutMap WIR_Compilation::getDataLayout( void );

```

- a) If the symbol refers to a function or basic block, the default section `.text` of the first processor core within the [WIR](#) system is returned.
- b) Otherwise, the symbol refers to a data object. In this case,
 - section `.rodata` of the system's first processor core is returned if the symbol is constant;
 - otherwise, section `.data` of the system's first processor core is returned if the data object is initialized;
 - otherwise, section `.bss` of the system's first processor core is returned.

Finally, method `getBaseAddress` (Line 11) can be used to conveniently determine a symbol's start address in memory.

The interface shown in Listing 10.9 now enables [WIR](#) application code to easily perform explicit memory allocation of code or data fragments. Listing 10.2 on Page 122 showed an example of a system configuration file supporting both cached and uncached parts of a memory hierarchy. This hierarchy can be exploited by some [Link-Time Optimization \(LTO\)](#) that explicitly allocates code fragments either to the cached or the uncached memory region. Such memory allocation within [WIR](#) is as easy as a simple call of method `setSymbol` and passing either the section named `.text_cached` or `.text_uncached`.

Methods `getDataLayout` shown in Listing 10.10 provide a convenient means to obtain the section assignments specifically for data objects. They both return a map that maps a given [WIR](#) section used as key to the list of references to all data objects currently assigned to the given section. In the case of the method from Line 2, this map contains all data objects of the whole [WIR](#) system, while the variant from Line 5 does the same but just per compilation unit. This map is basically declared as `std::map<std::reference_wrapper<WIR_Section>, std::list<std::reference_wrapper<WIR_Data>>>`.

Such section assignments are not only useful in the context of [LTO](#), but it can also be used for the convenient allocation of entire software tasks to processor cores in multi-core systems. In such a multi-core setup, [WIR](#) assumes that independent [ELF](#) executable files are generated for all the cores of a system. As discussed in Section 9.1.1.8, these executables all consist of various logical sections owned by their unique processor core, and all these sections are assigned to their respective physical memory regions (cf. Section 9.2). These relations are also depicted in Fig. 10.1 where independent sections `.text`, `.data`, `.bss` and `.rodata` exist for each processor core within a system. By now assigning, e.g., a [WIR](#) function to the `.text` section associated with processor core *p*, method `setSection` from Line 2 of Listing 10.9 is also key to the allocation of software to cores.

Now that all aspects of memory layout and allocation at the [WIR](#) system level have been introduced, the only things that remain to be discussed are some details of system configuration files (cf. Section 10.1) in the context of multi-core systems. Here, it is important to note that all the specified sections of a memory region must also be related to the system's cores. Listing 10.1 on Page 122 showed a system configuration for a memory region shared between two processor cores named `CORE0` and `CORE1`. Line 7 specifies that this region gets sections for code and data assigned. Since such sections need to exist within [WIR](#) for each involved processor core, the

meaning of Listing 10.1 is that region MAINMEM accomodates a total of 4 sections, i.e., .text and .data for both cores CORE0 and CORE1.

Generally speaking, logical sections are created by class WIR_System per region for all items listed in a region's section directive AND for all cores occuring in a region's hierarchy directives. Effectively, the cross-product over all sections and cores from a region's section and hierarchy directives is constructed.

Table 10.2 furthermore revealed that basic section attributes can also be specified in system configuration files. Now that it became evident that sections are inherently created per region and per involved core, the finally remaining open question is how to specify section attributes, e.g., specifically for the data section associated with CORE1 from Listing 10.1. For single-core architectures, this is very simple by just providing a paragraph starting with the name [.data] in the configuration file. This paragraph is unambiguous, since there is only one processor core in a single-core system and this data section can thus only be assigned to that one core. For multi-core systems, however, it has to be specified unambiguously which section of which processor core is meant by a configuration file's paragraph. Thus, both section and core name need to be specified, separated by a whitespace, e.g., [.data CORE1] for the above example referring to Listing 10.1.

11 Basic Set and Map Data Structures

The APIs of the various classes presented in the previous Chapters 8 to 10 regularly use sets containing WIR objects like, e.g., WIR_BasicBlockSet, WIR_SectionSet or WIR_SymbolSet. In general, sets of objects as well as maps that associate specific additional data with objects used as keys play an important role. As the WIR library is implemented in C++17, it is natural to use standard STL containers like `std::set` and `std::map` for this purpose. Of course, such containers should not contain the actual WIR objects as this would result in severe copying overheads and the risk of corrupt data structures. Thus, a straightforward realization of, e.g., a set of basic blocks would be to use a set of pointers to basic blocks such as `std::set<WIR_BasicBlock*>`. However, WIR is designed to never publicly expose such kinds of data structures for the following reasons:

- As motivated in Section 1.2 on Page 6, the API of WIR is intentionally kept free of pointers as much as possible. Instead, wrapped references are heavily used in order to avoid memory mismanagement, which is why the standard class `std::reference_wrapper` regularly appeared in earlier chapters.
- Such sets or maps of WIR objects are regularly traversed and iterated. This, however, is non-deterministic if pointers are used as keys. Internally, these STL sets and maps are ordered by the actual addresses of the pointers. These addresses are determined by the host system's operating system and cannot be controlled within the WIR library. Thus, depending on whether a WIR application gets executed on this or the other host machine, or whether the host's current memory usage is high or low, completely different addresses are assigned to WIR objects. This may lead to varying, non-deterministic orders of objects within their sets or maps and thus in different orders how WIR iterates and thus processes them. Such behavior opens the door to numerous kinds of undesired behaviors and bugs that are extremely hard to debug.

In order to address the issues above, WIR features deterministic mechanisms that allow to obtain a unique numerical identifier for (almost) all kinds of WIR objects. These identifiers and their underlying API are specified in Section 11.1. Section 11.2 describes the way how these IDs are used to declare deterministic set- and map-based data structures.

11.1 Unique Numerical Identifiers

In a straightforward data structure like, e.g., the abovementioned `std::set<WIR_BasicBlock*>`, the actual address of a pointer serves as unique key. This bases on the assumption that WIR objects such as basic blocks all have different and unique addresses in memory so that they are uniquely identified by their pointers' addresses. In order to remove pointers as unique identifiers from deterministic WIR data structures, some other mechanism is required that allows to clearly distinguish between two WIR objects and to arrange them properly in ordered data structures such as sets or maps.

This mechanism providing unique numerical identifiers for WIR objects is realized by class WIR_ID_API. This is a very simple base class from which all the classes presented in Chapters 6 to 10 (and many others) publicly inherit so that its API shown in Listing 11.1 is available (almost) everywhere within WIR. This class internally maintains a static counter that is incremented every time that a new object of a derived class is constructed. The current counter value is stored for each newly constructed object, thus representing a unique ID of each WIR object.

Listing 11.1: Handling of Unique Object Identifiers by Class WIR_ID_API

```

1 // Type WIR_id_t represents unique numerical IDs of WIR objects.
2 using WIR_id_t = unsigned long long;
3
4 // Determine whether the IDs of two WIR objects are equal.
5 virtual bool WIR_ID_API::operator == ( const WIR_ID_API &i ) const;
6
7 // Determine whether the IDs of two WIR objects are unequal.
8 virtual bool WIR_ID_API::operator != ( const WIR_ID_API &i ) const;
9
10 // Get a WIR object's unique ID.
11 WIR_id_t WIR_ID_API::getID( void ) const;

```

The data type `WIR_id_t` that is used everywhere within `WIR` to represent IDs is declared to be `unsigned long long` (cf. Line 2). Two operators `==` and `!=` for equality and inequality tests are declared in Lines 5 and 8. These operators do not do any kind of semantical test whether two `WIR` objects are equal or unequal. Instead, only the IDs of both objects are compared. If this behavior is undesired, both operators can be overridden in derived classes, since they are declared to be virtual. Finally, method `getID` (Line 11) can be used to retrieve an object's unique identifier.

11.2 Deterministic WIR Data Structures

Whenever sets of `WIR` objects have been introduced in earlier chapters, this guide on purpose used the formulation “is basically defined as `std::set<std::reference_wrapper<...>>` within `WIR`”, see e.g., the running example `WIR_BasicBlockSet` on Page 46. This, however, is not the complete truth, as this formulation lacks any implementation details making such a set (or map) deterministic.

To achieve this property, `WIR` exploits that the standard `STL` containers for sets and maps can be used in combination with a custom C++ comparison class that in turn determines the ordering of elements within a set or map.

Listing 11.2: Declaration of Deterministic Data Structures in file `wirtypes.h`

```

1 // Comparison class to sort sets and maps of WIR objects uniquely by their IDs.
2 template<typename T> struct WIR_Compare
3 {
4     inline bool operator()( const std::reference_wrapper<T> &lhs, const std::reference_wrapper<T> &rhs ) const
5     {
6         return( lhs.get().getID() < rhs.get().getID() );
7     };
8 };
9
10 // WIR_BasicBlockSet represents deterministic sets of references to WIR basic blocks.
11 using WIR_BasicBlockSet = std::set<std::reference_wrapper<WIR_BasicBlock>, WIR_Compare<WIR_BasicBlock>>;
12
13 // WIR_SectionSet represents deterministic sets of references to WIR sections.
14 using WIR_SectionSet = std::set<std::reference_wrapper<WIR_Section>, WIR_Compare<WIR_Section>>;
15
16 // WIR_SymbolSet represents deterministic sets of references to WIR symbols.
17 using WIR_SymbolSet = std::set<std::reference_wrapper<WIR_Symbol>, WIR_Compare<WIR_Symbol>>;
18
19 ...

```


This comparison class as well as all basic deterministic data structures of [WIR](#) are declared centrally in the source directory in file `WIR/wir/wirtypes.h`. An excerpt of this header file is given in Listing [11.2](#).

The comparison class shown in Lines [2](#) to [8](#) is one of the very few cases of a template-based class within [WIR](#). It allows to perform generic less-than comparisons for any kind of class `T` that is derived from `WIR_ID_API`. This comparison class gets two (wrapped) references as argument in Line [4](#) and simply performs a numerical less-than comparison of the objects' unique and deterministic IDs (Line [6](#)).

This comparison class is then used in the declarations of [WIR](#) sets of, e.g., basic blocks, sections or symbols. This is shown in Lines [11](#) to [17](#) where the generic comparison class is specialized towards, e.g., class `WIR_BasicBlock`. The [STL](#) containers resulting from these declarations now internally sort all their entries according to their IDs which, as motivated in Section [11.1](#), no longer on [WIR](#)-external effects such as host memory addresses.

The entire [WIR](#) library internally uses only such deterministic data structures whenever sets or maps need to be iterated. It is good practice that application code using [WIR](#) also follows this scheme and uses the comparator class from Listing [11.2](#).

12 Code Generation

While all the previous chapters of this guide described how to construct, use and manipulate objects of the various classes forming the [WIR IR](#), another central aspect of an [IR](#) is to generate actual code from it for further processing by, e.g., assemblers or linkers. Section [12.1](#) thus puts a focus on the generic mechanisms of [WIR](#) to emit its code in a processor-independent and easily comprehensible format. Manipulators that allow to increase or decrease the level of detail of such code output are presented in Section [12.2](#). Section [12.3](#) discusses how to adapt this code output such that syntactically correct assembly output for the processor architectures presented in Sections [9.1.3](#) to [9.1.5](#) can be obtained. The [WIR](#) system model features aspects of a global memory layout and allocation (cf. Section [10.4](#)) which are taken up in Section [12.4](#) that describes the generation of tailored linker scripts for a given memory allocation.

12.1 Generic Stream Output

Generating code from a given [WIR](#) structure is as easy as a "Hello world" exercise in C++. This is because all classes forming the [WIR](#) software model (cf. Chapter [8](#)) are equipped with `<<` stream operators that allow to dump the corresponding objects to an output stream in a highly flexible and configurable way. More precisely, these operators exist for the following classes: `WIR_System`, `WIR_CompilationUnit`, `WIR_Function`, `WIR_BasicBlock`, `WIR_Instruction`, `WIR_Operation`, `WIR_Parameter` and all derived parameter classes, and `WIR_Data`. They are all declared in analogy to Listing [12.1](#) showing, e.g., the [API](#) of class `WIR_BasicBlock`.

Listing 12.1: Generic Stream Output of Class `WIR_BasicBlock`

```

1 class WIR_BasicBlock
2 {
3     public:
4
5         [...]
6
7         // Dump a basic block to an output stream.
8         friend std::ostream & operator << ( std::ostream &os, const WIR_BasicBlock &b );
9 };

```

An example program showing the use of such stream output and of various other concepts introduced in Chapters [8](#) and [9](#) is shown in Listing [12.2](#). This example makes use of the RV32IMC processor model from Section [9.1.4](#) (cf. Line [3](#)). As already introduced in Chapter [5](#), the [WIR](#) library is initialized first in Line [10](#). Hereafter, an RV32IMC-based system is created (Line [13](#)) and its (one and only) processor core is obtained in Line [14](#). Lines [17](#) to [23](#) generate two compilation units containing one function each. In the first function `foo` of compilation unit `file1.s`, three RISC-V virtual registers are created (Lines [26](#) to [28](#)) where two of them are precolored with the processor's physical registers `x0` and `x1`, resp. (Lines [29](#) and [30](#)). Next, one basic block containing one instruction is added to this function `foo` (Line [32](#)), and this instruction contains two operations (Lines [33](#) and [36](#)).

Listing 12.2: Simple WIR Program Showing Stream Output

```

1 // Include WIR headers.
2 #include <wir/wir.h>
3 #include <arch/riscv/rv32imc.h>
4
5 using namespace WIR;
6
7 int main( void )
8 {
9     // Initialize WIR.
10    WIR_Init();
11
12    // Create RISC-V WIR system and get the system's RV32IMC processor core.
13    WIR_System sys( "rudol.v.sys", WIR_TaskManager() );
14    auto &p = sys.getComponents<RV32IMC>().begin()->get();
15
16    // Create two compilation units with one function each.
17    auto &c1 = sys.pushBackCompilationUnit( WIR_Function( "foo" ) );
18    c1.setName( "file1.s" );
19    auto &c2 = sys.pushBackCompilationUnit( WIR_Function( "bar" ) );
20    c2.setName( "file2.s" );
21
22    auto &f1 = c1.getFunctions().front().get();
23    auto &f2 = c2.getFunctions().front().get();
24
25    // Create various WIR objects in first compilation unit.
26    auto &r1 = f1.pushBackVirtualRegister( WIR_VirtualRegister( RV32I::RegisterType::reg ) );
27    auto &r2 = f1.pushBackVirtualRegister( WIR_VirtualRegister( RV32I::RegisterType::reg ) );
28    auto &r3 = f1.pushBackVirtualRegister( WIR_VirtualRegister( RV32I::RegisterType::reg ) );
29    f1.insertPreColor( r2, p.x0() );
30    f1.insertPreColor( r3, p.x1() );
31
32    f1.pushBackBasicBlock( { WIR_Instruction(
33        WIR_Operation( RV32I::OpCode::ADDI, RV32I::OperationFormat::RRC12_1,
34            WIR_RegisterParameter( r1, WIR_Usage::def ), WIR_RegisterParameter( r2, WIR_Usage::use ),
35            RV_Const12_Signed( 42 ) ),
36        WIR_Operation( RV32I::OpCode::JAL, RV32I::OperationFormat::RL_1,
37            WIR_RegisterParameter( r3, WIR_Usage::def ), WIR_LabelParameter( f2 ) ) } );
38
39    // Create various WIR objects in second compilation unit.
40    auto &r4 = f2.pushBackVirtualRegister( WIR_VirtualRegister( RV32I::RegisterType::reg ) );
41    auto &d = c2.pushBackData( { WIR_Data( "dataObject" ) } );
42    d.setSize( 4 );
43    d.pushBackInitData( WIR_DataInit( WIR_DataInitType::iword, "4711" ) );
44
45    f2.pushBackBasicBlock(
46        { WIR_Instruction( { RV32I::OpCode::LUI, RV32I::OperationFormat::RL_2,
47            WIR_RegisterParameter( r4, WIR_Usage::def ), WIR_LabelParameter( d ) } ),
48        WIR_Instruction( { RV32I::OpCode::LW, RV32I::OperationFormat::RLR_1,
49            WIR_RegisterParameter( p.x10(), WIR_Usage::def ), WIR_LabelParameter( d ),
50            WIR_RegisterParameter( r4, WIR_Usage::use ) } ),
51        WIR_Instruction( { RV32I::OpCode::JALR, RV32I::OperationFormat::RRC12_1,
52            WIR_RegisterParameter( p.x0(), WIR_Usage::def ), WIR_RegisterParameter( p.x1(), WIR_Usage::use ),
53            RV_Const12_Signed( 0 ), WIR_RegisterParameter( p.x10(), WIR_Usage::use, true ) } } );
54
55    // Generate code for the current WIR system.
56    std::cout << sys << std::endl;
57
58    return( 0 );
59 }

```

In a similar fashion, another virtual register is created within the second function bar of compilation unit `file2.s` (Line 40). Lines 41 to 43 show the construction of a data object inside `file2.s` that is initialized with 4711. Function bar itself obtains one basic block (Line 45) featuring three instructions where each instruction consists of exactly one operation each (Lines 46, 48 and 51).

Finally, the entire code structure accommodated by the WIR system is simply printed out to `cout` in Line 56.

Compiling and executing this example program from Listing 12.2 according to Chapter 5 yields the following WIR output:

```

1 FILE file1.s
2 {
3
4 SECTION .text
5 {
6
7 FUNCTION foo
8 {
9 foo:
10     addi RRC12_1 x_5733, x_5735, 42;
11     || jal RL_1 x_5737, bar;
12 };
13
14 };
15
16 };
17
18
19 FILE file2.s
20 {
21
22 SECTION .data
23 {
24
25 DATA dataObject
26 {
27     4711
28 };
29 };
30 };
31
32 SECTION .text
33 {
34
35 FUNCTION bar
36 {
37 bar:
38     lui RL_2 x_5758, dataObject;
39     lw RLR_1 x10, dataObject, x_5758;
40     jalr RRC12_1 x0, x1, 0;
41 };
42
43 };
44
45 };

```

This output is entirely generic in that sense that it is unaware of any RISC-V specific code conventions. As can be seen from, e.g., Lines 2, 5 and 8 and Lines 12 to 16, such generic WIR output consists of a dump of nested,

hierarchical scopes that are all enclosed in curly brackets { ... }. Each such scope begins with a keyword specifying the scope's nature, e.g., FILE, SECTION, FUNCTION or DATA, followed by the (optional) name of the corresponding component.

The output of a WIR function consists of the printout of all basic blocks of that function. Each basic block is printed out beginning with its name followed by a colon (cf. Lines 9 and 37). Line by line, all instructions contained by a basic block are emitted next (e.g., Lines 38 to 40). For each instruction, its operations are dumped again line by line, where the operation's output starts (indented by 8 whitespaces) with its mnemonic string followed by the operation format's name. Finally, all explicit parameters of an operation are printed out as comma-separated list. The output of an operation ends with a semicolon (Line 39). For instructions containing several operations, the dump of all operations except the very first one always begins with a "parallel" operator || as can be seen in Line 11. According to Section 8.4, all operations of an instruction are supposed to be executed in parallel, which is indicated in the WIR output this way.

12.2 Stream Manipulators

The generic WIR output from the previous section reveals that certain code properties that are specified in the example program from Listing 12.2 are not printed out by default:

- Function foo contains virtual registers x_5733, x_5735 and x_5737 (Lines 26 to 28), and function bar features virtual register x_5758 (Line 40);
- Virtual registers x_5735 and x_5737 are precolored with certain physical registers (Lines 29 and 30);
- Various register parameters of the shown operations are defined (e.g., Line 37), while others are used (e.g., Line 50);
- Implicit register parameters such as p.x10 in Line 53 are not printed at all in the default WIR output.

This default behavior is motivated by the objective to keep the output as simple as possible and to primarily focus on the WIR software model's code hierarchy. However, the output of additional details can selectively be turned on or off by using the C++ stream manipulators declared in file WIR/wir/wirio.h. A stream manipulator is a simple function that receives an output stream as argument, modifies it internally so that different output will be created, and returns a reference to the modified stream. Line 2 of Listing 12.3 shows a sample declaration of the defuse stream manipulator. This manipulator enables the output of additional def/use properties for register parameters. In analogy to the standard C++ API, each WIR manipulator activating a certain output property comes with a manipulator to deactivate this behavior again. The name of such manipulators always begins with "no" which can be seen from the example nodefuse from Line 5.

Listing 12.3: Declaration of defuse and nodefuse Stream Manipulators in file wirio.h

```

1 // Switch the output of def/use characteristics for register parameters on.
2 std::ostream &defuse( std::ostream &os );
3
4 // Switch the output of def/use characteristics for register parameters off.
5 std::ostream &nodefuse( std::ostream &os );

```

In order to control the level of detail of generic WIR output, the manipulators shown in Table 12.1 are declared in file wirio.h. They allow to address all the abovementioned issues. In addition to that, the output of jump targets that are explicitly annotated for jump operations (cf. Listing 8.19) can also be controlled. Finally, the output of potentially existing comments and source file information can also be activated or deactivated.

Table 12.1: Generic Stream Manipulators from file `wirio.h`

Manipulators	Enabled/Disabled Output Property
<code>comment, nocomment</code>	Comments attached to WIR objects
<code>defuse, nodefuse</code>	Def/use characteristics of register parameters
<code>fileinfo, nofileinfo</code>	Source code file information attached to WIR objects
<code>functionregisters, nofunctionregisters</code>	Virtual registers owned by functions
<code>implicitparams, noimplicitparams</code>	Implicit parameters of operations
<code>jumptargets, nojumptargets</code>	Explicitly specified targets of jump operations
<code>precolors, noprecolors</code>	Precoloring information of functions

When changing Line 56 from Listing 12.2 to

```

55 // Generate code for the current WIR system.
56 std::cout << defuse << functionregisters << implicitparams << precolors << sys << std::endl;

```

the produced [WIR](#) output for the two functions `foo` and `bar` from the previous example program changes to:

```

1  [...]
2
3  FUNCTION foo
4  {
5      {
6          registers: x_5733 x_5735 x_5737;
7          precolors: x_5735=x0 x_5737=x1;
8      }
9  foo:
10      addi RRC12_1 x_5733 (def, ID = 5747), x_5735 (use, ID = 5748), 42;
11      || jal RL_1 x_5737 (def, ID = 5741), bar;
12  };
13
14  [...]
15
16  FUNCTION bar
17  {
18      {
19          registers: x_5755;
20          precolors: ;
21      }
22  bar:
23      lui RL_2 x_5755 (def, ID = 5762), dataObject;
24      lw RLR_1 x10 (def, ID = 5769), dataObject, x_5755 (use, ID = 5771);
25      jalr RRC12_1 x0 (def, ID = 5778), x1 (use, ID = 5779), 0, x10 (use, ID = 5781);
26  };
27
28  [...]

```

As can be seen from Lines 5 to 8 and Lines 18 to 21, each function now begins with another scope containing information about the function's registers and its specified precolorings. This information starts with the keywords `registers:` and `precolors:`, resp., followed by a whitespace-separated list of registers or precolorings. These lines of output always end with a semicolon. Lines 10, 11 and 23 to 25 furthermore show that the output for each register parameter now is extended by parentheses (. . .) containing the information whether a parameter is

Listing 12.4: Constructors and Methods for Comment Text Handling of Class WIR_Comment

```

1 // Default constructor creating a single-line comment from a copied string.
2 explicit WIR_Comment::WIR_Comment( const std::string &s );
3
4 // Default constructor creating a single-line comment from a moved string.
5 explicit WIR_Comment::WIR_Comment( std::string &&s );
6
7 // Copy constructor.
8 WIR_Comment::WIR_Comment( const WIR_Comment &c );
9
10 // Move constructor.
11 WIR_Comment::WIR_Comment( WIR_Comment &&c );
12
13 // Set comment text from a copied string.
14 void WIR_Comment::setText( const std::string &s );
15
16 // Set comment text from a moved string.
17 void WIR_Comment::setText( std::string &&s );
18
19 // Get a comment's text.
20 std::string WIR_Comment::getText( void ) const;

```

defined, used, or both, plus the parameter's unique ID. A closer look at Line 25 also shows that the previously omitted implicit parameter is now printed, too.

WIR uses so-called containers in order to attach arbitrary kinds of meta-data to all parts of its software model. The discussion of this container API is beyond the scope of this chapter, details will be provided later in Section 13.1.

Listing 12.5: Constructors and Methods for File Location Handling of Class WIR_FileInfo

```

1 // Default constructor creating file information from a copied string.
2 WIR_FileInfo::WIR_FileInfo( const std::string &s, unsigned long long l );
3
4 // Default constructor creating file information from a moved string.
5 WIR_FileInfo::WIR_FileInfo( std::string &&s, unsigned long long l );
6
7 // Copy constructor.
8 WIR_FileInfo::WIR_FileInfo( const WIR_FileInfo &f );
9
10 // Move constructor.
11 WIR_FileInfo::WIR_FileInfo( WIR_FileInfo &&f );
12
13 // Set file name from a copied string.
14 void WIR_FileInfo::setFileName( const std::string &s );
15
16 // Set file name from a moved string.
17 void WIR_FileInfo::setFileName( std::string &&s );
18
19 // Get the file name.
20 std::string WIR_FileInfo::getFileName( void ) const;
21
22 // Set line number.
23 void WIR_FileInfo::setLineNumber( unsigned long long l );
24
25 // Get line number.
26 unsigned long long WIR_FileInfo::getLineNumber( void ) const;

```


However, the previously mentioned comments and source file information are very simple examples of such containers.

Class `WIR_Comment` can be found in file `WIR/containers/wircomment.h`, and it represents single-line string comments. The public interface of such comments (cf. Listing 12.4) is extremely straightforward and simple so that a more detailed discussion is omitted here. However, it is worthwhile to mention that comment containers represent single-line comments so that any newline character passed to class `WIR_Comment` is removed.

File information objects intend to establish a link between a certain construct within the `WIR` software model and a location in some source code from which the `WIR` object has been generated. Such code locations usually come in the form of pairs of file names and line numbers, and this is exactly what is represented by class `WIR_FileInfo` from file `WIR/containers/wirfileinfo.h`, cf. Listing 12.5. Both a file name and a line number can be specified during construction or can be set later via the set methods. As above, file names are assumed to be single-line strings so that line breaks are removed.

Coming back to the example program from Listing 12.2, such comments and file information can be added and printed out, e.g., as follows:

```

55  f1.insertContainer( WIR_FileInfo( "file1.c", 37 ) );
56  f2.begin()->get().insertContainer( WIR_Comment( "A commented WIR basic block." ) );
57
58  // Generate code for the current WIR system.
59  std::cout << comment << fileinfo << sys << std::endl;

```

Line 55 attaches a file location to function `foo`, referring to some C file `file1.c` and line number 37. The first basic block of function `bar` is extended by a comment in Line 56. The output finally resulting from Line 59 is:

```

1  [...]
2
3  ; file "file1.c", line 37
4  FUNCTION foo
5  {
6  foo:
7      addi RRC12_1 x_5733, x_5735, 42;
8      || jal RL_1 x_5737, bar;
9  };
10
11  [...]
12
13  FUNCTION bar
14  {
15  ; A commented WIR basic block.
16  bar:
17      lui RL_2 x_5755, dataObject;
18      lw RLR_1 x10, dataObject, x_5755;
19      jalr RRC12_1 x0, x1, 0;
20  };
21
22  [...]

```

As can be seen, both the file information (cf. Line 3) and the comment (Line 15) are printed before their respective `WIR` objects (here: function `foo` and basic block `bar`) are printed. The output of both containers starts with a semicolon followed by the container's data in the rest of the line.

12.3 Processor-Specific Assembly Code Generation

The discussion of code generation in the previous sections of this chapter solely focussed on generic and processor-independent output. This section thus shows how to produce processor-specific output that obeys the assembly-level code conventions of the architectures currently supported by [WIR](#).

From a user's perspective, generating processor-specific assembly code is as simple as already introduced in Section 12.2 by using the stream manipulators summarized in Table 12.2. They are named according to their respective processor families shown in the second column and aim to provide GNU assembler-compliant output. As shown in the right column, they are declared inside the architectures' various sub-directories. Since the MIPS SPIM architecture just serves as a demonstrator within [WIR](#) and is currently not used for real code generation, the `mips` manipulator only produces rudimentary valid assembly output. For the other manipulators `arm`, `riscv` and `tricore`, the [WIR](#) unit tests (cf. Chapter 5) ensure the generation of valid assembly. The `wir` stream manipulator switches a stream back to the generic output described in the previous sections.

When revisiting the previous example, generating RISC-V code can be achieved easily by changing Line 59 to:

```
58 // Generate RISC-V code for the current WIR system.
59 std::cout << riscv << comment << fileinfo << sys << std::endl;
```

As a result, [WIR](#) now produces the following assembly output:

```
1      .file          "file1.s"
2      # Generated using wir release 2.0 (2024-03-24)
3      # for RISC-V ISA RV32IMC
4
5      .section       .text
6      .balign       8
7      # file "file1.c", line 37
8      .type         foo, @function
9  foo:
10     addi          x_5733, x_5735/x0, 42
11     jal          x_5737/x1, bar
12     .size        foo, .-foo
13
14     .file          "file2.s"
15     # Generated using wir release 2.0 (2024-03-24)
16     # for RISC-V ISA RV32IMC
17
18     .section       .data, "aw"
19     .balign       8
20     .type         data0object, @object
21     .size        data0object, 4
22  data0object:
23     .word         4711
24
25     .section       .text
26     .balign       8
27     .type         bar, @function
28     # A commented WIR basic block.
29  bar:
30     lui          x_5755, %hi(data0object)
31     lw          x10, %lo(data0object)(x_5755)
32     jalr         x0, x1, 0
33     .size        bar, .-bar
```

Table 12.2: Processor-Specific Stream Manipulators

Manipulator	Processor Families	File of Declaration
arm	All models from ARMv4 to ARMv6 from Section 9.1.5	WIR/arch/arm/armio.h
mips	MIPS SPIM model from Section 9.1.2	WIR/arch/generic/mipsio.h
riscv	All models from RV32I to RV32IMFDC from Section 9.1.4	WIR/arch/riscv/rvio.h
tricore	TC13 and TC131 models from Section 9.1.3	WIR/arch/tricore/tcio.h
wir	WIR generic output from Section 12.1	WIR/wir/wirio.h

As can be seen, the various scopes of the previous, generic WIR output representing, e.g., compilation units, sections and data objects are now printed as valid assembly directives. The code of the functions themselves is also formatted in a proper manner where, e.g., the output of operation formats is omitted, or the specifiers `%hi(...)` and `%lo(...)` are used in Lines 30 and 31. The only aspect in the above example output that is not fully RISC-V assembly compliant is the use of virtual registers and their output together with their precolorings. This, however, is not an issue of WIR's code generation mechanisms. Instead, this is solely due to the use of virtual instead of physical registers in the running example.

For all classes of the WIR software model, their respective `<<` stream operators invoke so-called dumper functions, where such a dumper performs the actual task of printing out a WIR object. File `WIR/wir/wirio.h` declares a couple of data types that represent function pointers for all these available dumper functions. As Listing 12.1 showed the example of the `<<` operator for basic blocks, Listing 12.6 now shows the declaration of the corresponding basic block dumper type. Functions of this type thus expect a reference to an output stream as well as a constant reference to a basic block to be printed out.

Listing 12.6: Declaration of Basic Block Dumper Type in file `wirio.h`

```

1 // Type WIR_BasicBlockDumper represents pointers to functions for processor-specific output of basic blocks.
2 using WIR_BasicBlockDumper = void (*)( std::ostream &, const WIR_BasicBlock & );
```

For all processor families listed in Table 12.2, there now exist actual dumper functions that realize the processor-specific output of WIR objects. For example, the RISC-V-specific output of basic blocks is realized by function `dumpRVBasicBlock` declared in file `WIR/arch/riscv/rvio.h` (cf. Listing 12.7). As can be seen, this actual function is precisely of type `WIR_BasicBlockDumper` as declared in Listing 12.6.

Listing 12.7: Declaration of RISC-V-Specific Basic Block Dumper

```

1 // Dump a basic block to an output stream in a RISC-V-specific fashion.
2 void dumpRVBasicBlock( std::ostream &os, const WIR_BasicBlock &b );
```

The WIR library now finally uses an internal map that returns the appropriate function pointers to these processor-specific dumpers, depending on the stream manipulator from Table 12.2 that has been applied to an output stream before. Section 9.1.1.5 has already introduced that certain aspects of a processor model need to be registered centrally within WIR during the initialization of the entire library. This previous discussion of processor-specific initialization revolved around the registration of operation formats and opcodes, cf. Listing 9.23 on Page 81. The last aspect of WIR initialization now is the proper registration of the various processor-specific dumpers in this internal map. The corresponding excerpt of the initialization for the RISC-V processor family is shown in Listing 12.8. Here, all the existing dumpers are registered, and Line 6 showcases the concrete example of the RISC-V-specific basic block dumper registration.

Listing 12.8: Dumper Function Registration (here for Class RV32I)

```

1 void RV32I::init( void )
2 {
3     [...]
4
5     // Registration of RV32I dumpers.
6     registerBasicBlockDumper( getProcessorTypeID(), dumpRVBasicBlock );
7     registerCompilationUnitDumper( getProcessorTypeID(), dumpRVCompilationUnit );
8     registerDataDumper( getProcessorTypeID(), dumpRVData );
9     registerDataSectionDumper( getProcessorTypeID(), dumpRVDataSection );
10    registerFunctionDumper( getProcessorTypeID(), dumpRVFunction );
11    registerOperationDumper( getProcessorTypeID(), dumpRVOperation );
12    registerRegisterParameterDumper( getProcessorTypeID(), dumpRVRegisterParameter );
13    registerCommentDumper( getProcessorTypeID(), dumpRVComment );
14    registerFileInfoDumper( getProcessorTypeID(), dumpRVFileInfo );
15
16    // RV32I operation formats.
17
18    [...]
19 };

```

All registrations of dumpers from Lines 6 to 14 use method `getProcessorTypeID` (cf. Listing 9.3 on Page 67) in order to associate the given dumpers with the unique constant ID of the currently initialized processor family. The stream manipulators from Table 12.2 now finally associate exactly this unique processor ID with an output stream so that all the << operators of the WIR software model can obtain the proper processor-specific dumpers from the abovementioned internal map.

12.4 Linker Script Generation

One particular stream manipulator has not yet been introduced in the previous sections, as its purpose is not directly related to the generation of actual code from the WIR software model. As discussed earlier in Section 10.4, WIR systems provide mechanisms for the mapping of logical sections to physical memory regions, and for the allocation of symbols to sections. In other words, the allocation of code or data to a system's different memories can be determined at the system level.

WIR is designed to be an assembly-level IR. In contrast, such memory allocation decisions are regularly enforced at the level of binary executable code by applying some linker. The code generated using the concepts described in the previous sections of this chapter does not contain any provisions to enforce some memory allocation specified by a WIR system. For this purpose, the manipulator `ldscript` (cf. Line 2 of Listing 12.9) can be used. It enables the generation of a GNU linker-compliant script that in turn forces the linker to respect the memory allocations defined by the WIR system. This manipulator has only effect on the << output operator of class `WIR_System`, and the manipulator `noldscript` (Line 5) switches a stream back to the generation of regular code.

Listing 12.9: Declaration of `ldscript` and `noldscript` Stream Manipulators in file `wirio.h`

```

1 // Switch the output of a linker script for WIR systems on.
2 std::ostream &ldscript( std::ostream &os );
3
4 // Switch the output of a linker script for WIR systems off.
5 std::ostream &noldscript( std::ostream &os );

```

13 Code Analysis

This chapter briefly describes all static code analyses that are integral part of the [WIR](#) library. Section [13.1](#) first introduces all generic base classes that form the analysis framework within [WIR](#). Hereafter, Sections [13.2](#) to [13.6](#) are devoted to the control flow analyses, while Sections [13.7](#) to [13.11](#) present all data flow analyses.

13.1 Generic Analysis Framework

All code analyses presented in the rest of this chapter inherit from the base class `WIR_Analysis` declared in file `WIR/analyses/generic/wiranalysis.h` so that all actual analyses share the very same public [API](#) and thus are easy to use. This public interface is shown in Listing [13.1](#).

Listing 13.1: Code Analysis API of Class `WIR_Analysis`

```

1 // Default constructor for system-level analysis.
2 explicit WIR_Analysis::WIR_Analysis( WIR_System &s );
3
4 // Default constructor for compilation unit-level analysis.
5 explicit WIR_Analysis::WIR_Analysis( WIR_CompilationUnit &c );
6
7 // Default constructor for function-level analysis.
8 explicit WIR_Analysis::WIR_Analysis( WIR_Function &f );
9
10 // Default constructor for basic block-level analysis.
11 explicit WIR_Analysis::WIR_Analysis( WIR_BasicBlock &b );
12
13 // Default constructor for instruction-level analysis.
14 explicit WIR_Analysis::WIR_Analysis( WIR_Instruction &i );
15
16 // Default constructor for operation-level analysis.
17 explicit WIR_Analysis::WIR_Analysis( WIR_Operation &o );
18
19 // Performs the actual code analysis.
20 void WIR_Analysis::analyze( void );

```

As can be seen, various constructors can be chosen which determine whether a code analysis shall be carried out for a complete [WIR](#) system (cf. Line [2](#)), or just for a single compilation unit, function, basic block etc. (Lines [5](#) to [17](#)). After an analysis has been set up this way, only the `analyze` method from Line [20](#) needs to be invoked in order to apply the actual analysis to the [WIR](#) object used during construction.

Listing [13.1](#) furthermore shows that there are no methods at all that allow to get and to inspect analysis data that has been produced by the `analyze` method. This in turn is motivated by the fact that any kind of meta-data that provides additional information about a system's code properties is modeled within [WIR](#) by so-called containers. Consequently, all code analyses presented in the following produce such containers and attach them to the various classes of the [WIR](#) software model from Fig. [1.2](#), ranging from a top-level system down to the

Table 13.1: Analysis Containers and their Fixing Points Within WIR

	Analysis	Container Class	Fixing Points
Control Flow	Reachability	WIR_Reachability	Basic Blocks
	Domination	WIR_Domination	Basic Blocks and Instructions
	Immediate Domination	WIR_ImmediateDomination	Basic Blocks
	Structural Control Flow	WIR_ControlTree	Basic Blocks
	Flow Facts	WIR_FlowFactRef	Basic Blocks
Data Flow	Reaching Definitions	WIR_ReachingDefinitions	Instructions
	Available Definitions	WIR_AvailableDefinitions	Instructions
	Def-Use/Use-Def Chains	WIR_DUUDChain	Register Parameters
	Liveness	WIR_LiveOut	Instructions
	Bit-True Data Flow	WIR_BitValues	Register and Immediate Parameters

various flavors of [WIR](#) parameters. Table 13.1 briefly summarizes all [WIR](#) analyses and provides an overlook over the actual container classes that are generated by the respective analyses as well as over those [WIR](#) classes to which these containers get attached. The following sections will provide in-depth information about the meaning of all these containers and how to query them for their analysis data in detail.

Note

Whenever containers are attached to [WIR](#) objects by an analysis' `analyze` method, the analysis data stored in these containers reflects the state of the control and data flow at exactly that point when the analysis was carried out. If [WIR](#) objects are modified after an invocation of `analyze`, potentially existing containers are not automatically updated so that the state of [WIR](#) and that of the attached containers can be incoherent. Application code thus either explicitly has to re-run an analysis from scratch, or the data inside attached containers needs to be updated incrementally after some [WIR](#) modification.

All [WIR](#) classes acting as “fixing points” for such containers share a common [API](#) in order to generically attach or access containers. This is achieved by having all these classes inherit from the base class `WIR_Container_API` as depicted in Fig. 13.1. This [API](#) is declared in file `WIR/wir/API/wircontainerapi.h`, and it provides the methods shown in Listing 13.2 for the insertion and destruction of containers.

The `insert` methods from Lines 2 to 8 allow to attach a new container to some [WIR](#) object. Similar to Chapter 6, there are dedicated methods for the copy- and move-insertion of containers. Furthermore, one of the very rare cases within [WIR](#) where a pointer shows up in the public [API](#) can be seen in Line 8. Here, the user is responsible to dynamically create a new container and to pass a pointer to this container to `insertContainer`.

Warning

After such a pointer-based insertion, class `WIR_Container_API` takes over full control over the ownership of that pointer. In particular, it automatically destroys the object pointed to during erasure. Thus, the pointer passed to `insertContainer` should not be used by application code any more afterwards.

Method `eraseContainers` (cf. Line 11) can be used to remove all containers of a given container class shown in Table 13.1. For example, `f.eraseContainers( WIR_Domination::getContainerTypeID(), true )` would have the effect that all containers holding domination information will be removed from [WIR](#) function `f`. The Boolean flag serving as second argument of this method has the effect that the code hierarchy underneath, e.g., function `f` will also be inspected and containers be removed. Method `eraseContainer` (Line 14) can be

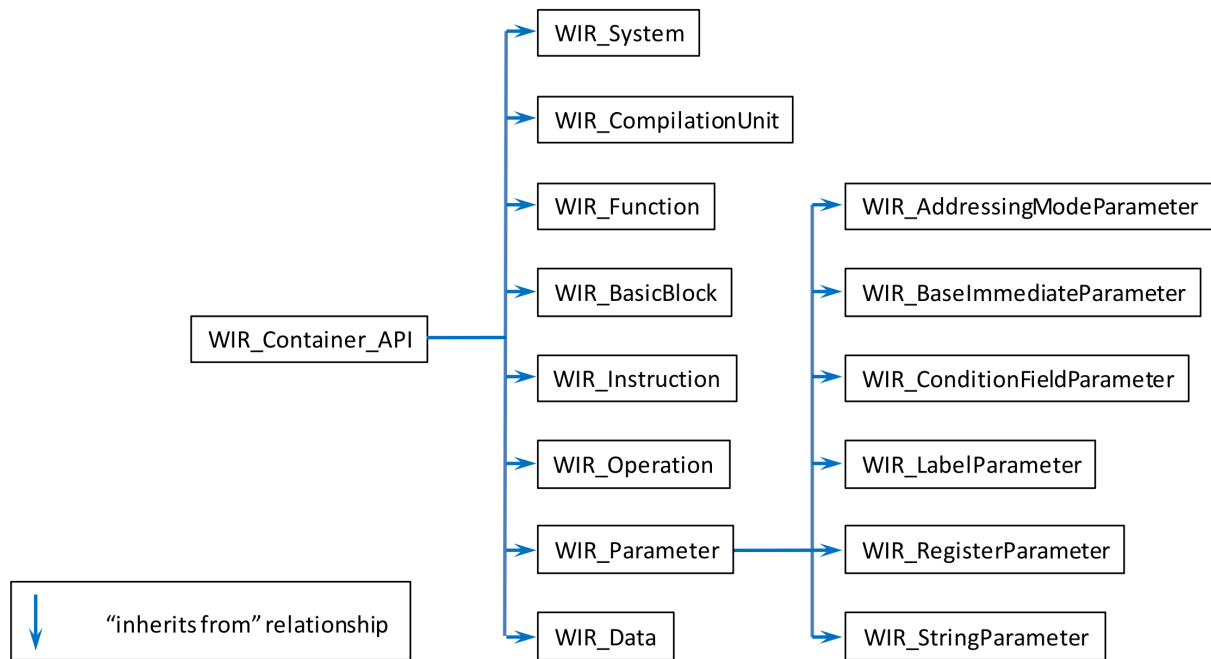


Figure 13.1: Inheritance Diagram of Class WIR_Container_API

used to remove one given container, and `clearContainers` (Line 17) removes all containers attached to some `WIR` object. All these erase methods automatically delete the corresponding container objects from memory.

All methods to look up containers attached to some `WIR` object are summarized in Listing 13.3. `getContainerTypes` from Line 2 provides the set of unique numerical IDs of all different container classes that are currently attached. The full set of all currently attached containers can be retrieved using method `getContainers` (Line 5), its return type `WIR_ContainerSet` is declared as `std::set<std::reference_wrapper<WIR_BaseContainer>, WIR_Compare<WIR_BaseContainer>>` (see Chapter 11). In analogy to method `WIR_System::getComponents` (cf. Line 5 of Listing 10.4), `getContainers` from Line 9 provides a set of (wrapped) references to only those containers that can properly be casted to the specified type name `T`. The different contains methods can eventually be used to check whether containers are attached to some `WIR` object at all (Line 12), to check whether containers

Listing 13.2: Insertion and Removal of WIR Containers

```

1 // Copy-insert a new container.
2 void WIR_Container_API::insertContainer( const WIR_BaseContainer &c );
3
4 // Move-insert a new container.
5 void WIR_Container_API::insertContainer( WIR_BaseContainer &&c );
6
7 // Insert a new container.
8 void WIR_Container_API::insertContainer( WIR_BaseContainer *p );
9
10 // Remove all containers of the specified container type (recursively).
11 void WIR_Container_API::eraseContainers( WIR_id_t id, bool r = false );
12
13 // Remove the specified container.
14 void WIR_Container_API::eraseContainer( const WIR_BaseContainer &c );
15
16 // Remove all containers.
17 void WIR_Container_API::clearContainers( void );

```


Listing 13.3: Lookup of WIR Containers

```

1 // Determine all types of containers currently attached to some WIR object.
2 std::set<WIR_id_t> WIR_Container_API::getContainerTypes( void ) const;
3
4 // Get the complete set of references to containers.
5 const WIR_ContainerSet WIR_Container_API::getContainers( void ) const;
6
7 // Get the set of all containers of a specified type.
8 template<typename T>
9 const std::set<std::reference_wrapper<T>, WIR_Compare<T>> WIR_Container_API::getContainers( void ) const;
10
11 // Determine whether some containers are attached at all.
12 bool WIR_Container_API::containsContainers( void ) const;
13
14 // Determine whether containers of a specified type are attached.
15 bool WIR_Container_API::containsContainers( WIR_id_t id ) const;
16
17 // Determine whether a given container is attached.
18 bool WIR_Container_API::containsContainer( const WIR_BaseContainer &c ) const;

```

of a given class from Table 13.1 are attached (Line 15), or whether a specific container is attached (Line 18).

Listings 13.2 and 13.3 show that generic containers of the abstract base class `WIR_BaseContainer` are used. Thus, all analysis containers from Table 13.1 are derived from this base class. Listing 13.4 shows the relevant parts of the public interface of this base class. As `WIR_BaseContainer` contains purely virtual methods, no actual objects of this class can be created. We thus omit the description of constructors here for the sake of brevity.

Some of the previously introduced methods like, e.g., `eraseContainers` (cf. Line 11 of Listing 13.2) or `getContainerTypes` (Line 2 of Listing 13.3) dealt with unique numerical IDs of the types of containers. These container type IDs have the effect that, e.g., `WIR_ControlTree` containers can generally be distinguished from `WIR_LiveOut` containers without having to do dynamic type casts in C++. The two variants of the `getContainerType` method from Listing 13.4 now provide access to these type IDs. The first one from Line 2 is a purely virtual method that gets implemented within each derived container class. The second one (Line 5) comes in the form of a static method, allowing to check container types using, e.g., `WIR_Domination::getContainerTypeID()`.

Finally, the uniqueness of `WIR` containers can be checked using method `isUnique` (Line 8). Uniqueness denotes whether at most one instance of a container type can be attached to some `WIR` object. All containers from Table 13.1 are unique so that it is not possible to attach, e.g., several domination containers to the very same basic block. If a new unique container is attached to a `WIR` object using `insertContainer` (cf. Listing 13.2), any possibly already attached containers of the same type are silently erased before. Class `WIR_Comment` (cf. Section 12.2) is a non-unique container so that many comments can be attached to the same `WIR` object.

Listing 13.4: Type Handling of Class `WIR_BaseContainer`

```

1 // Get the unique ID of a container type.
2 virtual WIR_id_t WIR_BaseContainer::getContainerType( void ) const = 0;
3
4 // Get the unique ID of a container type.
5 static WIR_id_t WIR_BaseContainer::getContainerType( void );
6
7 // Determine whether a container is unique.
8 virtual bool WIR_BaseContainer::isUnique( void ) const = 0;

```


13.2 Basic Intraprocedural Control Flow Analysis

Analysis Class	Analysis Level	Container Class	Fixing Points	Architectures
WIR_ControlFlowAnalysis	Functions	None	None	Generic, all

The foundations for control flow analysis are realized within **WIR** by class `WIR_ControlFlowAnalysis` (cf. file `WIR/analyses/controlflow/wircfganalysis.h`) that actually models a **CFG** according to Definition 8.1 on Page 46. This **WIR** analysis automatically generates an internal graph data structure based on a function's basic blocks and their predecessor and successor sets (cf. Listing 8.10). This graph is then traversed and the main results of this traversal are various orders in which basic blocks were visited, and classifications of **CFG** nodes and edges [22, chapter 7.2]. Since basic blocks and their successor/predecessor relations are modeled in a generic, processor-independent fashion within **WIR**, class `WIR_ControlFlowAnalysis` is also generic and does not require any adaptations for specific processor architectures.

The public **API** of class `WIR_ControlFlowAnalysis` is shown in Listing 13.5. This class analyzes the intraprocedural control flow so that caller/callee relationships between functions are not considered. Due to this, the constructor from Line 2 expects a reference to a **WIR** function as argument and then builds and traverses the **CFG** for this one function. **CFG** traversals always start at the very first basic block of a function, as this is a function's unique entry point. In the case that a function contains unreachable basic blocks, these blocks are also traversed and visited afterwards.

The information produced during these **CFG** traversals is not considered to be “real” analysis data providing a higher level of understanding of a **WIR** function. Instead, it basically serves as basis for other, more sophisticated analyses presented in the following sections. For this reason, class `WIR_ControlFlowAnalysis` does not produce any kind of analysis containers as introduced in Section 13.1. Instead, a few getter methods can be used to query **CFG** traversal information.

Methods `getDFSOrder`, `getReversePostOrder` and `getReverseTopologicalOrder` (Lines 5 to 13) can then be used to obtain the lists of **CFG** nodes according to the graph's DFS order, postorder or topological order, resp.,

Listing 13.5: Public Interface of Class `WIR_ControlFlowAnalysis`

```

1 // Default constructor for function-level analysis.
2 explicit WIR_ControlFlowAnalysis( WIR_Function &f );
3
4 // Get the list of visited nodes in depth-first traversal order.
5 const std::list<std::reference_wrapper<WIR_BasicBlock>> &WIR_ControlFlowAnalysis::getDFSOrder( void ) const;
6
7 // Get the list of visited nodes in reverse postorder.
8 const std::list<std::reference_wrapper<WIR_BasicBlock>>
9     &WIR_ControlFlowAnalysis::getReversePostOrder( void ) const;
10
11 // Get the list of visited nodes in reverse-topological order.
12 const std::list<std::reference_wrapper<WIR_BasicBlock>>
13     &WIR_ControlFlowAnalysis::getReverseTopologicalOrder( void ) const;
14
15 // Get the set of all CFG start nodes.
16 const WIR_BasicBlockSet &WIR_ControlFlowAnalysis::getStartNodes( void ) const;
17
18 // Get the set of all back edges in the CFG.
19 const WIR_BasicBlockPairSet &WIR_ControlFlowAnalysis::getBackEdges( void ) const;
20
21 // Visualize a CFG using xdot.
22 void WIR_ControlFlowAnalysis::visualize( bool i = false ) const;

```

Listing 13.6: Declaration of Basic Block Pairs in file `wirtypes.h`

```

1 // WIR_BasicBlockPair represents pairs of references to WIR basic blocks.
2 using WIR_BasicBlockPair =
3     std::pair<std::reference_wrapper<WIR_BasicBlock>, std::reference_wrapper<WIR_BasicBlock>>;
4
5 // WIR_BasicBlockPairSet represents deterministic sets of basic block pairs.
6 using WIR_BasicBlockPairSet = std::set<WIR_BasicBlockPair, WIR_Compare_Pair>;

```

where the last two orders are reversed in order to support backward analyses. The set of start nodes of the CFG, i.e., basic blocks having no predecessors, is provided by method `getStartNodes` (Line 16).

Back edges inside a CFG play an important role, since they close cycles in the graph and thus indicate the presence of loops. The set of all identified back edges is returned by method `getBackEdges` (Line 19). CFG edges are represented as pairs of basic blocks such that the first element of a pair denotes the edge's source node, and the second element refers to the target node. The declaration of such basic block pairs can be seen in Line 3 of Listing 13.6. Based on the concepts introduced in Chapter 11 and by exploiting a dedicated comparator class for such pairs, Line 6 shows the declaration of deterministic CFG edge sets.

Finally, method `visualize` from Line 22 of Listing 13.5 can be used to visualize the created CFG for human inspection. This method converts a CFG to the Graphviz DOT language [7], exports it into a temporary file, and invokes the interactive graph visualization tool `xdot` on it. The Boolean argument `in` of this method defaults to `false` which means that the internals of CFG nodes are not visualized. By setting this argument explicitly to `true`, all instructions contained by basic blocks are displayed.

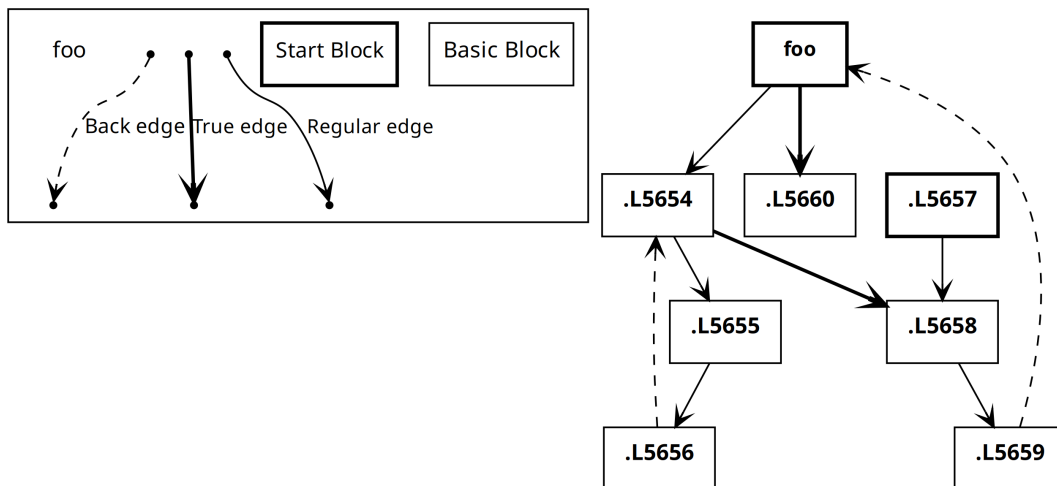
Figure 13.2: Control Flow Graph Visualized using Class `WIR_ControlFlowAnalysis`

Fig. 13.2 shows a CFG visualization created using the `WIR` unit test from file `WIR/tests/wirDFS2.cc`. As can be seen, basic blocks are represented by boxes, where start blocks (here: `foo` and `.L5657`) are formatted in bold. Regular graph edges representing an implicit successorship between two nodes (cf. Section 8.3 on Page 45) are displayed in normal style, e.g., the edge from `.L5654` to `.L5655`. So-called true edges are edges where an explicit branch needs to be taken, and they are represented by bold edges (e.g., the edge from `foo` to `.L5660` in Fig. 13.2). Back edges are represented in dashed style, see, e.g., the edge from `.L5656` to `.L5654`.

13.3 Reachability Analysis

Analysis Class	Analysis Level	Container Class	Fixing Points	Architectures
WIR_ReachabilityAnalysis	Functions	WIR_Reachability	Basic Blocks	Generic, all

Reachability analysis as realized by [WIR](#) class `WIR_ReachabilityAnalysis` (cf. file `WIR/analyses/reachability/wirreachabilityanalysis.h`) aims to answer the question which basic blocks may be reached from a given start block in a [CFG](#). In other words, it determines for each basic block b_s the set of basic blocks b_t such that a directed path from b_s to b_t exists in the [CFG](#). Reachability analysis bases on the generic control flow analysis from [Section 13.2](#) so that only the intraprocedural [CFG](#) is considered. The analysis is entirely generic and does not require adaptations for specific processor architectures.

Reachability analysis is carried out at the level of single [WIR](#) functions so that the public [API](#) of class `WIR_ReachabilityAnalysis` boils down to a function-level constructor as can be seen in [Line 8](#) of [Listing 13.1](#) and the `analyze` method from [Line 20](#).

Reachability information is attached to basic blocks using container class `WIR_Reachability`, cf. file `WIR/containers/wirreachability.h`. A basic block to which a reachability container is attached is supposed to be a start block b_s , and the container provides information about all basic blocks b_t reachable from b_s . The relevant parts of its public interface are shown in [Listing 13.7](#).

Listing 13.7: Reachability Handling of Container Class `WIR_Reachability`

```

1 // Add a basic block to the set of reachable blocks.
2 void WIR_Reachability::addReachableBlock( WIR_BasicBlock &b, bool be = true );
3
4 // Get the set of all reachable basic blocks.
5 const WIR_BasicBlockSet &WIR_Reachability::getReachableBlocks( bool be = true ) const;
6
7 // Determine whether a given basic block is reachable from that basic block holding this container.
8 bool WIR_Reachability::isReachable( const WIR_BasicBlock &b, bool be = true ) const;
9
10 // Determine whether the latter instruction is reachable from the former.
11 static bool WIR_Reachability::isReachable( const WIR_Instruction &i1, const WIR_Instruction &i2,
                                             bool be = true ) const;
```

A basic block can be added to the set of reachable blocks using method `addReachableBlock` (cf. [Line 2](#)). This method (as well as all others shown in [Listing 13.7](#)) accepts a Boolean flag defaulting to `true` as optional last argument. This flag indicates whether reachability across back edges in the [CFG](#) shall be considered or not, i.e., whether the analyzed paths from b_s to b_t are allowed to include back edges or not. The set of all blocks b_t that are reachable from the block hosting the container can be obtained via method `getReachableBlocks` ([Line 5](#)). The variant of the `isReachable` method from [Line 8](#) can be used to query whether a given basic block is part of the set of reachable blocks. A static variant of `isReachable` exists (cf. [Line 11](#)) in order to simply check reachability between two given instructions. This method takes the basic blocks holding the given instructions and checks the reachability of these blocks. If both instructions reside in the same basic block, `isReachable` considers which of the two instructions comes first in that basic block.

13.4 Domination and Immediate Domination Analysis

Analysis Class	Analysis Level	Container Class	Fixing Points	Architectures
WIR_DominationAnalysis	Functions	WIR_Domination	Basic Blocks Instructions	Generic, all
WIR_ImmediateDomination- Analysis	Functions	WIR_Immediate- Domination	Basic Blocks	

The domination and immediate domination analyses within [WIR](#) are realized by classes `WIR_DominationAnalysis` (cf. file `WIR/analyses/domination/wirdominationanalysis.h`) and `WIR_ImmediateDominationAnalysis` (file `WIR/analyses/immediatedomination/wirimmediatedominationanalysis.h`). According to [22, chapter 7.3], domination is a relation *dom* between two [CFG](#) nodes *d* and *n* such that

$$d \text{ dom } n \Leftrightarrow \text{Every path from the CFG's start node to } n \text{ includes } d \quad (13.1)$$

Immediate domination is a sub-relation *idom* such that

$$\forall i \neq n : i \text{ idom } n \Leftrightarrow i \text{ dom } n \wedge \nexists c : (c \neq i) \wedge (c \neq n) \wedge (i \text{ dom } c) \wedge (c \text{ dom } n) \quad (13.2)$$

Nodes *d* that (immediately) dominate some *n* are called (immediate) dominators. As was the case in the previous sections, (immediate) domination analysis takes place at the function-level, and both analyses are entirely generic. Their implementations closely follow the algorithms from [1, chapter 18.1] and [22, chapter 7.3].

Usually, domination relates to individual basic blocks as [CFG](#) nodes *d*, *i* or *n* in the above equations. Thus, analysis results are regularly attached to basic blocks using container classes `WIR_Domination` and `WIR_ImmediateDomination`, resp. For domination analysis, however, it might be useful to break the basic block-level relation *dom* down also to the level of single [WIR](#) instructions. Then, `WIR_Domination` containers are attached to both basic blocks and instructions. This behavior of class `WIR_DominationAnalysis` can be controlled using its constructor shown in Line 2 of Listing 13.8. Besides the function to be analyzed, it obtains a Boolean defaulting to `false` that specifies whether instruction-level domination shall be computed. Except this Boolean flag, the [API](#) of class `WIR_ImmediateDominationAnalysis` is identical to Listing 13.8.

Listing 13.8: Public Interface of Class `WIR_DominationAnalysis`

```

1 // Default constructor for function-level analysis.
2 explicit WIR_DominationAnalysis( WIR_Function &f, bool i = false );
3
4 // Performs the actual code analysis.
5 void WIR_DominationAnalysis::analyze( void );
```

Listing 13.9 shows the interfaces for (immediate) domination handling of both container classes. Dominators in the form of basic blocks or instructions can be added to class `WIR_Domination` using the `addDominator` methods from Line 2 or Line 5, resp. Likewise, either the set of dominator blocks or instructions can be retrieved using `getDominatorBlocks` (Line 8) or `getDominatorInstructions` (Line 11). Relation *idom* is unique so that a basic block *n* has exactly one immediate dominator *i*. For this reason, container class `WIR_ImmediateDomination` simply handles a reference to one single immediate dominator block. Method `setImmediateDominator` (Line 15) can be used to set this immediate dominator, while `getImmediateDominator` (Line 18) retrieves the immediate dominator.

Listing 13.9: Domination Handling of Container Classes WIR_Domination and WIR_ImmediateDomination

```

1 // Add a basic block to the set of dominators.
2 void WIR_Domination::addDominator( const WIR_BasicBlock &b );
3
4 // Add an instruction to the set of dominators.
5 void WIR_Domination::addDominator( const WIR_Instruction &i );
6
7 // Get the set of all dominator basic blocks.
8 const WIR_BasicBlockSet &WIR_Domination::getDominatorBlocks( void ) const;
9
10 // Get the set of all dominator instructions.
11 const WIR_InstructionSet &WIR_Domination::getDominatorInstructions( void ) const;
12
13
14 // Set a basic block as immediate dominator.
15 void WIR_ImmediateDomination::setImmediateDominator( const WIR_BasicBlock &b );
16
17 // Get the immediate dominator basic block.
18 const WIR_BasicBlock &WIR_ImmediateDomination::getImmediateDominator( void ) const;

```

13.5 Structural Control Flow Analysis

Analysis Class	Analysis Level	Container Class	Fixing Points	Architectures
WIR_StructuralAnalysis	Functions	WIR_ControlTree	Basic Blocks	Generic, all

The goal of structural control flow analysis is to decompose a general CFG of a function into (possibly nested) hierarchical regions that provide more high-level, structural control flow information. Regularly, the information provided by a CFG only models predecessor and successor relations between nodes. It remains unclear at this level, which groups of nodes form, e.g., an if-else construct or a tail-controlled loop. This changes during structural control flow analysis, where typical control flow structures well-known from high-level programming languages are systematically identified.

This analysis is realized by WIR class WIR_StructuralAnalysis declared in file WIR/analyses/structural-controlflow/wirstructuralanalysis.h. It closely follows the concepts presented in [22, chapter 7.7]. In analogy to all control flow analyses presented before, structural analysis is performed at the level of single functions, too. Furthermore, it requires no particular processor-specific extensions.

Listing 13.10 shows the API of class WIR_StructuralAnalysis. Besides the WIR function to be analyzed, its constructor (cf. Line 2) features two optional Boolean arguments that steer some technical aspects of method visualize from Line 8. Similar to Section 13.2, this method serves to visualize the outcomes of structural

Listing 13.10: Public Interface of Class WIR_StructuralAnalysis

```

1 // Default constructor for function-level analysis.
2 explicit WIR_StructuralAnalysis( WIR_Function &f, bool verbose = false,
3                                 bool tmpFiles = false );
4
5 // Performs the actual code analysis.
6 void WIR_StructuralAnalysis::analyze( void );
7
8 // Visualize the hierarchical CFG using xdot.
9 void WIR_StructuralAnalysis::visualize( void ) const;

```

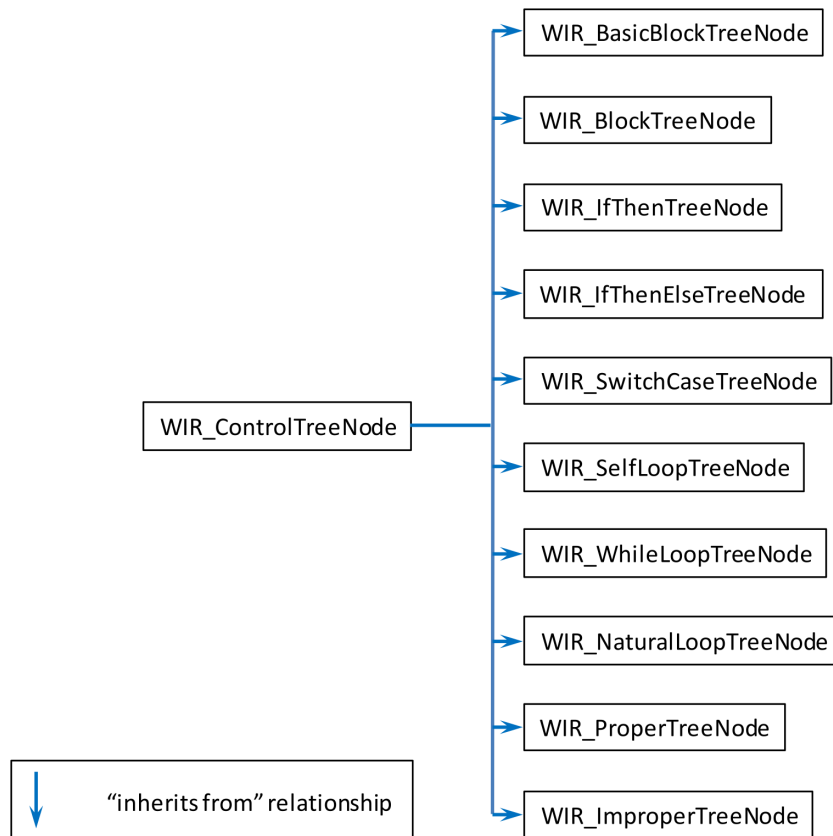


Figure 13.3: Inheritance Diagram of Control Regions

analysis for a human inspection. This again is done by plotting the hierarchical CFG in the external tool `xdot`. The constructor's parameter `verbose` controls whether the command lines used to invoke `xdot` are printed. Flag `tmpFiles` has the effect that the temporary DOT file that gets displayed using `xdot` is not erased. All subsequent figures illustrating the various kinds of control regions identified during structural analysis have been generated using the `visualize` method.

All these regions have in common that they have exactly one entry point. This property has the effect that any arbitrary unstructured CFG becomes reducible. Reducibility means that a series of transformations can be applied to the initial CFG clustering individual nodes into hierarchical nodes so that, in the end, the entire CFG gets reduced to a single (hierarchical) node [22, chapter 7.5]. Due to this property, the result produced by structural analysis no longer is a general graph, but a tree instead. The root node of this hierarchical control tree represents the entire function that has been analyzed. The tree's leaf nodes represent all the individual basic blocks of the original, unstructured CFG. In the following, we will use the terms "control region" and "tree node" interchangeably for this reason.

All the different control regions identified during structural analysis are represented by WIR classes that inherit from the abstract base class `WIR_ControlTreeNode` as shown in Fig. 13.3. The general properties of this base class are shown in Listing 13.11. As usual within WIR, there is a `getType` method (cf. Line 2) that allows to conveniently determine a region's type using the enumeration type `WIR_CTNodeType` from Line 32. Structural analysis generally distinguishes between acyclic and cyclic control regions, and these properties can be queried using methods `isAcyclic` (Line 5) and `isCyclic` (Line 8), resp. A region's abovementioned unique entry can be obtained with method `getEntry` (Line 11) that returns a constant reference to the identified entry node. Whether or not a region actually is some entry of another, surrounding region can be checked using `isEntry` (Line 14).

Listing 13.11: Public Interface of Class WIR_ControlTreeNode

```

1 // Get a control region's type.
2 virtual WIR_CTNodeType WIR_ControlTreeNode::getType( void ) const = 0;
3
4 // Determine whether a control region is acyclic or not.
5 bool WIR_ControlTreeNode::isAcyclic( void ) const;
6
7 // Determine whether a control region is cyclic or not.
8 virtual bool WIR_ControlTreeNode::isCyclic( void ) const = 0;
9
10 // Get a control region's unique entry node.
11 virtual const WIR_ControlTreeNode &WIR_ControlTreeNode::getEntry( void ) const = 0;
12
13 // Determine whether a region is the entry node of some parent or not.
14 bool WIR_ControlTreeNode::isEntry( void ) const;
15
16 // Get the set of all child regions.
17 const WIR_ControlTreeNodeSet &WIR_ControlTreeNode::getChildds( void ) const;
18
19 // Get a reference to a region's immediate parent in the hierarchy, or a reference to itself.
20 WIR_ControlTreeNode &WIR_ControlTreeNode::getParent( void ) const;
21
22 // Get a reference to a region's top-level root in the hierarchy, or a reference to itself.
23 WIR_ControlTreeNode &WIR_ControlTreeNode::getRoot( void ) const;
24
25 // Get the set of all basic blocks in the hierarchy below this regions.
26 WIR_BasicBlockSet WIR_ControlTreeNode::getBasicBlocks( void ) const;
27
28 // Get a region's set of back edges.
29 const WIR_ControlTreeEdgeSet &WIR_ControlTreeNode::getBackEdges( void ) const;
30
31 // Enum WIR_CTNodeType represents different types of WIR control regions.
32 enum class WIR_CTNodeType : char
33 {
34     bb,           // A single basic block.
35     block,        // A block schema.
36     ifthen,       // An if-then region.
37     ifthenelse,   // An if-then-else region.
38     switchcase,   // A switch-case region.
39     selfloop,     // A self loop.
40     whileloop,    // A regular for- or while-do loop.
41     naturalloop,  // A natural do-while loop.
42     proper,       // An acyclic proper region.
43     improper      // A cyclic improper region.
44 };

```

The next methods `getChildds` (Line 17), `getParent` (Line 20) and `getRoot` (Line 23) permit to navigate within the hierarchy of nested control regions in analogy to hierarchical registers (cf. Section 9.1.1.2). Method `getBasicBlocks` (Line 26) provides the set of (wrapped) references to all basic blocks that are somehow (hierarchically) contained by a control region. For cyclic regions, `getBackEdges` (Line 29) returns a reference to the set of identified back edges, and the returned type `WIR_ControlTreeEdgeSet` is declared similar to the standard `CFG` edges from Listing 13.6.

Fig. 13.4 shows the acyclic control structures that are recognized by the analysis. We quickly introduce the meaning and the key properties of these regions in the following.

Basic blocks: Basic block tree nodes form the leafs of the hierarchical control tree produced by structural analy-

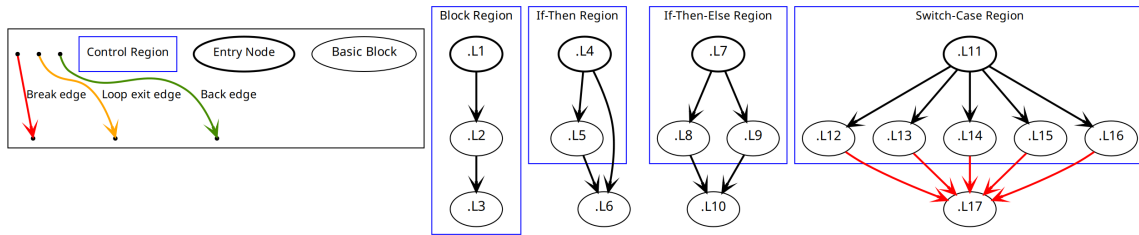


Figure 13.4: Acyclic, Structured Control Regions

sis. In Fig. 13.4, they are represented by the ellipses labeled with the block names. Due to their simplicity, class `WIR_BasicBlockTreeNode` provides no particular interface except method `getBasicBlock` that allows to retrieve the actual `WIR` basic block:

```
1 // Get the basic block represented by this region.
2 WIR_BasicBlock &WIR_BasicBlockTreeNode::getBasicBlock( void ) const;
```

Block regions: Block regions are not to be confused with the basic block nodes above, as block regions represent sequences of maximal length of tree nodes. Control flow enters a block region at its entry node and only follows the sequence of nodes contained by the block region. When reaching the block region's very last node, control flow leaves the region. Due to this sequential behavior, block regions have both the single-entry and single-exit property, see Fig. 13.4. Access to the sequence of nodes contained by a block region is provided using method `getBlockList` that returns the list of (wrapped) references to the region's nodes in their sequential order:

```
1 // Get the list of all contained nodes in their sequential order.
2 const std::list<std::reference_wrapper<WIR_ControlTreeNode>> &WIR_BlockTreeNode::getBlockList( void )
   const;
```

If-then structures: An if-then region contains exactly two nodes, a condition node and a branch node. The condition node is the region's entry and depending on its evaluation of some condition, control flow proceeds either to the branch node or to some other node outside the if-then. All involved nodes (condition, branch and outside node) are different, the condition node has exactly two successors, and the branch node has exactly one predecessor (the branch node) and one successor. Both the condition and the branch node share the same node outside the region as successor (cf. Fig. 13.4). The region's condition and branch nodes can be accessed using the below methods of class `WIR_IfThenTreeNode`:

```
1 // Get an if-then's condition node.
2 const WIR_ControlTreeNode &WIR_IfThenTreeNode::getCondition( void ) const;
3
4 // Get an if-then's then-branch node.
5 const WIR_ControlTreeNode &WIR_IfThenTreeNode::getBranch( void ) const;
```

If-then-else structures: Similar to if-then regions, the if-then-else region consists of a condition node branching to two different successors. However, these successors represent the two different branches of an if-then-else construct and thus are both part of the region. The two branches continue with the very same exit node outside the if-then-else, see Fig. 13.4. As a consequence, class `WIR_IfThenElseTreeNode` provides the methods `getCondition`, `getBranch1` and `Branch2` to access its internal nodes:


```

1 // Get an if-then-else's condition node.
2 const WIR_ControlTreeNode &WIR_IfThenElseTreeNode::getCondition( void ) const;
3
4 // Get an if-then-else's first branch node.
5 const WIR_ControlTreeNode &WIR_IfThenElseTreeNode::getBranch1( void ) const;
6
7 // Get an if-then-else's second branch node.
8 const WIR_ControlTreeNode &WIR_IfThenElseTreeNode::getBranch2( void ) const;

```

Switch-cases: Are a generalization of if-then-else structures in that sense that the condition node can branch to more than 2 cases within this region, cf. Fig. 13.4. All the case nodes share the condition node as predecessor. A case node can break out of the switch-case to a node outside the region as successor, where all cases share this very same node as common exit node. Alternatively, a case node can proceed to one other case node in the presence of a fall-through (not shown in Fig. 13.4). All predecessors of case nodes must be either the condition node or some other case node. The public API of class `WIR_SwitchCaseTreeNode` is quite straightforward, methods `getCondition` and `getCases` below return the region's condition node and the list of all identified case nodes, resp. Method `getFallthroughCases` helps to identify all those case nodes that exit with a fallthrough. It returns a list of iterators, where each iterator refers to the position of a fallthrough case in the list provided by `getCases`.

```

1 // Get a switch-case's condition node.
2 const WIR_ControlTreeNode &WIR_SwitchCaseTreeNode::getCondition( void ) const;
3
4 // Get the list of all contained case nodes in their sequential order.
5 const std::list<std::reference_wrapper<WIR_ControlTreeNode>>
6   &WIR_SwitchCaseTreeNode::getCases( void ) const;
7
8 // Get the list of all fall-through case nodes in their sequential order.
9 std::list<std::list<std::reference_wrapper<WIR_ControlTreeNode>>::const_iterator>
10  WIR_SwitchCaseTreeNode::getFallthroughCases( void ) const;

```

All control regions presented so far and shown in Fig. 13.4 are acyclic regions that do not possess any back edges. Fig. 13.5 now depicts the regular cyclic regions recognized by the structural analysis.

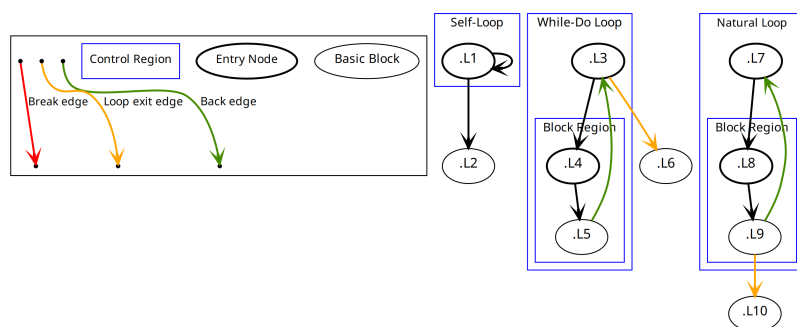


Figure 13.5: Cyclic, Structured Control Regions

Self-loops: The most simple cyclic control region is a self-loop that consists of exactly one node that either branches back to itself or to some outside node (cf. Fig. 13.5). The corresponding class `WIR_SelfLoopTreeNode` features no particular methods that go beyond that of its base class shown in Listing 13.11. Especially, method `getEntry` (cf. Line 11 of Listing 13.11) can be used to retrieve a self-loop's only node.

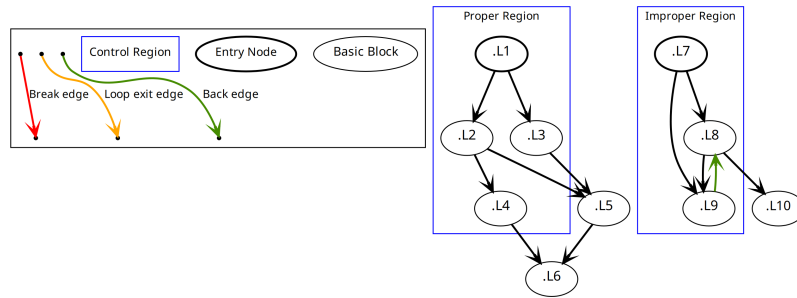


Figure 13.6: Unstructured Control Regions

While-do loops: This region represents classical head-controlled loops like, e.g., `for` or `while-do` loops. A head-controlled loop is characterized by an entry node that checks the loop's condition and either continues with the loop body or some node outside the loop region. Thus, this control region consists of two nodes, the entry node and the loop body. Only predecessor of the loop body is the entry node, and the loop body jumps back to the entry node via a back edge. The body can break out of its loop via break edges (not shown in Fig. 13.4). In this case, the target of such a break edge must be the same node outside the loop that is also reached from the loop's entry. Again, the region's class `WIR_WhileLoopTreeNode` is simple – besides method `getEntry` to obtain the loop's entry node, it only comes with method `getNodes` returning the set of all nodes in this region:

```
1 // Get a while loop's inner nodes.
2 const WIR_ControlTreeNodeSet &WIR_WhileLoopTreeNode::getNodes( void ) const;
```

The returned set of type `WIR_ControlTreeNodeSet` is defined in analogy to the concepts discussed in Chapter 11.

Natural loops: Represent tail-controlled loops like, e.g., `do-while` loops, where the loop's exit condition is checked after the execution of the loop body. This implies that the region's entry node represents the loop body, and the body's successor is the loop's condition node. This condition node either branches back to the body, or to some exit node outside the loop (cf. Fig. 13.5). Class `WIR_NaturalLoopTreeNode` has exactly the same public API as while-do loops have so that we omit its discussion for the sake of brevity.

The loop regions shown in Fig. 13.5 all have a unique single entry point, but can have multiple exits in the case of breaks. For a structural control flow analysis, it is difficult to unambiguously distinguish the loop's regular exit edge from potential break edges. In order to support the analysis, so-called break containers of class `WIR_Break` (cf. file `WIR/containers/wirbreak.h`) can be attached to branch operations that jump to a target outside a loop. If, e.g., a compiler's code selector creates this kind of hints whenever it exhibits a loop break, WIR's structural analysis can produce loop regions that precisely match the source code's structure. The container class `WIR_Break` comes with absolutely no specific properties or methods, the structural analysis simply checks whether such a container exists for a given branch or not.

All regions shown in Figs. 13.4 and 13.5 are well-structured in that sense that they inherently possess the single-entry property and thus lead to CFG reducibility. However, certain programming languages like, e.g., ANSI-C, permit to generate unstructured code where certain regions can be entered via several nodes. Clearly, such unstructured constructs should be avoided whenever possible, but structural analysis needs to be able to cope with such unstructured regions. For this reason, the analysis features two additional control regions which are depicted in Fig. 13.6.

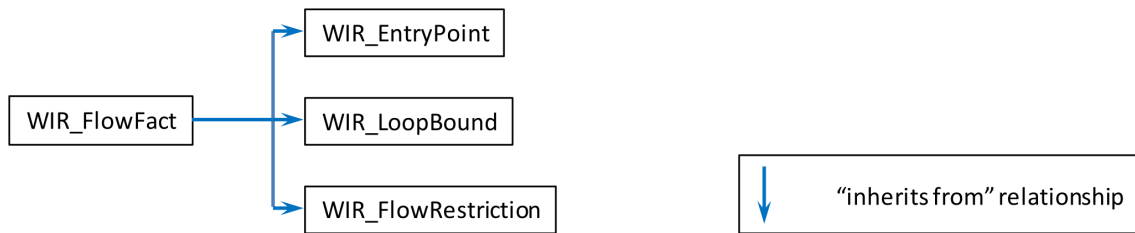


Figure 13.7: Inheritance Diagram of Flow Facts

Proper regions: Such regions are defined to be acyclic, single-entry regions of minimal size that cannot be reduced to any of the well-structured acyclic regions from Fig. 13.4. When considering only the basic blocks .L2, .L4, .L5 and .L6 from Fig. 13.6, then this part of the CFG resembles an if-then-else structure. However, branch .L5 can be reached via a second entry node, .L3. Thus, there are several entries into this unstructured “if-then-else” construct. In such a case, WIR’s structural analysis still enforces that only single-entry regions are created. Thus, the common dominator node (cf. Section 13.4) that is closest to the multiple entries (here: .L2 and .L3) is determined and taken as single entry node for the created unstructured, proper region shown in Fig. 13.6. The nodes contained by a proper region can be retrieved using methods `getEntry` and `getNodes` below:

```

1 // Get a proper region's entry node.
2 virtual const WIR_ControlTreeNode &WIR_ProperTreeNode::getEntry( void ) const;
3
4 // Get a proper region's inner nodes.
5 const WIR_ControlTreeNodeSet &WIR_ProperTreeNode::getNodes( void ) const;

```

Improper regions: Are the same as proper regions, but just for cyclic unstructured CFG regions. Fig. 13.6 shows an example of a while-do loop consisting of the entry .L8 and the loop body .L9. However, the loop body can also be reached from some other node .L7 outside the loop so that this code structure cannot be reduced to any of the regular structures from Fig. 13.5. Improper regions are represented by WIR class `WIR_ImproperTreeNode`, its interface is identical to that of proper regions above.

13.6 Flow Facts for Execution Time Analysis

Analysis Class	Analysis Level	Container Class	Fixing Points	Architectures
Does not apply	Does not apply	WIR_FlowFactRef	Basic Blocks	Generic, all

Static execution time analysis aims to estimate the actual execution time of a piece of code. The analysis’ results usually are the numbers of clock cycles or the physical runtimes of CFG nodes, functions, or entire tasks. Such an execution time analysis regularly inspects the set of possible paths through a task’s CFG. Especially in the presence of cyclic code structures, the number of iterations through each cycle needs to be estimated. This, however, is impossible in general due to the undecidability of the halting problem. For this reason, static timing analyses require human-provided meta-information giving hints about how often control regions are executed. These hints are called flow facts.

WIR does not feature a static timing analysis by itself. However, it contains mechanisms for the annotation and query of flow facts, and this section describes all the underlying concepts. All involved flow fact classes can be found in folder `WIR/flowfacts`, and they are all generic and processor-independent.

Listing 13.12: Public Interface of Class WIR_FlowFact

```

1 // Get a flow fact's type.
2 virtual WIR_FlowFactType WIR_FlowFact::getType( void ) const = 0;
3
4 // Dump a flow fact to an output stream in a human-readable way.
5 friend std::ostream & operator << ( std::ostream &os, const WIR_FlowFact &ff );
6
7 // Determine whether a flow fact is inserted into some system.
8 bool WIR_FlowFact::isInserted( void ) const;
9
10 // Get the system containing a flow fact.
11 WIR_System &WIR_FlowFact::getSystem( void ) const;
12
13 // Enum WIR_FlowFactType represents different types of WIR flow facts.
14 enum class WIR_FlowFactType : char
15 {
16     entrypoint,          // An entry point.
17     flowrestriction,     // A flow restriction.
18     loopbound            // A loop bound.
19 };

```

WIR currently supports three different kinds of flow facts: entry points, loop bounds, and flow restrictions, which all inherit from the abstract base class WIR_FlowFact (cf. file `wirflowfact.h`). Fig. 13.7 shows the inheritance diagram of all flow fact classes. The most relevant properties of this abstract base class are summarized in Listing 13.12. As usual, a method `getType` (cf. Line 2) can be used to quickly determine the type of a flow fact. The result is provided by enumeration type `WIR_FlowFactType` from Line 14. There also exists an output `<<` operator in order to print out flow facts (Line 5). Flow facts are centrally managed by a WIR system. Thus, method `isInserted` (Line 8) can be used to determine whether a flow fact is properly inserted into a system. If so, the system associated with a flow fact can be obtained using `getSystem` (Line 11).

Before going into the details of the various classes of flow facts and their semantics, the public interface of class

Listing 13.13: Flow Fact Handling of Class WIR_System

```

1 // Copy-add a new flow fact at the end of the list of flow facts.
2 WIR_FlowFact &WIR_System::pushBackFlowFact( const WIR_FlowFact &ff );
3
4 // Get the complete list of references of all attached flow facts.
5 const std::list<std::reference_wrapper<WIR_FlowFact>> &WIR_System::getFlowFacts( void ) const;
6
7 // Get the set of all flow facts of a specified type.
8 template<typename T> const std::list<std::reference_wrapper<T>> WIR_System::getFlowFacts( void ) const;
9
10 // Determine whether a system contains a flow fact with the given ID.
11 bool WIR_System::containsFlowFact( WIR_id_t id ) const;
12
13 // Remove the flow fact from the specified list position.
14 void WIR_System::eraseFlowFact( std::list<std::reference_wrapper<WIR_FlowFact>>::const_iterator pos );
15
16 // Remove the flow fact with the given ID.
17 void WIR_System::eraseFlowFact( WIR_id_t id );
18
19 // Remove all flow facts.
20 void WIR_System::clearFlowFacts( void );

```

Listing 13.14: Public Interface of Class WIR_EntryPoint

```

1 // Default constructor.
2 WIR_EntryPoint::WIR_EntryPoint( const WIR_Function &f, const std::map<std::string, std::string> &a );
3
4 // Copy constructor.
5 WIR_EntryPoint::WIR_EntryPoint( const WIR_EntryPoint &ep );
6
7 // Set the function this entry point describes.
8 void WIR_EntryPoint::setFunction( const WIR_Function &f );
9
10 // Get the function marked as entry point.
11 WIR_Function &WIR_EntryPoint::getFunction( void ) const;
12
13 // Get the task attributes associated with an entry point.
14 const std::map<std::string, std::string> &WIR_EntryPoint::getAttributes( void ) const;

```

WIR_System for flow fact management is shown in Listing 13.13. Flow facts can be added to a system using `pushBackFlowFact` (cf. Line 2). This method returns a reference to the newly added flow fact. A list with (wrapped) references to all flow facts of a system can be obtained using `getFlowFacts` from Line 5. The list of already correctly casted flow facts of a specific type can be retrieved by the variant of method `getFlowFacts` from Line 8. `containsFlowFact` (Line 11) returns whether a flow fact with a specific numerical ID exists within a system or not. The first variant of method `eraseFlowFact` from Line 14 erases the flow fact to which a given iterator in the list of all flow facts refers. The second `eraseFlowFact` variant (Line 17) erases the flow fact with a given ID. Finally, method `clearFlowFacts` (Line 20) erases all flow facts attached to a system. As usual within WIR, erased flow facts are automatically destroyed and removed from memory.

The most simple class of flow facts are the entry points represented by class `WIR_EntryPoint`. Their purpose is to mark a given function as entry of a software task modeled within WIR. For single-task systems created from, e.g., C code, the one and only entry point is the function named `main`. For multi-task systems, however, there can be various functions that get executed by a system's scheduler. These functions represent task entries and thus should be identified using entry point flow facts.

Note

As already mentioned on Page 2, WIR currently does not contain any well-designed system-level task model. This will probably change in the future. Since entry point flow facts inherently address task-level information, it is very likely that the entire class `WIR_EntryPoint` will become obsolete since it gets subsumed into the future task managers mentioned on Page 2. Thus, the API of entry points discussed in the following should be considered unstable/obsolete and should not actively be used if possible.

Listing 13.14 shows the relevant parts of the interface of class `WIR_EntryPoint`. As entry points aim to mark WIR functions as task entries, the constructor from Line 2 expects a function being task entry as first argument. The map being the constructor's second argument intends to specify attributes of a task like, e.g., periods or deadlines. This attribute map is fully unstructured as it only uses plain strings (e.g., key "period" could be mapped to value "50ms"), and such task attributes are currently not used and supported within WIR (see above note). Thus, it is sufficient to just pass the empty map to this constructor. Line 5 shows that the class comes with a copy constructor, too. The function marked as entry point can explicitly be set using `setFunction` (Line 8) and retrieved via `getFunction` (Line 11). The (useless) attribute map could be obtained with method `getAttributes` from Line 14.

Loop bound flow facts intend to specify the minimum and maximum number of times that a loop iterates, as

illustrated in the following code examples. Line 1 shows a loop with a constant number of iterations that could be annotated with a loop bound flow fact carrying exactly this iteration count both as minimum and maximum value. A data dependent loop whose number of iterations can vary but still is known to be bounded can be represented by a loop bound as can be seen in Line 5.

```

1 for ( i = 100; i > 0; --i )           /* loopbound min=100 max=100 */
2   [...]
3
4 j = 0;
5 do {                               /* loopbound min=42 max=1234 */
6   [...]
7   ++j;
8 } while ( j < n );

```

Listing 13.15 shows the public interface of class `WIR_LoopBound`. Its constructor (cf. Line 2) gets the values for the minimum and maximum iteration counts as first arguments. Next, the `WIR` basic block `b` that is the unique entry of the annotated loop according to the structural control flow analysis from Section 13.5 needs to be given.

Listing 13.15: Public Interface of Class `WIR_LoopBound`

```

1 // Default constructor.
2 WIR_LoopBound::WIR_LoopBound( int min, int max, const WIR_BasicBlock &b,
                               WIR_LoopBound::LoopControlType ctrl = LoopControlType::unknown,
                               bool chk = true );
3
4 // Copy constructor.
5 WIR_LoopBound::WIR_LoopBound( const WIR_LoopBound &lb );
6
7 // Set the minimum number of iterations of a loop.
8 void WIR_LoopBound::setMin( int min );
9
10 // Get the minimum number of iterations of a loop.
11 int WIR_LoopBound::getMin( void ) const;
12
13 // Set the maximum number of iterations of a loop.
14 void WIR_LoopBound::setMax( int max );
15
16 // Get the maximum number of iterations of a loop.
17 int WIR_LoopBound::getMax( void ) const;
18
19 // Set the loop that this loop bound is referring to.
20 void WIR_LoopBound::setLoop( const WIR_BasicBlock &b, LoopControlType ctrl = LoopControlType::unknown,
                               bool chk = true );
21
22 // Get the entry block of the loop that this loop bound is referring to.
23 const WIR_BasicBlock &WIR_LoopBound::getLoop( void ) const;
24
25 // Get the control type of the annotated loop.
26 WIR_LoopBound::LoopControlType WIR_LoopBound::getLoopControlType( void ) const;
27
28 // Enum LoopControlType represents the type of a loop.
29 enum class WIR_LoopBound::LoopControlType : char
30 {
31     headcontrolled,    // A head-controlled loop (for and while-do loops).
32     tailcontrolled,    // A tail-controlled loop (do-while loops).
33     unknown            // Unknown loop control.
34 };

```

The class of the annotated loop, i.e., whether it is head- or tail-controlled, can be specified using enumeration type `LoopControlType` (Line 29). Finally, the Boolean argument `chk` defaulting to `true` activates or deactivates a sanity check carried out by the constructor. This sanity check applies structural analysis and verifies that the given basic block `b` definitely is entry point of a loop. If this is not the case, the constructor fails with a runtime error. There is nothing special to say about the copy constructor from Line 5.

Methods `setMin` and `getMin` (Lines 8 and 11) set and retrieve the minimum iteration count of a loop, resp., the equivalent methods `setMax` and `getMax` (Lines 14 and 17) exist for the maximum iteration count. The loop entry block can be specified using `setLoop` (Line 20), the arguments related to the loop's control type and the sanity check are identical to those of the default constructor from Line 2. The loop's entry block can be retrieved via method `getLoop` (Line 23), and likewise for the loop control type using `getLoopControlType` from Line 26.

Flow restrictions are the most powerful and general class of flow facts. They allow to specify linear inequations relating execution frequencies of different locations in the code with each other. A typical use case is the annotation of triangular loops where the number of iterations of the inner `j`-loop below depends on an outer `i`-loop:

```

1  outside: i = 0;
2      while ( i < 10 ) {
3          j = i;
4          while ( j < 10 ) {
5              [...]
6  inside:  ++j;
7          }
8          ++i;
9      }
                                     /* flowrestriction 1*inside <= 55*outside */

```

The flow restriction from Line 9 upper-bounds the execution frequency at label `inside` by 55 times the execution frequency at label `outside`. As can be seen, flow restrictions do a less-equal comparison of terms on the left-hand side with terms on the right-hand side. These terms consist of a positive integer constant C that is multiplied by the supposed execution frequency $freq$ at some code location. In general, there can be many such terms on both sides, and they are all summed up on their respective sides of the inequation: $\sum_i C_i * freq_i \leq \sum_j C_j * freq_j$.

The code locations to which a flow restriction's summands refer usually reside inside and outside of some loop (nest) as in the above example. However, these locations can also be spread over different functions, which allows to annotate recursive functions as shown in the following:

```

1      int fib( int n )
2      {
3          if ( ( n == 0 ) || ( n == 1 ) )
4              return( 1 );
5          else
6              return( fib( n - 1 ) + fib( n - 2 ) );
7      }
8
9      int main()
10     {
11         [...]
12
13  call:  res = fib( 10 );
14
15         [...]
16     }
                                     /* flowrestriction 1*fib <= 177*call */

```


Here, the flow restriction from Line 13 specifies an upper limit of the execution frequency of function fib which is 177 times the execution frequency at label call.

Listing 13.16 shows the public API of class WIR_FlowRestriction. Individual summands are represented by pairs of int numbers and (wrapped) references to WIR basic blocks that denote the execution frequency at the block's location. Lists of such pairs finally represent an inequation's left- or right-hand side, resp. Thus, the constructor from Line 2 obtains two such lists as arguments and builds the flow restriction's left- and right-hand sides from them. It is enforced that only positive constant factors C can be used in both sides of the inequation. As usual, a copy constructor is provided in Line 5.

Adding summands to the left-hand side can be done using method addToLHS (Line 8). It checks whether a summand referring to the same basic block b already exists on the left-hand side. If so, its current constant factor

Listing 13.16: Public Interface of Class WIR_FlowRestriction

```

1 // Default constructor with copy-added left- and right-hand sides.
2 WIR_FlowRestriction::WIR_FlowRestriction( const std::list<
      std::pair<
        int,
        std::reference_wrapper<const WIR_BasicBlock>>> &lhs,
      const std::list<
        std::pair<
          int,
          std::reference_wrapper<const WIR_BasicBlock>>> &rhs );
3
4 // Copy constructor.
5 WIR_FlowRestriction::WIR_FlowRestriction( const WIR_FlowRestriction &fr );
6
7 // Add a summand to a flow restriction's left-hand side.
8 void WIR_FlowRestriction::addToLHS( int c, const WIR_BasicBlock &b );
9
10 // Add a summand to a flow restriction's right-hand side.
11 void WIR_FlowRestriction::addToRHS( int c, const WIR_BasicBlock &b );
12
13 // Remove a summand from a flow restriction.
14 int WIR_FlowRestriction::eraseSummand( const WIR_BasicBlock &b );
15
16 // Replace a summand in a flow restriction.
17 void WIR_FlowRestriction::replaceSummand( const WIR_BasicBlock &bOld, const WIR_BasicBlock &bNew );
18
19 // Get the left-hand side of a flow restriction.
20 const std::list<std::pair<int, std::reference_wrapper<const WIR_BasicBlock>>> &getLHS( void ) const;
21
22 // Get the right-hand side of a flow restriction.
23 const std::list<std::pair<int, std::reference_wrapper<const WIR_BasicBlock>>> &getRHS( void ) const;
24
25 // Determine whether a basic block belongs to a flow restriction's left-hand side.
26 bool WIR_FlowRestriction::isPartOfLHS( const WIR_BasicBlock &b ) const;
27
28 // Determine whether a basic block belongs to a flow restriction's right-hand side.
29 bool WIR_FlowRestriction::isPartOfRHS( const WIR_BasicBlock &b ) const;
30
31 // Determine whether a basic block belongs to a flow restriction at all.
32 bool WIR_FlowRestriction::isPartOfFlowFact( const WIR_BasicBlock &b ) const;
33
34 // Determine whether two flow restrictions are equal.
35 bool WIR_FlowRestriction::isEqual( const WIR_FlowRestriction &fr ) const;

```


Listing 13.17: Flow Fact Handling of Container Class WIR_FlowFactRef

```

1 // Statically create and get a FlowFactRef container for a given basic block.
2 static WIR_FlowFactRef &WIR_FlowFactRef::get( WIR_BasicBlock &b );
3
4 // Get the list of all flow facts referring to the basic block to which this container is attached.
5 const std::list<std::reference_wrapper<WIR_FlowFact>> &WIR_FlowFactRef::getFlowFacts( void ) const;
6
7 // Get the list of all flow facts of a specified type.
8 template<typename T> const std::list<std::reference_wrapper<T>> WIR_FlowFactRef::getFlowFacts( void ) const;

```

is incremented by the value provided in argument *c*. Otherwise, a completely new summand will be added. If the resulting constant factor of the summand becomes negative, the summand is moved from the left- to the right-hand side so that always positive constants show up in a flow restriction. If the referred basic block *b*, however, already shows up on the right-hand side, its current constant factor is decremented by *c* and possibly moved onto the left-hand side if it becomes negative. Method `addToRHS` (Line 11) behaves the same, just with opposite right- and left-hand sides. Summands whose constant factor becomes zero during `addToLHS` or `addToRHS` are erased from the flow restriction, as they are no longer significant. Method `eraseSummand` (Line 14) can be used to remove a summand referring to a given basic block from a flow restriction, regardless of whether the summand is part of the left- or right-hand side. The summand's current constant factor is returned by this method. Replacing an old basic block that some summand referred to by a new basic block can be achieved using `replaceSummand` (Line 17). During such a replacement, the previous constant factor of the modified summand remains unchanged.

Access to the full left- and right-hand sides of a flow restriction is provided by methods `getLHS` and `getRHS` (Lines 20 and 23), resp. Whether or not a certain basic block is referred to by some summand on the left- or right-hand side, or any of these, can be checked using `isPartOfLHS` (Line 26), `isPartOfRHS` (Line 29) and `isPartOfFlowFact` (Line 32), resp. Finally, method `isEqual` (Line 35) compares two flow restrictions for strict equality which means that all summands of both flow restrictions must be at exactly the same position in either their left- or right-hand sides, and the summands all must have the same constant factor and must refer to the very same basic block.

This quick overview about the different flow fact classes shows that they all refer to certain positions within **WIR** code – entry points refer to the beginning of a function, i.e., to a function's very first basic block; loop bounds refer to the entry block of a loop; and a flow restriction refers to an arbitrary number of basic blocks. This property motivates that solely accessing flow facts from their **WIR** system as shown in Listing 13.13 is not always ideal. Instead, an additional mechanism to access all flow facts referring to a certain basic block might be convenient. Such a mechanism is provided by **WIR** container class `WIR_FlowFactRef` which automatically gets attached to basic blocks. Whenever a flow fact is inserted into some **WIR** system, it is ensured that appropriate containers are created if they do not yet exist for the referred basic block(s), and that the newly inserted flow fact is introduced to the corresponding container(s). Upon a removal of a flow fact from a system, it furthermore also gets removed from all `WIR_FlowFactRef` containers.

The public interface of this container class is shown in Listing 13.17. It does not feature any public constructors, as the management of these containers is exclusively delegated to **WIR** systems and the individual flow facts themselves. However, the static `get` method from Line 2 should be used in order to obtain a `WIR_FlowFactRef` instance for a given basic block. If no such container is attached to the specified basic block yet, an empty container is created and returned. Access to all flow facts that somehow refer to that basic block to which a `WIR_FlowFactRef` container is attached is given by method `getFlowFacts` (Line 5). The template-based variant of this method from Line 8 can be used if only a certain type of flow facts like, e.g., loop bounds, referring to the container's basic block shall be inspected.

13.7 Reaching Definitions Analysis

Analysis Class	Analysis Level	Container Class	Fixing Points	Architectures
WIR_ReachingDefinitions-Analysis	Functions	WIR_Reaching-Definitions	Instructions	Generic, all

The first [WIR](#) data flow analysis presented in this chapter is the reaching definitions analysis realized by class `WIR_ReachingDefinitionsAnalysis` (see file `WIR/analysis/reachingdefinitions/wirreachingdefinitionsanalysis.h`). For each instruction i , it determines the set of definitions, i.e., of writes to some register r , that may reach the instruction i without being overwritten by some other re-definition in between. [WIR](#)'s reaching definitions analysis closely follows the concepts from [1, chapter 17.2] and [22, chapter 8.1].

Reaching definitions are computed at the scope of individual [WIR](#) functions f . For each individual register r , set $defs(r)$ contains all those parameters within the entire function f that somehow modify r , i.e., that either define or define-use r (cf. Section 8.6.1):

$$defs(r) := \{ \text{register parameters } rp \in f \mid (rp.getRegister() = r) \wedge (rp.isDefined() \vee rp.isDefUsed()) \} \quad (13.3)$$

For some instruction i , set $defs(i)$ contains all the registers that are modified by i :

$$defs(i) := \{ \text{registers } r \in f \mid \exists \text{register parameter } rp \in i : (rp.getRegister() = r) \wedge (rp.isDefined() \vee rp.isDefUsed()) \} \quad (13.4)$$

Note the difference between the two sets: the former contains locations in [WIR](#) code where a register is overwritten, while the latter contains all registers that somehow get overwritten by an instruction.

An instruction i generates new definitions, i.e., by the end of i , the parameters in set $gen(i)$ contain newly written values:

$$gen(i) := \{ \text{register parameters } rp \in i \mid rp.isDefined() \vee rp.isDefUsed() \} \quad (13.5)$$

An instruction not only generates new definitions, but it also kills previous definitions. If a definition of some register r is assumed to reach the beginning of i , but i overwrites this register with a new value, then this previous definition never reaches the end of i . i thus kills the previous definition of r . All potential definitions of some register r are captured in set $defs(r)$ from Eq. (13.3). Subtracting the set $gen(i)$ of newly generated definitions yields set $kill(i)$:

$$kill(i) := \bigcup_{\text{registers } r \in defs(i)} defs(r) - gen(i) \quad (13.6)$$

Based on the sets gen and $kill$ per instruction i , the sets of definitions that reach the beginning and the end of i , resp., are defined by the following two data flow equations:

$$ReachDefsIn(i) := \bigcup_{p \in pred(i)} ReachDefsOut(p) \quad (13.7)$$

$$ReachDefsOut(i) := gen(i) \cup (ReachDefsIn(i) - kill(i)) \quad (13.8)$$

Eq. (13.8) states that everything that is generated by i reaches the end of i , as well as everything that reaches the beginning of i but is not killed by i . Whatever reaches the end of some predecessors p of i also reaches the beginning of i , as stated by Eq. (13.7).

These data flow equations are solved by iteration over a function's [CFG](#) in reverse postorder within class `WIR_ReachingDefinitionsAnalysis`. Since the mechanisms within [WIR](#) to model [CFG](#) predecessors and definitions of registers are generic and processor-independent, reachability analysis does not require any adaptations for specific architectures. Besides the default analysis [API](#) from Listing 13.1, the two methods shown in

Listing 13.18: Public Interface of Class WIR_ReachingDefinitionsAnalysis

```

1 // Set whether reaching definitions of virtual registers only are analyzed.
2 void WIR_ReachingDefinitionsAnalysis::setVirtualRegistersOnly( bool vRegsOnly = true );
3
4 // Determine whether reaching definitions of virtual registers only are analyzed.
5 bool WIR_ReachingDefinitionsAnalysis::getVirtualRegistersOnly( void ) const;

```

Listing 13.18 are provided. They control whether the analysis only considers definitions of virtual registers, or whether both virtual and physical registers as well as their precolorings are analyzed. For this purpose, method `setVirtualRegistersOnly` (cf. Line 2) sets this analysis behavior using the Boolean argument defaulting to `true`. The current status of the analysis can be retrieved using `getVirtualRegistersOnly` (Line 5). In general, reaching definitions analysis is inherently able to deal with arbitrary mixtures of virtual and physical registers, as well as with arbitrary hierarchies of registers (cf. Section 9.1.1.2 on Page 69). Furthermore, registers defined outside some function `f` but which serve as inputs to `f` (cf. Listing 8.5 on Page 44) are also properly considered during this analysis.

The analysis stores the determined sets *ReachDefsIn* from Eq. (13.7) in `WIR_ReachingDefinitions` containers that are attached to `WIR` instructions. The public interface of this container class is shown in Listing 13.19. Found reaching definitions can be added using method `insertReachingDefinition` (cf. Line 2). This method does not do any sanity check whether the given register parameter actually is a definition or not, it should thus be used with extreme care. All identified incoming reaching definitions can be obtained via method `getReachingDefinitions` (Line 5). Similar `insert` and `get` methods furthermore exist for incoming reaching function inputs, see Lines 8 and 11. The sets of register parameters or physical registers returned by `getReachingDefinitions` and `getReachingInputs`, resp., are declared along the lines described earlier in Chapter 11.

Listing 13.19: Reaching Definitions and Inputs Handling of Container Class WIR_ReachingDefinitions

```

1 // Add a new definition to the set of incoming reaching definitions.
2 void WIR_ReachingDefinitions::insertReachingDefinition( const WIR_RegisterParameter &rp );
3
4 // Get the set of incoming reaching definitions.
5 const WIR_RegisterParameterSet &WIR_ReachingDefinitions::getReachingDefinitions( void );
6
7 // Add a new input register to the set of incoming reaching function inputs.
8 void WIR_ReachingDefinitions::insertReachingInput( const WIR_PhysicalRegister &p );
9
10 // Get the set of incoming reaching function inputs.
11 const WIR_PhysicalRegisterSet &WIR_ReachingDefinitions::getReachingInputs( void );

```

13.8 Available Definitions Analysis

Analysis Class	Analysis Level	Container Class	Fixing Points	Architectures
WIR_AvailableDefinitions-Analysis	Functions	WIR_Available-Definitions	Instructions	Generic, all

Available definitions (cf. file `WIR/analysis/availabledefinitions/wiravailabledefinitions.h`) analysis is very similar to reaching definitions analysis. While reaching definitions is a may analysis, available definitions analysis answers the question which register definitions definitely must reach an instruction `i` along all control flow paths [1, chapter 17.2].

The previous definitions of the *gen* and *kill* sets from Eqs. (13.5) and (13.6) remain unchanged, only the data flow equations that propagate analysis results through a function's CFG change slightly:

$$AvailDefsIn(i) := \bigcap_{p \in pred(i)} AvailDefsOut(p) \quad (13.9)$$

$$AvailDefsOut(i) := gen(i) \cup (AvailDefsIn(i) - kill(i)) \quad (13.10)$$

As can be seen from Eq. (13.9), available definitions analysis applies the intersection to all predecessor's outgoing available definitions, which is the natural formalization for a must analysis. The previous use of the union operator in Eq. (13.7), however, is the distinguishing feature of a may analysis. Other than that, WIR class `WIR_AvailableDefinitionsAnalysis` behaves identical to what has been discussed previously in Section 13.7. In analogy to Listing 13.18, the available definitions analysis can also be configured to consider virtual register definitions only.

For the sake of completeness, Listing 13.20 shows the public API of the analysis' container class `WIR_AvailableDefinitions`, which gets attached to WIR instructions again. There is no difference at all compared to Listing 13.19 so that we omit the presentation of the container's methods here.

Listing 13.20: Available Definitions and Inputs Handling of Container Class `WIR_AvailableDefinitions`

```

1 // Add a new definition to the set of incoming available definitions.
2 void WIR_AvailableDefinitions::insertAvailableDefinition( const WIR_RegisterParameter &rp );
3
4 // Get the set of incoming available definitions.
5 const WIR_RegisterParameterSet &WIR_AvailableDefinitions::getAvailableDefinitions( void );
6
7 // Add a new input register to the set of incoming available function inputs.
8 void WIR_AvailableDefinitions::insertAvailableInput( const WIR_PhysicalRegister &p );
9
10 // Get the set of incoming available function inputs.
11 const WIR_PhysicalRegisterSet &WIR_AvailableDefinitions::getAvailableInputs( void );

```

13.9 Def-Use/Use-Def Chains Analysis

Analysis Class	Analysis Level	Container Class	Fixing Points	Architectures
<code>WIR_DUUDChainAnalysis</code>	Functions	<code>WIR_DUUDChain</code>	Register Parameters	Generic, all

The combined def-use and use-def chain analysis of WIR realized by class `WIR_DUUDChainAnalysis` (see file `WIR/analyses/wirduudchainanalysis.h`) bases on the reaching definitions analysis from Section 13.7. While reaching definitions provides the set of all register definitions that may reach the beginning of an instruction, def-use and use-def chains represent the same information but limited to only one given register parameter *rp* [1, chapter 17.4] and [22, chapter 8.10]. More precisely, a def-use chain is defined as follows:

Definition 13.1 (Def-Use Chain)

For a register parameter rp_{def} defining some register *r*, its def-use chain is the set of all register parameters rp_{use}

using the very same register r such that the definition from rp_{def} may reach rp_{use} .

$$\begin{aligned} \forall \text{register parameters } rp_{def} : & \quad rp_{def}.getUsage() \in \{\text{def}, \text{defuse}\} : \\ DU\text{-}Chain(rp_{def}) := & \{ \text{register parameters } rp_{use} \mid (rp_{use}.getUsage() \in \{\text{use}, \text{defuse}\}) \wedge \\ & (rp_{def}.getRegister() = rp_{use}.getRegister()) \wedge \\ & (rp_{def} \in ReachDfsIn(rp_{use}.getOperation().getInstruction())) \} \end{aligned} \quad (13.11)$$

Similarly, a use-def chain for a used register parameter consists of all definitions of the very same register that may reach the use:

Definition 13.2 (Use-Def Chain)

For a register parameter rp_{use} using some register r , its use-def chain is the set of all register parameters rp_{def} defining the very same register r such that the definitions from rp_{def} may reach rp_{use} .

$$\begin{aligned} \forall \text{register parameters } rp_{use} : & \quad rp_{use}.getUsage() \in \{\text{use}, \text{defuse}\} : \\ UD\text{-}Chain(rp_{use}) := & \{ \text{register parameters } rp_{def} \mid (rp_{def}.getUsage() \in \{\text{def}, \text{defuse}\}) \wedge \\ & (rp_{use}.getRegister() = rp_{def}.getRegister()) \wedge \\ & (rp_{def} \in ReachDfsIn(rp_{use}.getOperation().getInstruction())) \} \end{aligned} \quad (13.12)$$

The def-use/use-def chain analysis thus first performs a reaching definitions analysis and then restructures its results such that the chains from Definitions 13.1 and 13.2 are formed. Of course, hierarchies and precolorings of registers are properly considered again, as well as registers serving as function input. Not surprisingly, def-use/use-def chain analysis is also fully generic and does not require any processor-specific adaptations. Due to its strong relationship to reaching definitions, the public interface of class `WIR_DUUDChainAnalysis` is identical to that already shown earlier in Listing 13.18.

Since def-use and use-def chains are defined for individual register parameters, the analysis attaches containers of class `WIR_DUUDChain` to all such register parameters within an analyzed `WIR` function. Listing 13.21 shows the public interface of this container class. The class provides the method `insertDUChain` to add a use to the def-use chain of the register parameter to which the container is attached (cf. Line 2), while method `getDUChains` (Line 5) retrieves the register parameter set forming a def-use chain. Similarly, methods `insertUDChain` (Line 8) and

Listing 13.21: Def-Use/Use-Def Chain and Inputs Handling of Container Class `WIR_DUUDChain`

```

1 // Add a new use to the def-use chain.
2 void WIR_DUUDChain::insertDUChain( const WIR_RegisterParameter &rp );
3
4 // Get the def-use chain.
5 const WIR_RegisterParameterSet &WIR_DUUDChain::getDUChains( void );
6
7 // Add a new definition to the use-def chain.
8 void WIR_DUUDChain::insertUDChain( const WIR_RegisterParameter &rp );
9
10 // Get the use-def chain.
11 const WIR_RegisterParameterSet &WIR_DUUDChain::getUDChains( void );
12
13 // Add a new defined input register to the use-def chain.
14 void WIR_DUUDChain::insertUDInput( const WIR_PhysicalRegister &p );
15
16 // Get the use-def chain for external function inputs.
17 const WIR_PhysicalRegisterSet &WIR_DUUDChain::getUDInputs( void );

```

getUDChain (Line 11) can be used to handle use-def chains. Since use-def chains also need to consider external function inputs provided by dedicated physical registers, methods insertUDInput (Line 14) and getUDInputs (Line 17) exist to extend and retrieve these sets of input registers belonging to a use-def chain.

13.10 Liveness Analysis

Analysis Class	Analysis Level	Container Class	Fixing Points	Architectures
WIR_LivenessAnalysis	Functions	WIR_LiveOut	Instructions	Generic, all

Liveness analysis [1, chapter 10] and [22, chapter 14.1.3] is a prerequisite for any kind of register allocation. It determines from where to where within an analyzed WIR function certain registers are alive, i.e., contain useful values that might be needed as input by some later instruction. Using this information, a register allocator can determine whether the liveness ranges of two virtual registers overlap somewhere and can then ensure that these two virtual registers are never allocated to the same physical register.

The analysis is realized by class WIR_LivenessAnalysis which can be found in file WIR/analyses/liveness/wirlivenessanalysis.h. Within WIR, liveness is considered at the level of individual instructions. In analogy to reaching definitions analysis (cf. Section 13.7), it bases on the sets of registers that are defined or used by some instruction i :

$$defs(i) := \{ \text{registers } r \in f \mid \exists \text{register parameter } rp \in i : (rp.getRegister() = r) \wedge (rp.isDefined() \vee rp.isDefUsed()) \} \quad (13.13)$$

$$uses(i) := \{ \text{registers } r \in f \mid \exists \text{register parameter } rp \in i : (rp.getRegister() = r) \wedge (rp.isUsed() \vee rp.isDefUsed()) \} \quad (13.14)$$

A register r is said to be alive along a CFG edge if there is a directed path from that edge to some use of r that contains no other re-definition of r . The sets of registers that are alive at the beginning and the end of an instruction i are defined by the following two data flow equations:

$$LiveIn(i) := uses(i) \cup (LiveOut(i) - defs(i)) \quad (13.15)$$

$$LiveOut(i) := \bigcup_{s \in succ(i)} LiveIn(s) \quad (13.16)$$

Eq. (13.15) states that everything that is used by i is live-in, as well as everything that is live-out at i and that is not re-defined by i . Whatever is live-in at the beginning of some successors s of i is also live-out at i which is modeled by Eq. (13.16).

These explanations show that liveness gets propagated through a function's CFG in the reverse sense of the graph's edges, i.e., liveness analysis is a backward analysis. These equations are thus solved by iteration through the CFG in reverse-topological order within class WIR_LivenessAnalysis. Once more, liveness analysis is generic and entirely processor-independent. The public API of the analysis contains no additional methods beyond the default interface from Listing 13.1.

The results of liveness analysis are stored in objects of the container class WIR_LiveOut that are attached to individual WIR instructions. As the name suggests, only the live-out sets are stored for the sake of efficiency. Listing 13.22 shows the public API of this container class, which is kept very similar to the concepts already introduced in Chapter 6.

Method insertRegister (cf. Line 2) can be used to manually add a register to a live-out set. The complete live-out set can be retrieved using getRegisters (Line 5). The containsRegister methods (Lines 8 and 11)

Listing 13.22: Liveness Handling of Container Class WIR_LiveOut

```

1 // Add a register to the live-out set.
2 void WIR_LiveOut::insertRegister( const WIR_BaseRegister &r );
3
4 // Get the live-out set.
5 const WIR_RegisterSet &WIR_LiveOut::getRegisters( void ) const;
6
7 // Determine whether the live-out set contains an item with the given ID.
8 bool WIR_LiveOut::containsRegister( WIR_id_t id ) const;
9
10 // Determine whether the live-out set contains the given register.
11 bool WIR_LiveOut::containsRegister( const WIR_BaseRegister &r ) const;
12
13 // Find the register with the given ID in the live-out set.
14 WIR_RegisterSet::const_iterator WIR_LiveOut::findRegister( WIR_id_t id ) const;
15
16 // Find the given register in the live-out set.
17 WIR_RegisterSet::const_iterator WIR_LiveOut::findRegister( const WIR_BaseRegister &r ) const;
18
19 // Remove the specified register from the live-out set.
20 void WIR_LiveOut::eraseRegister( const WIR_BaseRegister &r );
21
22 // Remove all registers from the live-out set.
23 void WIR_LiveOut::clearRegisters( void );

```

and the `findRegister` methods (Lines 14 and 17) can be used to check whether a register is part of a live-out set, and to find it in the set, resp. Finally, elements can be removed from a live-out set using `eraseRegister` (Line 20) and `clearRegisters` (Line 23), resp.

13.11 Bit-True Data Flow Analysis

Analysis Class	Analysis Level	Container Class	Fixing Points	Architectures
WIR_BitDFA RV_BitDFA, TC_BitDFA	Functions	WIR_BitValues	Register Parameters, Immediate Parameters	RV32IMC, TC1.3, TC1.3.1

While all the data flow analyses presented in the previous Sections 13.7 to 13.10 considered complete registers as a whole, it sometimes might be useful to obtain more fine-grained information about the potential contents of individual bits of a register. This is the purpose of the bit-true data flow analysis [32] whose generic implementation is provided by the abstract base class `WIR_BitDFA` from file `WIR/analyses/bit/wirbitdfa.h`. In contrast to all analyses introduced previously in this chapter, bit-true data flow analysis requires processor-specific implementation details. At the time of writing of this document, parts of the analysis exist for the RISC-V RV32IMC ISA (class `RV32_BitDFA` from file `WIR/arch/riscv/analyses/bit/rv32bitdfa.h`) and for both Infineon TriCore models (class `TC_BitDFA` from file `WIR/arch/tricore/analyses/bit/tcbitdfa.h`).

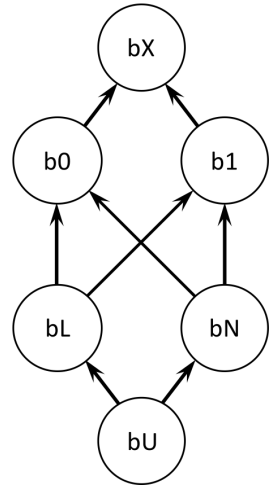
Information about a single bit is represented using the enumeration class `WIR_L4` (see file `WIR/analyses/bit/wirL4.h`) shown in Listing 13.8a. The elements of this enumeration form a partial order depicted in Fig. 13.8b. A completely unknown bit where neither its value nor its origin are known is represented by element `bU` (cf. Line 9). If it is known that a bit provably contains the value 0 or 1, it is represented by elements `b0` or `b1` (cf. Lines 5 and 6), resp. A bit whose value is irrelevant so that it does not have any influence on the result computed by some `WIR` code is modeled by `bX` (Line 4). If the actual value of a bit cannot be determined, but if it is known that the bit must

```

1 // Enum WIR_L4 represents the L4 partial order.
2 enum class WIR_L4 : char
3 {
4     bX,      // Arbitrary bit value, don't care.
5     b0,      // Exactly known bit value of '0'.
6     b1,      // Exactly known bit value of '1'.
7     bL,      // Unknown bit value but known location of the bit's origin.
8     bN,      // Unknown bit value but negated known location of the bit's origin.
9     bU,      // Unknown bit value and unknown origin.
10 };

```

(a) Enumeration Class WIR_L4



(b) Hasse Diagram

Figure 13.8: Partial Order L4

be identical with another bit at a different location, it is represented by bL (Line 7). Finally, enumeration element bN (Line 8) models that a bit's value is unknown, but the bit is the negation of another bit at a different location.

According to the Hasse diagram shown in Fig. 13.8b, element bU represents the least degree of information about a bit. Not knowing a bit's value but being aware of its original location is useful so that bL and bN have a higher position in the Hasse diagram. Even more information is represented by the next level of the partial order, consisting of the actual bit values b0 and b1. Finally, the largest amount of information about a bit is given if it does not care at all so that bX forms the top in Fig. 13.8b. These levels of the partial order represent a < operator along the directed edges of Fig. 13.8b that is also declared in file wir_l4.h.

(Negated) locations of a bit's origin are represented by class WIR_Location from file WIR/analyses/bit/wir_location.h. Similar to classical, register-level data flow analysis, such a location regularly refers to an operation's register parameter that defines the currently considered bit. But in addition to this, the position of the bit from its original location is furthermore provided.

Example

Consider the following code snippet based on the RISC-V RV32 processor model introduced in Section 9.1.4 as example:

```

1      srli      x_3, x10, 2      # x_3 := x10 >> 2
2      andi      x_4, x_3, 7      # x_4 := x_3 & 7

```

Line 1 takes an external function argument provided in the physical argument register x10, shifts it right by 2 bit positions, and puts the result of this shift in virtual register x_3. Hereafter, Line 2 applies a bitwise AND operation by the constant 7 to this virtual register x_3. The result of this AND operation is put in virtual register x_4.

Here and in the following, the bits of a register are enumerated from the least-significant position 0 to the most-significant position 31. Then, the actual value of bit 0 of register x_4 is unknown, since the bit-true data flow analysis of WIR is an intraprocedural analysis that does not track external function arguments. However, it becomes evident from the above code that this bit 0 of x_4 must be the very same as bit 2 of register x10 due to the nature of the srli and andi operations. Likewise, bit 1 of x_4 is equal to bit 3 of x10, and bit 2 of x_4 matches with bit 4 of x10.

The additional line

```
3 xori      x_5, x_4, 4           # x_5 := x_4 ^ 4
```

has the effect that one bit of `x_4` gets inverted. Thus, bit 2 of `x_5` finally is the negation of bit 4 of `x10`.

The interface of class `WIR_Location` is shown in Listing 13.23. A location referring to a certain bit position within some defined register parameter can be created with the constructor shown in Line 2. A location's register parameter can be retrieved using `getRegisterParameter` (Line 17), and its bit position is returned by `getBitPosition` (Line 23).

Locations not necessarily refer to defined register parameters only, but they can also refer to [WIR](#) symbols, i.e., to a single bit of some memory address. For this purpose, the constructor from Line 5 can be used, and method `getSymbol` (Line 20) retrieves a location's symbol.

Whether a location refers either to a register parameter or a symbol can be checked using methods `getType`, `isRegisterParameter` or `isSymbol` (Lines 8 to 14), resp., where the enumeration type returned by `getType` is

Listing 13.23: Location Handling by Class `WIR_Location`

```
1 // Default constructor creating a location referring to a bit produced by a given register parameter.
2 WIR_Location::WIR_Location( const WIR_RegisterParameter &rp, unsigned int b );
3
4 // Default constructor creating a location referring to a bit of a symbol's address.
5 WIR_Location::WIR_Location( const WIR_Symbol &s, unsigned int b );
6
7 // Get a location's type.
8 WIR_LocationType WIR_Location::getType( void ) const;
9
10 // Determine whether a location refers to a register parameter.
11 bool WIR_Location::isRegisterParameter( void ) const;
12
13 // Determine whether a location refers to a symbol.
14 bool WIR_Location::isSymbol( void ) const;
15
16 // Get a location's register parameter.
17 const WIR_RegisterParameter &WIR_Location::getRegisterParameter( void ) const;
18
19 // Get a location's symbol.
20 const WIR_Symbol &WIR_Location::getSymbol( void ) const;
21
22 // Get a location's bit position.
23 unsigned int WIR_Location::getBitPosition( void ) const;
24
25 // Equality operator for locations.
26 bool WIR_Location::operator == ( const WIR_Location &l ) const;
27
28 // Inequality operator for locations.
29 bool WIR_Location::operator != ( const WIR_Location &l ) const;
30
31 // Enum WIR_LocationType represents different types of locations.
32 enum class WIR_LocationType : char
33 {
34     reg,          // A location referring to a register parameter.
35     sym           // A location referring to a symbol.
36 };
```

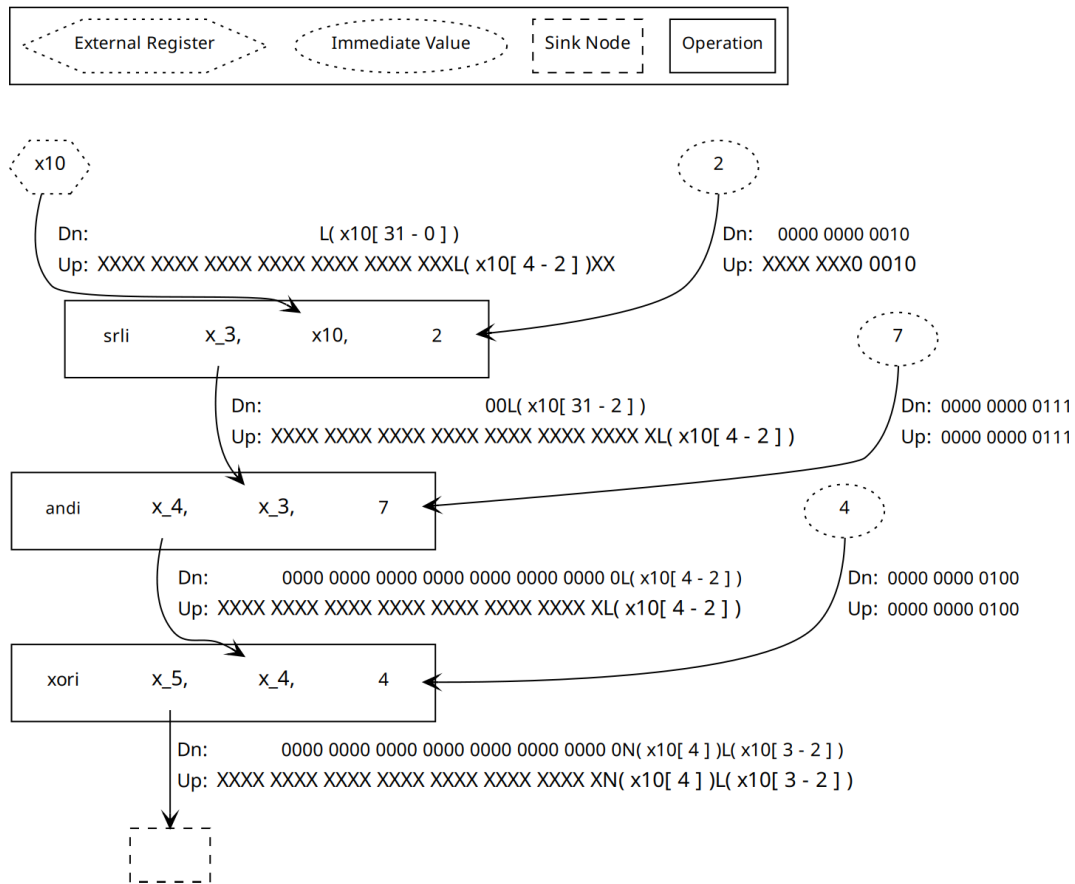


Figure 13.9: Example of Bit-True Data Flow Graph

shown in Line 32. As usual within WIR, getRegisterParameter fails with a runtime error if the location refers to a symbol, and vice versa for getSymbol.

Finally, the operators == and != from Lines 26 and 29 can be used to compare two locations for equality or inequality, resp. Here, two locations are equal iff they both have the same type and refer to the very same bit of the very same register parameter or symbol, resp.

The concepts regarding the partial order L4 and locations refer to one single bit to be analyzed. As WIR operations regularly use complete registers of a certain bit width as parameters (see also the previous example on Page 174), single bit values from L4 are combined to form values that represent all bits of a register. These combined values (called down and up values in the following) are associated with the edges of the bit-true DFG which is defined as follows:

Definition 13.3 (Bit-true Data Flow Graph)

A bit-true data flow graph $DFG = (V, E, Dn, Up)$ is a directed graph with

- $V = \{v_1, \dots, v_n\}$ being the set of all nodes, where each node represents either
 - an input value (external registers or immediates), or
 - a single WIR operation, or
 - an output value,
- $E = \{(v_i^x \rightarrow v_j^y) \mid \text{parameter } x \text{ of node } i \text{ defines a value used by parameter } y \text{ of node } j\}$,

- $D_n : E \rightarrow L4^k$ and $U_p : E \rightarrow L4^k$ are the down and up values representing bit-true analysis results per graph edge where the determined values are k bits wide.

Example

For the three operations from the previous example, the bit-true DFG and all analyzed down and up values are depicted in Fig. 13.9.

Here, the immediate value 7 required by the `andi` operation is represented as “0000 0000 0111” which is the straightforward outcome due to the binary representation of the number 7 using a width of 12 bits, as mandated by the RISC-V ISA. The immediate 2 used by `srl` is also modeled using 12 bits, but the most-significant 7 bits are set to `bX` here: “XXXX XXX0 0010”. This is because the analysis determines that shift amounts in the RISC-V ISA can only take values between 0 and 31 so that only the least-significant 5 bits are relevant. The most-significant bits of shift amounts thus don’t care.

Values analyzed for DFG edges representing RV32I registers are all 32 bits wide. The edge from `andi` to `xori` along virtual register `x_4` shows that only the least-significant 3 bits of `x_4` are relevant for the given example, all other most-significant bits are set to `bX`. Furthermore, these least-significant 3 bits of `x_4` are identical with bits 2-4 of input register `x10` which is represented by the locations “L(x10[4 - 2])” in Fig. 13.9.

As already discussed in the previous example, the result finally computed by `xori` in `x_5` contains the original bits 2 and 3 of `x10` in the least-significant positions, and the inverted bit 4 of `x10` next: “N(x10[4])L(x10[3 - 2])”.

All down and up values determined during bit-true data flow analysis are modeled by class `WIR_UpDownValue` (cf. file `WIR/analyses/bit/wirupdownvalue.h`). The constructors of this class are shown in Listing 13.24. The most simple constructor from Line 2 creates an unsigned up/down value of a given bit width whose bits are all initialized with `bU`. If some other L4 value shall be used for initialization, the variant from Line 5 can be used that accepts this L4 value as argument, as well as its bit width and a Boolean flag defaulting to `true` that indicates whether the up/down value is interpreted as signed or unsigned value, resp. Up/down values representing immediate values can conveniently be created with the constructor from Line 8. The actual value as well as its bit width and signedness are directly taken from the given immediate parameter. The constructor from Line 11 creates an up/down value that refers to all bits of a given register parameter so that all elements of

Listing 13.24: Constructors of Class `WIR_UpDownValue`

```

1 // Default constructor for unsigned up/down values of a given width.
2 explicit WIR_UpDownValue( WIR_UpDownValue( unsigned int w );
3
4 // Default constructor for up/down values with all bits set to some L4 value and given width and signedness.
5 WIR_UpDownValue( WIR_UpDownValue( WIR_L4 v, unsigned int w, bool s = true );
6
7 // Default constructor for up/down values initialized to some given immediate value.
8 explicit WIR_UpDownValue( WIR_UpDownValue( const WIR_BaseImmediateParameter &i );
9
10 // Default constructor for up/down values initialized to some register location and signedness.
11 WIR_UpDownValue( WIR_UpDownValue( const WIR_RegisterParameter &rp, bool s = true, bool n = false );
12
13 // Copy constructor.
14 WIR_UpDownValue( WIR_UpDownValue( const WIR_UpDownValue &v );
15
16 // Move constructor.
17 WIR_UpDownValue( WIR_UpDownValue( WIR_UpDownValue &&v );

```

Listing 13.25: Value Handling of Class WIR_UpDownValue

```

1 // Set all bits to a given L4 value different from bL and bN.
2 void WIR_UpDownValue::setAllBits( WIR_L4 v );
3
4 // Set all bits to a given L4 value bL or bN
5 void WIR_UpDownValue::setAllBits( WIR_L4 v, const WIR_Location &l );
6
7 // Set a single bit at a given position to an L4 value different from bL and bN.
8 void WIR_UpDownValue::setBit( unsigned int p, WIR_L4 v );
9
10 // Set a single bit at a given position to an L4 bL or bN.
11 void WIR_UpDownValue::setBit( unsigned int p, WIR_L4 v, const WIR_Location &l );
12
13 // Get the L4 value of a single bit at a given position.
14 WIR_L4 &WIR_UpDownValue::operator [] ( unsigned int p );
15
16 // Get the L4 value of a single bit at a given position.
17 const WIR_L4 &WIR_UpDownValue::at( unsigned int p ) const;
18
19 // Determine whether an up/down value contains a given L4 value.
20 bool WIR_UpDownValue::containsBit( const WIR_L4 v ) const;
21
22 // Determine whether an up/down value contains at least one L4 value from the given set.
23 bool WIR_UpDownValue::containsBits( WIR_L4Set &&v ) const;
24
25 // Determine whether an up/down value contains only the given L4 value.
26 bool WIR_UpDownValue::containsOnlyBit( const WIR_L4 v ) const;
27
28 // Determine whether an up/down value contains only L4 values from the given set.
29 bool WIR_UpDownValue::containsOnlyBits( WIR_L4Set &&v ) const;
30
31 // Determine whether a bit at a given position is either bL or bN.
32 bool WIR_UpDownValue::isLocationBit( unsigned int p ) const;
33
34 // Get the location at a given position.
35 const WIR_Location &WIR_UpDownValue::getLocation( unsigned int p ) const;
36
37 // Get an up/down value's bit width.
38 unsigned int WIR_UpDownValue::getBitWidth( void ) const;
39
40 // Set a value's signedness.
41 void WIR_UpDownValue::setSignedness( bool s = true );
42
43 // Determine whether an up/down value is signed.
44 bool WIR_UpDownValue::isSigned( void ) const;
45
46 // Determine whether an up/down value is unsigned.
47 bool WIR_UpDownValue::isUnsigned( void ) const;

```

such an up/down value are initialized with bL. The second argument of this constructor again denotes the value's signedness, and its last argument denotes whether bN shall be used for initialization. Finally, the usual copy and move constructors also exist, see Lines 14 and 17.

Listing 13.25 shows all methods to set and retrieve the basic properties of up/down values, beginning with setting bits to some L4 value. Methods `setAllBits` (cf. Lines 2 and 5) set all bits to the same given L4 value, where the first variant of this method is applicable only for values other than bL and bN. Since bL and bN necessarily require the specification of a location, the second variant is provided for this purpose. Both methods produce a runtime

Listing 13.26: Value Interpretation by Class WIR_UpDownValue

```

1 // Determine whether an up/down value contains only b0, b1 or bX bits.
2 bool WIR_UpDownValue::isInteger( void ) const;
3
4 // Determine whether an up/down value contains only b0 or b1 bits.
5 bool WIR_UpDownValue::isBinaryInteger( void ) const;
6
7 // Determine whether an up/down value is a positive integer number.
8 bool WIR_UpDownValue::isPositive( void ) const;
9
10 // Determine whether an up/down value is a negative integer number.
11 bool WIR_UpDownValue::isNegative( void ) const;
12
13 // Get an up/down value's numerical interpretation.
14 signed long long WIR_UpDownValue::getSignedValue( void ) const;

```

error if wrong L4 values are provided. Similarly, methods `setBit` (Lines 8 and 11) can be used to set a single bit specified by its numerical position. Bit positions within up/down values always start with 0 denoting the value's least-significant bit. Retrieving the L4 value of a bit at a given position can be done using the `[]` operator from Line 14 or the `at` method (Line 17) if the up/down value is a `const` object. Checking whether a given L4 value occurs in some bit is done by method `containsBit` (Line 20). By providing a set of L4 bits to `containsBits` (Line 23), the existence of at least one of these values in some bits can be tested. The set type `WIR_L4Set` follows the principles discussed earlier in Chapter 11. Whether an up/down value contains only a given (set of) L4 value(s) can be determined using `containsOnlyBit` (Line 26) and `containsOnlyBits` (Line 29), resp. A convenient test whether a bit at a given position is a location `bL` or a negated location `bN` is realized by `isLocationBit` (Line 32). The location at a bit position can be obtained using `getLocation` (Line 35). It fails with a runtime error if the specified bit position is neither `bL` nor `bN`.

An up/down value's bit width can only be set during its construction. For this reason, there is no method to change the bit width afterwards. A value's bit width can be obtained using `getBitWidth` (Line 38). The interpretation of an up/down value depends on whether it represents signed or unsigned contents. A value's signedness can explicitly be set using `setSignedness` (Line 41) that expects a Boolean argument defaulting to `true`. Methods `isSigned` and `isUnsigned` (Lines 44 and 47) determine whether a value is signed or unsigned, resp.

An up/down value can be interpreted as a classical number according to some number representation known from computer arithmetics. However, the fact that L4 values form a superset of standard Boolean bits needs to be considered. Listing 13.26 shows all methods of class `WIR_UpDownValue` that can be used to numerically interpret an up/down value. Method `isInteger` (cf. Line 2) determines whether an up/down value represents an integer number. Integer numbers of course contain only bits `b0` and `b1`. Since, however, `bX` bits are irrelevant and do not care by definition, such bits are also accepted by `isInteger`. If a test for a standard binary integer shall be performed, method `isBinaryInteger` (Line 5) should be used that exclusively checks for `b0` and `b1` bits. `isPositive` (Line 8) determines whether a value represents a positive number. This is the case if the value either is unsigned, or if it is signed and it is an integer number whose most-significant bit (i.e., the sign bit) is `b0`. Likewise, `isNegative` (Line 11) checks for a negative number, i.e., if a value is signed and is an integer with most-significant bit equal to `b1`. An up/down value's numerical interpretation can be retrieved via method `getSignedValue` (Line 14). The use of this method of course makes only sense if `isBinaryInteger` is `true`. If an up/down value contains bits other than `b0` and `b1`, such bits are silently ignored, i.e., assumed to be `b0`. For positive numbers, `getSignedValue` applies the standard unsigned number representation where all `b1` bits are multiplied by the power of 2 according to their bit positions, and all these partial products get accumulated. For negative numbers, the two's complement representation is applied. The so-determined numerical interpretation

Listing 13.27: Comparison Operators of Class WIR_UpDownValue

```

1 // Determine whether two up/down values are provably, strictly bitwise equal.
2 WIR_L4 WIR_UpDownValue::operator == ( const WIR_UpDownValue &v ) const;
3
4 // Determine whether two up/down values are provably, strictly bitwise unequal.
5 WIR_L4 WIR_UpDownValue::operator != ( const WIR_UpDownValue &v ) const;
6
7 // Determine whether one up/down value is provably greater-equal than another.
8 WIR_L4 WIR_UpDownValue::operator >= ( const WIR_UpDownValue &v ) const;
9
10 // Determine whether one up/down value is provably less than another.
11 WIR_L4 WIR_UpDownValue::operator < ( const WIR_UpDownValue &v ) const;

```

is then returned by this method as value of type signed long long, irrespective of the up/down value's actual signedness (cf. methods isSigned and isUnsigned from Listing 13.25).

Comparing up/down values for equality and other properties can be done using the operators shown in Listing 13.27. Again, the particularities of the L4 partial order have to be considered so that these comparison operators behave differently from standard operators. Due to the nature of L4, there can be situations where no clear result can be determined for a comparison. For this reason, all the operators from Listing 13.27 do not return a simple Boolean result, but instead return an L4 value that is either b1 if a comparison definitely is true, or b0 if a comparison provably is false, or bU if no safe statement can be made.

For the equality test of the == operator from Line 2, the following rules apply:

- If the compared up/down values have different bit widths, they are not equal and b0 is returned.
- Otherwise, both values are equal and b1 is returned iff, for each bit position, both compared bits are b0 or both are b1 or both are bL or both are bN, where, in the last two cases, both bits refer to exactly the same bit location.
- Otherwise, both values are unequal and b0 is returned iff, for some bit position, one bit is b0 and the other is b1, or one bit is bL and the other is bN where, in this case, both bits refer to exactly the same bit location.
- Otherwise, no clear statement can be made and bU is returned.

The analog rules (with inverted return values) are applied by the inequality operator != from Line 5.

The >= and < operators from Lines 8 and 11 are unable to make a safe statement and return bU if some bit of the two values to be compared is bU, bL or bN. In other words, if some up/down value is not an integer (see isInteger from Line 2 of Listing 13.26), both >= and < return bU. If both values to be compared are integers, their numerical interpretations are obtained using getSignedValue and are compared with each other. This way, either b1 or b0 are returned, depending on whether this numerical comparison holds true or not.

Class WIR_UpDownValue comes with the full set of bitwise and arithmetical operators in order to conveniently apply the usual computations to such up/down values and their featured L4 bits. Listing 13.28 shows the declarations of all bitwise operators. Since most of them are straightforward, we omit their detailed discussion for the sake of brevity. However, a few explanations regarding the shift operators from Lines 14 to 23 are required. The << operator from Line 14 shifts an up/down value by the number of bits specified as integer shift amount. If this shift amount is positive, a left-shift is applied, while negative shift amounts lead to a right-shift. When shifting left, the << operator fills the least-significant bits with b0. When doing a right shift, the most-significant bits get filled with either b0 (logical right-shift) or with the original up/down value's sign bit (arithmetical right-shift), depending

Listing 13.28: Bitwise Operators of Class WIR_UpDownValue

```

1 // Bitwise AND operator.
2 WIR_UpDownValue WIR_UpDownValue::operator & ( const WIR_UpDownValue &v ) const;
3
4 // Bitwise OR operator.
5 WIR_UpDownValue WIR_UpDownValue::operator | ( const WIR_UpDownValue &v ) const;
6
7 // Bitwise XOR operator.
8 WIR_UpDownValue WIR_UpDownValue::operator ^ ( const WIR_UpDownValue &v ) const;
9
10 // Bitwise negation operator.
11 WIR_UpDownValue WIR_UpDownValue::operator ~ ( const WIR_UpDownValue &v ) const;
12
13 // Bitwise left-shift operator by integer shift amount.
14 WIR_UpDownValue WIR_UpDownValue::operator << ( int s ) const;
15
16 // Bitwise right-shift operator by integer shift amount.
17 WIR_UpDownValue WIR_UpDownValue::operator >> ( int s ) const;
18
19 // Bitwise left-shift operator by another up/down value.
20 WIR_UpDownValue WIR_UpDownValue::operator << ( const WIR_UpDownValue &v ) const;
21
22 // Bitwise right-shift operator by another up/down value.
23 WIR_UpDownValue WIR_UpDownValue::operator >> ( const WIR_UpDownValue &v ) const;

```

on the value's signedness. The same applies for the >> operator from Line 17 – for positive shift amounts, either logical or arithmetical right-shifts are applied, while negative shift amounts are equivalent to a left-shift.

The shift operators from Lines 20 and 23 allow to use a shift amount being an up/down value itself. If such a shift amount contains any bU, bL or bN bits, these shift operators produce a result containing only bU bits. If, otherwise, the shift amount is positive, potential bX bits in the shift amount are interpreted as b1, or as b0 for negative shift amounts. This interpretation of bX bits maximizes the absolute value of the shift amount so that a maximal number of bits gets shifted out. As before, left-shifting by a negative value de facto is a right-shift (either arithmetical or logical), and right-shifting by a negative value is a left-shift.

Finally, Listing 13.29 summarizes all remaining operators of class WIR_UpDownValue which mostly perform the standard arithmetical computations such as addition or subtraction. The addition from Line 2 is realized as ripple-carry adder, while the subtraction (Line 5) exploits the two's complement and thus gets reduced to an addition. The unary minus operator (Line 8) negates an up/down value by again applying the two's complement. Multiplication (Line 11) uses Booth's algorithm, while division and remainder from Lines 14 and 17 go back to a restoring division.

The abs function (Line 20) provides the negated value for negative arguments, and the identity for positive arguments. Extending the bit width of an up/down value can be achieved using extend (Line 23). This method gets the new bit width as argument (which must be larger than the value's current width) and returns either the zero- or the sign-extended up/down value. From a bit position p on, extract (Line 26) extracts the next w consecutive bits and returns this extracted value. The insert function (Line 29) takes the bits of value v2 and inserts them into v1 at bit position p.

The bit-true data flow analysis of class WIR_BitDFA constructs the DFG of a WIR function according to Definition 13.3 and traverses it in forward and backward direction. The first pass produces the down values D_n , while the second pass yields the U_p values. Without going into the details of the analysis itself, the down values can

Listing 13.29: Arithmetical and Miscellaneous Operators of Class WIR_UpDownValue

```

1 // Bitwise plus operator.
2 WIR_UpDownValue WIR_UpDownValue::operator + ( const WIR_UpDownValue &v ) const;
3
4 // Bitwise minus operator.
5 WIR_UpDownValue WIR_UpDownValue::operator - ( const WIR_UpDownValue &v ) const;
6
7 // Bitwise unary minus operator.
8 WIR_UpDownValue WIR_UpDownValue::operator - ( void ) const;
9
10 // Bitwise multiplication operator.
11 WIR_UpDownValue WIR_UpDownValue::operator * ( const WIR_UpDownValue &v ) const;
12
13 // Bitwise division operator.
14 WIR_UpDownValue WIR_UpDownValue::operator / ( const WIR_UpDownValue &v ) const;
15
16 // Bitwise remainder operator.
17 WIR_UpDownValue WIR_UpDownValue::operator % ( const WIR_UpDownValue &v ) const;
18
19 // Get the absolute value.
20 friend WIR_UpDownValue abs( const WIR_UpDownValue &v );
21
22 // Get the width-extended value.
23 WIR_UpDownValue WIR_UpDownValue::extend( unsigned int w ) const;
24
25 // Get a consecutive bit sequence from an up/down value.
26 WIR_UpDownValue WIR_UpDownValue::extract( unsigned int p, unsigned int w ) const;
27
28 // Insert a value somewhere into another up/down value.
29 friend void insert( WIR_UpDownValue &v1, const WIR_UpDownValue &v2, unsigned int p );

```

never contain `bX` bits by construction. Thus, the up values represent the final result of the whole analysis. Besides the standard interface of `WIR` analyses (cf. Listing 13.1), this class does not provide any additional relevant methods.

Analysis results are stored in `WIR_BitValues` containers (see file `WIR/containers/wirbitvalues.h`) which are attached to register parameters and to immediate parameters. This is motivated by the fact that `DFG` edges connect these types of parameters with each other as can be seen from Fig. 13.9.

Consider, e.g., register parameter `x_3` of the `andi` operation from Fig. 13.9. This parameter is used by its operation so that data is flowing into it along an incoming `DFG` edge. The handling of incoming edges by container class `WIR_BitValues` is shown in Listing 13.30. For parameter `x_3` of `andi`, the incoming edge gets added by the analysis using `insertInValues` (Line 2) where the incoming edge's source parameter (here: `x_3` of `srli`) and the associated down and up values are provided. This triplet of an incoming edge's source parameter and its up/down values is represented by struct `edge` from Line 20. A reference to the complete list of incoming edges can be retrieved via `getInValues` (Line 5), while `findInValues` (Line 8) provides an iterator into this list referring to the edge having the given parameter as source. Methods `eraseInValues` (Lines 11 and 14) remove the incoming edge either starting at a given parameter or referred by a given iterator. Finally, `clearInValues` (Line 17) erases all incoming edges.

The `andi` operation of the previous example from Fig. 13.9 produces register parameter `x_4` as a result so that this register parameter is source of an outgoing `DFG` edge. In order to manage outgoing edges, container class `WIR_BitValues` features the equivalent set of methods as already shown in Listing 13.30, which are just named `insertOutValues`, `getOutValues`, `findOutValues`, `eraseOutValues` and `clearOutValues`, resp. Due to

Listing 13.30: Handling of Incoming Up/Down Values by Container Class WIR_BitValues

```

1 // Add a down and an up value as incoming DFG edge.
2 void WIR_BitValues::insertInValues( WIR_Parameter &p, WIR_UpDownValue &d, WIR_UpDownValue &u );
3
4 // Get the list of incoming DFG edges with their down and up values.
5 std::list<edge> &WIR_BitValues::getInValues( void );
6
7 // Find the incoming DFG edge starting at a given parameter.
8 std::list<edge>::iterator WIR_BitValues::findInValues( const WIR_Parameter &p );
9
10 // Remove the incoming DFG edge starting at the given parameter and its down and up values.
11 void WIR_BitValues::eraseInValues( const WIR_Parameter &p );
12
13 // Remove the incoming DFG edge at the given iterator position and its down and up values.
14 void WIR_BitValues::eraseInValues( std::list<edge>::iterator pos );
15
16 // Remove all incoming DFG edges and their down and up values.
17 void WIR_BitValues::clearInValues( void );
18
19 // struct edge represents a DFG edge between two WIR parameters.
20 struct WIR_BitValues::edge
21 {
22     // rp points to the parameter being an edge's source or target node, resp.
23     WIR_Parameter *rp;
24
25     // downVal stores an edge's down value.
26     WIR_UpDownValue downVal;
27
28     // upVal stores an edge's up value.
29     WIR_UpDownValue upVal;
30 };

```

their similarity with Listing 13.30, we omit their detailed presentation here. For an outgoing edge, struct edge from Line 20 stores the register parameter being the edge's target node.

14 Code Optimization

This chapter briefly describes all available optimizations that are integral part of the [WIR](#) library. Section [14.1](#) first introduces the generic base classes that form the optimization framework within [WIR](#). Hereafter, Sections [14.2](#) to [14.7](#) present entirely generic and processor-independent cleanup and optimizations passes: unused virtual registers, empty blocks, redundant blocks, unreachable blocks, loop-invariant code motion, webs. Processor-specific optimizations for which realizations for the RISC-V and TriCore [ISAs](#) exist are subject to Sections [14.8](#) to [14.10](#): register allocation, dead code, constant folding. Finally, the currently entirely TriCore-specific optimizations are presented in Sections [14.11](#) to [14.17](#): constant propagation, redundant code, jump correction, operation bit widths, predicated execution, peephole, instruction scheduling.

14.1 Generic Optimization Framework

Similar to the code analyses from Chapter [13](#), all [WIR](#) optimizations also inherit from a common base class, which is `WIR_Optimization` declared in file `WIR/optimizations/generic/wiroptimization.h`. The public interface of this base class is shown in Listing [14.1](#).

Listing 14.1: Code Optimization API of Class `WIR_Optimization`

```

1 // Default constructor for system-level optimization.
2 explicit WIR_Optimization::WIR_Optimization( WIR_System &s );
3
4 // Default constructor for compilation unit-level optimization.
5 explicit WIR_Optimization::WIR_Optimization( WIR_CompilationUnit &c );
6
7 // Default constructor for function-level optimization.
8 explicit WIR_Optimization::WIR_Optimization( WIR_Function &f );
9
10 // Default constructor for basic block-level optimization.
11 explicit WIR_Optimization::WIR_Optimization( WIR_BasicBlock &b );
12
13 // Default constructor for instruction-level optimization.
14 explicit WIR_Optimization::WIR_Optimization( WIR_Instruction &i );
15
16 // Default constructor for operation-level optimization.
17 explicit WIR_Optimization::WIR_Optimization( WIR_Operation &o );
18
19 // Performs the actual code optimization.
20 void WIR_Optimization::optimize( void );
21
22 // Set whether an optimization shall generate 16 bits wide operations.
23 void WIR_Optimization::setGenerate16BitOperations( bool f = true );
24
25 // Determine whether an optimization shall generate 16 bits wide operations.
26 bool WIR_Optimization::getGenerate16BitOperations( void ) const;

```

Listing 14.2: Bit-Level Optimization API of Class WIR_BitOpt

```

1 // Default constructor for system-level optimization.
2 explicit WIR_BitOpt::WIR_BitOpt( WIR_System &s );
3
4 // Default constructor for compilation unit-level optimization.
5 explicit WIR_BitOpt::WIR_BitOpt( WIR_CompilationUnit &c );
6
7 // Default constructor for function-level optimization.
8 explicit WIR_BitOpt::WIR_BitOpt( WIR_Function &f );
9
10 // Set whether an optimization shall generate 16 bits wide operations.
11 void WIR_BitOpt::setRunDFA( bool f = true );

```

Again, various constructors can be chosen which determine whether an optimization shall be carried out for a complete **WIR** system (cf. Line 2), or just for a single compilation unit, function, basic block etc. (Lines 5 to 17). Hereafter, only the optimize method from Line 20 needs to be invoked in order to apply the actual optimization to the **WIR** object used during construction.

The **ISAs** of many processor architectures allow to freely mix instructions of different bit widths with each other. Whether or not such mixtures shall be exploited by a **WIR** optimization can be specified using method `setGenerate16BitOperations` (Line 23), and the current status of this optimization property can be checked via `getGenerate16BitOperations` (Line 26).

Some of the optimizations presented in the following base on the bit-true data flow analysis presented in Section 13.11. Instead of base class **WIR_Optimization** above, such bit-level optimizations inherit from a base class **WIR_BitOpt**, which in turn is derived from **WIR_Optimization**. Its public **API** is shown in Listing 14.2. Besides the straightforward constructors from Lines 2 to 8, only method `setRunDFA` (Line 11) is novel for bit-level optimizations. It can be used to specify whether a bit-true data flow analysis shall explicitly be carried out prior to the actual optimization, or not.

The data flow analysis from Section 13.11 can be computationally expensive so that reapplying it after each code optimization would not be ideal. For this reason, the bit-level optimizations discussed in this chapter – esp. dead code elimination (Section 14.9), constant folding (Section 14.10) and propagation (Section 14.11), redundant code elimination (Section 14.12) – can be told not to do a data flow analysis on their own. Instead, they reuse **WIR_BitValue** containers that were already attached to **WIR** constructs earlier, and the bit-level optimizations are designed such that these existing containers simply get updated by novel down and up values after some code modification.

14.2 Unused Virtual Register Elimination

Optimization Class	Optimization Level	Architectures
WIR_UnusedVRegs	Systems – Functions	Generic, all

As introduced earlier in Section 8.2 (cf. Page 42), virtual registers are managed at the level of code functions within **WIR**. Class **WIR_UnusedVRegs** (see file `WIR/optimizations/unusedvregs/wirunusedvregs.h`) now provides a pass that removes all virtual registers from their functions that are not at all defined or used somewhere, i.e., which do not occur as register parameter in some **WIR** operation. This pass intends to keep some **WIR**-internal data structures as small as possible in order to speed up subsequent liveness analyses and register allocation passes.

This cleanup pass can be applied to [WIR](#) systems, compilation units or functions. Due to its simplicity, the public [API](#) of class `WIR_UnusedVRegs` contains no extensions beyond what is shown in [Listing 14.1](#). This class does not require any processor-specific adaptations, it can be applied generically for all processor models from [Chapter 9](#).

14.3 Empty Basic Block Elimination

Optimization Class	Optimization Level	Architectures
<code>WIR_EmptyBlocks</code>	Systems – Functions	Generic, all

Empty basic blocks are blocks that contain no instructions, or only empty instructions without operations. Such empty blocks are fully redundant and are eliminated by class `WIR_EmptyBlocks` from file `WIR/optimizations/emptyblocks/wiremptyblocks.h`. For this purpose, the first non-empty successor block is determined, and all occurrences of the empty block's label are replaced by this successor. Likewise, flow facts referring to the empty block (cf. [Section 13.6](#)) are also shifted towards the successor, as are any other containers that are attached to the empty block. This pass simplifies a function's [CFG](#) and thus aims to speed up the iterative data flow analyses from [Chapter 13](#).

There is nothing to mention regarding the interface of class `WIR_EmptyBlocks`, [Listing 14.1](#) describes all available methods.

14.4 Redundant Basic Block Elimination

Optimization Class	Optimization Level	Architectures
<code>WIR_RedundantBlocks</code>	Systems – Functions	Generic, all

According to [Definition 1.1](#), a basic block is a *maximal* instruction sequence that always gets executed sequentially from its very first to its very last instruction. Two basic blocks b_1 and b_2 that directly follow each other within their owning [WIR](#) function are redundant if

- b_1 has exactly one successor, which is b_2 , and
- b_2 has exactly one predecessor, which is b_1 , and
- b_2 is the immediate successor of b_1 .

b_2 being immediate successor of b_1 means that after the execution of the very last instruction of b_1 , directly the first instruction of b_2 is executed without any other control flow deviations in between. For example, b_1 could end with a jump explicitly branching to b_2 , and b_2 would then be immediate successor of b_1 . If, however, b_1 ends with a function call, b_2 is not immediate successor since the code of the called function gets executed in between.

The presence of redundant blocks implies that the requirement of [Definition 1.1](#) to be a maximal instruction sequence is violated. The simplification pass from class `WIR_RedundantBlocks` (see file `WIR/optimizations/redundantblocks/wirredundantblocks.h`) thus removes the potentially existing branch from b_1 to b_2 and moves all instructions from b_2 to the tail of b_1 afterwards. This way, b_2 becomes empty and gets eliminated next, see [Section 14.3](#).

14.5 Unreachable Basic Block Elimination

Optimization Class	Optimization Level	Architectures
WIR_UnreachableBlocks	Systems – Functions	Generic, all

A basic block b is called unreachable if there is no directed path in a function's CFG from its unique entry point to b . Such unreachable blocks never get executed and thus can be eliminated completely.

This is done by class WIR_UnreachableBlocks (cf. file WIR/optimizations/unreachableblocks/wirunreachableblocks.h). The optimization can be applied to WIR systems, compilation units, or functions. It again is processor-independent and generic, and the class has no specific public API that requires a discussion here.

14.6 Loop-Invariant Code Motion

Optimization Class	Optimization Level	Architectures
WIR_LoopInvariantCM	Systems – Functions	Generic, all

Loop-invariant code motion is a very powerful optimization that identifies instructions within loops that are loop-invariant, i.e., that do not at all depend on the surrounding loop's behavior. In such a situation, repetitively executing loop-invariant code is highly inefficient since it is sufficient to execute such code just once before the loop. The goal of this optimization thus is to move as many instructions as possible out of loops.

Within WIR, loop-invariant code motion is realized by class WIR_LoopInvariantCM (cf. file WIR/optimizations/loopinvariantcm/wirloopinvariantcm.h) as an entirely generic and processor-independent optimization. Its implementation closely follows the concepts of [22, chapter 13.2] according to which an operation is loop-invariant if, for each of its parameters:

1. the parameter is constant, or
2. all definitions that reach this use of the parameter are located outside the current loop, or
3. there is exactly one definition of the parameter that reaches the operation, and that definition is an operation inside the loop that is itself loop-invariant.

Operations that realize function calls or returns, jumps, or that have any other side effects are never considered loop-invariant.

The public interface of class WIR_LoopInvariantCM is identical to Listing 14.1, its constructors allow to apply loop-invariant code motion to WIR systems, compilation units or individual functions.

14.7 Web Generation

Optimization Class	Optimization Level	Architectures
WIR_Webs	Systems – Functions	Generic, all

A web for a virtual register is the maximal union of intersecting def-use chains (see Definition 13.1 on Page 170) for that register [22, chapter 8.10]. Hereby, two def-use chains intersect if they have a use in common [22, chapter 16.3.3]. The web generation pass of WIR detects webs for a virtual register and then replaces that original register by a novel virtual register for each individual web found. Webs are particularly useful in global

Listing 14.3: Public Interface of Class WIR_Webs

```

1 // Default constructor for system-level optimization.
2 explicit WIR_Webs::WIR_Webs( WIR_System &s );
3
4 // Default constructor for compilation unit-level optimization.
5 explicit WIR_Webs::WIR_Webs( WIR_CompilationUnit &c );
6
7 // Default constructor for function-level optimization.
8 explicit WIR_Webs::WIR_Webs( WIR_Function &f );
9
10 // Get the list of all identified and transformed webs.
11 std::list<struct WIR_Webs::Web> WIR_Webs::getWebs( void ) const;
12
13 // Struct Web represents a web for some virtual register and distinguishes between its definitions and uses.
14 struct WIR_Webs::Web
15 {
16     public:
17
18         WIR_RegisterParameterSet defs;
19         WIR_RegisterParameterSet uses;
20 };

```

register allocation by graph coloring (see Section 14.8) – they are the units that are candidates for allocation to physical registers.

The main effect of constructing webs is to separate uses of a variable with a name like, e.g., `i` that may be used over and over again in a procedure as, say, a loop index, but whose uses are disjoint from each other. This can significantly improve register allocation by reducing the range over which a virtual register may require a physical register and can improve the effectiveness of other optimizations as well.

Class `WIR_Webs` exploits `WIR`'s generic reaching definitions and def-use chain analyses from Sections 13.7 and 13.9 so that this optimization in turn is entirely generic and processor-independent. Its public interface is documented in Listing 14.3. Besides the default constructors from Lines 2 to 8, there is only method `getWebs` (cf. Line 11) which returns the list of all identified webs. A web is modeled by the struct seen in Line 14, i.e., it consists of the set of all register parameter being definitions of the web, and the parameter set of all its uses.

14.8 Graph Coloring-Based Register Allocation

Optimization Class	Optimization Level	Architectures
WIR_GraphColoring, RV_GraphColoring, TC_GraphColoring	Systems – Functions	RV32IMFC, TC1.3, TC1.3.1

The `WIR` register allocator is a generic, processor-independent reference implementation of the graph coloring allocator according to [5]. The graph to be colored is the **Interference Graph (IG)** which is defined as follows:

Definition 14.1 (Interference Graph)

Let f be a `WIR` function and $R_V = \{r_1, \dots, r_n\}$ be the set of all virtual registers of f . Let $R_P = \{R_1, \dots, R_K\}$ be the set of all physical registers of a processor architecture.

An interference graph $IG = (V, E)$ is an undirected graph with

Listing 14.4: Constructors of Class WIR_GraphColoring

```

1 // Default constructor for system-level optimization.
2 WIR_GraphColoring::WIR_GraphColoring( WIR_System &s, bool v = false );
3
4 // Default constructor for compilation unit-level optimization.
5 WIR_GraphColoring::WIR_GraphColoring( WIR_CompilationUnit &c, bool v = false );
6
7 // Default constructor for function-level optimization.
8 WIR_GraphColoring::WIR_GraphColoring( WIR_Function &f, bool v = false );
9
10 // Determine whether verbose messages shall be printed.
11 bool WIR_GraphColoring::getVerbosity( void );

```

- $V = R_V \cup R_P$,
- $E = \{ \{v, w\} \mid v \text{ and } w \text{ may never share the same physical register, i.e., } v \text{ and } w \text{ interfere} \}$.

The nodes of the **IG** represent all virtual and physical registers, and edges are introduced whenever either liveness analysis (cf. Section 13.10) or processor-specific constraints mandate that two graph nodes must not be assigned to the same physical register.

Using this **IG**, a Graph Coloring is defined as follows:

Definition 14.2 (Graph Coloring)

Let $IG = (V, E)$ be an interference graph, $K \in \mathbb{N}$ be a non-zero positive integer.

Graph Coloring denotes the problem to assign each node $v \in V$ one color $k_v \in \{1, \dots, K\}$ such that

$$\forall e = \{v, w\} \in E : k_v \neq k_w$$

i.e., no two adjacent nodes in the **IG** may have the same color.

A register allocator based on graph coloring now assumes that each physical register of a processor architecture represents one specific color so that a total of K colors is available where K equals the number of physical processor registers. A graph colored according to Definition 14.2 thus provides a valid mapping of virtual registers to colors, i.e., to physical registers. In the real life, many **IGs** are not colorable due to high degrees of nodes. In such situations, the register allocator must take a decision which node to spill into the system's main memory in order to render the remaining graph colorable.

The implementation of the **WIR** register allocator closely follows the concepts and algorithms presented in [22, chapter 16.3] and [1, chapter 11]. Its main features are:

Maximal genericity of all key algorithms regarding interference graph creation, simplification, decisions on coloring and spilling etc. The allocator supports any kind of register hierarchies and precolorings as described in Chapter 8. Processor-specific details like, e.g., specification of caller- or callee-saved registers, processor-specific interferences, generation of actual spill code, are properly separated from the allocator's main algorithms.

Live range splitting of virtual registers is integrated into register allocation by exploiting the web generating pass from Section 14.7.

Spill cost computation takes the places where a virtual register is defined, used, or copied into account.

Various spilling heuristics can be used to decide which virtual register actually gets spilled, such as Chaitin's rule [5, Eq. (2.1)] considering the degree of a virtual register in the interference graph, or the three heuristics proposed by Bernstein [3] that consider the degrees of nodes in the interference graph as well as the loop nesting depths of operations or the number of live webs. A best-of-three spilling based on all heuristics originally proposed by Bernstein can also be activated.

Stack coalescing re-uses stack locations for the spilling of several virtual registers that are not simultaneously alive.

Coalescing of virtual registers involved in register-register moves takes loop nesting depths into account. Safe coalescing according to Briggs' and George's heuristics can be applied, which eliminates a vast amount of register-register moves from WIR code.

All these concepts are realized within the abstract base class `WIR_GraphColoring` (cf. file `WIR/optimizations/gcregallocation/wirgraphcoloring.h`). Its constructors are depicted in Listing 14.4 – as can be seen, they enable register allocation in complete WIR systems, compilation units, or in individual functions. All constructors support an optional Boolean argument defaulting to `false` that controls whether verbose or progress messages shall be produced. Whether or not such messages will be produced can be queried via method `getVerbosity` (cf. Line 11).

The various features of the WIR register allocator can selectively be turned on or off using the methods shown in Listing 14.5. The various `set` methods all require a Boolean argument defaulting to `true` which means that a certain feature of the register allocator gets activated. The current activation state of some feature can be retrieved with the help of the `get` methods from Listing 14.5.

For instance, live range splitting by generating webs can be activated or deactivated using method `setGenerateWebs` (cf. Line 2). By default, the generation of webs is always activated during the construction of a register allocator object.

Method `setBestOfThreeSpilling` (Line 8) controls whether Bernstein's best-of-three spilling heuristic is applied. It computes spill costs and takes spilling decisions using three different heuristics and finally keeps the decisions of the best heuristic leading to overall minimum spill costs for a WIR function. By default, this best-of-three approach is not activated due to its compile-time overheads. Instead, Chaitin's simple spill heuristic is activated during construction.

The full spilling of all virtual registers onto the system's stack without any kind of intelligent register allocation can be achieved via method `setSpillAll` (Line 14). The code resulting from this somehow resembles the output of GCC when no register allocation is performed at its optimization level `-O0`. Obviously, this "feature" regularly is turned off by the constructor of class `WIR_GraphColoring`. In order to conveniently recognize spill loads or stores, they can be marked using dedicated WIR comments (see Listing 12.4 on Page 142). This behavior of the register allocator can be controlled using method `setMarkSpillCode` (Line 20), it is deactivated by default.

The behavior of stack coalescing (default: activated) can be adjusted using `setCoalesceStackLocations` (Line 26).

The different flavors of coalescing of register-register moves can be turned on or off independently via methods `setCoalescingBriggs` (Line 32) and `setCoalescingGeorge` (Line 38), resp. By default, the class' constructors activate both Briggs' and George's coalescing. Method `getCoalescing` (Line 44) returns whether any of Briggs' or George's coalescing is turned on.

Finally, a very simple approach for rematerialization can be activated with the help of method `setRematerialization` (Line 47). Rematerialization reconstructs or rematerializes a value if this is less expensive than spill-storing and -loading it. Within the WIR register allocator, the bit-true data flow analysis from Section 13.11 is

Listing 14.5: Feature Activation of Class WIR_GraphColoring

```

1 // Set whether the splitting of virtual registers into webs shall be activated.
2 void WIR_GraphColoring::setGenerateWebs( bool w = true );
3
4 // Determine whether the splitting of virtual registers into webs is activated.
5 bool WIR_GraphColoring::getGenerateWebs( void );
6
7 // Set whether the best-of-three spilling heuristic by Bernstein shall be activated.
8 void WIR_GraphColoring::setBestOfThreeSpilling( bool b = true );
9
10 // Determine whether the best-of-three spilling heuristic by Bernstein is activated.
11 bool WIR_GraphColoring::getBestOfThreeSpilling( void );
12
13 // Set whether full spilling of all virtual registers shall be activated.
14 void WIR_GraphColoring::setSpillAll( bool s = true );
15
16 // Determine whether full spilling of all virtual registers is activated.
17 bool WIR_GraphColoring::getSpillAll( void );
18
19 // Set whether all generated spill instructions shall be marked with a dedicated comment.
20 void WIR_GraphColoring::setMarkSpillCode( bool m = true );
21
22 // Determine whether all generated spill instructions are marked with a dedicated comment.
23 bool WIR_GraphColoring::getMarkSpillCode( void );
24
25 // Set whether stack coalescing of spills shall be activated.
26 void WIR_GraphColoring::setCoalesceStackLocations( bool c = true );
27
28 // Determine whether stack coalescing of spills is activated.
29 bool WIR_GraphColoring::getCoalesceStackLocations( void );
30
31 // Set whether coalescing according to Briggs shall be activated.
32 void WIR_GraphColoring::setCoalescingBriggs( bool c = true );
33
34 // Determine whether coalescing according to Briggs is activated.
35 bool WIR_GraphColoring::getCoalescingBriggs( void );
36
37 // Set whether coalescing according to George shall be activated.
38 void WIR_GraphColoring::setCoalescingGeorge( bool c = true );
39
40 // Determine whether coalescing according to George is activated.
41 bool WIR_GraphColoring::getCoalescingGeorge( void );
42
43 // Determine whether any kind of coalescing is activated.
44 bool WIR_GraphColoring::getCoalescing( void );
45
46 // Set whether rematerialization shall be activated.
47 void WIR_GraphColoring::setRematerialization( bool r = true );
48
49 // Determine whether rematerialization is activated.
50 bool WIR_GraphColoring::getRematerialization( void );

```

used to identify spill candidates that contain only constant values. For such virtual registers, re-computing these constants usually is cheap so that no spill code is produced, but rematerialization code instead that simply loads the analyzed constant value. Due to the complexity of the bit-true data flow analysis, rematerialization is in general turned off by default.

Listing 14.6: Public Interface of RISC-V Register Allocator from Class RV_GraphColoring

```

1 // Default constructor for system-level optimization.
2 explicit RV_GraphColoring::RV_GraphColoring( WIR_System &s, bool v = false );
3
4 // Default constructor for compilation unit-level optimization.
5 explicit RV_GraphColoring::RV_GraphColoring( WIR_CompilationUnit &c, bool v = false );
6
7 // Default constructor for function-level optimization.
8 explicit RV_GraphColoring::RV_GraphColoring( WIR_Function &f, bool v = false );
9
10 // Set whether 32 bits wide floating-point registers from RV32IF shall be used.
11 void RV_GraphColoring::setRV32IF( bool if = true );

```

Class `RV_GraphColoring` (see file `WIR/arch/riscv/optimizations/gcregallocation/rvgraphcoloring.h`) provides the derivation of the `WIR` register allocator for the RISC-V RV32IMFC ISA. Its constructors are identical to those of the base class shown in Listing 14.4. The register allocator currently supports the 32 general-purpose integer registers `x0 – x31` and the 32 bits wide floating-point registers `f0 – f31`. The use of the floating-point registers can be enabled with method `setRV32IF` (cf. Line 11 of Listing 14.6).

Future Work

Register allocation for the RISC-V floating-point registers `f0 – f31` is currently partially supported. Adding support for the 64 bits wide floating-point registers defined by the RV32IFD ISA extension to the register allocator is part of our future work.

The port of the `WIR` register allocator to the Infineon TriCore architecture also inherits from class `WIR_GraphColoring` and can be found in class `TC_GraphColoring` from file `WIR/arch/tricore/optimizations/gcregallocation/tcgraphcoloring.h`. Its public interface is summarized in Listing 14.7. The constructors are very similar to those from Listing 14.4, except an additional optional Boolean argument `u` that defaults to false. This argument controls whether the TriCore allocator will use only physical registers from its so-called upper context. The TriCore's upper context is automatically saved by the processor's hardware during function calls so that their use is highly efficient. By default, however, the register allocator uses lower context registers if the register pressure is high. Then, however, the saving and restoration of these lower context-registers during function calls requires extra code to be generated by the register allocator.

Listing 14.7: Public Interface of TriCore Register Allocator from Class TC_GraphColoring

```

1 // Default constructor for system-level optimization.
2 explicit TC_GraphColoring::TC_GraphColoring( WIR_System &s, bool v = false, bool u = false );
3
4 // Default constructor for compilation unit-level optimization.
5 explicit TC_GraphColoring::TC_GraphColoring( WIR_CompilationUnit &c, bool v = false, bool u = false );
6
7 // Default constructor for function-level optimization.
8 explicit TC_GraphColoring::TC_GraphColoring( WIR_Function &f, bool v = false, bool u = false );
9
10 // Set whether only physical registers from the TriCore's upper context will be used.
11 void TC_GraphColoring::setUseOnlyUC( bool u = true );
12
13 // Determine whether only physical registers from the TriCore's upper context will be used.
14 bool TC_GraphColoring::getUseOnlyUC( void );

```

Besides the constructors' argument `u`, this behavior can also be specified using method `setUseOnlyUC` (Line 11). The Boolean argument of this method defaults to `true` which means that the register allocator will only use upper context registers. The current configuration of the allocator regarding the upper context can be retrieved with method `getUseOnlyUC` (Line 14).

14.9 Dead Code Elimination

Optimization Class	Optimization Level	Architectures
WIR_DeetCode, RV_DeetCode, TC_DeetCode	Systems – Functions	RV32IMC, TC1.3, TC1.3.1

Dead code denotes [WIR](#) operations that compute only values that are not used on any executable path leading from the operation [22, chapter 18.10]. When relating this to the bit-true [DFG](#) from Definition 13.3, dead operations are primarily characterized by nodes that have no outgoing edge. Of course, certain classes of machine operations that, by construction, have no outgoing [DFG](#) edges – e.g., function calls or returns, memory stores or other operations with side effects – are not considered dead.

However, this text book definition of dead code can be extended by exploiting that the bit-true data flow analysis from Section 13.11 is able to reason about potential values of registers. If the analysis reveals that all bits along some [DFG](#) edge provably do not care, i.e., all bits of the edge's up value are `bX`, then this means that a value flows along this edge whose individual bits are not used on the edge's executable path. Such dead [DFG](#) edges can naturally be eliminated, and if all outgoing edges of an operation get eliminated this way, the operation itself becomes dead. The entire process of marking edges and operations as dead, which in turn leads to other edges being dead etc. is iteratively applied until no more dead edges and operations were identified. All dead operations are finally removed from [WIR](#).

This behavior is realized by the [WIR](#) dead code elimination. Its base implementation is given by class `WIR_DeetCode` from file `WIR/optimizations/deadcode/wirdeadcode.h`, which in turn inherits from class `WIR_BitOpt` from Listing 14.2. ISA-specific variants plug the appropriate bit-true data flow analysis into this generic dead code elimination so that the final, processor-specific optimizations for RISC-V RV32IMC and Infineon TriCore are provided by classes `RV_DeetCode` and `TC_DeetCode`, resp. (see files `WIR/arch/riscv/optimizations/deadcode/rvdeadcode.h` and `WIR/arch/tricore/optimizations/deadcode/tcdeadcode.h`). The interfaces of all these optimization classes offer no particular features beyond Listing 14.2 so that no listings are provided here again.

14.10 Constant Folding

Optimization Class	Optimization Level	Architectures
WIR_ConstFold, RV_ConstFold, TC_ConstFold	Systems – Functions	RV32IMC, TC1.3, TC1.3.1

According to [22, chapter 12.1], constant folding refers to the evaluation at compile time of operations whose operands are known to be constant. Translating this to [WIR](#) and its bit-true data flow analysis, constant folding then replaces operations that provably only produce known constant values as output by new operations that explicitly generate these pre-computed constants.

Example

Consider the following code snippet based on the RISC-V RV32 processor model introduced in Section 9.1.4 as example:

```

1      addi      x_3, x0, 20          # x_3 := 20
2      addi      x_4, x0, 42          # x_4 := 42
3      mul       x_5, x_3, x_4        # x_5 := x_3 * x_4
4
5      add       x_6, sp, x_5         # x_6 := stack pointer + x_5
6      lw        x_7, 0(x_6)          # x_7 := memory[ x_6 ]

```

It shows typical, unoptimized code for address computations: Line 1 loads the number of an array element into virtual register `x_3`. The size of a single array element is loaded into `x_4` in Line 2. The multiplication of both values (Line 3) results in `x_5` holding the byte offset of the relevant array element, relative from the array's base. Adding the stack pointer (Line 5) provides the stack pointer-relative address in `x_6`, which is then finally used by the `lw` operation from Line 6.

The bit-true data flow analysis from Section 13.11 determines that register `x_5` defined by the multiplication always contains the constant value 840. Constant folding exploits this information as follows:

```

4      addi      x_5, x0, 840         # x_5 := 840
5      add       x_6, sp, x_5         # x_6 := stack pointer + x_5
6      lw        x_7, 0(x_6)          # x_7 := memory[ x_6 ]

```

This optimization exists within **WIR** for the RISC-V RV32IMC architecture (class `RV_ConstFold` from file `WIR/arch/riscv/optimizations/constfold/rvconstfold.h`) and the TriCore TC1.3 and TC1.3.1 ISAs (class `TC_ConstFold`, file `WIR/arch/tricore/optimizations/constfold/tcconstfold.h`). In general, it puts one (or maybe two) new operation immediately after each original operation that provably only produces constants along all outgoing **DFG** edges. This new operation (the `addi` from Line 4 in the above example) defines the very same register as `mul` from Line 3 does, and it is placed directly after it within the **WIR** code, without actively erasing the operations from Lines 1 to 3. This deletion of the original operations that only produce constants as result is left to a subsequent dead code elimination (cf. Section 14.9). Constant folding within **WIR** is realized such that it never attempts to fold operations that it has created itself. I.e., a second folding pass will not fold the previously created `addi` from Line 4 above again.

As the constant folding of **WIR** is a bit-level optimization, it also exploits that bits produced by some constant operation can be `bX`, i.e., they do not care. If the corresponding up value in the **DFG** is positive, such useless bits are interpreted as `b0`, or as `b1` for negative values. This way, the absolute value of the constant subject to folding is minimized, which increases the likelihood that only one machine operation is required for folding, instead of two.

In addition, this optimization furthermore features:

Branch folding: Whenever the condition of a conditional branch operation is known to be always true or false, the conditional branch gets substituted by an unconditional branch or gets eliminated completely.

Move folding: Operations that provably have no arithmetical effect but instead copy one register into another one (e.g., `xori x_8, x_7, 0` for RV32I) are replaced by a dedicated register-register move operation, allowing to coalesce it during a subsequent register allocation (cf. Section 14.8).

Similar to all optimizations previously presented in this chapter, the interfaces of the constant folding classes do not provide any specific features so that their discussion is omitted here.

14.11 Constant Propagation

Optimization Class	Optimization Level	Architectures
WIR_ConstProp, RV_ConstProp, TC_ConstProp	Systems – Functions	RV32IMC TC1.3, TC1.3.1

Constant propagation is a transformation that, given an assignment ' $x := c$ ' for a variable x and a constant c , replaces later uses of x with uses of c as long as intervening assignments have not changed the value of x [22, chapter 12.6]. In the context of [WIR](#), this optimization pass actually replaces register parameters of operations by immediate parameters, in order to make constant operands explicit as much as possible.

Example

For the RISC-V constant folding example from Section 14.10

```

4      addi      x_5, x0, 840           # x_5 := 840
5      add       x_6, sp, x_5          # x_6 := stack pointer + x_5
6
7      lw        x_7, 0(x_6)           # x_7 := memory[ x_6 ]
```

propagation of the constant 840 into the use of register x_5 of Line 5 has the effect that another operation is inserted after Line 5 that has a format supporting an immediate parameter, such as:

```

6      addi      x_6, sp, 840           # x_6 := stack pointer + 840
7      lw        x_7, 0(x_6)           # x_7 := memory[ x_6 ]
```

Again, constant propagation only adds new operations and leaves the removal of original, obsolete operations (e.g., the `addi` and `add` operations from Lines 4 and 5 above) to a subsequent dead code elimination. And propagation supports constants consisting of `b0`, `b1` and `bX` bits out of the L4 partial order described in Section 13.11.

Within [WIR](#), constant propagation is realized by the abstract base class `WIR_ConstProp` from file `WIR/optimizations/constprop/wirconstprop.h`. Its port to the RISC-V architecture resides in class `RV_ConstProp` from file `WIR/arch/riscv/optimizations/constprop/rvconstprop.h`. The derivation for the TriCore architecture is provided by class `TC_ConstProp` (file `WIR/arch/tricore/optimizations/constprop/tcconstprop.h`). The [APIs](#) of the constant propagation classes are identical to that shown in Listing 14.2.

14.12 Redundant Code Elimination

Optimization Class	Optimization Level	Architectures
WIR_RedundantCode, RV_RedundantCode, TC_RedundantCode	Systems – Functions	RV32IMC TC1.3, TC1.3.1

According to [22, chapter 13.1], an expression in a program is a common subexpression if there is another occurrence of the expression whose evaluation always precedes this one in execution order and if the operands of the expression remain unchanged between the two evaluations. In the context of [WIR](#), this notion is generalized and subsequently called redundant code. The key property of common subexpressions is that they always produce the very same results, and this is exploited using the bit-true data flow analysis of [WIR](#) (cf. Section 13.11). Redundant code thus is characterized by pairs of [WIR](#) operations $o1$ and $o2$ such that

- $o1$ defines a register parameter $p1$ and $o2$ uses a register parameter $p2$ where the bit-true data flow along all outgoing edges of $p1$ is equivalent to that of all incoming edges of $p2$, and

- the definition of p1 always reaches the use in p2, i.e., o1 dominates o2, and
- the registers defined and used by p1 and p2 are different, but are of the same type.

Redundant operations are then optimized such that the register used by parameter p2 is replaced by the register defined in p1. This way, a direct data flow from p1 to p2 is achieved, probably turning the initially incoming edges of p2 dead.

The generic part of this optimization is provided by class `WIR_RedundantCode` from file `WIR/optimizations/redundantcode/wirredundantcode.h`. Its port to the RISC-V architecture can be found in class `RV_RedundantCode` (file `WIR/arch/riscv/optimizations/redundantcode/rvredundantcode.h`), the TriCore class is `TC_RedundantCode` (file `WIR/arch/tricore/optimizations/redundantcode/tcredundantcode.h`). All these classes do not come with any special API features worthwhile mentioning here.

14.13 Jump Correction

Optimization Class	Optimization Level	Architectures
<code>WIR_JumpCorrection</code> , <code>TC_JumpCorrection</code>	Systems	TC1.3, TC1.3.1

As discussed earlier in Chapter 10, **WIR** is an **IR** that is explicitly designed to model memory layouts of computer systems and to perform memory allocation of, e.g., functions or basic blocks. However, jump and function call operations of real-life processor **ISAs** are regularly limited in the distance that they can bridge. According to the vendor-specific format of some jump operation, a maximum distance of, e.g., $\pm 2^{15}$ bytes between the jump and its target might be allowed. If the jump and its target are accidentally allocated to memory regions that are far away from each other in a processor's address space, the jump distance might become too large so that the jump operation in its given format is invalid.

Such situations are detected and corrected by the jump correction pass. By exploiting the **WIR** memory layout (cf. Section 10.4), it computes the addresses of a jump and its target and determines whether it is valid or not. The latter case is corrected by, e.g., replacing a short jump by a long jump, by inserting indirect jumps, or by creating trampolines etc. The correction of one jump can have the effect that the addresses of following basic blocks change, potentially turning other jumps or calls invalid. Thus, jump correction in general is an iterative pass that keeps on working until all jumps and calls are valid in the end.

Obviously, this pass is heavily **ISA**-specific so that its abstract base class `WIR_JumpCorrection` from file `WIR/optimizations/jumpcorrection/wirjumpcorrection.h` provides very generic functionality only. The main sophistication lies in the processor-specific classes that inherit from this base class. Currently, jump correction is realized for the Infineon TriCore **ISAs**, see class `TC_JumpCorrection` (file `WIR/arch/tricore/optimizations/jumpcorrection/tcjumpcorrection.h`).

Listing 14.8 shows the public interface of class `WIR_JumpCorrection`. As can be seen, there is only one constructor (cf. Line 2) that expects a reference to a full **WIR** system as argument. More fine-grained optimization, e.g., at the level of single functions, is not supported as the memory layout properties introduced in Section 10.4 require the presence of a complete system. If just a single correction pass over all jumps present in a system shall be performed instead of iterating until all jumps are valid, this behavior can be specified using `setSinglePass` (Line 5). The method's Boolean argument defaults to true so that only a single pass is carried out. This jump correction property can be queried with method `getSinglePass` (Line 8).

The jump costs methods `getJumpCostsSize` and `getJumpCostsCycles` from Lines 11 and 14 are purely virtual as they are inherently processor-specific. Their intention is to provide the estimated number of bytes and clock

Listing 14.8: Public Interface of Class WIR_JumpCorrection

```

1 // Default constructor for system-level optimization.
2 explicit WIR_JumpCorrection( WIR_System &s );
3
4 // Set whether jump correction is applied just once or if it repeats until a fixed point is reached.
5 void WIR_JumpCorrection::setSinglePass( bool p = true );
6
7 // Determine whether jump correction is applied just once or if it repeats until a fixed point is reached.
8 bool WIR_JumpCorrection::getSinglePass( void ) const;
9
10 // Get the number of bytes introduced to a basic block due to correction of a jump from s to t.
11 virtual int WIR_JumpCorrection::getJumpCostsSize( const WIR_BasicBlock &s, const WIR_BasicBlock &t,
12                                                    bool src ) = 0;
13
14 // Get the number of clock cycles introduced to a basic block due to correction of a jump from s to t.
15 virtual int WIR_JumpCorrection::getJumpCostsCycles( const WIR_BasicBlock &s, const WIR_BasicBlock &t,
16                                                     bool src ) = 0;

```

cycles, resp., that will be added due to the correction of a jump from a given source to a target basic block. The Boolean argument of both methods specifies whether the additional costs relating to either the source (`true`) or to the target (`false`) basic block shall be provided. These methods are not required for the actual task of jump correction per se. Instead, they can be used, e.g., by advanced memory allocation optimizations so that they can properly consider potential overheads if they decide to allocate two basic blocks to different memories, making a jump correction necessary.

The TriCore-specific variant of jump correction does not extend this [API](#) from Listing 14.8 so that we omit its presentation for the sake of brevity.

14.14 Operation Bit Width Reduction

Optimization Class	Optimization Level	Architectures
RV_OperationBitWidths, TC_OperationBitWidths	Systems – Functions	RV32IMC TC1.3, TC1.3.1

Many processor architectures feature a mixed-width [ISA](#) where operations having different bit widths can freely be used. The operation bit width reduction of [WIR](#) is a pass that consequently replaces operations of large bit widths by more compact operations in order to reduce code size. In many processor [ISAs](#), this width reduction is achieved by saving one explicit parameter denoting an operation's argument and target register. In other words, such compact operations regularly use the very same register parameter both as input and as output, i.e., the register gets overwritten by a compact operation.

Example

Consider the following 16 bits wide add operation of the TriCore [ISA](#) as example:

```

1      add          d_3, d_4          # d_3 := d_3 + d_4, 16 bits wide

```

As can be seen, it only requires two registers as parameters, turning it into a compact 16-bits operation. The first register parameter `x_3` is both used and defined afterwards.

If some [WIR](#) code contains a 32 bits wide addition that also overwrites its first input register like, e.g.:


```

2  add      d_3, d_3, d_4      # d_3 := d_3 + d_4, 32 bits wide

then this operation's bit width can naturally be reduced by exploiting the 16-bits variant of add. If further-
more the 32-bits operation performs a commutative computation, as a classical addition does, then

3  add      d_3, d_4, d_3      # d_3 := d_4 + d_3, 32 bits wide

can also be reduced to the add from Line 1.

```

Class `RV_OperationBitWidths` (cf. file `WIR/arch/riscv/optimizations/operationbitwidths/rvoperationbitwidths.h`) performs this kind of bit width reductions for the RISC-V RV32IMC architecture by systematically matching all 32 bits wide operations with scenarios allowing to use compact 16 bits wide operations instead. The TriCore port of this optimization is realized by class `TC_OperationBitWidths` from file `WIR/arch/tricore/optimizations/operationbitwidths/tcoperationbitwidths.h`. This class has no specific additional features so that its public interface is identical with Listing 14.1.

14.15 Predicated Execution

Optimization Class	Optimization Level	Architectures
TC_PredicatedExecution	Systems – Functions	TC1.3, TC1.3.1

Predicated execution is a concept supported by many ISAs in order to avoid branching, especially for architectures with deep pipelines. In classical, unoptimized code, if-then regions (see Fig. 13.4 on Page 158) are realized using a conditional branch that evaluates some condition. Operations inside the branch node are either executed or skipped via the conditional branch.

When using predicated execution (also known as conditional execution), the conditional branch is removed so that the operations within the branch node are always executed. However, these operations are extended by a condition parameter that holds the result of the evaluated condition of the if-then region. Depending on these condition parameters, the always-executed, predicated operations either keep their computed results, or silently discard them as if they had never been executed.

Within [WIR](#), predicated execution is supported for the TriCore architectures. This pass is realized by class `TC_PredicatedExecution` (file `WIR/arch/tricore/optimizations/predicatedexecution/tcpredicatedexecution.h`). Its implementation closely follows the suggestions from [29, sections 1.2.8 and 1.2.9]. According to that, predicated execution can be applied to an if-then region if

- a condition basic block ends with a conditional branch, and
- the branch node consists of a single basic block only, and
- all operations inside this one basic block are suitable for predicated execution, i.e., can be transformed into the TriCore `cmov`, `cmovn`, `sel`, `seln`, `cadd`, `caddn`, `csub` or `csubn` operations.

14.16 Peephole Optimization

Optimization Class	Optimization Level	Architectures
WIR_Peephole, WIR_SiliconBugs, TC_Peep_EXTRST – TC_Peep_PreIncr, TC1796_CPU_TC_013 – TC179x_CPU_TC_105	Systems – Functions	TC1.3, TC1.3.1

Peephole optimization is a very simple approach where a sliding window of fixed size is moved over the code of [WIR](#) functions. Within the currently observed peephole, some specific code pattern is searched and if this pattern matches, a simple local transformation of the peephole's code is done. This general behavior to move a sliding window and to match some code pattern is realized by the abstract base class `WIR_Peephole` from file `WIR/optimizations/peephole/wirpeephole.h`. The generic interface of this base class is presented in Listing 14.9.

Besides the standard constructors from Lines 2 to 8, methods `addPeepholeSize` and `getPeepholeSizes` (Lines 11 and 14) are provided for the handling of several peephole sizes. Peephole sizes represent the numbers of consecutive [WIR](#) instructions that form a peephole. By supporting several peephole sizes, various code patterns can be matched in one pass in a very efficient manner. The set of all specified peephole sizes can be retrieved using `getPeepholeSizes`. Another characteristic property of peepholes is whether they may span over several [WIR](#) basic blocks or not. This behavior can be controlled using method `setCrossBasicBlocks` (Line 17). By default, peepholes are not allowed to cross basic block boundaries so that all instructions of a peephole belong to the very same basic block. Method `getCrossBasicBlocks` (Line 20) can be used to query the status of this property.

The peephole optimizer then starts creating peepholes for a given [WIR](#) function beginning with its very first instruction. From this starting point on, the next n instructions form a peephole if they exist in the current function and if n is a previously specified peephole size. Depending on `setCrossBasicBlocks`, the inspection of the next n instructions moves over and covers all successor blocks if the last instruction of the current basic block is reached. This way, all possible peepholes for one instruction acting as starting point are formed, code patterns

Listing 14.9: Public Interface of Class `WIR_Peephole`

```

1 // Default constructor for system-level optimization.
2 explicit WIR_Peephole::WIR_Peephole( WIR_System &s );
3
4 // Default constructor for compilation unit-level optimization.
5 explicit WIR_Peephole::WIR_Peephole( WIR_CompilationUnit &c );
6
7 // Default constructor for function-level optimization.
8 explicit WIR_Peephole::WIR_Peephole( WIR_Function &f );
9
10 // Add another peephole size to be considered during optimization.
11 void WIR_Peephole::addPeepholeSize( unsigned int s );
12
13 // Get the set of all peephole sizes to be considered during optimization.
14 const std::set<unsigned int> &WIR_Peephole::getPeepholeSizes( void ) const;
15
16 // Set whether peepholes may cross basic block boundaries or not.
17 void WIR_Peephole::setCrossBasicBlocks( bool c = true );
18
19 // Determine whether peepholes may cross basic block boundaries or not.
20 bool WIR_Peephole::getCrossBasicBlocks( void ) const;

```

Table 14.1: Examples of TriCore Peephole Transformations

Peephole Optimizer	Original Code Pattern	Transformed Code Pattern
TC_Peep_EXTRST	EXTR dA, dB, 0, 8 ST.B [aX], dA	ST.B [aX], dB
TC_Peep_LDEXTR	LD.B dA, [aX] EXTR dB, dA, 0, 8	LD.B dB, [aX]
TC_Peep_LDST	LD.B dA, [aB] ST.B [aB], dA	LD.B dA, [aB]
TC_Peep_PreIncr	ADD.A aA, 6 ST.B [aA]0, dX	ST.B [+aA]6, dX

are matched and transformations applied. After the current starting instruction has been processed, its successor instruction is considered as next starting point, etc.

For the Infineon TriCore architecture, a total of 4 peephole optimizers exist within [WIR](#) (cf. directory `WIR/arch/tricore/optimizations/peephole`). Their behavior is exemplified in Table 14.1. In TriCore code, the EXTR operation regularly serves for casting the contents of some register dB to 8 bits of a byte or 16 bits of a half-word. Such casts are redundant before memory stores or after memory loads of bytes or half-words, resp., because these loads and stores of bytes and half-words only transmit 8 or 16 bits to/from memory by construction. Thus, the TriCore peephole optimizers TC_Peep_EXTRST and TC_Peep_LDEXTR detect such situations, remove the EXTR operation and patch the load/store such that the originally casted register dB is stored or loaded as byte or half-word.

A very simple form of redundant load/store elimination is realized by class TC_Peep_LDST. It searches for two directly following load and store operations of the same bit width that load and store using the very same address register aB. If, furthermore, the very same data register dA that gets loaded is directly written back to the very same address, then the store operation is redundant and is removed.

Finally, the peephole optimizer TC_Peep_PreIncr exploits the pre-increment addressing mode of the TriCore architecture. It looks for address register increments by a constant using, e.g., an ADD.A operation, followed by a subsequent memory load or store that uses this incremented address register as base address with constant offset 0. Then, the same effect can be achieved by eliminating the ADD.A operation and modifying the memory access such that it uses pre-increment addressing by the original constant increment.

A very special variant of peephole optimization is provided by the silicon bug optimizer of [WIR](#). It is generically realized by class `WIR_SiliconBugs` (see file `WIR/optimizations/siliconbugs/wirsiliconbugs.h`) that is derived from `WIR_Peephole`. The motivation of this is that occasionally, real hardware implementations of some architecture behave slightly different than initially specified in the [ISA](#). Such behavioral deviations are documented by the hardware manufacturer, and possible workarounds are given by providing code sequences that do not trigger such silicon bugs. The silicon bugs optimizer searches for code snippets that might trigger such a bug and replaces them by the suggested workaround.

The general optimization approach (detecting a given code pattern in some peephole, and locally transforming the peephole's contents) is identical to that of class `WIR_Peephole`. The constructor of class `WIR_SiliconBugs` is shown in Line 2 of Listing 14.10. It receives a reference to a [WIR](#) function to be optimized as first argument, followed by a string containing some ID of the detected silicon bug according to the manufacturer's documentation. This string is used to attach a comment to the optimized code snippet that indicates that this particular code structure prevents a certain silicon bug. Finally, the constructor expects the peephole's size and whether peepholes may span over several basic blocks, see Listing 14.9.

Listing 14.10: Constructor of Class WIR_SiliconBugs

```

1 // Default constructor for function-level optimization.
2 WIR_SiliconBugs::WIR_SiliconBugs( WIR_Function &f, std::string &n, unsigned int s, bool c = false );

```

For the Infineon TriCore TC1796 chip, the [WIR](#) silicon bugs optimizers consider errata sheet [26] and work around bug IDs CPU_TC.013, CPU_TC.048, CPU_TC.060, CPU_TC.065, CPU_TC.069, CPU_TC.070, CPU_TC.071, CPU_TC.072, CPU_TC.081, CPU_TC.082, CPU_TC.083, CPU_TC.094, CPU_TC.095, CPU_TC.096 and CPU_TC.105. Silicon bug ID CPU_TC.106 of the TC1797 chip is fixed according to [27]. In many cases, the workaround consists of inserting a NOP operation at specific places in a critical code sequence, which underlines the very local, peephole style of transformation. All TriCore silicon bug optimizers reside as dedicated classes in directory WIR/arch/tricore/optimizations/siliconbugs.

14.17 Local Instruction Scheduling

Optimization Class	Optimization Level	Architectures
WIR_LocalScheduling, TC_LocalScheduling	Systems – Functions	TC1.3, TC1.3.1

Local instruction scheduling rearranges the order in which [WIR](#) operations occur locally within single basic blocks. In general, the goals of this re-scheduling of operations are twofold: first, the flow of operations through a processor's pipeline is optimized such that, e.g., pipeline stalls are avoided; second, [ILP](#) provided by a processor architecture can be exploited such that several operations get executed in parallel. Within the [WIR](#) software model, [ILP](#) is modeled by class WIR_Instruction (see Section 8.4 on Page 47). Thus, the instruction scheduler in general attempts to group operations together within one single instruction which models that all these operations are supposed to be executed within the very same execution cycle.

Naturally, instruction scheduling must obey dependencies between operations, i.e., [Read-After-Write \(RAW\)](#), [Write-After-Read \(WAR\)](#), [Write-After-Write \(WAW\)](#) and control dependencies. For this purpose, the scheduler creates the dependence graph for each basic block to be optimized. The nodes represent the block's operations,

Listing 14.11: Public Interface of Class WIR_LocalScheduling

```

1 // Default constructor for system-level optimization.
2 WIR_LocalScheduling::WIR_LocalScheduling( WIR_System &s, const WIR_SchedulingPriority &p, bool v = false,
3                                           bool t = false );
4
5 // Default constructor for compilation unit-level optimization.
6 WIR_LocalScheduling::WIR_LocalScheduling( WIR_CompilationUnit &c, const WIR_SchedulingPriority &p,
7                                           bool v = false, bool t = false );
8
9 // Default constructor for function-level optimization.
10 WIR_LocalScheduling::WIR_LocalScheduling( WIR_Function &f, const WIR_SchedulingPriority &p, bool v = false,
11                                           bool t = false );
12
13 // Set that as-soon-as-possible scheduling is applied.
14 void WIR_LocalScheduling::setASAP( void );
15
16 // Set that as-late-as-possible scheduling is applied.
17 void WIR_LocalScheduling::setALAP( void );

```

Listing 14.12: Relevant Interface of Class WIR_SchedulingPriority

```

1 // Default constructor.
2 WIR_SchedulingPriority::WIR_SchedulingPriority( void );
3
4 // Copy constructor.
5 WIR_SchedulingPriority::WIR_SchedulingPriority( const WIR_SchedulingPriority &p );
6
7 // Get an operation's scheduling priority.
8 virtual long long WIR_SchedulingPriority::getPriority( const WIR_Operation &o ) const = 0;
9
10 // Get the operation sequence with best scheduling priority for a given execution cycle.
11 std::list<WIR_Operation *> WIR_SchedulingPriority::getBestOperations( long long c, WIR_Instruction *i );

```

and directed edges are inserted in the presence of the above dependencies. This graph is then traversed in topological order and operations are scheduled into execution cycles such that this order is preserved.

The generic and processor-independent parts of the instruction scheduler are realized by the abstract base class `WIR_LocalScheduling` (cf. file `WIR/optimizations/scheduling/wirlocalscheduling.h`). Its public interface is depicted in Listing 14.11. As can be seen, the constructors (Lines 2 to 8) allow to apply scheduling for all levels from a complete system down to single functions. A mandatory parameter of all constructors is a reference to a scheduling priority object. Its purpose is to select one operation with high priority to be scheduled next in the current execution cycle, out of several operations that could potentially be scheduled. Last but not least, all constructors accept two optional Boolean flags defaulting to `false` as last arguments. They serve for debugging purposes during the potential visualization of the dependence graph using the graph visualization tool `xdot`. The first Boolean `v` prints a verbose message showing the issued `xdot` command line, while argument `t` keeps the temporary DOT file containing the dependence graph on the hard disk.

Schedulers classically distinguish between **ASAP** and **ALAP** scheduling which arranges operations at their earliest or latest convenience, resp. **ASAP** scheduling strives to schedule an operation as soon as its predecessors in the dependence graph have finished execution. In **ALAP** scheduling, an operation is scheduled such that it gets executed whenever its successors require its data. By default, class `WIR_LocalScheduling` performs **ASAP** scheduling. This strategy can explicitly be controlled via methods `setASAP` (cf. Line 11) and `setALAP` (Line 14) which require no further explanation.

Class `WIR_SchedulingPriority` (see file `WIR/optimizations/scheduling/wirschedulingpriority.h`) is an abstract base class that generically models different priorities that can be assigned to **WIR** operations during scheduling. The relevant parts of its interface are shown in Listing 14.12. There is nothing to say about the constructors from Lines 2 and 5. Priorities are modeled using the standard data type `long long` so that `getPriority` (Line 8) is a purely virtual method that returns such a priority value for a given operation. Priorities are interpreted literally, i.e., the higher the `long long` value determined by `getPriority`, the higher the operation's priority. Based on such priorities, `getBestOperations` (Line 11) determines those operations that should best be grouped together for some given execution cycle.

For each basic block, the **WIR** scheduler begins with the starting execution cycle 0, determines those operations to be scheduled in this current cycle, and continues with the next cycle. This scheme is directly supported by method `getBestOperations`: it gets the current execution cycle into which operations shall be scheduled as first argument. Furthermore, the **WIR** instruction containing all operations scheduled in the previous cycle (if any) serves as second argument. A list of pointers to operations is finally returned that should be scheduled together and that thus should form the next instruction inside the current basic block.

Currently, **WIR** supports the following heuristics to determine scheduling priorities:

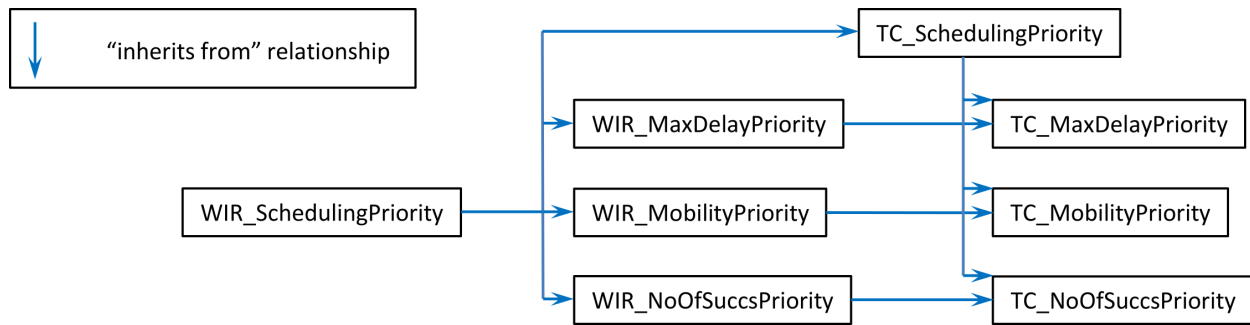


Figure 14.1: Inheritance Diagram of Scheduling Priority Classes

Maximum Delays: The maximum delay represents the length of the longest path from an operation to the basic block's very last operation within the dependence graph. The higher the maximum delay is, the higher is a node's scheduling priority.

Mobility: Mobility models the flexibility that an operation has within the dependence graph to be scheduled. More precisely, an operation's mobility is the difference between its latest- and earliest-possible execution cycle. Operations with a high mobility are not severely constrained in the dependence graph and thus get a high scheduling priority.

Number of Successors: This measure aims to quantify how many other, dependent operations potentially get enabled and their dependencies met by executing some other operation. Consequently, operations with a large fan-out in the dependence graph obtain a high scheduling priority.

These heuristics are realized by classes `WIR_MaxDelayPriority`, `WIR_MobilityPriority` and `WIR_NoOfSuccsPriority` (located in directory `WIR/optimizations/scheduling`) that all inherit from the above base class as depicted in Fig. 14.1. The interfaces of all these derived classes do not differ from what is shown in Listing 14.12.

Processor-specific realizations of this instruction scheduler for the Infineon TriCore models are in turn derived using inheritance. Thus, the overall TriCore scheduler comes with class `TC_LocalScheduling` located in file `WIR/arch/tricore/optimizations/scheduling/tclocalscheduling.h`. The TriCore-specific scheduling priorities (i.e., classes `TC_MaxDelayPriority`, `TC_MobilityPriority` and `TC_NoOfSuccsPriority` shown in Fig. 14.1) exploit detailed pipelining and timing information taken from the manufacturer's data sheets.

15 Introspection of WIR

The classes, data structures and algorithms of the [WIR](#) library are complex so that users might wish to get a deeper understanding of its inner workings without having to do a deep dive into its C++ code base and without having to rebuild [WIR](#) completely. Typical questions that come up when using [WIR](#) are, e.g.: When are objects of a certain class created or destroyed? How does a complex analysis or optimization algorithm from Chapters [13](#) and [14](#) behave in more detail? How does the control or data flow graph look like?

[WIR](#) comes with a convenient mechanism allowing to do such kinds of introspection of the library's behavior without having to change the code base. These mechanisms are called *debug macros* in the following. By using a simple configuration file, the desired output can selectively be turned on or off, giving the user full control over the amount of created introspection output.

The entire debug macro infrastructure gets activated only if a [WIR](#) build is configured with option `--enable-wir-debugmacros` (see Table [4.1](#) on Page [23](#)). By default, the whole infrastructure is deactivated since it has a noticeable performance impact. The rest of this chapter thus assumes a [WIR](#) build configured with option `--enable-wir-debugmacros`.

Note

For release quality builds of the [WIR](#) software package at the highest optimization level of the host compiler, users typically should not activate the debug macros.

Section [15.1](#) first describes the general infrastructure provided by the debug macros. After that, Section [15.2](#) presents how to use the debug macros integrated into the [WIR](#) code for an in-depth introspection.

15.1 Debug Macro Framework

The term debug macros originates from a couple of C++ preprocessor macros that are all declared in file `WIR/LIBUSEFUL/libuseful/debugmacros.h`. The entire framework consists of two parts:

- the consequent and pervasive use of the debug macros in the C++ code of [WIR](#) by the developers, and
- the dynamic and flexible configuration of actual introspection output to be produced for a [WIR](#) user based on configuration file `wir.conf` that by default resides in directory `WIR/DEBUGMACRO_CONF` (see configuration option `--with-debugmacro-configuration` from Table [4.1](#)).

The following discussion and the excerpt from `debugmacros.h` shown in Listing [15.1](#) focuses on the most frequently used debug macros only which are heavily used by [WIR](#). For a full description of all available debug macros, the interested reader is referred to header file `WIR/LIBUSEFUL/libuseful/debugmacros.h`.

The most important debug macro is `DINIT` shown in Line [2](#). It expects a string literal as argument which serves as so-called activation name. The purpose of `DSTART` is to selectively activate the debugging of a complete C++ scope if the user has specified this in configuration file `wir.conf`. Here, a C++ scope is any region in the C++ code base of [WIR](#) between curly brackets `{ ... }`. For such a scope featuring `DSTART`, a message is printed out

Listing 15.1: Most Relevant Debug Macros from WIR/LIBUSEFUL/libuseful/debugmacros.h

```

1 // Start debugging for the currently active C++ scope identified by the given activation name.
2 #define DSTART( ACTIVATIONNAME ) [...]
3
4 // Produce debug output for the currently active C++ scope.
5 #define DOUT( ... ) [...]
6
7 // Execute any arbitrary debugging code for the currently active C++ scope.
8 #define DACTION( ACTIONCODE ) [...]

```

whenever this scope is entered during the execution of a user's [WIR](#)-based application. Likewise, an exit message is printed whenever such a scope is left, i.e., the user's application reaches the scope's closing `}` bracket.

In order to give the user control for which scopes such enter and exit messages shall be printed, the activation name is used which serves as a unique identifier for debugging-relevant scopes. The string literals used as activation names for `DSTART` all-over [WIR](#) are hard-coded in the C++ code of [WIR](#). By putting the activation name strings of the user's interest into the abovementioned configuration file `wir.conf`, debugging of the so-specified scope is activated and enter and exit messages are created. If a scope's activation name is not present in the user's configuration file, no debug output will be produced for a scope.

Internally, the debug macros dynamically organize activation names in a stack. Whenever a scope featuring `DSTART` is entered while executing a user's application, the scope's activation name is put on top of the current stack. Similarly, the top of the stack is removed when leaving a `DSTART` scope. This allows to properly keep track of the nesting of scopes while debugging [WIR](#) applications that very frequently execute one scope from within another in a deeply nested fashion.

Besides such enter and exit messages, it regularly is desirable to print additional messages for debugging purposes. This can include, e.g., simple messages ("Hello world.") indicating that a certain position in the code has been reached, or messages printing the current contents of important variables etc. For this purpose, the `DOUT` macro (Line 5) is used within [WIR](#). It accepts a variadic number of arguments that are all treated as regular C++ code serving to produce output to a `std::ostream`. This way, complex debug messages can be created as follows:

```

1 DOUT( "Current address: 0x" << std::hex << addr << "." << std::dec << std::endl );

```

`DOUT` only generates actual output if it is part of a scope featuring `DSTART` and if the activation name provided to `DSTART` shows up in the user's configuration file.

Finally, it might be necessary to execute complex code for debugging purposes instead of printing simple messages. This usually is the case when data structures shall be inspected and checked for sanity. This kind of debugging is supported by macro `DACTION` from Line 8. It expects any kind of C++ code to be executed only if, once again, the activation name of the scope is configured by the user. The following example displays the names of all childs of some register `reg` only if debugging of the scope containing this `DACTION` is desired by the user:

```

1 DACTION(
2     for ( WIR_VirtualRegister &child : reg.getChilds() )
3         DOUT( "Current child register: " << child.getName() << std::endl ); );

```

For [WIR](#) builds configured with disabled debug macros, all preprocessor definitions from file `debugmacros.h` get replaced by empty dummies so that the manifold occurrences of `DSTART`, `DOUT` and `DACTION` within the [WIR](#) library leave absolutely no trace in this case.

Debug macro configuration files are simple plain-text files that can be edited by the user in any text editor. They are interpreted line by line, empty lines and comment lines starting with "#" or "/" are ignored. One line in a debug macro configuration file now simply contains exactly one activation name of some scope to be debugged. The strings used in the context of DSTART and within a user's configuration file are always case-sensitive, and they are interpreted literally, i.e., spacing or punctuation also matters. Thus, the configuration file needs to list the exact and identical strings as they show up in the DSTART directives to be debugged.

15.2 WIR Debug Macros

As mentioned earlier, **WIR** by default considers the debug macro configuration file `WIR/DEBUGMACRO_CONF/wir.conf`. For the user's convenience, a configuration template listing all currently existing and supported activation names is provided in file `WIR/DEBUGMACRO_CONF/wir.conf.bak`. Activation names are all grouped by their associated **WIR** classes. This template gives an overview for which parts of **WIR** debugging can be activated. By copying and pasting strings from the template to `wir.conf`, an appropriate configuration can be created easily.

As can be seen, these activation names regularly refer to the various methods of **WIR** classes, including their argument and return types. This results from the fact that within the entire C++ code base of the **WIR** library, all methods always consequently start with a dedicated DSTART directive. Like this, the begin and end of execution of individual **WIR** methods can be traced by the user.

Listing 7.2 on Page 39 showed an example for the variadic construction of object hierarchies within **WIR**. When using the following debug macro configuration:

```

1 WIR_CompilationUnit::WIR_CompilationUnit(WIR_Function&&, Args&& ...)
2 WIR_Function::WIR_Function(string&&)
3 WIR_BasicBlock& WIR_Function::pushBackBasicBlock(WIR_BasicBlock&&)
4 WIR_BasicBlock::WIR_BasicBlock(WIR_Instruction&&, Args&& ...)
5 WIR_Instruction::WIR_Instruction(WIR_Operation&&, Args&& ...)
6 WIR_Operation::WIR_Operation(const WIR_BaseProcessor::OpCode&, const WIR_BaseProcessor::OperationFormat&,
  Args&& ...)
```

the creation of the various objects by the example program can be observed. Compiling and executing the example from Listing 7.2 with the above configuration file in place produces the following output:

```

1 --> enter WIR_Function::WIR_Function(string&&) (0)
2 <-- exit WIR_Function::WIR_Function(string&&) (0)
3 --> enter WIR_CompilationUnit::WIR_CompilationUnit(WIR_Function&&, Args&& ...) (0)
4 <-- exit WIR_CompilationUnit::WIR_CompilationUnit(WIR_Function&&, Args&& ...) (0)
5 --> enter WIR_Operation::WIR_Operation(const WIR_BaseProcessor::OpCode&, const
  WIR_BaseProcessor::OperationFormat&, Args&& ...) (0)
6 <-- exit WIR_Operation::WIR_Operation(const WIR_BaseProcessor::OpCode&, const
  WIR_BaseProcessor::OperationFormat&, Args&& ...) (0)
7 --> enter WIR_Instruction::WIR_Instruction(WIR_Operation&&, Args&& ...) (0)
8 <-- exit WIR_Instruction::WIR_Instruction(WIR_Operation&&, Args&& ...) (0)
9 --> enter WIR_Operation::WIR_Operation(const WIR_BaseProcessor::OpCode&, const
  WIR_BaseProcessor::OperationFormat&, Args&& ...) (0)
10 <-- exit WIR_Operation::WIR_Operation(const WIR_BaseProcessor::OpCode&, const
  WIR_BaseProcessor::OperationFormat&, Args&& ...) (0)
11 --> enter WIR_Instruction::WIR_Instruction(WIR_Operation&&, Args&& ...) (0)
12 <-- exit WIR_Instruction::WIR_Instruction(WIR_Operation&&, Args&& ...) (0)
13 --> enter WIR_BasicBlock::WIR_BasicBlock(WIR_Instruction&&, Args&& ...) (0)
14 <-- exit WIR_BasicBlock::WIR_BasicBlock(WIR_Instruction&&, Args&& ...) (0)
15 --> enter WIR_BasicBlock& WIR_Function::pushBackBasicBlock(WIR_BasicBlock&&) (0)
16 <-- exit WIR_BasicBlock& WIR_Function::pushBackBasicBlock(WIR_BasicBlock&&) (0)
```

Changing the previous debug macro configuration to:

```
1 void WIR_System::createComponents(const string&)
```

and re-executing the very same test program now yields the following output:

```
1 --> enter void WIR_System::createComponents(const string&) (1)
2   Reading system configuration file '/tmp/WIR/arch/generic/genericmips.sys'.
3   Created new processor core 'CORE0'
4     Instruction set architecture: 'WIR generic'
5     Clock frequency: 150000000 Hz
6     Voltage: 3.3 V
7   Created new memory region 'RAM'
8     Base address: 0x0
9     Length: 0x100000
10    Attributes: 0xf
11    Delay: 1
12    Memory hierarchy: { 'CORE0' }
13 <-- exit void WIR_System::createComponents(const string&) (1)
```

As can be seen from this example, just editing the plain-text configuration file helps to display completely different output, now showing the details how a [WIR](#) system is set up for the generic MIPS architecture from Section [9.1.2](#).

Besides activation names referring to complete methods of the [WIR](#) code base, a few particular ones are provided for the introspection of complex analyses and optimizations. They are summarized in Listing [15.2](#). The control tree iteratively constructed during structural control flow analysis (see Section [13.5](#) at Page [155](#)) can be visualized using the activation names from Lines [1](#) and [2](#). The former shows the current control tree after each individual iteration of the analysis, while the latter shows only the finally created result.

Lines [4](#) and [5](#) show the activation names relating to the visualization of the bit-true [DFG](#) from Definition [13.3](#) on Page [176](#). The first one enables the graph visualization right after the top-down analysis, i.e., the forward data flow analysis. The final bit-true [DFG](#) resulting from both top-down and bottom-up backward analysis can be shown using the activation name from Line [5](#).

Due to its inherent complexity, various debugging facilities are provided for the graph coloring-based register allocation described in Section [14.8](#) on Page [189](#). Most of them serve to visualize the interference graph during the various steps of the graph coloring algorithm: immediately after having built the initial, uncolored interference graph (Line [7](#)); after graph simplification, once showing only the remaining non-simple nodes (Line [8](#)), or once showing both simple and non-simple nodes (Line [9](#)); after coalescing of graph nodes (Line [10](#)); after coloring the graph (Line [11](#)); and finally when applying stack coalescing (Line [12](#)).

According to [[22](#), chapter 16.3], the [WIR](#) graph coloring is implemented as a typical worklist algorithm. The processing of the various worklists, i.e., when operations or registers are added to or removed from certain worklists can be monitored by configuring the activation name from Line [13](#). Furthermore, [[22](#), chapter 16.3] and [[1](#), chapter 11] formulate a couple of invariants, i.e., structural properties of the interference graph and/or the worklists that must always hold. The explicit verification of all these invariants (and many others, going beyond the textbooks cited above) and thus in-depth sanity checks of all algorithms can be turned on using the activation name from Line [14](#). However, the computational overhead of this introspection of worklists and invariants is extremely high. Even if Lines [13](#) and [14](#) are not active, the very frequent executions of `DSTART` and the corresponding comparisons of activation names with configuration file contents still lead to a noticeable performance degradation. For this reason and in order to obtain a reasonable performance of register allocation even for [WIR](#) builds configured with `--enable-wir-debugmacros`, these two activation names are “mechanically” turned off completely by default. Should a [WIR](#) user wish to have a closer look at worklists or invariants, it thus is not sufficient to just copy

Listing 15.2: Activation Names for the In-Depth Introspection of WIR Analyses and Optimizations

```

1 virtual void WIR_StructuralAnalysis::runAnalysis(WIR_Function&).visualize
2 virtual void WIR_StructuralAnalysis::runAnalysis(WIR_Function&).final.visualize
3
4 virtual void WIR_BitDFA::runAnalysis(WIR_Function&).topdown.visualize
5 virtual void WIR_BitDFA::runAnalysis(WIR_Function&).bottomup.visualize
6
7 void WIR_GraphColoring::build(WIR_Function&, WIR_InterferenceGraph&).visualize
8 virtual void WIR_GraphColoring::runOptimization(WIR_Function&).simplified.visualize
9 virtual void WIR_GraphColoring::runOptimization(WIR_Function&).simplified.visualizeFull
10 WIR_BaseRegister& WIR_GraphColoring::combine(const WIR_BaseRegister&, const WIR_BaseRegister&,
    WIR_InterferenceGraph&).visualize
11 virtual void WIR_GraphColoring::runOptimization(WIR_Function&).colored.visualizeFull
12 void WIR_GraphColoring::computeStackLocations(const WIR_Function&).visualize
13 WIR_GraphColoring.worklists
14 WIR_GraphColoring.invariants
15
16 WIR_JumpCorrection.invariants
17 TC_JumpCorrection.invariants
18
19 void WIR_SchedulingRegion::buildDG().visualize

```

Lines 13 and 14 into configuration file `wir.conf`. Instead, the definition of macro `ENABLE_INVARIANTS` at the top of file `WIR/optimizations/gcregallocation/wirgraphcoloring.cc` needs to be activated and the `WIR` library rebuilt once afterwards.

The very same applies to invariants that check the memory layout and start addresses of basic blocks during jump correction (see Section 14.13 on Page 197). This checking of invariants is in general supported by the activation names from Lines 16 and 17. But again and for the same reasons as discussed in the previous paragraph, the macro `ENABLE_INVARIANTS` must be defined in files `WIR/optimizations/jumpcorrection/wirjumpcorrection.cc` and `WIR/arch/tricore/optimizations/jumpcorrection/tcjumpcorrection.cc`, resp.

Last but not least, the activation name from Line 19 serves to visualize the dependence graph showing `RAW`, `WAR` and many other dependencies between `WIR` operations during instruction scheduling, cf. Section 14.17 on Page 202.

Part III

Retargeting Guide

Outline

Retargetability refers to the concept that a model of a system's architecture or of an [Instruction Set Architecture \(ISA\)](#) is not hard-wired inside a compiler, but instead allows for simple and lightweight extensions and changes. Purpose of this guide thus is to describe the various steps required to extend the [WIR](#) library by support for new systems or target [ISAs](#).

The individual steps in order to retarget [WIR](#) towards a completely new processor model are presented in Chapter [16](#) (Page [213](#) ff.). Novel processor models should of course also benefit from the manifold analyses and optimizations that are integral part of the [WIR](#) library. While most of them are entirely generic and processor-independent, some others need to be retargeted. Chapter [17](#) (Page [243](#) ff.) covers the retargeting aspects of the [WIR](#) code analyses, while Chapter [18](#) (Page [257](#) ff.) does the same for its code optimizations.

16 Retargeting of Processor Models

This chapter describes the steps needed to extend the [WIR](#) library by new processor and system models. These descriptions cover the two scenarios where either a completely new processor model is added from scratch, or where a new model is derived from already existing ones – see Fig. 9.1 for an illustration of both scenarios. In general, retargeting [WIR](#) for new processors consists of the following major steps:

- creation of a rudimentary C++ class that will hold the new processor model (cf. Section 16.1);
- addition of the new class to the initialization and registration mechanisms of [WIR](#) (cf. Section 16.2);
- extension of this rudimentary processor class by specific [ISA](#) features such as register types and classes, immediates, opcodes and operation formats, addressing modes etc. (cf. Section 16.3);
- implementation and registration of processor-specific stream output functions (cf. Section 16.4);
- design of new system configuration files involving the novel processor model (cf. Section 16.5);
- provision of both positive and negative unit tests for all newly created C++ classes (cf. Section 16.6).

16.1 Setup of a New Processor Model's Class

In case that a completely novel processor architecture is modeled, it is first necessary to create a new architecture-specific directory and to integrate this into the GNU Build System of [WIR](#). The individual steps for this task are:

1. Change into the arch sub-directory of the [WIR](#) software package (cf. Fig. 3.1) and create a new directory therein with a speaking name for the processor architecture to be modeled.

Note

Let *archDir* be the name of this directory where the newly created processor model will reside.

2. Extend file `WIR/arch/README` by a short description.
3. Add *archDir* to the `SUBDIRS` directive of file `WIR/arch/Makefile.am`.
4. Create a new file `WIR/arch/archDir/Makefile.am` with the following contents:

```
1 SUBDIRS          =
2 EXTRA_DIST      =
```

5. Add a new line

```
    arch/archDir/Makefile
```

to the `AC_CONFIG_FILES` directive at the very end of file `WIR/configure.ac` in the top-level directory of the whole [WIR](#) software package.

6. Add a new line

`include arch/archDir/Makefile.src`
to file `WIR/arch/Makefile.src`.

7. Create a new file `WIR/arch/archDir/Makefile.src` with the following contents:

```
1 libwir_la_SOURCES +=
2
3 archDirsysconfdir = ${sysconfdir}/archDir
4 archDirsysconf_DATA =
```

The above steps require a complete rebuild of the GNU Build System so that the bootstrapping, configuration and build procedures (cf. Sections 4.1 and 4.2 and Chapter 5, resp.) need to be carried out once again.

For the case that a new processor model shall be created by inheriting from an already existing one, none of the above steps are required.

Note

Alternatively, let *archDir* be the name of the directory underneath the *arch* sub-directory where the already existing processor model resides.

The next task consists of setting up a rudimentary header file that declares the C++ class of the newly created processor model.

Note

Let *newModel* be a speaking name for the actual *ISA* to be modeled.

8. Create a new file `WIR/arch/archDir/newModel.h` according to the general structure depicted in Listing 16.1. Line 9 shows the scenario for a completely new processor model that thus inherits from the generic base class `WIR_Processor` included in Line 5. Alternatively, the new class needs to inherit from some already existing model that needs to be included instead of `wirprocessor.h` in Line 5.

The new class initially only comes with its standard constructors (Lines 14 to 20), a virtual destructor (Line 23), assignment operators (Lines 26 and 29) and a static initialization method (Line 32). Furthermore, a private `clone` method is required for type-safe deep copies of inherited processor models (Line 37).

9. Create a new file `WIR/arch/archDir/newModel.cc` according to the rudimentary structure depicted in Listing 16.2. Again, this code example refers to the scenario that an entirely new architecture inherits from base class `WIR_Processor`. If the new model is derived from an already existing one, appropriate changes of the base class need to be applied to Lines 12, 19, 23, 33 and 40.

The implementation of the constructors, the destructor and the assignment operators is rather straightforward, the only noteworthy aspect is that the default constructor should assign meaningful names as processor and *ISA* names (cf. Lines 14 and 15). The currently empty `init` method will be responsible for the proper initialization of the new processor model. Finally, the `clone` method must be provided as shown in Line 51 such that it calls the copy constructor of the new class. The purpose is to have one single virtual method available in the entire class hierarchy derived from `WIR_Processor` that is used internally to obtain appropriate deep copies of actual processor models.

10. Add

`arch/archDir/newModel.cc arch/archDir/newModel.h`
to the `libwir_la_SOURCES` directive of file `WIR/arch/archDir/Makefile.src`.

Listing 16.1: Rudimentary Header File for a New Processor Model's Class

```

1  #ifndef _newModel_H
2  #define _newModel_H
3
4  // Include WIR headers
5  #include <wir/wirprocessor.h>
6
7  namespace WIR {
8
9  class newModel : public WIR_Processor<newModel>
10 {
11     public:
12
13         // Default constructor.
14         newModel( void );
15
16         // Copy constructor.
17         newModel( const newModel &o );
18
19         // Move constructor.
20         newModel( newModel &&o );
21
22         // Destructor.
23         virtual ~newModel( void );
24
25         // Copy-assignment operator.
26         newModel & operator = ( const newModel &o );
27
28         // Move-assignment operator.
29         newModel & operator = ( newModel &&o );
30
31         // Globally initialize a processor model.
32         static void init( void );
33
34     private:
35
36         // Create a deep copy of a processor.
37         virtual WIR_BaseProcessor *clone( void ) const override;
38 };
39
40 } // namespace WIR
41
42 #endif

```

16.2 Integration of a New Processor Model Into the WIR Library

In order to be able to use the new model set up in the previous section, it needs to be initialized and registered within the [WIR](#) library. This requires the following steps:

11. Extend the currently empty `init` method by the structure depicted in Listing 16.3. It is important to register the new processor model as shown in Line 11 because without this, [WIR](#) will fail to create actual systems using the new model from system configuration files.

The interested reader is strongly encouraged to directly put the comments from Lines 4 to 8 and Line 13 in the code, since they provide a required internal structure to the `init` method that will be filled with contents in the upcoming sections of this document.

Listing 16.2: Rudimentary Implementation of a New Processor Model's Class

```

1  ifndef HAVE_CONFIG_H
2  #include <config_wir.h>
3  #endif
4
5  // Include WIR headers
6  #include <wir/wir.h>
7  #include <arch/archDir/newModel.h>
8
9  namespace WIR {
10
11  // Default constructor.
12  newModel::newModel( void ) : WIR_Processor<newModel> {}
13  {
14      setProcessorName( "some name of the processor architecture" );
15      setISAName( "some name of the modeled processor ISA" );
16  };
17
18  // Copy constructor.
19  newModel::newModel( const newModel &o ) : WIR_Processor<newModel> { o }
20  {};
21
22  // Move constructor.
23  newModel::newModel( newModel &&o ) : WIR_Processor<newModel> { std::move( o ) }
24  {};
25
26  // Destructor.
27  newModel::~newModel( void )
28  {};
29
30  // Copy-assignment operator.
31  newModel & newModel::operator = ( const newModel &o )
32  {
33      WIR_Processor<newModel>::operator = ( o );
34      return( *this );
35  };
36
37  // Move-assignment operator.
38  newModel & newModel::operator = ( newModel &&o )
39  {
40      WIR_Processor<newModel>::operator = ( std::move( o ) );
41      return( *this );
42  };
43
44  // Globally initialize a processor model.
45  void newModel::init( void )
46  {};
47
48  // Create a deep copy of a processor.
49  WIR_BaseProcessor *newModel::clone( void ) const
50  {
51      return( new newModel( *this ) );
52  };
53
54  } // namespace WIR

```

Listing 16.3: General Structure of a New Processor Model's `init` Method

```

1 // Globally initialize a processor model.
2 void newModel::init( void )
3 {
4     // Register I/O functions.
5
6     // Register operation formats.
7
8     // Register opcode to operation format mappings.
9
10    // Register this current processor model. This MUST be the second-last section in this method.
11    registerProcessor( newModel() );
12
13    // Finally, initialize derived processor models.
14 };

```

Warning

It has to be ensured that the registration of the processor model shown in Line 11 of Listing 16.3 always comes after all other registrations of I/O functions (cf. Section 16.4), operation formats and opcode mappings (cf. Section 16.3), and before the initialization of any other processor models derived from the current class.

12. If the new processor model is not derived from an already existing one, function `WIR_Init` from file `wir/wirmisc.cc` has to be extended. `WIR_init` is the central point for initialization of the whole `WIR` library. One crucial aspect of `WIR_init` is to trigger the initialization of all processor models known to `WIR`, which is shown in Line 10 of Listing 16.4 and below. As a consequence, the initialization of any completely new processor model has to be added to `WIR_init` as depicted in Line 15.

In order to introduce the new processor model to `WIR_init`, its header file `newModel.h` has to be included at the top of file `wirmisc.cc` (cf. Line 3).

13. In the case that the new processor model inherits from some already existing class within the `arch` directory hierarchy, this already existing base class must come with a full-fledged `init` method itself. Thus, this `init`

Listing 16.4: Extension of function `WIR_Init`

```

1 // Include WIR headers
2 [...]
3 #include <arch/archDir/newModel.h>
4
5 // WIR_Init performs some global initialization tasks for the entire WIR library.
6 void WIR_Init( void )
7 {
8     [...]
9
10    // Initialize WIR processor models.
11    ARMv4::init();
12    MIPS::init();
13    RV32I::init();
14    TC13::init();
15    newModel::init();
16 };

```

Listing 16.5: Declaration of a New Processor Model's Register Types

```

1  class newModel : public baseClass
2  {
3      public:
4
5          [...]
6
7      class RegisterType : public baseClass::RegisterType
8      {
9          public:
10
11             static const RegisterType regType1;
12
13             static const RegisterType regType2;
14
15             [...]
16
17         protected:
18
19             // Inherit the constructors from the base class.
20             using baseClass::RegisterType::RegisterType;
21     };
22
23     [...]
24 };

```

method of the base class needs to be extended by `newModel::init()`; at its very end in that section that is responsible to initialize derived processor models (compare Line 13 of Listing 16.3). Of course, `newModel.h` needs to be included by the base class for this purpose.

16.3 Addition of Processor-Specific ISA Features

The main effort of retargeting [WIR](#) for new processors goes in the careful analysis of the new architecture's [ISA](#) specifications and their precise transfer into a [WIR](#) processor model as described in Section 9.1.1. Since the concepts described in Section 9.1.1 and all its subsections form the base of the generic [WIR](#) processor model, all retargeting steps described in the sequel follow the very same structure.

Note

Let `baseClass` be the name of the base class from which the new processor model inherits.

16.3.1 Register Types

The following steps describe how to extend the new architecture's model by types of different registers.

14. Edit file `WIR/arch/archDir/newModel.h` and extend the new class by a novel, public class `RegisterType` that inherits from `WIR_Processor::RegisterType` (cf. Listing 16.5). For all different classes of registers that the new architecture features and that can occur as parameters of machine operations, add a new member declaration to class `RegisterType` as shown in Lines 11 and 13.

Listing 16.6: Implementation of a New Processor Model's Register Types

```

1  ifdef HAVE_CONFIG_H
2  #include <config_wir.h>
3  #endif
4
5  // Include WIR headers
6  #include <wir/wir.h>
7  #include <arch/archDir/newModel.h>
8
9  namespace WIR {
10
11  const newModel::RegisterType newModel::RegisterType::regType1 { [...] };
12  const newModel::RegisterType newModel::RegisterType::regType2 { [...] };
13
14  [...]
15
16  }           // namespace WIR

```

15. Create a new file `WIR/arch/archDir/newModelregistertypes.cc` according to the structure depicted in Listing 16.6. For each register type declared in Listing 16.5, a dedicated instance needs to be provided as shown in Lines 11 and 12 of Listing 16.6. Ensure that at the positions indicated by `[...]` in Lines 11 and 12, the constructor of class `RegisterType` is called with the proper strings for prefixes and suffixes of virtual and physical registers, as well as with the register type's bit width (cf. Line 25 of Listing 9.4).

16. Add

```

    arch/archDir/newModelregistertypes.cc
to the libwir_la_SOURCES directive of file WIR/arch/archDir/Makefile.src.

```

16.3.2 Registers

Based on the register types introduced in the previous section, the next step is to provide classes modelling virtual registers of a certain type.

Note

Let *regType* be some short name of a register type declared in Section 16.3.1.

17. Create a new file `WIR/arch/archDir/newModelregTypevirtual.h` following the structure shown in Listing 16.7. In analogy to Section 16.1, such a new virtual register class simply comes with its standard constructors, a virtual destructor, assignment operators, and a virtual `clone` method for deep copies.

Have this header file included by file `WIR/arch/archDir/newModel.h`.

18. Create a new file `WIR/arch/archDir/newModelregTypevirtual.cc` implementing this new virtual register according to Listing 16.8.

Listing 16.7: Header File for a New Virtual Register's Class

```

1  #ifndef _newModel_regTypeVIRTUAL_H
2  #define _newModel_regTypeVIRTUAL_H
3
4  // Include WIR headers
5  #include <wir/wirvirtualregister.h>
6
7  namespace WIR {
8
9  class newModel_regTypeV final : public WIR_VirtualRegister
10 {
11     public:
12
13         // Default constructor.
14         newModel_regTypeV( void );
15
16         // Copy constructor.
17         newModel_regTypeV( const newModel_regTypeV &o );
18
19         // Move constructor.
20         newModel_regTypeV( newModel_regTypeV &&o );
21
22         // Destructor.
23         virtual ~newModel_regTypeV( void );
24
25         // Copy-assignment operator.
26         newModel_regTypeV & operator = ( const newModel_regTypeV &o );
27
28         // Move-assignment operator.
29         newModel_regTypeV & operator = ( newModel_regTypeV &&o );
30
31     protected:
32
33         // Create a deep copy of a virtual register.
34         virtual newModel_regTypeV *clone( void ) const;
35 };
36
37 } // namespace WIR
38
39 #endif

```

If the newly created register class models hierarchical registers (cf. Section 9.1.1.2), the default constructor from Line 12 needs to create and push back the required number of virtual child registers of the appropriate sub-register class:

```

1  // Default constructor.
2  newModel_regTypeV::newModel_regTypeV( void ) : WIR_VirtualRegister { newModel::RegisterType::regType }
3  {
4      // Create two virtual child registers of type childRegType.
5      pushBackChild( newModel_childRegTypeV() );
6      pushBackChild( newModel_childRegTypeV() );
7  };

```

19. Add

arch/archDir/newModelregTypevirtual.cc arch/archDir/newModelregTypevirtual.h
to the libwir_la_SOURCES directive of file WIR/arch/archDir/Makefile.src.

Listing 16.8: Implementation of a New Virtual Register's Class

```

1  ifdef HAVE_CONFIG_H
2  #include <config_wir.h>
3  #endif
4
5  // Include WIR headers
6  #include <wir/wir.h>
7  #include <arch/archDir/newModel.h>
8
9  namespace WIR {
10
11  // Default constructor.
12  newModel_regTypeV::newModel_regTypeV( void ) : WIR_VirtualRegister { newModel::RegisterType::regType }
13  {};
14
15  // Copy constructor.
16  newModel_regTypeV::newModel_regTypeV( const newModel_regTypeV &o ) : WIR_VirtualRegister { o }
17  {};
18
19  // Move constructor.
20  newModel_regTypeV::newModel_regTypeV( newModel_regTypeV &o ) : WIR_VirtualRegister { std::move( o ) }
21  {};
22
23  // Destructor.
24  newModel_regTypeV::~newModel_regTypeV( void )
25  {};
26
27  // Copy-assignment operator.
28  newModel_regTypeV & newModel_regTypeV::operator = ( const newModel_regTypeV &o )
29  {
30      WIR_VirtualRegister::operator = ( o );
31      return( *this );
32  };
33
34  // Move-assignment operator.
35  newModel_regTypeV & newModel_regTypeV::operator = ( newModel_regTypeV &o )
36  {
37      WIR_VirtualRegister::operator = ( std::move( o ) );
38      return( *this );
39  };
40
41  // Create a deep copy of a virtual register.
42  newModel_regTypeV *newModel_regTypeV::clone( void ) const
43  {
44      return( new newModel_regTypeV( *this ) );
45  };
46
47  } // namespace WIR

```

Following a similar approach, individual classes for physical registers of each newly declared register type need to be provided. More precisely, this involves the following steps for non-hierarchical registers that do not feature any child registers:

20. Create a new file `WIR/arch/archDir/newModel_regType_physical.h` following the structure shown in Listing 16.9. The public interface of physical register classes is restricted to the virtual destructor only (cf. Line 14). All constructors or operators that create new instances of physical registers are private, since physical registers are created only when new processor models are created. This is also the reason behind

Listing 16.9: Header File for a New Physical Register's Class

```

1  #ifndef _newModel_regTypePHYSICAL_H
2  #define _newModel_regTypePHYSICAL_H
3
4  // Include WIR headers
5  #include <wir/wirphysicalregister.h>
6
7  namespace WIR {
8
9  class newModel_regTypeP final : public WIR_PhysicalRegister
10 {
11     public:
12
13         // Destructor.
14         virtual ~newModel_regTypeP( void );
15
16     private:
17
18         friend class WIR_BaseProcessor;
19         friend class newModel;
20
21         // No standard construction allowed, only the below default constructor may be used.
22         newModel_regTypeP( void ) = delete;
23
24         // Default constructor.
25         explicit newModel_regTypeP( const std::string &n, bool isSP = false );
26
27         // Copy constructor.
28         newModel_regTypeP( const newModel_regTypeP &o );
29
30         // Copy-assignment operator.
31         newModel_regTypeP & operator = ( const newModel_regTypeP &o );
32
33         // Create a deep copy of a physical register.
34         virtual newModel_regTypeP *clone( void ) const;
35 };
36
37 } // namespace WIR
38
39 #endif

```

the two friend declarations from Lines 18 and 19.

The default constructor without any arguments is disabled (Line 22) so that new physical registers can only be instantiated by providing a string holding the register's name according to the new processor model's ISA and an optional Boolean flag indicating whether the register to be created is the architecture's stack pointer (Line 25). Please note that this constructor does not expect the full name of a physical register as argument. If, e.g., an architecture's registers are named from x0 to x31, then their common prefix "x" is already specified during the setup of the corresponding register type in Step 15. Thus, the physical register's constructor only gets the unique part of its name as argument, such as "12" for register "x12".

Also, a copy constructor, copy-assignment operator and a clone method shall be provided (Lines 28 to 34).

Have this header file included by file WIR/arch/archDir/newModel.h.

21. Create a new file WIR/arch/archDir/newModel_regTypephysical.cc implementing this new virtual register according to Listing 16.10.

Listing 16.10: Implementation of a New Physical Register's Class

```

1  ifndef HAVE_CONFIG_H
2  #include <config_wir.h>
3  #endif
4
5  // Include WIR headers
6  #include <wir/wir.h>
7  #include <arch/archDir/newModel.h>
8
9  namespace WIR {
10
11  // Destructor.
12  newModel_regTypeP::~newModel_regTypeP( void )
13  {};
14
15  // Default constructor.
16  newModel_regTypeP::newModel_regTypeP( const std::string &n, bool isSP ) : WIR_PhysicalRegister {
17      newModel::RegisterType::regType, n, isSP }
18  {};
19
20  // Copy constructor.
21  newModel_regTypeP::newModel_regTypeP( const newModel_regTypeP &o ) : WIR_PhysicalRegister { o }
22  {};
23
24  // Copy-assignment operator.
25  newModel_regTypeP & newModel_regTypeP::operator = ( const newModel_regTypeP &o )
26  {
27      WIR_PhysicalRegister::operator = ( o );
28      return( *this );
29  };
30
31  // Create a deep copy of a physical register.
32  newModel_regTypeP *newModel_regTypeP::clone( void ) const
33  {
34      return( new newModel_regTypeP( *this ) );
35  };
36  } // namespace WIR

```

If the newly created register class models hierarchical registers (cf. Section 9.1.1.2), another private method `addChilds` needs to be added to Listings 16.9 and 16.10 that takes the required number of physical child registers as arguments and sets up the register hierarchy accordingly. Below, an example for `addChilds` is given where a physical register hierarchy consisting of two child registers is established:

```

1  // Add the specified physical register as childs.
2  newModel_regTypeP::addChilds( const WIR_PhysicalRegister &c1, const WIR_PhysicalRegister &c2 )
3  {
4      pushBackChild( const_cast<WIR_PhysicalRegister &>( c1 ) );
5      pushBackChild( const_cast<WIR_PhysicalRegister &>( c2 ) );
6  };

```

22. Add

`arch/archDir/newModelregTypephysical.cc` `arch/archDir/newModelregTypephysical.h`
to the `libwir_la_SOURCES` directive of file `WIR/arch/archDir/Makefile.src`.

As soon as all classes for physical registers of all required types have been implemented this way, the processor

Listing 16.11: Construction of Physical Registers by a New Processor Model's Class

```

1 // Default constructor.
2 newModel::newModel( void ) : WIR_Processor<newModel> {}
3 {
4     setProcessorName( "some name of the processor architecture" );
5     setISAName( "some name of the modeled processor ISA" );
6
7     // Create physical register bottom-up, i.e., from the leafs upwards to complex hierarchical registers.
8     addPhReg<newModel_regType1>( "Unique name part of first register of first type" );
9     addPhReg<newModel_regType1>( "Unique name part of second register of first type" );
10    [...]
11    addPhReg<newModel_regType2>( "Unique name part of first register of second type" );
12    [...]
13 };

```

model from Section 16.2 needs to be extended by all the hardware registers of the new architecture. For this purpose, the following steps are required:

23. Extend the new processor model's constructor from Line 12 of Listing 16.2 by addPhReg directives (declared in file WIR/wir/wirbaseprocessor.h) for each actual hardware register of each modeled register type. Start by adding non-hierarchical physical registers to the constructor first, as shown in Listing 16.11.

When it comes to the addition of the stack pointer register, extend the addPhReg call by a true Boolean flag, e.g., for the RISC-V RV32I architecture from Section 9.1.4:

```

1 addPhReg<RV_RegP>( "2", true );

```

24. Next, add public getter methods for all physical registers created so far to the new processor model's class. For this purpose, extend header file WIR/arch/archDir/newModel.h as shown in Listing 16.12.

Here, *regName* stands for the unique and complete name of a processor's register according to its ISA specification so that these access methods are named exactly as defined by the ISA (i.e., the methods for the RISC-V model should be x0() to x31()).

The realization of these access methods is shown in Listing 16.13. The addPhReg calls from Listing 16.11 store all the newly created physical registers at the tail of some internal std::vector named mPhReg-References. By using the appropriate numerical indices 0, 1, ... in Listing 16.13, references to the correct physical registers can be retrieved, dynamically casted to the required result type, and returned (cf. Lines 4

Listing 16.12: Declaration of Access Methods for Physical Registers

```

1 class newModel : public WIR_Processor<newModel>
2 {
3     public:
4
5     [...]
6
7     const newModel_regType1P &regName1( void ) const;
8     const newModel_regType1P &regName2( void ) const;
9     [...]
10    const newModel_regType2P & [...]
11 };

```

Listing 16.13: Implementation of Access Methods for Physical Registers

```

1 // Access to first register of first type.
2 const newModel_regType1P &newModel::regName1( void ) const
3 {
4     return( dynamic_cast<newModel_regType1P &>( mPhRegReferences.at( 0 ).get() ) );
5 };
6
7 // Access to second register of first type.
8 const newModel_regType1P &newModel::regName2( void ) const
9 {
10    return( dynamic_cast<newModel_regType1P &>( mPhRegReferences.at( 1 ).get() ) );
11 };
12
13 [...]

```

and 10). It is thus a good idea to order the access methods in Listings 16.12 and 16.13 in exactly the same way as the addPhReg calls so that simply increasing constant indices can be used.

25. Next, hierarchical physical registers consisting of already modeled and implemented child registers need to be created. Their creation inside the new processor model's constructor consists of a combination of addPhReg from Listing 16.11 and addChilds calls from Step 21. The excerpt from the new processor model's constructor below shows the construction of the TriCore model's physical extended register E0 (cf. Table 9.5) from its child data registers D0 and D1:

```

1 // Create hierarchical physical registers.
2 addPhReg<TC_ERegP>( "0" );
3 dynamic_cast<TC_ERegP &>( mPhRegReferences.back().get() ).addChilds( D0(), D1() );

```

As can be seen, a new physical register of appropriate type is created first using addPhReg so that this new register shows up at the tail of vector mPhRegReferences. This tail is then retrieved and casted to the correct register class so that its addChilds method can be invoked, taking the references to two child registers provided by the appropriate getter methods from Step 24 as arguments.

26. Finally, the physical hierarchical registers established during the previous step need to be supported by their dedicated getter methods in analogy to Step 24.

16.3.3 immediates

This section describes how to add all the required classes of immediate parameters for some ISA to a new processor's model. For this purpose, a careful analysis of an architecture's ISA specification is required. For each individual machine operation of some actual architecture, all the possibly involved immediates, their bit widths, their signedness, and any other specific properties should be extracted from the specifications and noted elsewhere.

Note

Let *bitWidth* denote the bit width of some immediate value.

Hereafter, the WIR processor model can be extended by tailored immediate classes as follows:

Listing 16.14: Header File for a New Signed Immediate Parameter's Class

```

1  #ifndef _newModel_CONSTbitWidth_SIGNED_H
2  #define _newModel_CONSTbitWidth_SIGNED_H
3
4  // Include WIR headers
5  #include <wir/wirsignedimmediateparameter.h>
6
7  namespace WIR {
8
9  class newModel_ConstbitWidth_Signed final : public
    WIR_SignedImmediateParameter<newModel_ConstbitWidth_Signed>
10 {
11     public:
12
13         // No standard construction allowed, only the below default constructor may be used.
14         newModel_ConstbitWidth_Signed( void ) = delete;
15
16         // Default constructor.
17         explicit newModel_ConstbitWidth_Signed( signed long long v );
18
19         // Copy constructor.
20         newModel_ConstbitWidth_Signed( const newModel_ConstbitWidth_Signed &o );
21
22         // Move constructor.
23         newModel_ConstbitWidth_Signed( newModel_ConstbitWidth_Signed &&o );
24
25         // Destructor.
26         virtual ~newModel_ConstbitWidth_Signed( void );
27
28         // Copy-assignment operator.
29         newModel_ConstbitWidth_Signed & operator = ( const newModel_ConstbitWidth_Signed &o );
30
31         // Move-assignment operator.
32         newModel_ConstbitWidth_Signed & operator = ( newModel_ConstbitWidth_Signed &&o );
33 };
34
35 } // namespace WIR
36
37 #endif

```

27. For each identified signed immediate parameter, create a new file `WIR/arch/archDir/newModelconstbitWidthsigned.h` following the structure shown in Listing 16.14. The default constructor without arguments is disabled, signed immediates can only be created by explicitly passing a signed value to the constructor from Line 17.

Other than that, only the usual copy/move constructors and assignment operators as well as the virtual destructor need to be provided.

Have this header file included by file `WIR/arch/archDir/newModel.h`.

28. Create a new file `WIR/arch/archDir/newModelconstbitWidthsigned.cc` implementing this new virtual register according to Listing 16.15. The only noteworthy point here is that the immediate's actual bit width is passed as integer argument to the constructor of base class `WIR_SignedImmediateParameter` in Line 12. This way, the `WIR` library is able to ensure that any actual signed value `v` used to construct an immediate parameter lies in the range of values that can be represented with `bitWidth` bits, assuming a two's-complement representation.

Listing 16.15: Implementation of a New Signed Immediate Parameter's Class

```

1  #ifndef HAVE_CONFIG_H
2  #include <config_wir.h>
3  #endif
4
5  // Include WIR headers
6  #include <wir/wir.h>
7  #include <arch/archDir/newModel.h>
8
9  namespace WIR {
10
11  // Default constructor.
12  newModel_ConstbitWidth_Signed::newModel_ConstbitWidth_Signed( signed long long v ) :
13      WIR_SignedImmediateParameter<newModel_ConstbitWidth_Signed> { v, bitWidth }
14  {};
15
16  // Copy constructor.
17  newModel_ConstbitWidth_Signed::newModel_ConstbitWidth_Signed( const newModel_ConstbitWidth_Signed &o ) :
18      WIR_SignedImmediateParameter<newModel_ConstbitWidth_Signed> { o }
19  {};
20
21  // Move constructor.
22  newModel_ConstbitWidth_Signed::newModel_ConstbitWidth_Signed( newModel_ConstbitWidth_Signed &&o ) :
23      WIR_SignedImmediateParameter<newModel_ConstbitWidth_Signed> { std::move( o ) }
24  {};
25
26  // Destructor.
27  newModel_ConstbitWidth_Signed::~newModel_ConstbitWidth_Signed( void )
28  {};
29
30  // Copy-assignment operator.
31  newModel_ConstbitWidth_Signed & newModel_ConstbitWidth_Signed::operator = ( const
32      newModel_ConstbitWidth_Signed &o )
33  {
34      WIR_SignedImmediateParameter<newModel_ConstbitWidth_Signed>::operator = ( o );
35      return( *this );
36  };
37
38  // Move-assignment operator.
39  newModel_ConstbitWidth_Signed & newModel_ConstbitWidth_Signed::operator = ( newModel_ConstbitWidth_Signed
40      &&o )
41  {
42      WIR_SignedImmediateParameter<newModel_ConstbitWidth_Signed>::operator = ( std::move( o ) );
43      return( *this );
44  };
45
46  } // namespace WIR

```

29. Add

arch/archDir/newModelconstbitWidthsigned.cc arch/archDir/newModelconstbitWidthsigned.h
to the libwir_la_SOURCES directive of file WIR/arch/archDir/Makefile.src.

30. For each identified unsigned immediate parameter, perform Steps 27 to 29 analogously, but based on the code shown in Listings 16.16 and 16.17. The constructor realized in Line 12 of Listing 16.17 assumes a value range from 0 to $2^{bitWidth-1}$ for unsigned values v used to construct an unsigned immediate.

Listing 16.16: Header File for a New Unsigned Immediate Parameter's Class

```

1  #ifndef _newModel_CONSTbitWidth_UNSIGNED_H
2  #define _newModel_CONSTbitWidth_UNSIGNED_H
3
4  // Include WIR headers
5  #include <wir/wirunsignedimmediateparameter.h>
6
7  namespace WIR {
8
9  class newModel_ConstbitWidth_Unsigned final : public
    WIR_UnsignedImmediateParameter<newModel_ConstbitWidth_Unsigned>
10 {
11     public:
12
13         // No standard construction allowed, only the below default constructor may be used.
14         newModel_ConstbitWidth_Unsigned( void ) = delete;
15
16         // Default constructor.
17         explicit newModel_ConstbitWidth_Unsigned( unsigned long long v );
18
19         // Copy constructor.
20         newModel_ConstbitWidth_Unsigned( const newModel_ConstbitWidth_Unsigned &o );
21
22         // Move constructor.
23         newModel_ConstbitWidth_Unsigned( newModel_ConstbitWidth_Unsigned &&o );
24
25         // Destructor.
26         virtual ~newModel_ConstbitWidth_Unsigned( void );
27
28         // Copy-assignment operator.
29         newModel_ConstbitWidth_Unsigned & operator = ( const newModel_ConstbitWidth_Unsigned &o );
30
31         // Move-assignment operator.
32         newModel_ConstbitWidth_Unsigned & operator = ( newModel_ConstbitWidth_Unsigned &&o );
33 };
34
35 } // namespace WIR
36
37 #endif

```

31. If required by the architecture's [ISA](#), additional constraints on the value range accepted by the constructors can be added by extending the body of the constructor from Line 13 of Listing 16.17 with specific assertions. For example, [ISAs](#) sometimes require that certain immediates used for particular machine operations have to be non-zero, or that immediates need to be multiples of 4. Such behavior can be enforced as follows:

```

1  {
2      // Body of an immediate's default constructor.
3      ufAssert( v != 0 );
4      ufAssert( v % 4 == 0 );
5  }

```

As motivated in Chapter 12, [WIR](#) is designed to model assembly code with the purpose to generate actual code for further processing by subsequent assembler tools. However, [ISA](#) specifications that serve as the basis for the modelling of immediate parameters regularly refer to an architecture's binary executable interface, and not to some assembly-level interface. This can sometimes lead to the situation that an assembler for some architecture accepts different immediate parameters for some machine operation as originally specified by the [ISA](#).

Listing 16.17: Implementation of a New Unsigned Immediate Parameter's Class

```

1  #ifndef HAVE_CONFIG_H
2  #include <config_wir.h>
3  #endif
4
5  // Include WIR headers
6  #include <wir/wir.h>
7  #include <arch/archDir/newModel.h>
8
9  namespace WIR {
10
11  // Default constructor.
12  newModel_ConstbitWidth_Unsigned::newModel_ConstbitWidth_Unsigned( unsigned long long v ) :
13      WIR_UnsignedImmediateParameter<newModel_ConstbitWidth_Unsigned> { v, bitWidth }
14  {};
15
16  // Copy constructor.
17  newModel_ConstbitWidth_Unsigned::newModel_ConstbitWidth_Unsigned( const newModel_ConstbitWidth_Unsigned &o
18      ) : WIR_UnsignedImmediateParameter<newModel_ConstbitWidth_Unsigned> { o }
19  {};
20
21  // Move constructor.
22  newModel_ConstbitWidth_Unsigned::newModel_ConstbitWidth_Unsigned( newModel_ConstbitWidth_Unsigned &&o ) :
23      WIR_UnsignedImmediateParameter<newModel_ConstbitWidth_Unsigned> { std::move( o ) }
24  {};
25
26  // Destructor.
27  newModel_ConstbitWidth_Unsigned::~newModel_ConstbitWidth_Unsigned( void )
28  {};
29
30  // Copy-assignment operator.
31  newModel_ConstbitWidth_Unsigned & newModel_ConstbitWidth_Unsigned::operator = ( const
32      newModel_ConstbitWidth_Unsigned &o )
33  {
34      WIR_UnsignedImmediateParameter<newModel_ConstbitWidth_Unsigned>::operator = ( o );
35      return( *this );
36  };
37
38  // Move-assignment operator.
39  newModel_ConstbitWidth_Unsigned & newModel_ConstbitWidth_Unsigned::operator = (
40      newModel_ConstbitWidth_Unsigned &&o )
41  {
42      WIR_UnsignedImmediateParameter<newModel_ConstbitWidth_Unsigned>::operator = ( std::move( o ) );
43      return( *this );
44  };
45
46  } // namespace WIR

```

Example

According to the RISC-V RV32IC ISA [24], the operation C.ADDI16SP requires a non-zero 6-bits immediate, and this immediate is processor-internally scaled to represent multiples of 16. This internal scaling is done by left-shifting the 6-bits immediate by 4 bits, effectively widening the immediate from 6 bits to 10 bits. However, the RISC-V GNU assembler riscv32-unknown-elf-as does not accept arbitrary non-zero immediates of 6 bits width as parameter for C.ADDI16SP. Instead, the assembler expects the immediate to be already scaled up to 10 bits while being multiples of 16.

Listing 16.18: Declaration of a New Processor Model's Condition Fields

```

1  class newModel : public baseClass
2  {
3      public:
4
5          [...]
6
7      class Condition : public baseClass::Condition
8      {
9          public:
10
11             static const Condition cond1;
12
13             static const Condition cond2;
14
15             [...]
16
17         protected:
18
19             // Inherit the constructors from the base class.
20             using baseClass::Condition::Condition;
21     };
22
23     [...]
24 };

```

The above example shows that [ISA](#) documents and assemblers can differ in the handling of certain immediate parameters. This implies that care needs to be taken during the design of immediate classes for a newly modeled architecture within [WIR](#). Depending on the intended use of a new [WIR](#) processor model, immediates might be realized according to a machine's [ISA](#), or according to the capabilities of some particular assembler tool.

16.3.4 Addressing Modes

The modelling of an [ISA](#)'s addressing modes within [WIR](#) follows the very same pattern as already introduced for register types in Section [16.3.1](#):

32. Edit file `WIR/arch/archDir/newModel.h` and extend the new class by a novel, public class `AddressingMode` that inherits from `baseClass::AddressingMode`. For all addressing modes to be modeled within [WIR](#) for the new architecture, add a new member declaration to class `AddressingMode`.
33. Create a new file `WIR/arch/archDir/newModeladdressingmodes.cc` and provide a dedicated instance per addressing mode declared in Step [32](#).
34. Add


```
arch/archDir/newModeladdressingmodes.cc
```

 to the `libwir_la_SOURCES` directive of file `WIR/arch/archDir/Makefile.src`.

For the sake of brevity, we omit detailed listings here and simply refer to Section [9.1.1.6](#) and Listing [9.25](#).

Listing 16.19: Implementation of a New Processor Model's Condition Fields

```

1  ifdef HAVE_CONFIG_H
2  #include <config_wir.h>
3  #endif
4
5  // Include WIR headers
6  #include <wir/wir.h>
7  #include <arch/archDir/newModel.h>
8
9  namespace WIR {
10
11  const newModel::Condition newModel::Condition::cond1 { "conditionAcronym1" };
12  const newModel::Condition newModel::Condition::cond2 { "conditionAcronym2" };
13
14  [...]
15
16  }           // namespace WIR

```

16.3.5 Condition Fields

The modelling of an ISA's condition fields within WIR follows the very same pattern as already introduced for register types or addressing modes in Sections 16.3.1 and 16.3.4:

35. Edit file WIR/arch/archDir/newModel.h and extend the new class by a novel, public class Condition that inherits from `baseClass::Condition` (cf. Listing 16.18). For all condition fields to be modeled within WIR for the new architecture, add a new member declaration to class Condition.
36. Create a new file WIR/arch/archDir/newModelconditions.cc and provide a dedicated instance per condition declared in Step 35 (cf. Listing 16.19).
37. Add
`arch/archDir/newModelconditions.cc`
to the `libwir_la_SOURCES` directive of file WIR/arch/archDir/Makefile.src.

16.3.6 Opcodes

After register types, registers, immediates and other types of parameters for a new processor model have been realized according to the previous sections, the next steps center around the specification of an ISA's opcodes within the WIR library. Once again, this requires a careful analysis of the architecture's ISA specification in order to understand which operations are native machine operations directly realized by the hardware, or which other operations are just pseudo-operations that boil down to specific flavors of native machine operations.

Note

It is strongly recommended to specify all native machine operations as WIR opcodes, and to keep the amount of pseudo-opcodes within WIR at an absolute minimum. Whenever possible, make the use of a processor's native opcodes explicit and avoid the introduction and usage of pseudo-opcodes in general. The only exception of this rule of thumb are pseudo-opcodes that help the various WIR analyses (cf. Chapter 13) to better understand certain code properties. Prime examples here are control flow operations realizing branches or function calls/returns, or operations used for register-register moves. If an ISA does not feature dedicated native opcodes for these purposes, it is definitely recommended to introduce a few

Listing 16.20: Declaration of a New Processor Model's OpCodes

```

1  class newModel : public baseClass
2  {
3      public:
4
5          [...]
6
7      class OpCode : public baseClass::OpCode
8      {
9          public:
10
11             static const OpCode opCode1;
12
13             static const OpCode opCode2;
14
15             [...]
16
17         protected:
18
19             // Inherit the constructors from the base class.
20             using baseClass::OpCode::OpCode;
21     };
22
23     [...]
24 };

```

pseudo-opcodes.

The following steps describe how to extend the new architecture's model by types of different registers.

38. Edit file `WIR/arch/archDir/newModel.h` and extend the new class by a novel, public class `OpCode` that inherits from `baseClass::OpCode` (cf. Listing 16.20). For all identified opcodes that shall be modeled within [WIR](#) for the new architecture, add a new member declaration to class `OpCode` as shown in Lines 11 and 13.
39. Create a new file `WIR/arch/archDir/newModelopcodes.cc` according to the structure depicted in Listing 16.21. For each opcode declared in Listing 16.20, a dedicated instance needs to be provided as shown in Lines 11 and 12 of Listing 16.21. Ensure that the constructor of class `OpCode` is called with the proper mnemonic string, as well as – if applicable – with the proper opcode classification(s) according to Listing 9.19.
40. Add


```
arch/archDir/newModelopcodes.cc
```

 to the `libwir_la_SOURCES` directive of file `WIR/arch/archDir/Makefile.src`.

16.3.7 Operation Formats

The following steps describe how to specify the operation formats (cf. Section 9.1.1.5) required to model a new [ISA](#). For this purpose, a common naming scheme should be defined that will then be used to create the acronyms of all required operation formats such that it becomes evident which kinds of parameters, registers, immediates etc. an operation of a certain format expects.

Listing 16.21: Implementation of a New Processor Model's Opcodes

```

1  ifndef HAVE_CONFIG_H
2  #include <config_wir.h>
3  #endif
4
5  // Include WIR headers
6  #include <wir/wir.h>
7  #include <arch/archDir/newModel.h>
8
9  namespace WIR {
10
11  const newModel::OpCode newModel::OpCode::opCode1 { "mnemonic1" [...] };
12  const newModel::OpCode newModel::OpCode::opCode2 { "mnemonic2" [...] };
13
14  [...]
15
16  }           // namespace WIR

```

41. Edit file `WIR/arch/archDir/newModel.h` and extend the new class by a novel, public class `OperationFormat` that inherits from `baseClass::OperationFormat` (cf. Listing 16.22). For all identified formats that shall be modeled within `WIR` for the new architecture, add a new member declaration to class `OperationFormat` as shown in Lines 11 and 13.
42. Create a new file `WIR/arch/archDir/newModeloperationformats.cc` according to the structure depicted in Listing 16.23. For each format declared in Listing 16.22, a dedicated instance needs to be provided as shown in Lines 11 and 12 of Listing 16.23. Ensure that the constructor of class `OperationFormat` is called with the proper bit width of the format as well as with a string containing the format's acronym that is

Listing 16.22: Declaration of a New Processor Model's Operation Formats

```

1  class newModel : public baseClass
2  {
3      public:
4
5          [...]
6
7      class OperationFormat : public baseClass::OperationFormat
8      {
9          public:
10
11              static const OperationFormat formatAcronym1;
12
13              static const OperationFormat formatAcronym2;
14
15              [...]
16
17          protected:
18
19              // Inherit the constructors from the base class.
20              using baseClass::OperationFormat::OperationFormat;
21      };
22
23      [...]
24  };

```

Listing 16.23: Implementation of a New Processor Model's Operation Formats

```

1  ifndef HAVE_CONFIG_H
2  #include <config_wir.h>
3  #endif
4
5  // Include WIR headers
6  #include <wir/wir.h>
7  #include <arch/archDir/newModel.h>
8
9  namespace WIR {
10
11  const newModel::OperationFormat newModel::OperationFormat::formatAcronym1 { bitwidth1, "formatAcronym1" };
12  const newModel::OperationFormat newModel::OperationFormat::formatAcronym2 { bitwidth2, "formatAcronym2" };
13
14  [...]
15
16  }           // namespace WIR

```

used for [WIR](#) code dumps.

43. Add

`arch/archDir/newModeloperationformats.cc`
to the `libwir_la_SOURCES` directive of file `WIR/arch/archDir/Makefile.src`.

44. Next, the structure of all implemented operation formats needs to be registered, i.e., their number of parameters and the different kinds of parameters in their specific order according to the new architecture's [ISA](#). For this purpose, the new model's `init` method from Listing 16.3 needs to be extended by calls of the global [WIR](#) registry in analogy to Listing 9.23.

Edit the `init` method in file `WIR/arch/archDir/newModel.cc` and create virtual registers of the various types realized in Section 16.3.1 (cf. Line 7 of Listing 16.24).

Listing 16.24: Operation Format Registration in a New Processor Model's `init` Method

```

1  // Globally initialize a processor model.
2  void newModel::init( void )
3  {
4      [...]
5
6      // Register operation formats.
7      const auto *reg1 = new newModel_childRegTypeV;
8      const auto *reg2 = new newModel_childRegTypeP( "5" );
9      [...]
10
11     registerOperationFormat( OperationFormat::formatAcronym1, { new param1, new param2, [...] } );
12     registerOperationFormat( OperationFormat::formatAcronym2, { [...] } );
13
14     [...]
15
16     // Register opcode to operation format mappings.
17     registerOpCode( OpCode::opCode1, OperationFormat::formatAcronymX );
18     registerOpCode( OpCode::opCode2, OperationFormat::formatAcronymY );
19
20     [...]
21 };

```

Listing 16.25: Rudimentary Header File for a New Processor Model's Stream Output

```

1  #ifndef _newModel_IO_H
2  #define _newModel_IO_H
3
4  // Include standard headers
5  #include <iostream>
6
7  // Include WIR headers
8  #include <wir/wirtypes.h>
9
10 namespace WIR {
11
12 // Class forward declarations
13
14 // Processor-specific stream manipulator.
15 std::ostream &newModel( std::ostream &os );
16
17 // Processor-specific dumper functions
18
19 } // namespace WIR
20
21 #endif

```

For each operation format declared in Step 41, add a call to `registerOperationFormat` afterwards (Lines 11 and 12). Pay attention that the exact amount of register, immediate or other parameters is provided to `registerOperationFormat` in exactly that order in which the ISA specification expects them. For register parameters, ensure that their usage characteristics (cf. type `WIR_Usage` from Listing 8.23) are correctly registered. If an operation format of the ISA supports any register of a particular type as parameter, use a virtual register of that type here. Otherwise, if an operation format requires a very specific processor register only as parameter (e.g., the architecture's stack pointer), create an instance of that particular physical register first (Line 8) and then use this physical register for `registerOperationFormat`.

A concrete example of this operation format registration for the TriCore TC13 architecture was given in Listing 9.23 on Page 81.

45. Finally, specify which opcodes can have which operation formats by adding corresponding `registerOpCode` directives to the very same `init` method as shown in Lines 17 and 18.

16.4 Realization of Processor-Specific Code Generation Mechanisms

According to Section 12.3, `WIR` provides a flexible concept to generate valid processor-specific assembly code. When a new processor architecture is modeled within `WIR`, these code generation mechanisms thus need to be adapted accordingly, which is described in the following steps.

46. Table 12.2 on Page 145 presented all processor-specific stream manipulators that have to be used in order to switch the code generated by `WIR` to a desired processor architecture. Consequently, a stream manipulator needs to be added for the newly modeled processor architecture.

Create a new file `WIR/arch/archDir/newModelio.h` according to the general structure depicted in Listing 16.25. This new header file initially only comes with the two required `include` directives from Lines 5 and 8 and the declaration of the new stream manipulator from Line 15.

Listing 16.26: Rudimentary Implementation of a New Processor Model's Stream Output

```

1  #ifndef HAVE_CONFIG_H
2  #include <config_wir.h>
3  #endif
4
5  // Include WIR headers
6  #include <wir/wir.h>
7  #include <arch/archDir/newModel.h>
8
9  namespace WIR {
10
11  // Processor-specific stream manipulator.
12  std::ostream &newModel( std::ostream &os )
13  {
14      os.iword( WIR_ProcessorIO() ) = newModel::getProcessorTypeID();
15      return( os );
16  };
17
18  } // namespace WIR

```

47. Create a new file `WIR/arch/archDir/newModelio.cc` according to the general structure depicted in Listing 16.26. The implementation of the stream manipulator (cf. Line 12) simply sets an output stream's `WIR_ProcessorIO` property to the unique ID of the new processor model. This ID is internally created and managed by the `WIR` library so that output streams invoke the correct dumper functions for the processor model associated with the given ID.

48. Add

```
arch/archDir/newModelio.cc arch/archDir/newModelio.h
```

to the `libwir_la_SOURCES` directive of file `WIR/arch/archDir/Makefile.src`.

49. Next, processor-specific dumper functions for individual components of the `WIR` software model need to be realized. These dumper functions must have a type as declared in file `WIR/wir/wirio.h`, see Listing 12.6 as an example of a basic block dumper's type.

Extend file `WIR/arch/archDir/newModelio.h` by the declarations of all processor-specific dumpers that need to be tailored for the newly modeled processor architecture. Listing 12.7 shows the declaration of a RISC-V-specific basic block dumper. The naming scheme of all these dumpers should follow the pattern `dumpnewModelComponent` where *Component* denotes the `WIR` component to be dumped in a processor-specific manner (e.g., `BasicBlock` or `Operation`).

50. Implement each dumper function declared in Step 49 by extending file `WIR/arch/archDir/newModelio.cc`. Since this retargeting guide cannot cover all issues that are potentially raised during code generation for some particular architecture, we refrain from giving code examples of some dumpers' implementations here.

Regularly, the dumpers for `WIR` basic blocks, comments, data objects and operations need to be tailored in a processor-specific way. Correctly generated code for basic blocks and data objects usually requires to print out `.section` directives in front of the actual code whenever the previous basic block/data object was allocated to some different section. The correct formatting of comments attached to `WIR` objects such that they are accepted by a subsequent assembler is another typical task to be realized here. Most importantly, processor-specific operation dumpers are frequently required in order to output pseudo-operations (cf. Page 231) in the form of correct native machine operations.

Listing 16.27: Rudimentary System Configuration File

```

1 #
2 # System specification of the newModel implementation newSystem.
3 #
4 [newSystem]
5 type          = system
6
7 #
8 # Processor Core (ISA newModel)
9 #
10 [CORE0]
11 type          = core
12 isa           = ISA name

```

For concrete examples of implementations of various dumper functions, the reader is referred to files `WIR/arch/riscv/rvio.cc` or `WIR/arch/tricore/tcio.cc`.

51. Finally, all the dumpers declared in Step 49 need to be registered under the unique ID of the new processor model.

Edit the `init` method in file `WIR/arch/archDir/newModel.cc` and add calls to the appropriate `registerComponentDumper` methods as already depicted earlier in Listing 12.8. The complete specifications of these registration methods can be found in file `WIR/wir/wirbaseprocessor.h`.

16.5 Setup of New System Configurations

As discussed in Section 10.1, **WIR** uses system configuration files in order to instantiate hardware components with specific properties and compose them to form a complete system. When it comes to retargeting **WIR** to a new processor architecture, it is thus necessary to provide at least one new such configuration file that instantiates and uses the new processor model created so far.

Note

At this point, the name `newModel` of the actual **ISA** that has been modeled so far needs to be distinguished from the name of some actual system that bases on the modeled **ISA** but adds a variety of other hardware components.

Let `newSystem` thus be a speaking name for the actual system to be modeled.

The setup of a new system configuration file and its integration into the **WIR** infrastructure is described in the following steps.

52. Create a new file `WIR/arch/archDir/newSystem.sys` according to the general structure shown in Listing 16.27. This file first specifies that new systems will carry the name `newSystem` in Lines 4 and 5.

The more relevant part, however, is the specification of the new system's processor core given in Lines 10 to 12. In Line 12, it is important to specify exactly that name that has previously been given to the **ISA** using `setISAName`, cf. Line 15 of Listing 16.2 on Page 216. This way, whenever this system configuration file is used to construct a **WIR** system, a processor core with the newly implemented **ISA** model gets instantiated.

53. Add
`arch/archDir/newSystem.sys`
to the `archDirsysconf_DATA` directive of file `WIR/arch/archDir/Makefile.src`.
54. Add
`newSystem.sys`
to the `EXTRA_DIST` directive of file `WIR/arch/archDir/Makefile.am`.
55. Add more paragraphs and `<key> = <value>` directives to file `WIR/arch/archDir/newSystem.sys` in order to specify more components and properties of the actual system to be modeled, see also Tables 10.1 and 10.2 for more details.

16.6 Implementation of Unit Tests

While the procedures described in this chapter up to this point are sufficient to realize a novel and fully operational system and ISA model within WIR, it is best practice to furthermore add unit tests that validate all critical properties of all new classes and models (cf. Listing 5.2 on Page 26). For this purpose, a dedicated folder needs to be set up and integrated into the build infrastructure first:

56. Change into the `arch/archDir` sub-directory of the WIR software package and create a new directory therein with the name `tests`.
57. Add
`tests`
to the `SUBDIRS` directive of file `WIR/arch/archDir/Makefile.am`.
58. Create a new file `WIR/arch/archDir/tests/Makefile.am` with the contents shown in Listing 16.28.
59. Add a new line
`arch/archDir/tests/Makefile`
to the `AC_CONFIG_FILES` directive at the very end of file `WIR/configure.ac` in the top-level directory of the whole WIR software package.
60. For each virtual register class implemented along the lines of Listings 16.7 and 16.8, create a new unit test `WIR/arch/archDir/tests/newModelregType.cc`. Purpose of this kind of tests is to verify the virtual registers' base properties (e.g., register type, bit width, the fact that it is virtual and not physical, register name), as well as to ensure that all constructors (default, copy, move) and assignment operators behave as expected.

For this purpose, the new unit test needs to create a new virtual register of a given type first. Using a series of assertions, the abovementioned properties are verified for this virtual register.

Next, various other virtual registers of the same type are created using the copy/move constructors and assignment operators. For all these other registers, their properties are again checked using assertions. Obviously, all these assertions must hold so that the entire unit test finally passes successfully.

A representative code example for this kind of unit tests can be found in file `WIR/arch/riscv/tests/rvReg1.cc`.

Listing 16.28: Rudimentary Makefile for Unit Tests

```

1  # Unit tests to be executed
2  check_PROGRAMS      =
3
4  failscripts         =
5  passscripts         =
6
7  check_SCRIPTS       = $(failscripts) $(passscripts)
8
9  XFAIL_TESTS         = $(failscripts)
10 TESTS               = $(passscripts) $(failscripts)
11
12 # Source files and build flags
13 CLEANFILES          = $(check_PROGRAMS) $(check_SCRIPTS) \
14                      a.out core
15 AM_DEFAULT_SOURCE_EXT = .cc
16 LDADD               = $(top_builddir)/libwir.la \
17                      @LIBUSEFUL_PREFIX@/libuseful/libuseful.la
18
19 # Specific rules
20 $(check_SCRIPTS): \
21     %.sh: %.bin
22     @echo "#!$(BASH) > $@";
23     @echo "$(builddir)"/$< > /dev/null 2>&1" >> $@;
24     @echo "exit \"$$\"?" >> $@;
25     @$ (CHMOD) ugo+x $@

```

61. For each unit test created in the previous step that is expected to pass, add a new line

newModelregType.bin

to the `check_PROGRAMS` directive of file `arch/archDir/tests/Makefile.am`, and a new line

newModelregType.sh

to the `passscripts` directive of the very same Makefile.

62. For the new processor model (cf. Listings 16.1 and 16.2), create a unit test `WIR/arch/archDir/tests/newModelPhRegs.cc` that aims to assert the existence and properties of all physical registers.

This test first creates an object of the new processor model's class. Next, it uses all the access methods from Listing 16.12 to check the correctness of all physical registers' names, whether they definitely are physical registers only, that they all have the correct register types and bit widths etc. All assertions placed in the code must hold in order to successfully pass.

Sample code for this kind of unit test is given in file `WIR/arch/riscv/tests/rvPhRegs1.cc`. Add these tests expected to pass to the [WIR](#) build infrastructure as described in Step 61.

63. For each new immediate parameter class (cf. Listings 16.14 to 16.17), create a new unit test `WIR/arch/archDir/tests/newModelConstbitWidthSignedness1.cc`. Motivation of this kind of unit tests is to ensure the correct modelling and implementation of immediates.

Such a test first creates some immediates of a given bit width and signedness using constants that lie in the immediate's value range. Assertions are again applied next to validate these immediates' types, values, bit widths, signedness etc. In analogy to Step 60, these immediate parameters are next duplicated using their copy/move constructors and operators, and the duplicates' properties are again checked using assertions. Finally, the smallest and largest possible values of the valid value range are used to construct some more

immediate parameters. This ensures that the entire value range is correctly covered and supported by some implemented immediate class.

A typical example for this kind of tests is `WIR/arch/riscv/tests/rvConst12Signed1.cc`. Obviously, these tests are expected to pass so that they need to be integrated into the build infrastructure according to Step 61.

64. In addition to the previous step, at least two more tests `WIR/arch/archDir/tests/newModelConst-bitWidthSignedness2.cc` and `newModelConstbitWidthSignedness3.cc` should be provided per immediate class that are all expected to fail. The rationale here is to verify that the classes implemented in the course of Section 16.3.3 actually reject constants that lie outside an immediate's value range.

Such a test attempts to create an immediate with, e.g., the smallest valid value minus one or the largest possible value plus one, which obviously must fail. If additional constraints on the value range are realized by an immediate's constructor (cf. Step 31), additional failing unit tests should be added for all these constraints.

Code examples can be found in files `WIR/arch/riscv/tests/rvConst10Signed*`.

65. For each unit test created in the previous step that is expected to fail, add a new line `newModelregType.bin` to the `check_PROGRAMS` directive of file `arch/archDir/tests/Makefile.am`, and a new line `newModelregType.sh` to the `failscripts` directive of the very same `Makefile`.
66. The next unit test aims to validate the correctness of all opcodes and operation formats realized for a newly modeled processor architecture. For this purpose, create a new test `WIR/arch/archDir/tests/new-ModelOperations1.cc`.

Tests of this kind regularly first declare some virtual registers of all available register classes, some basic blocks and functions etc. These objects will serve as parameters of all the machine operations created and validated in the following.

The main body of this test case next creates `WIR` operations using all modeled combinations of opcodes and suitable operation formats. For each such operation, the correct amount of register, immediate, or other parameters has to be provided in the correct order and with the correct def/use properties. After the creation of each such operation, its fundamental properties such as its byte size and its classifications according to Listing 8.18 are checked using assertions.

A typical code example can be found in file `WIR/arch/riscv/tests/rv32iOperations1.cc`. Integrate this test that is expected to pass into the build infrastructure in analogy to Step 61.

67. As discussed in Section 16.4, a `WIR` processor model should furthermore produce valid assembly code. Thus, the output generated by the processor-specific dumper functions also needs to be validated using an external assembler for the modeled processor architecture.

The `WIR` build infrastructure first of all needs to find the desired external assembler program. Open file `WIR/configure.ac` and locate that part (quite in the middle of the whole file) where the assemblers for the ARM, RISC-V and TriCore architectures (`arm-elf-as`, `riscv32-unknown-elf-as` and `tricore-as`) are already determined. Extend this part by the following two lines:

```
AC_PATH_PROG(newmodelas, assemblerExecutable, assemblerExecutable)
AC_SUBST(NEWMODELAS, [{newmodelas}])
```

In the first line, replace both occurrences of `assemblerExecutable` by the name of the assembler's executable program as it is installed on your machine, but without any paths to that executable. `NEWMODELAS`

Listing 16.29: Makefile for Unit Tests Extended by Assembler Tests

```

1  [...]
2
3  passscripts           = [...]
4  aspassscripts         =
5
6  check_SCRIPTS         = $(failscrip) $(passscripts)
7  check_ASSCRIPTS       = $(aspassscripts)
8
9  XFAIL_TESTS           = [...]
10 TESTS                 = $(passscripts) $(aspassscripts) $(failscrip)
11
12 # Source files and build flags
13 CLEANFILES             = $(check_PROGRAMS) $(check_SCRIPTS) $(check_ASSCRIPTS) \
14                        [...]
15
16 # Specific rules
17
18 [...]
19
20 $(check_ASSCRIPTS): \
21     %.sh: %.bin
22     @echo "#!$(BASH) > $@";
23     @echo 'TMPFILE=/tmp/`basename $$0`.$$$$.log' >> $@;
24     @echo $(builddir)"/$< > '$$TMPFILE' >> $@;
25     @echo 'ret=$$?' >> $@;
26     @echo 'if [ $$ret != 0 ]; then' >> $@;
27     @echo '  rm $$TMPFILE' >> $@;
28     @echo '  exit $$ret' >> $@;
29     @echo 'fi' >> $@;
30     @echo $(NEWMODELAS) ' $$TMPFILE' >> $@;
31     @echo 'ret=$$?' >> $@;
32     @echo 'rm $$TMPFILE' >> $@;
33     @echo 'exit $$ret' >> $@;
34     @$(CHMOD) ugo+x $@

```

and *newmodelas* use the acronym of the newly modeled processor architecture that was introduced already in the very beginning of this chapter, but once in upper-case and once in lower-case letters only.

68. Next, the Makefile *arch/archDir/tests/Makefile.am* needs to be extended towards assembly tests as shown in Listing 16.29. A new, empty *aspassscripts* directive needs to be added after the last entry of the already existing *passscripts* directive, cf. Line 4. After that, a *check_ASSCRIPTS* directive has to be added as shown in Line 7. The *TESTS* directive from Line 10 needs to be extended by *aspassscripts*, and *check_ASSCRIPTS* has to be added to *CLEANFILES* as shown in Line 13.

The most important part, however, is shown in Lines 20 to 34 that needs to be put at the very end of the Makefile. These lines create a wrapper script that executes the actual *WIR* test program and forwards its output into a temporary file. This temporary file is afterwards fed into the new architecture's assembler (cf. Line 30) as determined by the *NEWMODELAS* macro from the previous step. Should the assembler require any specific command-line flags, simply add them to Line 30 immediately before *\$\$TMPFILE*. Note that Line 23 uses regular apostrophe characters *'* twice, but two reversed apostrophes *`* (accent grave) in order to surround the *basename \$\$0* command.

69. Now, create a new test *WIR/arch/archDir/tests/newModel0operations2.cc*. Tests of this kind regularly first declare some instance of the new architecture's processor model. Using this, physical registers of

all register classes are usually declared next, followed by some basic blocks and functions etc. In analogy to Step 66, all these objects will serve as parameters of machine operations. But since these operations will be processed by a subsequent assembler, no virtual registers are used here at all.

After these declarations, the standard output stream `std::cout` is switched to assembly output for the new architecture using the stream manipulator from Line 15 of Listing 16.25. Next, the test case should declare a simple C++ lambda that receives a **WIR** operation, attaches a comment displaying its operation format to it and prints it to `std::cout`.

After this, the main body of this test case creates all possible operations of the new architecture in all valid formats and passes them to the above lambda. Like this, all modeled operations get printed to `std::cout` one after the other, and this output of the test case is forwarded to the architecture's assembler as discussed in the previous step.

Again, a convenient code example for this kind of unit tests can be found in file `WIR/arch/riscv/tests/rv32i0perations2.cc`.

70. Finally, add a new line

```
newModelOperations2.bin
```

to the `check_PROGRAMS` directive of file `arch/archDir/tests/Makefile.am`, and a new line

```
newModelOperations2.sh
```

to the `aspassscripts` directive of the very same `Makefile`.

17 Retargeting of Existing Code Analyses

As already mentioned earlier, the bit-true [Data Flow Analysis \(DFA\)](#) from [Section 13.11](#) is the only analysis that requires processor-specific adaptations. All other [WIR](#) analyses are generic and processor-independent. This chapter thus focuses on the steps that are required to retarget this one bit-true analysis to a novel processor architecture.

17.1 Bit-True Data Flow Analysis

The bit-true [DFA](#) performs a forward followed by a backward pass through the [DFG](#). These passes over a single [DFG](#) node, i.e., for a processor-specific [WIR](#) operation, are performed by the purely virtual methods `simulateTopDown` and `simulateBottomUp` of class `WIR_BitDFA`, cf. [Listing 17.1](#).

Listing 17.1: Interface of Class `WIR_BitDFA` to be Retargeted

```

1 class WIR_BitDFA : public WIR_Analysis,
2                   public WIR_DFG
3 {
4     public:
5
6         [...]
7
8     protected:
9
10        [...]
11
12        // Performs top-down forward simulation of a given operation.
13        virtual void simulateTopDown( const WIR_Operation &o,
14                                     std::map<WIR_id_t, WIR_UpDownValue> &operands,
15                                     std::map<WIR_id_t, WIR_UpDownValue> &results ) = 0;
16
17        // Performs bottom-up backward simulation of a given operation.
18        virtual void simulateBottomUp( const WIR_Operation &o,
19                                      std::map<WIR_id_t, WIR_UpDownValue> &in,
20                                      std::map<WIR_id_t, WIR_UpDownValue> &out,
21                                      std::map<WIR_id_t, WIR_UpDownValue> &results ) = 0;
22
23        [...]
24 };

```

Besides the operation `o` to be analyzed, both methods receive references to maps as arguments. These maps map the unique IDs of the operation's parameters to their associated bit-true up/down values. For example, map `operands` of method `simulateTopDown` (cf. [Line 13](#)) collects all parameters of `o` that somehow serve as input. This includes, e.g., used register parameters, immediate parameters or labels. For each of these input parameters, the associated down values as determined by the bit-true analysis can be retrieved from the map. Using all these input down values, method `simulateTopDown` then has to compute the bit-true outputs of operation `o`. These

Listing 17.2: Header File for a New Processor Model's Bit-True Data Flow Analysis

```

1  #ifndef _newModel_BITDFA_H
2  #define _newModel_BITDFA_H
3
4  // Include standard headers
5  #include <map>
6
7  // Include WIR headers
8  #include <wir/wirtypes.h>
9  #include <analyses/bit/wirbitdfa.h>
10
11 namespace WIR {
12
13 class WIR_Function;
14
15 class newModel_BitDFA : public WIR_BitDFA
16 {
17 public:
18
19     // Default constructor.
20     explicit newModel_BitDFA( WIR_Function & );
21
22     // Destructor.
23     virtual ~newModel_BitDFA( void );
24
25 protected:
26
27     // Performs top-down forward simulation of a given operation.
28     virtual void simulateTopDown( const WIR_Operation &, std::map<WIR_id_t, WIR_UpDownValue> &,
29                                   std::map<WIR_id_t, WIR_UpDownValue> & ) override;
30
31     // Performs bottom-up backward simulation of a given operation.
32     virtual void simulateBottomUp( const WIR_Operation &, std::map<WIR_id_t, WIR_UpDownValue> &,
33                                    std::map<WIR_id_t, WIR_UpDownValue> &,
34                                    std::map<WIR_id_t, WIR_UpDownValue> & ) override;
35 };
36
37 } // namespace WIR
38
39 #endif

```

outputs are again represented using down values, which then get associated with all output parameters of *o*, i.e., all defined register parameters. Consequently, map results of `simulateTopDown` maps the IDs of all parameters that get defined by *o* to their newly determined down values. The task of `simulateTopDown` is thus to populate map results with down values for all output parameters, depending on the down values along all input parameters and under consideration of the bit-true behavior of *o*'s opcode and its operation format.

The `simulateTopDown` forward analysis can never generate don't care bit values *bX* due to the definition of the basic operators `&`, `|` and `^` for class `WIR_L4` (cf. file `WIR/analyses/bit/wirL4.cc`). The truth tables used by these operators only generate *bX* if at least one of its input bits is already *bX*. The initialization of the bit-true DFG before the forward analysis only uses *bU*, *bL*, *bN*, *b0* or *b1*, but never *bX*. Thus, method `simulateTopDown` never gets *bX* bits as input and thus can never generate any *bX* bits as output. The goal of method `simulateBottomUp` is thus to determine don't care bits *bX*. If an operation *o* does not use all bits that it gets via some input register, these unused and irrelevant input bits can be set to *bX*. If, furthermore, the predecessor operation *p* that defines such a register now sees that it produces a value as output that is partially don't care for the rest of the following

Listing 17.3: Rudimentary Implementation of a New Processor Model's Bit-True Data Flow Analysis

```

1  #ifndef HAVE_CONFIG_H
2  #include <config_wir.h>
3  #endif
4
5  // Include standard headers
6  #include <iterator>
7  #include <utility>
8
9  // Include WIR headers
10 #include <wir/wir.h>
11 #include <arch/archDir/newModel.h>
12
13 // Include local headers
14 #include "newModelbitdfa.h"
15
16 namespace WIR {
17
18 // Default constructor.
19 newModel_BitDFA::newModel_BitDFA( WIR_Function &f ) : WIR_BitDFA { f }
20 {};
21
22 // Destructor.
23 newModel_BitDFA::~newModel_BitDFA( void )
24 {};
25
26 // Performs top-down forward simulation of a given operation.
27 void newModel_BitDFA::simulateTopDown( const WIR_Operation &o,
28                                         std::map<WIR_id_t, WIR_UpDownValue> &operands,
29                                         std::map<WIR_id_t, WIR_UpDownValue> &results )
30 {
31 };
32
33 // Performs bottom-up backward simulation of a given operation.
34 void newModel_BitDFA::simulateBottomUp( const WIR_Operation &o,
35                                          std::map<WIR_id_t, WIR_UpDownValue> &in,
36                                          std::map<WIR_id_t, WIR_UpDownValue> &out,
37                                          std::map<WIR_id_t, WIR_UpDownValue> &results )
38 {
39 };
40
41 } // namespace WIR

```

code, then these bX bits at p's output could propagate towards p's input operands.

Method `simulateBottomUp` (cf. Line 18) thus gets the operation `o` as parameter, as well as two maps that associate the IDs of `o`'s input and output parameters to their up values, resp. In analogy to `simulateTopDown` above, another map `results` is used to explicitly collect all newly generated up values.

This discussion shows that retargeting the bit-true analysis requires to provide processor-specific implementations of these two purely virtual methods `simulateTopDown` and `simulateBottomUp`. The following steps describe how to initially set up such a processor-specific analysis and how to integrate it into the [WIR](#) build infrastructure:

1. Create new directories `WIR/arch/archDir/analyses` and `WIR/arch/archDir/analyses/bit`.
2. Add a new line
`include arch/archDir/analyses/Makefile.src`

Listing 17.4: Definition of Lambdas for Top-Down Analysis

```

1 void newModel_BitDFA::simulateTopDown( [...] )
2 {
3     // firstOp retrieves the down value of operation o's first explicit parameter from map operands.
4     auto firstOp = [&]( void ) -> WIR_UpDownValue & {
5         auto &p = o.getExplicitParameters().front().get();
6
7         // Get down value from map operands, update it to the correct bit width needed by parameter p.
8         auto &v = operands.at( p.getID() );
9         updateOperand( p, v );
10
11         return( v );
12     };
13
14     // lastOp retrieves the down value of operation o's last explicit parameter from map operands.
15     auto lastOp = [&]( void ) -> WIR_UpDownValue & {
16         auto &p = o.getExplicitParameters().back().get();
17
18         // Get down value from map operands, update it to the correct bit width needed by parameter p.
19         auto &v = operands.at( p.getID() );
20         updateOperand( p, v );
21
22         return( v );
23     };
24
25     // nthOp retrieves the down value of operation o's n-th explicit parameter from map operands.
26     auto nthOp = [&]( unsigned int n ) -> WIR_UpDownValue & {
27         auto it = o.getExplicitParameters().begin();
28         std::advance( it, n );
29
30         // Get down value from map operands, update it to the correct bit width needed by n-th parameter.
31         auto &v = operands.at( it->get().getID() );
32         updateOperand( it->get(), v );
33
34         return( v );
35     };
36
37     [...]
38 };

```

to the beginning of file WIR/arch/archDir/Makefile.src.

3. Create a new file WIR/arch/archDir/analyses/Makefile.src with the following contents:

```
1 include arch/archDir/analyses/bit/Makefile.src
```

4. Create a new file WIR/arch/archDir/analyses/bit/Makefile.src with the following contents:

```
1 libwir_la_SOURCES      += arch/archDir/analyses/bit/newModelbitdfa.cc
   arch/archDir/analyses/bit/newModelbitdfa.h
```

5. Create a new file WIR/arch/archDir/analyses/bit/newModelbitdfa.h according to the general structure depicted in Listing 17.2.
6. Create a new file WIR/arch/archDir/analyses/bit/newModelbitdfa.cc according to the general structure depicted in Listing 17.3.

Listing 17.5: Definition of Lambdas for Top-Down Analysis (ctd.)

```

1 void newModel_BitDFA::simulateTopDown( [...] )
2 {
3     [...]
4
5     // lastReg retrieves the register associated with operation o's last explicit parameter.
6     auto lastReg = [&]( void ) -> WIR_BaseRegister & {
7         const auto &rp = dynamic_cast<const WIR_RegisterParameter &>( o.getExplicitParameters().back().get() );
8         return( rp.getRegister() );
9     };
10
11    // nthReg retrieves the register associated with operation o's n-th explicit parameter.
12    auto nthReg = [&]( unsigned int n ) -> WIR_BaseRegister & {
13        auto it = o.getExplicitParameters().begin();
14        std::advance( it, n );
15        return( dynamic_cast<const WIR_RegisterParameter &>( it->get() ).getRegister() );
16    };
17
18    // nthRes retrieves the down value of operation o's n-th explicit parameter from map results.
19    auto nthRes = [&]( unsigned int n ) -> WIR_UpDownValue & {
20        auto it = o.getExplicitParameters().begin();
21        std::advance( it, n );
22
23        // Check presence of n-th parameter in map results. Create U* if not yet present.
24        auto resIt = results.find( it->get().getID() );
25        if ( resIt == results.end() )
26            resIt =
27                results.insert(
28                    { it->get().getID(),
29                      { WIR_L4::bU,
30                        dynamic_cast<WIR_RegisterParameter &>(
31                          it->get() ).getRegister().getBitWidth() } } ).first;
32
33        return( resIt->second );
34    };
35 };

```

As motivated earlier, method `simulateTopDown` heavily uses maps `operands` and `results` to manage the down values during forward analysis. Efficient access to these down values and their associated registers is thus crucial. For this purpose, `simulateTopDown` usually defines a few lambdas that encapsulate the access to these maps.

7. Extend method `simulateTopDown` of file `WIR/arch/archDir/analyses/bit/newModelbitdfa.cc` by the lambdas as shown in Listings 17.4 and 17.5.

`firstOp` (cf. Line 4 of Listing 17.4) provides a reference to that down value of map `operands` that is associated with the first explicit parameter of operation `o`. For this purpose, it retrieves this first parameter (Line 5) and looks its unique ID up in map `operands` (Line 8). In the presence of hierarchical registers, it can happen that the obtained down value has a larger width than that of the currently considered parameter's register. The use of `updateOperand` in Line 9 helps here and ensures that the down value has an appropriate width afterwards. This updated down value is then finally returned.

In a similar manner, `lastOp` (Line 15) and `nthOp` (Line 26) can be used to get references to the down values of the operation's very last or n -th explicit parameter.

During top-down analysis, it will often be necessary to also check the actual registers serving as parameters of operation `o`. For this purpose, `lastReg` (cf. Line 6 of Listing 17.5) and `nthReg` (Line 12) provide

references to the registers used as very last or n -th explicit parameter of an operation.

Finally, `nthRes` (Line 19) returns a reference to some new down value from `map results` that is produced by operation `o` via its n -th explicit parameter. If `map results` does not yet contain an entry for this n -th parameter, the lambda creates and returns one that only contains unknown bU bits.

Warning

At no place, the lambdas shown in Listings 17.4 and 17.5 perform any kind of safety checks for efficiency reasons. They assume that an operation actually has a first, last or n -th explicit parameter, they rely on the presence of some entry in `map operands` in Listing 17.4, and they also assume in Listing 17.5 that the operation's parameters actually are register parameters.

The code of the top-down analysis still to be developed thus has to make sure that these lambdas are used only in appropriate contexts for correct opcodes and operation formats featuring the desired numbers and positions of (register) parameters.

8. Using the lambdas defined in the previous step, the main realization of method `simulateTopDown` can follow next. Basically, the body of this method consists of a sequence of if-conditions that, one after the other, check the opcode and possibly the operation format of the current operation `o`. And depending on `o`'s opcode (and format), the bit-true effects of the operation are computed.

Obviously, this document cannot cover all possible situations that might need to be considered during the implementation of method `simulateTopDown`. Instead, the following paragraphs simply focus on the most relevant classes of machine operations and their bit-true forward analysis. Code snippets from the RISC-V implementation (see file `WIR/arch/riscv/analyses/bit/rv32bitdfa.cc`) will serve as examples.

The most simple operations to be modeled in a bit-true way regularly are *move operations* that copy one register's contents to another register. Assuming the RISC-V RV32IC operation C.MV as example, its top-down analysis looks as follows:

```
1 if ( o.getOpCode() == RV32IC::OpCode::CMV )
2   nthRes( 0 ) = lastOp();
```

Lambda `lastOp` is used to obtain the down value of the register to be copied, and this down value is simply assigned to the target register using `nthRes`.

Another very straightforward class of operations to be modeled in a bit-true way are the *bitwise logical operations* such as AND, OR, XOR or NOT. Their simplicity arises from the fact that these bitwise operations have a one-to-one conformance with the very same operators defined for the partial order `WIR_L4` and also for bit vectors modeled by class `WIR_UpDownValue`. The top-down analysis of the RISC-V bitwise AND operation thus boils down to:

```
1 if ( o.getOpCode() == RV32I::OpCode::AND ) {
2   nthRes( 0 ) = nthOp( 1 ) & lastOp();
3   nthRes( 0 ).setSignedness( false );
4 }
```

The down value of the second register parameter of the AND is obtained using `nthOp` (note that parameter counting starts with number 0), and `lastOp` provides the down value of the third register parameter. Both down values are combined using `WIR_UpDownValue::operator &`, and its result is stored using `nthRes` so that it gets associated with the first register parameter of the AND. Bitwise machine operations usually treat all operands and results as unsigned values so that the signedness of the resulting down value produced above is explicitly reset.

The handling of immediates during top-down analysis can also be exemplified nicely using the RISC-V ANDI operation:

```

1  if ( o.getOpCode() == RV32I::OpCode::ANDI ) {
2      auto &imm = lastOp();
3      imm.setSignedness();
4      nthRes( 0 ) = nthOp( 1 ) & imm.extend( 32 );
5      nthRes( 0 ).setSignedness( false );
6  }

```

Here, `lastOp` gives a reference to the immediate's down value which, according to the RISC-V specification, is 12 bits wide. The ISA specification furthermore states that this immediate gets sign-extended before applying the bitwise operation itself. For this purpose, the immediate's down value needs to be set to signed and then extended to 32 bits. The rest is equivalent to the AND operation above. Other bitwise operations can be modeled in analogy to these two examples.

Loading constants of immediate parameters into a register is another rather simple task to be modeled in a bit-true fashion. The following code realizes the top-down analysis of the RISC-V LUI operation when it receives a 20 bits wide immediate parameter:

```

1  if ( o.getOpCode() == RV32I::OpCode::LUI ) {
2      if ( o.getOperationFormat() == RV32I::OperationFormat::RC20_1 )
3          nthRes( 0 ) = lastOp().extend( 32 ) << 12;
4      [...]
5  }

```

The down value of the immediate parameter is extended to 32 bits first. The resulting value is then left-shifted using the `<<` operator of class `WIR_UpDownValue` so that the least-significant 12 bits are filled with `b0`. The LUI's immediate then resides in the most-significant 20 bits, and this value is finally stored using `nthRes`.

The LUI operation also serves as a good example for operations that support labels as parameter. In this case, the most-significant 20 bits of an assembly label are loaded into a target register. This behavior can be modeled for top-down analysis as follows:

```

1  if ( o.getOpCode() == RV32I::OpCode::LUI ) {
2      [...]
3      else {
4          auto &label = dynamic_cast<WIR_LabelParameter &>( o.getExplicitParameters().back().get() );
5
6          if ( ( label.getLabelType() == WIR_SymbolType::data ) ||
7              ( label.getLabelType() == WIR_SymbolType::function ) ) {
8              const auto &f = o.getInstruction().getBasicBlock().getFunction();
9              const auto &sys = f.getCompilationUnit().getSystem();
10             auto &sym =
11                 label.getLabelType() == WIR_SymbolType::data ?
12                 sys.findSymbol( label.getData() ) : sys.findSymbol( label.getFunction() );
13
14             nthRes( 0 ).setAllBits( WIR_L4::b0 );
15             for ( unsigned int i = 12; i < 32; ++i )
16                 nthRes( 0 ).setBit( i, WIR_L4::bL, { sym, i } );
17         }
18     }
19 }

```

Here, the label is retrieved from the operation first. For the RISC-V LUI operation, a label may refer either to a data object or to a function. Depending on this, the symbol to which the label refers is retrieved from the `WIR` system. After that, all bits of the LUI's result register are set to `b0`. And the most-significant 20 bits of

the result register are finally set to bL such that each bit i of the resulting down value refers to the location given by the i -th bit of the symbol.

Shifting register contents is again simple to model in a bit-true way, as the following example for the logical left-shift of RISC-V shows:

```

1  if ( o.getOpCode() == RV32I::OpCode::SLL ) {
2      auto shamt = lastOp().extract( 0, 5 );
3      shamt.setSignedness( false );
4
5      // Enforce logical shifting.
6      auto &a = nthOp( 1 );
7      a.setSignedness( false );
8
9      nthRes( 0 ) = a << shamt;
10 }

```

In this example, the least-significant 5 bits of the last parameter of SLL are extracted first, as only they encode the shift amount. Since SLL realizes a logical left-shift, it is important to explicitly set the parameter `nthOp( 1 )` to be shifted to unsigned. After that, the operand register can simply be shifted by the shift amount, and the result is stored as down value associated with the result register.

Right-shifts usually distinguish between logical and arithmetical shifts which requires consideration also in the top-down analysis:

```

1  if ( ( o.getOpCode() == RV32I::OpCode::SRA ) || ( o.getOpCode() == RV32I::OpCode::SRL ) ) {
2      auto shamt = lastOp().extract( 0, 5 );
3      shamt.setSignedness( false );
4
5      // Enforce arithmetical/logical shifting.
6      auto &a = nthOp( 1 );
7      a.setSignedness( o.getOpCode() == RV32I::OpCode::SRA );
8
9      nthRes( 0 ) = a >> shamt;
10 }

```

In the above code example, the down value of the register parameter to be shifted is either set to signed or unsigned depending of the shift's opcode. This way, the `>>` operator of class `WIR_UpDownValue` can precisely determine which style of shift to apply.

The bit-true modelling of standard *integer arithmetical operations* is similarly simple as most of these operations are already provided as operators of class `WIR_UpDownValue`. Thus, an integer addition for the ongoing RISC-V example boils down to:

```

1  if ( o.getOpCode() == RV32I::OpCode::ADD ) {
2      nthRes( 0 ) = nthOp( 1 ) + lastOp();
3      nthRes( 0 ).setSignedness();
4  }

```

In analogy to this previous example, machine operations for integer subtraction, multiplication, division and modulo can be modeled for bit-true forward analysis.

Likewise, *integer comparison* operators of down values are also provided by class `WIR_UpDownValue` which facilitates the modelling of machine operations for greater-equal, less-than and other similar operations. The following example shows the realization of the RISC-V SLT and SLTU less-than comparisons, once signed and once unsigned, resp.:

```

1  if ( ( o.getOpCode() == RV32I::OpCode::SLT ) || ( o.getOpCode() == RV32I::OpCode::SLTU ) ) {
2      auto &a = nthOp( 1 );
3      a.setSignedness( o.getOpCode() == RV32I::OpCode::SLT );
4      auto &b = lastOp();
5      b.setSignedness( o.getOpCode() == RV32I::OpCode::SLT );
6
7      nthRes( 0 ) = { WIR_L4::b0, 32, false };
8      nthRes( 0 )[ 0 ] = ( a < b );
9  }

```

The important point here is to explicitly set or reset the signedness of the operands' down values, depending on whether a signed or unsigned comparison is desired. For many architectures, such operations produce a Boolean result in the target register where all bits are cleared and only the least-significant bit contains the binary comparison result. This behavior is mimicked in the above example where the result's down value is completely set to b0 first, and only its bit 0 is set to the comparison result.

The bit-true data flow analysis is not able to analyze the contents of memory addresses. Due to this, operations performing a *memory load* are regularly not modeled in the top-down analysis, because they only generate unknown bU bits by default. The only exception here are loads that load an unsigned quantity that is smaller than the machine's bit width. Usually, this applies for operations loading an unsigned byte or halfword. These quantities usually get zero-extended after being loaded so that it is known a priori that such load operations only produce zero b0 bits in the most-significant positions. This behavior is modeled for the RISC-V LBU (load byte unsigned) operation in the following example:

```

1  if ( o.getOpCode() == RV32I::OpCode::LBU ) {
2      // Least-significant 8 bits are unknown, most-significant bits are all 0.
3      nthRes( 0 ) = { WIR_L4::b0, 32, false };
4      WIR_UpDownValue u { WIR_L4::bU, 8, false };
5      insert( nthRes( 0 ), u, 0 );
6  }

```

As can be seen, only the least-significant 8 bits are unknown here, while the rest is set to b0.

Store operations are usually a sink in the data flow graph that have no outgoing edges. For this reason, they usually don't have to be modeled for top-down analysis. All other classes of machine operations need to be analyzed carefully in terms of their bitwise behavior. If this behavior is clear and can somehow be broken down to the fundamental operators of class `WIR_UpDownValue`, a bit-true modelling of operations not mentioned here is possible. The careful analysis of each operation's bitwise semantics as well as of the crucial header file `WIR/analyses/bit/wirupdownvalue.h` is indispensable, though.

As last step for retargeting the bit-true DFA of `WIR`, the backward analysis needs to be implemented. Similar to the top-down simulation, method `simulateBottomUp` makes use of a few lambdas for the efficient access to maps in and out.

9. Extend method `simulateBottomUp` of file `WIR/arch/archDir/analyses/bit/newModelbitdfa.cc` by the lambdas as shown in Listings 17.6 and 17.7.

The up values along all incoming DFG edges of an operation are stored in map `in`, while the up values of all outgoing edges reside in map `out`. The lambdas from Listings 17.6 and 17.7 provide simple access to these incoming and outgoing up values. Since they are extremely similar to the lambdas used during top-down analysis (cf. Step 7 above), we omit their detailed discussion here. The only difference that is worthwhile mentioning is that the bottom-up lambdas all return a pair consisting of the unique ID of the operation's parameter and a pointer to its associated up value.

Listing 17.6: Definition of Lambdas for Bottom-Up Analysis

```

1 void newModel_BitDFA::simulateBottomUp( [...] )
2 {
3     // firstIn retrieves the incoming up value of operation o's first explicit parameter from map in.
4     auto firstIn = [&]( void ) -> std::pair<WIR_id_t, WIR_UpDownValue *> {
5         auto &p = o.getExplicitParameters().front().get();
6
7         // Get up value from map in, update it to the correct bit width needed by parameter p.
8         auto &v = in.at( p.getID() );
9         updateOperand( p, v );
10
11         return( std::make_pair( p.getID(), &v ) );
12     };
13
14     // lastIn retrieves the incoming up value of operation o's last explicit parameter from map in.
15     auto lastIn = [&]( void ) -> std::pair<WIR_id_t, WIR_UpDownValue *> {
16         auto &p = o.getExplicitParameters().back().get();
17
18         // Get up value from map in, update it to the correct bit width needed by parameter p.
19         auto &v = in.at( p.getID() );
20         updateOperand( p, v );
21
22         return( std::make_pair( p.getID(), &v ) );
23     };
24
25     // nthIn retrieves the incoming up value of operation o's n-th explicit parameter from map in.
26     auto nthIn = [&]( unsigned int n ) -> std::pair<WIR_id_t, WIR_UpDownValue *> {
27         auto it = o.getExplicitParameters().begin();
28         std::advance( it, n );
29
30         // Get up value from map in, update it to the correct bit width needed by parameter p.
31         auto &v = in.at( it->get().getID() );
32         updateOperand( it->get(), v );
33
34         return( std::make_pair( it->get().getID(), &v ) );
35     };
36
37     [...]
38 };

```

10. Using the lambdas defined in the previous step, the main realization of method `simulateBottomUp` can follow next. Again, the body of this method consists of a sequence of if-conditions that, one after the other, check the opcode and possibly the operation format of the current operation `o`. And depending on `o`'s opcode (and format), the bottom-up simulation determines which bits of incoming up values can be set to don't care `bX`. The resulting up values including such `bX` bits are stored in `map results`.

Obviously, this document cannot cover all possible situations that might need to be considered during the implementation of method `simulateBottomUp`. Instead, the following paragraphs simply focus on the most relevant classes of machine operations and their bit-true backward analysis. Code snippets from the RISC-V implementation (see file `WIR/arch/riscv/analyses/bit/rv32bitdfa.cc`) will serve as examples.

The most simple phenomenon to be considered during bottom-up simulation is the backward propagation of `bX` bits from an operation's outgoing up value(s) to its incoming ones. This refers to a situation where it turned out that some outputs produced by operation `o` do not care. If `o` produces a result that is (partially) irrelevant, then it is very likely that also some bits of `o`'s inputs don't care. This concept motivates why the

Listing 17.7: Definition of Lambdas for Bottom-Up Analysis (ctd.)

```

1 void newModel_BitDFA::simulateBottomUp( [...] )
2 {
3     [...]
4
5     // firstOut retrieves the outgoing up value of operation o's first explicit parameter from map out.
6     auto firstOut = [&]( void ) -> std::pair<WIR_id_t, WIR_UpDownValue *> {
7         auto &p = o.getExplicitParameters().front().get();
8
9         // Get up value from map out, update it to the correct bit width needed by parameter p.
10        auto &v = out.at( p.getID() );
11        updateOperand( p, v );
12
13        return( std::make_pair( p.getID(), &v ) );
14    };
15
16    // nthOut retrieves the outgoing up value of operation o's n-th explicit parameter from map out.
17    auto nthOut = [&]( unsigned int n ) -> std::pair<WIR_id_t, WIR_UpDownValue *> {
18        auto it = o.getExplicitParameters().begin();
19        std::advance( it, n );
20
21        // Get up value from map out, update it to the correct bit width needed by parameter p.
22        auto &v = out.at( it->get().getID() );
23        updateOperand( it->get(), v );
24
25        return( std::make_pair( it->get().getID(), &v ) );
26    };
27
28    [...]
29 };

```

bit-true bottom-up simulation is part of a backwards [DFA](#), since the bX bits in general propagate against the sense of the [DFG](#)'s edges from a node's outputs to its inputs. A *move operation* is again the most simple example illustrating this idea. The bottom-up simulation of the RISC-V RV32IC's C.MV operation looks as follows:

```

1 if ( o.getOpCode() == RV32IC::OpCode::CMV ) {
2     auto &outUp = *(firstOut().second);
3     auto in1 = lastIn();
4     auto &inUp1 = *(in1.second);
5
6     for ( unsigned int i = 0; i < outUp.getBitWidth(); ++i )
7         if ( outUp.at( i ) == WIR_L4::bX )
8             inUp1.setBit( i, WIR_L4::bX );
9
10    results.insert( { in1.first, inUp1 } );
11 }

```

The up value of the target register which is the only output of a move is retrieved first. Next, the last parameter of the move being its only input is used to obtain its respective up value. As a move copies its input to the output register, don't care bits showing up at the output's up value thus are also don't care for the move's input. This backward propagation is realized by the above for-loop that sets individual input bits to bX if the bit at the same position of the output is already don't care. Finally, the up value obtained this way is stored in map `results` under the input parameter's unique ID.

Besides this backward propagation of don't care bits, certain operations can generate bX bits on their own.

Prime example here is a *bitwise logical* AND operation. Here, any zero b0 bit in one of its input parameters has the effect that the corresponding bit of the other input does not care, since 0 AND *<something>* is always 0 – irrespective of the other bit value. This behavior, combined with the already discussed backward propagation, is illustrated in the following RISC-V code example:

```

1  if ( o.getOpCode() == RV32I::OpCode::AND ) {
2      auto &outUp = *(firstOut().second);
3      auto in1 = nthIn( 1 );
4      auto &inUp1 = *(in1.second);
5      auto in2 = lastIn();
6      auto &inUp2 = *(in2.second);
7
8      for ( unsigned int i = 0; i < outUp.getBitWidth(); ++i ) {
9          if ( inUp2.at( i ) == WIR_L4::b0 )
10             inUp1.setBit( i, WIR_L4::bX );
11          else
12
13             if ( inUp1.at( i ) == WIR_L4::b0 )
14                 inUp2.setBit( i, WIR_L4::bX );
15
16             if ( outUp.at( i ) == WIR_L4::bX ) {
17                 inUp1.setBit( i, WIR_L4::bX );
18                 inUp2.setBit( i, WIR_L4::bX );
19             }
20         }
21
22         results.insert( { in1.first, inUp1 } );
23         results.insert( { in2.first, inUp2 } );
24     }

```

This code first checks for a b0 bit in the last input's up value and then sets the same bit of the first input to bX. Otherwise, the symmetric approach is applied and a bit of the second input is set to bX if the same bit of the first input is b0. In addition to this, both inputs' bits at position *i* are set to don't care if the *i*-th bit of the operation's output is already bX.

A similar approach can be applied for the bitwise OR where a b1 bit in one input sets its partner bit of the other input to bX.

For *shift operations*, the potential to generate don't cares during bottom-up analysis is also quite large. This is because those input bits that get shifted out can obviously be set to bX, in addition to backward propagation again. The following code shows the bottom-up analysis of the RISC-V SLLI operation:

```

1  if ( o.getOpCode() == RV32I::OpCode::SLLI ) {
2      auto &outUp = *(firstOut().second);
3      auto in1 = nthIn( 1 );
4      auto &inUp1 = *(in1.second);
5      auto &shamtUp = *(lastIn().second);
6
7      for ( unsigned int i = 1; i <= shamtUp.getSignedValue(); ++i )
8          inUp1.setBit( inUp1.getBitWidth() - i, WIR_L4::bX );
9
10     for ( unsigned int i = shamtUp.getSignedValue(); i < outUp.getBitWidth(); ++i )
11         if ( outUp.at( i ) == WIR_L4::bX )
12             inUp1.setBit( i - shamtUp.getSignedValue(), WIR_L4::bX );
13
14     results.insert( { in1.first, inUp1 } );
15 }

```

The first for-loop sets the input's most-significant bits that get shifted out to bX. And the second loop real-

izes the backward propagation, but here under consideration of the left-shift such that a bX in output bit i propagates backwards to input bit $i - \langle \text{shift amount} \rangle$.

For *integer arithmetical operations* such as additions or subtractions, backward propagation is not as simple as in all the previous examples. This is due to the fact that two input bits at position i to be added/subtracted do not affect output bit i in isolation. In the case of a carry-over, output bit $i+1$ might also be affected. If such a carry-over furthermore ripples onwards through the most-significant bit positions, then all bits from position i to the most-significant bit might be affected.

Thus, only if all the most-significant output bits from position i on are all bX, it is safe to set all bits of both inputs from i on to don't care. This is shown in the following example for the RISC-V ADD operation:

```

1  if ( o.getOpCode() == RV32I::OpCode::ADD ) {
2      auto &outUp = *(firstOut().second);
3      auto in1 = nthIn( 1 );
4      auto &inUp1 = *(in1.second);
5      auto in2 = lastIn();
6      auto &inUp2 = *(in2.second);
7
8      for ( unsigned int i = outUp.getBitWidth() - 1; ( i >= 0 ) && ( outUp[ i ] == WIR_L4::bX ); --i ) {
9          inUp1.setBit( i, WIR_L4::bX );
10         inUp2.setBit( i, WIR_L4::bX );
11     }
12
13     results.insert( { in1.first, inUp1 } );
14     results.insert( { in2.first, inUp2 } );
15 }

```

Here, the for-loop scans the output bits for don't cares beginning with the most-significant position. As long as bX bits are found in the output, the corresponding most-significant input bits are set to bX and the loop continues. The very same approach can be applied to integer subtractions.

Integer comparisons that produce only a Boolean value in the output's least-significant bit are simple to model for bottom-up analysis. If this least-significant output bit turns out to be irrelevant, then all bits of both inputs that are compared do not care, as shown in the below example for the RISC-V SLT comparison:

```

1  if ( o.getOpCode() == RV32I::OpCode::SLT ) {
2      auto &outUp = *(firstOut().second);
3      auto in1 = nthIn( 1 );
4      auto &inUp1 = *(in1.second);
5      auto in2 = lastIn();
6      auto &inUp2 = *(in2.second);
7
8      if ( outUp.at( 0 ) == WIR_L4::bX ) {
9          inUp1.setAllBits( WIR_L4::bX );
10         inUp2.setAllBits( WIR_L4::bX );
11     }
12
13     results.insert( { in1.first, inUp1 } );
14     results.insert( { in2.first, inUp2 } );
15 }

```

Conditional branches usually do not produce any visible result in some output register. But in certain situations, the branch condition can be exploited and don't cares be generated. If, e.g., for a branch-if-equal operation the analysis turns out that the branch condition is definitely not met, then it is sufficient to find one bit position where both inputs definitely differ. All other bits of both inputs can safely be set to bX, as shown in the following example for the RISC-V BEQ operation:

```

1  if ( o.getOpCode() == RV32I::OpCode::BEQ ) {
2      auto in1 = firstIn();
3      auto &inUp1 = *(in1.second);
4      auto in2 = nthIn( 1 );
5      auto &inUp2 = *(in2.second);
6
7      // Compare input up values for equality.
8      auto cmp = ( inUp1 == inUp2 );
9
10     if ( cmp == WIR_L4::b0 ) {
11         // The comparison definitely is false. Determine bit position where inputs differ.
12         unsigned int i = 0;
13
14         for ( ; i < inUp1.getBitWidth(); ++i )
15             if ( ( getLevel( inUp1[ i ] ) == 2 ) && ( getLevel( inUp2[ i ] ) == 2 ) &&
16                 ( inUp1[ i ] != inUp2[ i ] ) )
17                 break;
18         else
19
20             if ( ( getLevel( inUp1[ i ] ) == 1 ) && ( getLevel( inUp2[ i ] ) == 1 ) &&
21                 ( inUp1[ i ] != inUp2[ i ] ) && ( inUp1.getLocation( i ) == inUp2.getLocation( i ) ) )
22                 break;
23         else {
24             inUp1.setBit( i, WIR_L4::bX );
25             inUp2.setBit( i, WIR_L4::bX );
26         }
27
28         for ( ++i; i < inUp1.getBitWidth(); ++i ) {
29             inUp1.setBit( i, WIR_L4::bX );
30             inUp2.setBit( i, WIR_L4::bX );
31         }
32
33         results.insert( { in1.first, inUp1 } );
34         results.insert( { in2.first, inUp2 } );
35     }
36 }

```

First, the branch's condition is evaluated by applying the == operator of class `WIR_UpDownValue`. This operator returns `b1` if both compared operands are provably equal, `b0` if they both are provably unequal, and `bU` otherwise. If both operands definitely are unequal, one bit position is determined where both inputs provably are unequal. This is the case either if bit *i* of one input is `b0` and the same bit of the other input is `b1`, or vice versa. Or otherwise, if the one bit is `bL` and the other `bN`, and both refer to the very same location. These two cases are tested in the above code, and all bits except at this differing bit positions are set to `bX`.

For *store operations* saving half-words or bytes into memory, the most-significant bits that do not belong to the stored half-word/byte are obviously don't care. The following example shows the bottom-up analysis of the RISC-V SB operation:

```

1  if ( o.getOpCode() == RV32I::OpCode::SB ) {
2      auto in1 = firstIn();
3      auto &inUp1 = *(in1.second);
4
5      // The most-significant 24 bits are X.
6      for ( unsigned int i = 8; i < inUp1.getBitWidth(); ++i )
7          inUp1.setBit( i, WIR_L4::bX );
8
9      results.insert( { in1.first, inUp1 } );
10 }

```

18 Retargeting of Existing Code Optimizations

This chapter describes the steps that are required to retarget the existing [WIR](#) optimizations to new processor architectures. For a general understanding of the various optimizations' purpose, the interested reader should be familiar with [Chapter 14](#). Since some optimizations are generic and entirely processor-independent (i.e., unused virtual register elimination, empty basic block elimination, redundant basic block elimination, unreachable basic block elimination, loop-invariant code motion and web generation), they are not discussed in the following. This chapter thus focuses on the remaining optimizations ranging from graph coloring-based register allocation to local instruction scheduling.

18.1 Graph Coloring-Based Register Allocation

The by far most complex optimization that requires the largest retargeting effort is the [WIR](#) register allocator based on graph coloring. According to [\[5, 22, 1\]](#), the overall workflow of a graph coloring-based register allocator is:

1. *Build*: Create the [IG](#).
2. *Simplify*: Choose some node v from the [IG](#) with degree less than K that is neither source nor target of a move operation. Remove v and all its incident edges from the [IG](#), push v onto a stack S . Repeat this until no such non move-related nodes of degree less than K exist in [IG](#).
3. *Coalesce*: Merge nodes in the [IG](#) that represent source and target registers of move operations. If nodes were merged, go back to Step 2.
4. *Spill*: Select one remaining node w of the [IG](#) (which must have a degree greater or equal than K), mark w as potential spill, remove it and its incident edges from the [IG](#), push w onto S .
5. *Repeat*: If the [IG](#) still contains nodes, go back to Step 2.
6. *Select*: Pop the top node v from S , re-insert it and its incident edges into the [IG](#). Check among all nodes w adjacent to v whether there is a free color k_v available. If so, color v with k_v . Otherwise, mark v as actual spill. Repeat this until S is empty.
7. *Spill Code Generation*: For each actual spill v , insert a spill-load operation before each use of v and a spill-store operation after each definition of v .
8. *Moves*: Remove all register-register move operations with identical source and target registers.
9. *Start Over*: If actual spills have been determined during Step 6, go to Step 1.
10. *Uncolored Spills*: Color all actual spills still remaining in the [IG](#).
11. *Caller-Saving*: Add spill-stores and -loads for caller-saved registers alive across calls.
12. *Redundant Spills*: Remove all redundant spills.

Listing 18.1: Header File for a New Processor Model's Graph Coloring-Based Register Allocator

```

1  #ifndef _newModel_GRAPHCOLORING_H
2  #define _newModel_GRAPHCOLORING_H
3
4  // Include WIR headers
5  #include <wir/wirtypes.h>
6  #include <optimizations/gcregallocation/wirgraphcoloring.h>
7
8  namespace WIR {
9
10 class WIR_CompilationUnit;
11 class WIR_Function;
12 class WIR_System;
13
14 class newModel_GraphColoring : public WIR_GraphColoring
15 {
16 public:
17
18     // Default constructor for system-level optimization.
19     explicit newModel_GraphColoring( WIR_System &, bool = false );
20
21     // Default constructor for compilation unit-level optimization.
22     explicit newModel_GraphColoring( WIR_CompilationUnit &, bool = false );
23
24     // Default constructor for function-level optimization.
25     explicit newModel_GraphColoring( WIR_Function &, bool = false );
26
27     // Destructor.
28     virtual ~newModel_GraphColoring( void );
29
30 protected:
31
32 private:
33 };
34
35 } // namespace WIR
36
37 #endif

```

The generic and processor-independent implementation of the IG is provided by class `WIR_InterferenceGraph` (see file `WIR/optimizations/gcregallocation/wirinterferencegraph.cc`). Most of the above algorithm is also realized in a generic way within class `WIR_GraphColoring` (cf. file `WIR/optimizations/gcregallocation/wirgraphcoloring.cc`). However, several steps above require adaptations during retargeting for a new processor architecture. This regularly means to provide overloaded virtual methods by inheriting from the abstract base class `WIR_GraphColoring`. The upcoming subsections describe what needs to be done to retarget the individual steps of the above algorithm. But before coming to this, the following steps describe how to set up the infrastructure for inheriting from the register allocator's abstract base class:

1. Create new directories `WIR/arch/archDir/optimizations` and `WIR/arch/archDir/optimizations/gcregallocation`.
2. Add a new line
`include arch/archDir/optimizations/Makefile.src`
to the beginning of file `WIR/arch/archDir/Makefile.src`.
3. If not yet existing, create a new file `WIR/arch/archDir/optimizations/Makefile.src` with the following

Listing 18.2: Rudimentary Implementation of a New Processor Model's Graph Coloring-Based Register Allocator

```

1  ifdef HAVE_CONFIG_H
2  #include <config_wir.h>
3  #endif
4
5  #include <libuseful/io.h>
6
7  // Include WIR headers
8  #include <wir/wir.h>
9  #include <arch/archDir/newModel.h>
10
11 // Include local headers
12 #include "newModelgraphcoloring.h"
13
14 namespace WIR {
15
16 // Default constructor for system-level optimization.
17 newModel_GraphColoring::newModel_GraphColoring( WIR_System &s, bool verbosity ) :
18     WIR_Optimization { s },
19     WIR_GraphColoring { s, verbosity }
20 {};
21
22 // Default constructor for compilation unit-level optimization.
23 newModel_GraphColoring::newModel_GraphColoring( WIR_CompilationUnit &c, bool verbosity ) :
24     WIR_Optimization { c },
25     WIR_GraphColoring { c, verbosity }
26 {};
27
28 // Default constructor for function-level optimization.
29 newModel_GraphColoring::newModel_GraphColoring( WIR_Function &f, bool verbosity ) :
30     WIR_Optimization { f },
31     WIR_GraphColoring { f, verbosity }
32 {};
33
34 // Destructor.
35 newModel_GraphColoring::~newModel_GraphColoring( void )
36 {};
37
38 }           // namespace WIR

```

contents:

```
1 include arch/archDir/optimizations/gcregallocation/Makefile.src
```

4. Create a new file WIR/arch/archDir/optimizations/gcregallocation/Makefile.src with the following contents:

```
1 libwir_la_SOURCES      += arch/archDir/optimizations/gcregallocation/newModelgraphcoloring.cc
   arch/archDir/optimizations/gcregallocation/newModelgraphcoloring.h
```

5. Create a file WIR/arch/archDir/optimizations/gcregallocation/newModelgraphcoloring.h according to the general structure depicted in Listing 18.1.
6. Create file WIR/arch/archDir/optimizations/gcregallocation/newModelgraphcoloring.cc according to the general structure depicted in Listing 18.2.

The constructors from Lines 17, 23 and 29 of Listing 18.2 take a reference to a [WIR](#) system, compilation unit or function as first argument, resp. Within these objects, all virtual registers will be replaced by physical registers. The optional Boolean argument of all constructors controls whether progress messages are printed during register allocation. These arguments are delegated to the constructors of base classes `WIR_Optimization` and `WIR_GraphColoring`. In their initial state, the constructors and the destructor have no additional tasks, since the newly created class does not yet feature its very own attributes to be constructed or destroyed.

18.1.1 Initialization

Before realizing the graph coloring algorithm's steps from Page 257, a processor-specific initialization is needed that introduces a new processor architecture's physical registers to the allocator. This initialization is done by the protected, purely virtual method `createPhregs` that needs to be overloaded by the newly created register allocation class. For this purpose, extend file `WIR/arch/archDir/optimizations/gcregallocation/newModelgraphcoloring.h` by the following protected method declaration:

```
1 virtual void createPhregs( WIR_Function &f ) override;
```

Provide an initially empty implementation of this method in file `WIR/arch/archDir/optimizations/gcregallocation/newModelgraphcoloring.cc`. For the sake of simplicity, we will refer to the existing implementation of method `TC_GraphColoring::createPhregs` from file `WIR/arch/tricore/optimizations/gcregallocation/tcgraphcoloring.cc` for the TriCore processor models.

The first task to be done in `createPhregs` is to retrieve the underlying [WIR](#) processor model and to ensure that it corresponds to `newModel`. This is performed by obtaining the [WIR](#) system from the function provided as argument to `createPhregs`. The system can then be queried for its sections and associated processor models, and the obtained base processor can afterwards be dynamically casted to the desired class. For the running TriCore example, these steps look as follows:

```
1 // Determine the involved processor core.
2 const WIR_System &sys = f.getCompilationUnit().getSystem();
3 const WIR_Section &sec = sys.findSymbol( f ).getSection();
4 const WIR_BaseProcessor &p = sec.getProcessor();
5
6 // Ensure the correct processor model.
7 uassert( p.getISAName().find( "TC1.3" ) == 0 );
8 const auto &proc = dynamic_cast<const TC13 &>( p );
```

The register allocator uses a list `mPhregs` that contains all physical registers of a processor model that can be used by the allocator. This list is declared in file `WIR/optimizations/gcregallocation/wirgraphcoloring.h` as `std::list<std::reference_wrapper<const WIR_PhysicalRegister>>`. This list must be initialized by method `createPhregs` with the appropriate physical registers, below again shown for the TriCore example:

```
1 // TriCore registers subject to register allocation.
2 mPhregs = {
3     // TriCore data registers.
4     const_cast<TC_DRegP &>( proc.D0() ), const_cast<TC_DRegP &>( proc.D1() ), [...],
5     const_cast<TC_DRegP &>( proc.D15() ),
6
7     // TriCore address registers.
8     const_cast<TC_ARegP &>( proc.A2() ), [...], const_cast<TC_ARegP &>( proc.A7() ),
9     const_cast<TC_ARegP &>( proc.A12() ), [...], const_cast<TC_ARegP &>( proc.A15() ) };
```

As can be seen, not necessarily all physical registers of a processor model get listed in `mPhregs`. The above example purposively omits the TriCore address registers `A0`, `A1` and `A8 – A11`. This is due to the [ABI](#) that specifies

special roles of certain registers which must be respected during register allocation. For example, A8 and A9 are reserved for operating system use, and A10 serves as stack pointer.

However, **WIR** code regularly makes use of such special registers by introducing virtual registers that are pre-colored with the desired special-purpose physical register. In this case, the register allocator of course has to replace each occurrence of such a pre-colored virtual register by its associated physical register. In order to support this task, the base class `WIR_GraphColoring` declares a `WIR_PhysicalRegisterSet` named `mPhregsForPrecoloringOnly`. This set thus also needs to be initialized by method `createPhRegs`, e.g.:

```
1 // TriCore registers subject to pre-coloring but NOT to register allocation.
2 mPhregsForPrecoloringOnly = {
3     const_cast<TC_ARegP &>( proc.A0() ), const_cast<TC_ARegP &>( proc.A1() ),
4     const_cast<TC_ARegP &>( proc.A8() ), const_cast<TC_ARegP &>( proc.A9() ),
5     const_cast<TC_ARegP &>( proc.A10() ), const_cast<TC_ARegP &>( proc.A11() ) };
```

18.1.2 Interference Graph Creation

Most of the creation of the **IG** is done in a processor-independent way by exploiting the generic **WIR** liveness analysis and inserting edges whenever two registers are simultaneously alive. However, it might be required to add processor-specific interferences to the **IG** which is the purpose of the purely virtual method `buildProcessorSpecificInterferences`.

A typical scenario occurring for many processors is that the architecture features different types of physical registers which are incompatible with each other. Prime examples are general-purpose integer registers and dedicated floating-point registers, or the TriCore's data and address registers. Here, the register allocator must ensure that no virtual data register is ever allocated to some address register, and vice versa. This is ensured by adding extra **IG** edges between any node representing a virtual register of the one type and all nodes representing the physical registers of the other type.

Thus, file `WIR/arch/archDir/optimizations/gcregallocation/newModelgraphcoloring.h` has to be extended by the following protected method declaration:

```
1 virtual void buildProcessorSpecificInterferences( WIR_Function &f,
2                                                  WIR_InterferenceGraph &igraph ) override;
```

Next, provide an initially empty realization of this method in file `WIR/arch/archDir/optimizations/gcregallocation/newModelgraphcoloring.cc`. It receives references to the current **WIR** function where register allocation shall take place, and to the interference graph as arguments. For the aforementioned purpose, `buildProcessorSpecificInterferences` iterates over the function's virtual registers that are represented by an **IG** node and that are not precolored. Hereafter, it iterates over all physical registers listed in `mPhregs`, determines the types of the current virtual and physical register and adds an interference to the **IG** if necessary:

```
1 for ( const WIR_VirtualRegister &r : f.getVirtualRegisters() )
2     if ( !isChild() && igrph.containsNode( r ) && !isPrecolored( r ) )
3         for ( const WIR_VirtualRegister &l : r.getLeaves() )
4             for ( const WIR_PhysicalRegister &preg : mPhregs )
5                 if ( !igrph.interfere( l, preg ) && !igrph.areSameNodes( l, preg ) &&
6                     ( ( l.getType() == TC13::RegisterType::dReg ) &&
7                       ( preg.getType() == TC13::RegisterType::aReg ) ) ||
8                     ( ( l.getType() == TC13::RegisterType::aReg ) &&
9                       ( preg.getType() == TC13::RegisterType::dReg ) ) ) )
10                     igrph.addInterference( l, preg, 1 );
```

A very simple **IG** showing the effect of the above code for virtual TriCore data registers is depicted in Fig. 18.1. It

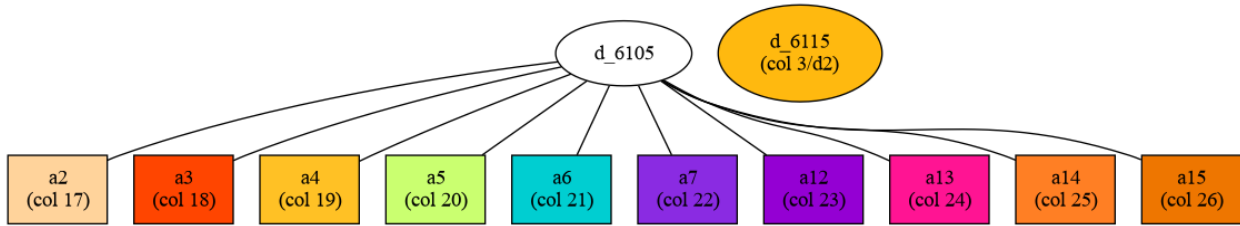


Figure 18.1: Example of Processor-Specific Interferences for Virtual TriCore Data Registers

shows the virtual data register `d_6115` which, due to a given precoloring, is constrained to physical register D2. Due to this constraint, it is not necessary to add interference edges between `d_6115` and all the address registers shown at the bottom of Fig. 18.1. This motivates why Line 2 above checks only for virtual registers without precoloring. For virtual register `d_6105`, however, the edges towards address registers A2 – A15 are generated in Line 10.

Method `addInterference` used in Line 10 expects the two IG nodes that interfere with each other as first arguments, followed by the number of edges to be added. Usually, only one edge needs to be added to the IG as shown in Line 10. In general, however, the IG can be a multi-graph with several edges between a pair of nodes. This situation can happen for hierarchical registers, which are fully supported by the IG itself as well as by the generic register allocator from class `WIR_GraphColoring`.

To be more specific, IG nodes can represent either non-hierarchical virtual registers such as `d_6105` or `d_6115` from Fig. 18.1. These are stand-alone virtual data registers that can freely be allocated to any of the available physical data registers. Alternatively, an IG node can represent the root of some hierarchical virtual register, such as an extended register of the TriCore architecture that spans over two neighboring data registers. For a hierarchical register, both the IG and the register allocator ensure that the childs of the hierarchical register are no longer freely allocated, but only in such a way that the entire hierarchy of virtual registers is correctly assigned to an appropriate hierarchy of physical registers.

Fig. 18.2 shows an example IG with two virtual hierarchical registers `e_6128` and `e_6115`. These TriCore extended registers form the roots of register hierarchies which contain virtual data registers `d_6130` and `d_6132` as childs (`d_6117` and `d_6119`, resp.). But these child registers are not represented by individual IG nodes, they are only visualized in Fig. 18.2 in order to make clear that the child registers are integral part of their respective root nodes representing `e_6128` and `e_6115`.

This explains the test `!isChild()` from Line 2 on Page 261 – any kind of virtual register being a child node is skipped in the above code so that only non-hierarchical virtual registers or roots of virtual hierarchical registers are considered. The loop from Line 3 then iterates over all leafs, e.g., registers `d_6117` and `d_6119` for hierarchical register `e_6115` from Fig. 18.2. And between any such virtual leaf register and the TriCore's address registers, interferences are finally added, which results in edges between node `e_6115` and all address registers depicted in Fig. 18.2.

Furthermore, the example from Fig. 18.2 shows that both hierarchical registers `e_6128` and `e_6115` interfere due to some liveness conflicts. Since these two graph nodes represent hierarchical registers with 2 leafs each, two interference edges between them are shown in Fig. 18.2, illustrating the fact that the IG can potentially be a multi-graph.

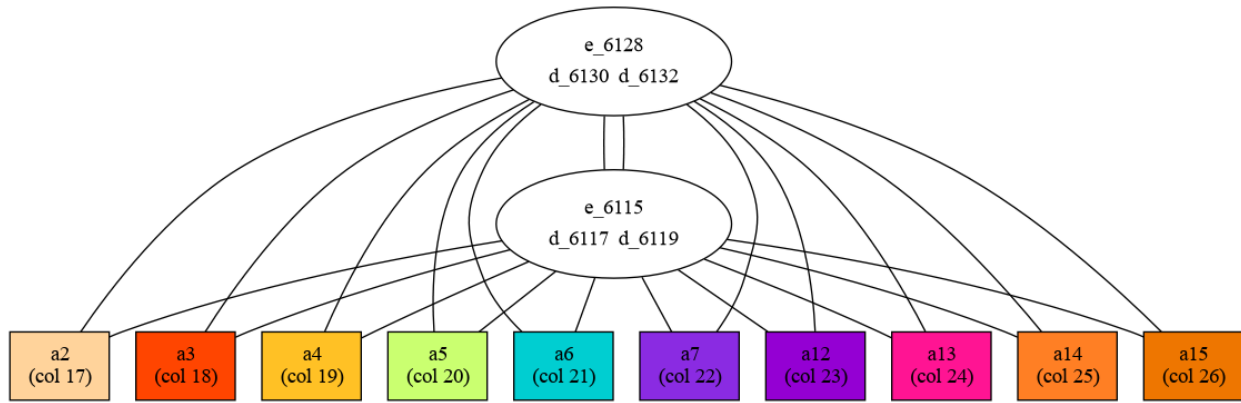


Figure 18.2: Example of Processor-Specific Interferences for Virtual TriCore Extended Data Registers

18.1.3 Coalescing

Coalescing is an optimization that is part of the register allocator and that aims to eliminate as many redundant move operations as possible. Move operations copy the contents of a source register to another target register. If the register allocator assigns both these move-related source and target registers to the same physical register, the move actually performs no action and can thus be removed. In order to systematically exploit this, the **WIR** register allocator checks for move operations in the code and ensures that no interference edges between the source and target registers of a move are added to the **IG**. Then, coalescing merges the **IG** nodes representing a move's source and target into one single node. And since this coalesced node will get exactly one color, both move-related registers are assigned the same physical register and the move gets eliminated.

The register allocator uses the purely virtual methods `getUseOfMove` and `getDefOfMove` to identify the source and target registers of a given move operation. In order to provide these methods for a new processor architecture, file `WIR/arch/archDir/optimizations/gcregallocation/newModelgraphcoloring.h` has to be extended as follows:

```
1 virtual WIR_BaseRegister &getUseOfMove( const WIR_Operation &o ) const override;
2
3 virtual WIR_BaseRegister &getDefOfMove( const WIR_Operation &o ) const override;
```

These methods get a reference to a **WIR** operation as argument that has safely been identified as register-register move. They return a reference to the move's source register (`getUseOfMove`) or to its target register (`getDefOfMove`), resp.

For most architectures that feature a dedicated move operation in their **ISA**, these methods regularly boil down to returning the register associated with the operation's first or second parameter, as shown in the following code example taken from the TriCore register allocator:

```
1 WIR_BaseRegister &TC_GraphColoring::getUseOfMove( const WIR_Operation &o ) const
2 {
3     return( dynamic_cast<WIR_RegisterParameter &>( o.getExplicitParameter( 2 ) ).getRegister() );
4 };
5
6 WIR_BaseRegister &TC_GraphColoring::getDefOfMove( const WIR_Operation &o ) const
7 {
8     return( dynamic_cast<WIR_RegisterParameter &>( o.getExplicitParameter( 1 ) ).getRegister() );
9 };
```

For other architectures where arithmetical operations need to be used in special ways in order to achieve a

register-register move (e.g., RISC-V where a move is only a pseudo-operation), more complex approaches can easily be realized inside `getUseOfMove` and `getDefOfMove`.

Besides this identification of a move's source and target registers, coalescing can be controlled in a fine-grained and processor-specific way via method `avoidCoalescing`. As the name suggests, this method can be used to avoid coalescing of two move-related registers in situations that depend on properties of the underlying processor architecture. If coalescing needs to be avoided, extend file `WIR/arch/archDir/optimizations/gcregallocation/newModelgraphcoloring.h` by:

```
1 virtual bool avoidCoalescing( const WIR_Operation &o, const WIR_BaseRegister &r1,
2                             const WIR_BaseRegister &r2,
3                             const WIR_InterferenceGraph &igraph ) const override;
```

This method gets references to a move operation and its two move-related registers as arguments, as well as a reference to the current [IG](#). Based on these inputs, `avoidCoalescing` returns `true` if the given move operation shall not be coalesced, or `false` otherwise.

A typical scenario where coalescing needs to be avoided are move operations that store the results of a function call. A processor architecture's [ABI](#) mandates which physical registers have to be used to pass results from a called function (callee) to the calling function (caller). Regularly, these registers are caller-saved, and special attention has to be paid to such caller-saved registers that are alive across function calls (cf. [Section 18.1.8.1](#)). Move operations immediately following a function call can then move the function's result from its dedicated caller-saved registers into callee-saved registers. In such a situation, it might be reasonable for a given processor architecture to keep these particular move operations.

For the running TriCore example, method `avoidCoalescing` thus checks whether

- the move's source register corresponds to physical registers D2, D3 or E2 (the return value registers according to the TriCore [ABI](#)), and
- the move operation resides at the very beginning of its basic block, and
- all basic blocks preceding the move in the [CFG](#) end with a function call operation, and
- all these preceding call operations define that register that is the move's source register.

In such a situation, `avoidCoalescing` returns `true` so that the move is kept.

18.1.4 Spill Cost Computation

The spilling Step 4 from [Page 257](#) selects one [IG](#) node of significant degree and marks it as potential spill. This means that the virtual register associated with the chosen node might not be kept in a physical register. Instead, it might have to be spilled out into a reserved place of the processor's main memory (usually a location in the current function's stack frame). A spilled virtual register thus cannot be directly accessed by machine operations, but dedicated load and store operations (so-called spill code) must load the register's contents from the stack into a register first, and new values of the register need to be written back using spill-stores.

The decision which virtual register becomes a potential spill is critical for several reasons. First, it has a significant impact on the total amount of spill code to be generated and on the resulting code size. Furthermore, the final code's performance can be heavily affected, as the frequent execution of large amounts of spill code is extremely inefficient.

Spilling within the [WIR](#) graph coloring allocator assigns a metric to all [IG](#) nodes that takes the positions where a register is used, defined or moved into account. In principle, each use of a virtual register results in costs for

a preceding spill-load, each definition adds costs for a subsequent spill-store, and spilling a move-related virtual register adds negative costs since the move can be eliminated again. Among all **IG** nodes of significant degree, the register allocator chooses that one leading to the least overall spill costs.

This general approach of spill cost computation is realized by the virtual method `WIR_GraphColoring::computeSpillCosts` which usually does not require a specific retargeting. By overloading it in derived register allocation classes, different cost computations can be implemented though.

What needs to be adapted for a new processor architecture are those methods that determine the actual costs of a spill-load, spill-store or a move eliminated due to a spill. This is realized by extending the header file of the new architecture's register allocator as follows:

```
1 virtual unsigned int getSpillLoadCosts( const WIR_VirtualRegisterParameter &p ) const override;
2
3 virtual unsigned int getSpillStoreCosts( const WIR_VirtualRegisterParameter &p ) const override;
4
5 virtual unsigned int getMoveCosts( const WIR_Operation &o ) const override;
```

For some operation's register parameter `p` whose associated register might become a potential spill, `getSpillLoadCosts` and `getSpillStoreCosts` return the processor-specific spill load and store costs, resp. If a move operation `o` gets eliminated due to spilling, its saved costs are returned by `getMoveCosts`.

What these costs shall represent for a new processor architecture is left to the developer. If execution time shall be modeled, the latencies of the involved physical memory regions are important. A spill accesses the stack so that the latency of the stack's memory region is relevant. Furthermore, an additional latency has to be considered, since each spill operation needs to be fetched from the memory region holding the binary executable code.

Using the **WIR** memory layout (cf. Section 10.4), memory access latencies can be exploited for spill cost estimation. For this purpose, the new allocator's header file can be extended by the following two private attributes:

```
1 private:
2
3 // mStackAccessLatency stores the latency of the memory region holding the stack.
4 unsigned int mStackAccessLatency;
5
6 // mBBAccessLatency stores the latency of the memory region where a basic block with a given ID resides.
7 std::map<WIR_id_t, unsigned int> mBBAccessLatency;
```

Attribute `mStackAccessLatency` should be initialized with the dummy value 0 by the constructors from Listing 18.2. Meaningful values can be given to both attributes inside method `createPhregs` (cf. Section 18.1.1). For the running **TriCore** example, `createPhregs` pre-computes the access latencies as follows:

```
1 // Determine memory access latency of the stack.
2 ufAssert( proc.containsSection( ".stack" ) );
3 const WIR_MemoryRegion &stackMem = proc.findSection( ".stack" )->get().getRegion();
4 mStackAccessLatency = stackMem.getMaxDelay();
5
6 // Determine the code access latencies of all basic blocks.
7 mBBAccessLatency.clear();
8 for ( const WIR_BasicBlock &b : f ) {
9     const auto &region = sys.findSymbol( b ).getSection().getRegion();
10    mBBAccessLatency[ b.getID() ] = region.getMaxDelay();
11 }
```

Line 2 ensures that a section `.stack` actually exists, which always holds for the **TriCore** systems modeled within **WIR**. Next, this section and the memory region to which it is allocated are retrieved. The stack region's memory latency is obtained using `getMaxDelay`, which is assigned to the attribute `mStackAccessLatency` in Line 4.

For the access latencies of all basic blocks, map `mBBAccessLatency` is cleared first. After that, each basic block of the current function is iterated, its logical section and then its memory region are determined (Line 9). Finally, the region's latency is stored in the map under the current basic block's ID (Line 10).

These pre-computed access latencies can finally be used for spill cost computation:

```

1 unsigned int TC_GraphColoring::getSpillLoadCosts( const WIR_RegisterParameter &p ) const
2 {
3     auto &b = p.getOperation().getInstruction().getBasicBlock();
4     unsigned int cost = mStackAccessLatency + mBBAccessLatency.at( b.getID() );
5
6     auto &r = dynamic_cast<WIR_VirtualRegister &>( p.getRegister() );
7
8     if ( ( r.getType() == TC13::RegisterType::eReg ) || ( r.getType() == TC13::RegisterType::pReg ) )
9         return( cost + mStackAccessLatency );
10
11     return( cost );
12 };
13
14 unsigned int TC_GraphColoring::getSpillStoreCosts( const WIR_RegisterParameter &p ) const
15 {
16     return( getSpillLoadCosts( p ) );
17 };

```

Line 4 computes a load's basic spill costs consisting of the stack's access latency plus the latency of that memory region to which the basic block containing register parameter `p` to be spilled is allocated. As loading a TriCore extended register from the stack performs a total of two memory accesses, the stack latency is added another time to the returned cost in this case in Line 9. Method `getSpillStoreCosts` simply calls `getSpillLoadCosts`, because the costs for loads and stored do not differ for the given example.

```

1 unsigned int TC_GraphColoring::getMoveCosts( const WIR_Operation &o ) const
2 {
3     ufAssert( ( o.getOpCode() == TC13::OpCode::MOV_RR ) || ( o.getOpCode() == TC13::OpCode::MOV_AA ) );
4     return( mBBAccessLatency.at( o.getInstruction().getBasicBlock().getID() ) );
5 };

```

Finally, `getMoveCosts` ensures that the given `WIR` operation is a move, and the returned costs again boil down to the access latency of the memory containing the move.

18.1.5 Color Selection

The selection of actual colors for an `IG` node is done by the purely virtual method `selectColors`. Thus, a retargeted allocator's header file has to be extended by the following declaration:

```

1 virtual WIR_GraphColoring::WIR_ColorMap selectColors( const std::vector<std::reference_wrapper<
2     WIR_VirtualRegister>> &leafs,
3     WIR_InterferenceGraph &igraph ) override;

```

According to Section 18.1.2, `IG` nodes can be either non-hierarchical virtual registers or roots of a hierarchical virtual register. In the latter case, such a node contains more than one leaf registers as shown in Fig. 18.2. `selectColors` gets a vector `leafs` as first argument that contains references to all leaf registers of the `IG` node to be colored. For the TriCore example from Fig. 18.1, `selectColors` would be called with a vector of length 1 containing only `d_6105`. For Fig. 18.2, however, `leafs` would contain the two elements `d_6117` and `d_6119` if `IG` node `e_6115` is supposed to be colored.

The current `IG` is passed as second argument to `selectColors`. The method returns a color map assigning a unique color to each register from vector `leafs`. A color map is basically defined as follows:

```
1 using WIR_ColorMap = std::map<std::reference_wrapper<WIR_VirtualRegister>, unsigned int>;
```

For a (reference to a) leaf register, the color map stores the number of the assigned color. The following invariants must hold for a color map returned by `selectColors`:

- The color map either contains exactly one entry for each leaf register from vector `leafs`, or it is completely empty. The latter situation means that no valid colors were found in the current `IG` for the considered `IG` node so that it will become an actual spill.
- No color stored in the color map must already be used for an `IG` node adjacent to the current node.

The implementation of `selectColors` is entirely left to the developer, as long as the above invariants are respected. It is good practice to use method `WIR_InterferenceGraph::getPossibleColors` in order to address the second invariant above. For an `IG` node represented by a virtual leaf register, `getPossibleColors` returns a set of unsigned integers/colors such that none of these returned colors is used for adjacent nodes.

For the TriCore architecture, `selectColors` first of all retrieves the set of possible colors exactly in this way. After that, the two cases from Figs. 18.1 and 18.2 are distinguished. For just one leaf node to be colored, `selectColors` uses a list of feasible physical data or address registers that is ordered according to a processor-specific precedence order. E.g., the physical data registers D15 down to D8 shall be used first in this order for coloring whenever possible. With lower priority, registers D1, D0 and so on should be used. By iterating through this precedence list and checking whether the physical register's color actually is possible, the color selection is performed this way. For hierarchical registers with two leafs, a similar precedence order of matching physical extended registers is iterated and it is checked whether the colors of both physical leaf registers are actually available within the current `IG`.

18.1.6 Spill Code Generation

The generation of spill code for an `IG` node marked as actual spill in principle boils down to creating a spill-store after each definition of a virtual register, and to inserting a spill-load before each use of a virtual register. This is illustrated using the following TriCore example where virtual register `d_42` is assumed to be an actual spill:

```
1 add    %d_42, %d_6115, #364      # d_42 := d_6115 + 364, definition of d_42.
2 [...]
3 mul    %d_6117, %d_42, %d_123    # d_6117 := d_42 * d_123, use of d_42.
4 [...]
5 sh     %d_42, #4                 # d_42 := d_42 >> 4, definition and use of d_42.
```

By inserting spill-stores after `add` and `sh` and spill-loads before `mul` and `sh`, the live range of the original virtual register `d_42` – which originally covers the entire code example above – gets split into three distinct live ranges which are much smaller. For each of these smaller live ranges of `d_42`, the `WIR` register allocator creates a new virtual register which is called a spill-clone of `d_42`. For the above example, we assume that spill-clones named `d_9000`, `d_9001` and `d_9002` are introduced. Spilling would thus transform the above code to:

```
1 add    %d_9000, %d_6115, #364    # d_9000 := d_6115 + 364.
2 st.w   [%a10] 0, %d_9000        # Spill-store d_9000 alias d_42 to stack-position 0.
3 [...]
4 ld.w   %d_9001, [%a10] 0        # Spill-load d_9001 alias d_42 from stack-position 0.
5 mul    %d_6117, %d_9001, %d_123  # d_6117 := d_9001 * d_123.
6 [...]
7 ld.w   %d_9002, [%a10] 0        # Spill-load d_9002 alias d_42 from stack-position 0.
8 sh     %d_9002, #4              # d_9002 := d_9002 >> 4.
9 st.w   [%a10] $0$, %d_9002      # Spill-store d_9002 alias d_42 to stack-position 0.
```

Note that spilling does not replace any virtual register marked as actual spill by some kind of physical register – the created spill-clones are all virtual registers which require some color assignment later on. Due to the fact that the original register's live range has been split significantly, it is quite likely that the virtual spill-clones can be colored successfully. This is the reason why the general graph coloring algorithm sketched on Page 257 features Step 9: since spill code generation introduces other virtual registers, a start over of the entire algorithm is required.

The actual generation of such spill code is done by the purely virtual methods `insertSpillLoad` and `insertSpillStore`. When retargeting the `WIR` register allocator, both of them thus have to be overloaded and the new register allocator's header file has to be extended by the following method declarations:

```

1 virtual void insertSpillLoad( const WIR_BaseRegister &clone, const WIR_BaseRegister &r, int stackPos,
2                             WIR_BasicBlock &b,
3                             std::list<
4                             std::reference_wrapper<WIR_Instruction>>::const_iterator pos ) override;
5
6 virtual void insertSpillStore( const WIR_BaseRegister &clone, const WIR_BaseRegister &r, int stackPos,
7                               WIR_BasicBlock &b,
8                               std::list<
9                               std::reference_wrapper<WIR_Instruction>>::const_iterator pos ) override;

```

These two methods expect references to both the spill-clone `clone` and the original actual spill register `r` as first two arguments. The latter register `r` is provided mostly for debugging purposes, the actual spill code generation usually only deals with `clone`. The next integer argument denotes the stack pointer-relative offset to or from which the register has to be spilled. The spill position, i.e., the location where the actual spill occurs as definition or use, is specified by arguments `b` and `pos`. The former is a reference to the `WIR` basic block containing the spill position, and the latter is an iterator that refers to the position in the basic block's instruction sequence before which the generated spill code shall be inserted. Using these arguments, `insertSpillLoad` inserts code realizing one single spill-load, e.g., the one from Line 4 above. Likewise, `insertSpillStore` generates code for spill-storing exactly one definition of an actual spill.

However, the actual task to create one single spill is more complex than it seems at first glance. A first aspect to be considered by `insertSpillLoad` and `insertSpillStore` are the various types of registers that an architecture might feature. E.g., the TriCore ISA provides different load and store operations for address, data or extended registers. Depending on the type of a virtual register to be spilled, different flavors of spill-loads and spill-stores need to be created.

Furthermore, the stack pointer-relative offset `stackPos` passed to `insertSpillLoad` and `insertSpillStore` is fully unaware of a machine's addressing modes and other ISA-related constraints such as bit widths etc. For example, the `st.w` and `ld.w` operations from the above code use the TriCore's base+offset addressing mode allowing signed stack offsets of up to 16 bits width. If, however, `stackPos` becomes larger than these 16 bits, both spilling methods must generate extra code computing a pointer to the desired spill location on the stack. Alternatively, the stack pointer might need to be adjusted temporarily so that the spill's stack location becomes within reach using 16 bits wide offsets. This stack pointer adjustment of course needs to be reverted after the spill instruction.

Finally, spilling might also become difficult depending on the position `pos` where to insert the spill code. This is especially the case if `pos` is the `std::end` iterator, i.e., if spill code shall be inserted at the very end of basic block `b`. In this case, care has to be taken if `b` ends with some branch or call operation. This prevents to insert the spill code into `b`, because branches or calls always have to be the very last operation of a basic block. The implication of this is that the spill code needs to be inserted into the successor block of `b` in the CFG. Attention then has to be paid if `b` has several successors, or if `b` has only one successor that can, however, be also reached from other basic blocks. Potentially, `insertSpillLoad` and `insertSpillStore` have to create a new basic block to accomodate the spill code, and this new basic block must be properly inserted into the CFG.

In any case, the [WIR](#) graph coloring algorithm requires that `insertSpillLoad` stores all newly created instructions for spill-loading a register in a map `mSpillLoads`, and similarly for `insertSpillStore` and map `mSpillStores`. This includes all instructions to compute large stack offsets or to set the stack pointer back and forth, in addition to the actual spill-loads and -stores themselves. Maps `mSpillLoads` and `mSpillStores` are declared as `std::map<WIR_id_t, std::list<std::list<std::reference_wrapper<WIR_Instruction>>>>`. They map the unique ID of the root register of the actual spill `r` to a list of spills. For each spill position of `r` in the [WIR](#) code, a new entry is added to this list of spills. And one such entry in turn consists of a list of (references to) potentially several instructions created for one spill position.

The graph coloring algorithm supports any arbitrary kind of hierarchical registers, also during spilling. This implies that proper stack locations have to be determined also for child registers if a hierarchical register should be spilled. This is the purpose of the purely virtual method `getStackPosOfSubReg`. The new register allocator's header file thus has to be extended by the following declaration:

```
1 virtual unsigned int getStackPosOfSubReg( const WIR_VirtualRegister &r,
2                                         unsigned int rootPos ) const override;
```

This method gets a reference to a virtual child register `r` as well as the stack position of the root of `r`'s hierarchy as argument. It returns the processor-specific stack position of child `r` based on where on the stack the whole register hierarchy resides.

In exceptional cases, it can happen that the graph coloring algorithm's main loop (from Steps 1 to 9 on Page 257) is unable to color to all [IG](#) nodes. This can happen only for the very short code sequences ranging from a spill-load to an actual spill's use, or from an actual spill's definition to its spill-store. The purpose of Step 10 thus is to also color these remaining uncolored actual spills. For each uncolored actual spill, this happens by selecting one currently occupied physical register, spilling it out onto the stack, using it to color the uncolored spill, and finally spilling in the original value of the physical register.

This task is supported by the purely virtual methods `getCandidatePhregs` and `getCandidatePhreg` which thus also have to be overloaded by a retargeted graph coloring-based register allocator. For this reason, the allocator's header file needs to be extended as follows:

```
1 virtual WIR_PhysicalRegisterSet getCandidatePhregs( const WIR_VirtualRegister &r ) override;
2
3 virtual const WIR_PhysicalRegister &getCandidatePhreg( const WIR_PhysicalRegisterSet &candidates ) override;
```

For a still uncolored actual spill represented by virtual register `r`, `getCandidatePhregs` returns a set of possible physical registers than can potentially be used to color `r`. It is important to note here that `getCandidatePhregs` does not consider any coloring constraints encoded by the [IG](#) (cf. Definition 14.2) at all. Instead, this method only checks processor-specific constraints such as register types. E.g., for the [TriCore](#) example, if the actually spilled register `r` represents a virtual data register, `getCandidatePhregs` returns a set containing all physical data registers of the [TriCore](#) architecture. Since the graph coloring algorithm failed to color the actual spill `r`, the [IG](#) node representing `r` must have so many (colored) neighbors so that no color remains for `r` itself. Due to this, it would not make sense to consider the current [IG](#) in `getCandidatePhregs`. Furthermore, `getCandidatePhregs` returns an unordered set of (references to) physical candidate registers. This implies that iterating through this returned set does not happen in a certain order, i.e., no kind of precedence of one physical candidate register over another is provided by `getCandidatePhregs`.

Among all possible candidates provided by `getCandidatePhregs`, the generic graph coloring algorithm eliminates those leading to high costs if they were used to color virtual register `r`. After this, the set of candidate registers contains one or several physical registers with the same, minimal costs. In order to break the tie, this current set of candidate registers is passed to `getCandidatePhreg` which selects one physical register from

the set of candidates and returns it. Here, processor-specific precedence rules can be implemented which give priority to one register over another.

After all uncolored spills have been colored, the last Step 12 of the algorithm from Page 257 is to optimize the finally generated spill code by removing redundant spills. Spills are redundant in the following situations:

- A physical register is only spill-stored to a certain stack location, but never spill-loaded from it (due to rematerialization, cf. Section 18.1.7). Then, the spill-stores are redundant.
- If a spill-load of a physical register from a stack location is immediately followed by a spill-store of the same physical register to the same stack location, the spill-store is redundant.
- Likewise for a spill-store directly followed by a spill-load, turning the spill-load redundant.
- Consecutive sequences of two spill-stores (or two spill-loads) of the same physical register to/from the same stack position are also redundant, one spill from the sequence can be eliminated.

As discussed earlier in this section, a spill can consist of several WIR instructions performing address arithmetics and/or stack pointer adjustment, besides the actual load or store operation. Due to the complexity of such potential code patterns, it is not clear a priori which physical register actually gets spilled by a sequence of spill instructions. For this reason, the purely virtual method `getPhregOfSpill` is used while removing redundant spills. As a consequence, the header file of a retargeted register allocator needs to be extended as follows:

```
1 virtual WIR_PhysicalRegister &getPhregOfSpill( const std::list<std::reference_wrapper<WIR_Instruction>>
    &spill ) override;
```

The processor-specific implementation of this method examines the list spill of spill instructions and compares it with all code patterns that methods `insertSpillLoad` or `insertSpillStore` could actually produce. By exploiting these code patterns, the physical register that actually gets spilled is returned by `getPhregOfSpill`.

18.1.7 Rematerialization

An improved variant of spilling is rematerialization. Spill-storing a virtual register and later loading it from the system's stack is quite costly. In some cases, it might be cheaper to simply recompute the virtual register's value instead of spilling it in and out. This especially holds if the operation(s) required for such a recomputation do not result in memory accesses themselves, i.e., if these rematerialization operations do not depend on spilled registers in turn.

A typical example where rematerialization is worthwhile are virtual registers that simply contain a constant value. Here, spill-storing this constant and loading it back from the system's stack is overly inefficient. Instead, just generating the one or two operations required to recreate the constant again is usually much cheaper.

In order to enable rematerialization for a retargeted graph coloring-based register allocator, the virtual methods `getRematerializationCosts` and `getRematerializationInstructions` have to be overloaded by extending its header file as follows:

```
1 virtual unsigned int getRematerializationCosts( const WIR_RegisterParameter &p ) const override;
2
3 virtual std::list<WIR_Instruction*> getRematerializationInstructions( const WIR_RegisterParameter &p )
    const override;
```

For a given virtual register parameter `p`, `getRematerializationCosts` returns the costs if `p` gets rematerialized. In analogy to the spill cost computation already discussed in Section 18.1.4, it is left fully open what such rematerialization costs shall represent for a concrete processor architecture.

Referring the running TriCore example, spill and rematerialization costs model the additional latency in clock cycles due to the newly inserted code. For the sake of simplicity, the TriCore implementation only rematerializes register parameters that contain a known constant number. In this case, the rematerialization costs boil down to the latencies of one or two TriCore operations that reload the register with its constant value.

`getRematerializationInstructions` returns the list of [WIR](#) instructions that are needed to rematerialize the given register parameter `p`. The code patterns that can be created by `getRematerializationInstructions` are finally also subject to the removal of redundant spill code as discussed at the end of Section [18.1.6](#). Thus, method `getPhregOfSpill` should also be extended such that it recognizes rematerialization code for a specific processor architecture and also returns the physical register that is actually rematerialized by some spill code.

18.1.8 Cross-Cutting Issues

All the retargeting actions described previously have a clear relation to one of the graph coloring algorithm's main steps listed on Page [257](#). However, porting the [WIR](#) register allocator to another processor architecture poses additional technical challenges that cannot be attributed to the algorithm's main steps. Instead, some issues have a cross-cutting nature so that various different parts of the allocator might need to be touched during retargeting. Prime examples are the correct handling of virtual registers that are alive during function calls (cf. Section [18.1.8.1](#)) and stack frame management (cf. Section [18.1.8.2](#)). Finally, Section [18.1.8.3](#) briefly lists additional places inside the [WIR](#) graph coloring algorithm where processor-specific code could be added.

18.1.8.1 Calling Conventions and Registers Alive During Function Calls

[ABI](#) specifications regularly define which registers have to be used to pass arguments and return values for a function call, and which registers are caller- and callee-saved. Such calling conventions of course must be respected by the [WIR](#) register allocator, requiring processor-specific extensions at various places.

In order to motivate the issues related to calling conventions and function calls, the below example shows a simple TriCore code snippet before register allocation:

```

1 main:  [...]
2      mov    %d_6077, #42
3      ld.w   %d_6082, <addr1>
4      ld.w   %d_6093, <addr2>
5      mov    %d_6104/D4, %d_6082
6      mov    %d_6109/D5, %d_6093
7      call   foo
8 .L6122: mov    %d_6133, %d_6126/D2
9      mul    %d_6138, %d_6077, %d_6133
10     [...]

```

The instructions from Lines [2](#) to [4](#) load virtual registers `d_6077`, `d_6082` and `d_6093` with some values. According to the TriCore calling conventions, physical registers `D4` and `D5` serve argument passing, `D2` for return value passing. Thus, the operations from Lines [5](#) and [6](#) provide the arguments for some function `foo` by moving virtual register `d_6082` (`d_6093`, resp.) to register `d_6104` which is precolored with `D4` (`d_6109` and `D5`, resp.). After that, function `foo` is called in Line [7](#).

The result returned by `foo` must reside in physical register `D2` so that Line [8](#) stores it by moving `d_6126` precolored with `D2` to virtual register `d_6133`. Finally, some other operations follow in Line [9](#) and later.

The first important point visible from this example is that the plain function call from Line [7](#) by itself does not provide any information about its adherence to some calling conventions. In particular, the `call` operation shown above is

fully unaware of which registers are read and written in the context of this function call. However, such information is crucial for a proper liveness analysis and the correctness of register allocation. The **WIR** register allocator has no means to reverse-engineer this information from the code and to validate that a call complies to a given calling convention. Due to this, the register allocator expects that every function call comes with implicit register parameters (cf. Section 8.5) that read and write all registers involved in argument and return value passing.

When including these implicit parameters, the output of the call operation would look as follows:

```
1      [...]
2      call    foo, %d_6104/D4 (use), %d_6109/D5 (use), %d_6126/D2 (def)
3      [...]
```

Note

As motivated above, the register allocator is unable to determine registers used and defined by a function call according to some calling convention. It thus lies in the responsibility of **WIR**-based compiler passes prior to register allocation to add this information in the form of implicit parameters to all function calls.

The second crucial issue demonstrated by the above example is that virtual registers can be alive across a function call. This is the case for `d_6077` which is defined in Line 2 before the call and used in Line 9 after the call. If the register allocator decides during Step 6 from Page 257 to allocate `d_6077` to some caller-saved physical register, e.g., `D0`, the function call might potentially overwrite `D0` and this way invalidate the contents of `d_6077`.

In order to prevent such an illegal overwriting of the caller-saved register, the register allocator has to insert a spill-store of `d_6077/D0` before the call, and a spill-load right after the call, as shown in the following code:

```
1 main:  [...]
2      mov     %d_6077/D0, #42
3      ld.w    %d_6082, <addr1>
4      ld.w    %d_6093, <addr2>
5      st.w    <addr3>, %d_6077/D0          # Spill-store of caller-saved register D0.
6      mov     %d_6104/D4, %d_6082
7      mov     %d_6109/D5, %d_6093
8      call    foo                          # Function call potentially overwriting D0.
9 .L6122: mov   %d_6133, %d_6126/D2
10     ld.w    %d_6077/D0, <addr3>          # Spill-load of caller-saved register D0.
11     mul     %d_6138, %d_6077/D0, %d_6133
12     [...]
```

By introducing the load and store operations from Lines 5 and 10, the register's contents is saved and restored in the context of the call which results in a proper implementation of the caller-saved policy mandated by the **ABI**.

For the proper handling of such situations, the register allocator obviously needs to be aware of whether a given register is caller-saved or not. For this purpose, the purely virtual method `isCallerSaved` has to be provided by extending the new allocator's header file as follows:

```
1 virtual bool isCallerSaved( const WIR_PhysicalRegister &r ) const override;
```

This method simply returns `true` if the given physical register `r` is caller-saved, and `false` otherwise.

Another required Boolean predicate to be overloaded is `isFunctionReturnMove` below:

```
1 virtual bool isFunctionReturnMove( const WIR_Operation &o ) const override;
```

It returns `true` iff the given operation `o` is a move that stores a function's return value in some other register. More precisely, this method indicates moves that immediately follow a call operation and where the used register parameter is a return value passing register according to some processor's **ABI**. E.g., this method is used to

identify the move from Line 9 so that caller-saving spill-loads can be placed right after it. Furthermore, coalescing has to be avoided for such particular moves, which is a second use case of `isFunctionReturnMove`.

The most important method that a retargeted register allocator requires for the handling of caller-saved registers is called `checkCallerSavedRegsAliveAcrossCall`. Since the argument list of this method already might depend on processor-specific details, this method is not a classical virtual method that needs to be overloaded. In the following, we are explaining the purpose of this method by using the TriCore implementation of the graph coloring-based register allocator as running example. Here, this method is declared as follows in the allocator's header file:

```
1 void checkCallerSavedRegsAliveAcrossCall( const WIR_PhysicalRegister &phreg,
2                                           const WIR_RegisterSet &vregs_c1,
3                                           const WIR_RegisterSet &vregs_c2 );
```

An **IG** node represents either a non-hierarchical virtual register or the root of a complex register hierarchy. In the latter case, such a root node can have several leaf registers. Method `checkCallerSavedRegsAliveAcrossCall` expects all these leafs of an **IG** node as arguments `vregs_c1`, `vregs_c2`, ... Furthermore, coalescing can have the effect that a single virtual register can have several aliases that stem from merging move-related **IG** nodes. For this reason, these leaf registers are given as sets of virtual registers.

For the above TriCore example, `vregs_c1` contains the virtual register being the first leaf of an **IG** node, including its coalescing-related aliases. If an **IG** node represents a non-hierarchical virtual register, set `vregs_c2` is empty. Otherwise, it contains the second virtual leaf register of the **IG** node and its respective aliases.

If the **IG** node specified this way is assigned to a caller-saved physical register `phreg`, and if the virtual registers from sets `vregs_c1` and `vregs_c2` are alive across a function call, then the caller-saving spills have to be created as discussed on Page 272. The purpose of method `checkCallerSavedRegsAliveAcrossCall` is to identify such situations and to keep track of which registers need to be spill-stored and -loaded before and after a call.

By exploiting the generic nature of **WIR** analyses, the register allocator's processor-independent base class computes a map `mVregsAliveAcrossCall` that maps each **WIR** instruction containing a function call to the set of virtual registers that are alive across this call. This map is basically declared as `std::map<std::reference_wrapper<WIR_Instruction>, WIR_VirtualRegisterSet>`. For the example from Page 271, map `mVregsAliveAcrossCall` contains an entry for the call from Line 7 that states that the registers from set `{d_6077}` are alive across this call.

If physical register `phreg` is caller-saved, method `checkCallerSavedRegsAliveAcrossCall` then traverses map `mVregsAliveAcrossCall` and checks whether any virtual register from sets `vregs_c1` or `vregs_c2` shows up in the current map entry. If this is the case, `checkCallerSavedRegsAliveAcrossCall` stores in map `mLocalCallerSavedRegsAliveAcrossCall` that the physical register assigned to the currently considered virtual leaf requires caller-saving at the call of the current map entry.

Note that `checkCallerSavedRegsAliveAcrossCall` does not generate the actual spill code implementing the caller-saved policy. This is performed during Step 11 of the entire algorithm from Page 257. However, a special case arises if `phreg` is one of the return-value passing registers and the function call implicitly overwrites this physical register as shown in the following example:

```
1 main:  [...]
2      mov    %d_6077/D2, #42
3      st.w   <addr3>, %d_6077/D2          # Spill-store of caller-saved register D2.
4      call   foo, %d_6126/D2 (def)        # Function call overwriting D2.
5 .L6122: mov    %d_6133, %d_6126/D2
6      ld.w   %d_6077/D2, <addr3>          # Spill-load of caller-saved register D2.
7      mul    %d_6138, %d_6077/D2, %d_6133
8      [...]
```

The spill-store and the spill-load from Lines 3 and 6 ensure the semantical correctness of the code during the function call in Line 4. However, while allocating IG nodes to physical registers, these two spills are not yet present in the WIR code, leading to wrong def-use relationships. In particular, the mov from Line 2 writes some value to d_6077/D2 that gets immediately overwritten by the call from Line 4. The definition of d_6077/D2 in Line 2 does not reach a proper use if the spill instructions were not present in the above code.

For this reason, `checkCallerSavedRegsAliveAcrossCall` also has the task to identify such special situations and to store that the function call shall be extended by another implicit parameter using d_6077/D2:

```

1      [...]
2      call    foo, %d_6126/D2 (def), D2 (use)
3      [...]

```

This is achieved by adding a new entry to list `mLocalDummyParameters` consisting of a pair with the physical register (D2 in the current example) and the call from Line 4.

This method `checkCallerSavedRegsAliveAcrossCall` now needs to be used in two places. The first is inside method `selectColors` where a color map is computed for the leaf registers of some IG node (cf. Section 18.1.5). For each entry that `selectColors` adds to the resulting color map, `checkCallerSavedRegsAliveAcrossCall` needs to be invoked.

The second place is inside method `getCandidatePhreg` that selects a physical register to be used for some uncolored actual spill (cf. Line 3 on Page 269). For the physical register determined here, again `checkCallerSavedRegsAliveAcrossCall` has to be called.

The actual generation of the loads and stores implementing a caller-saving policy is done in a processor-independent way by method `applyCallerSavingPolicy` of the generic base class `WIR_GraphColoring`. This method is virtual so that it can be overloaded by derived register allocators if the realized default implementation is insufficient for some actual processor architecture.

18.1.8.2 Stack Frame Management

The generation of spill code implies that additional space needs to be reserved within the stack frame of the currently allocated WIR function. This is the purpose of the purely virtual method `adjustStack` which thus has to be overridden as follows by derived register allocators:

```

1  virtual void adjustStack( WIR_Function &f ) override;

```

This method gets a reference to the currently allocated WIR function `f` as argument. Using internal attributes of the register allocator such as `mAdditionalStackSize` (an unsigned integer storing how many additional bytes are required in the stack frame for spilling) or `mInsertedSpillCode` (a list of references to all spill instructions inside `f`), the WIR code of `f` gets adjusted.

As already stated earlier (cf. the note on future work on Page 45), WIR currently provides no structured model of a function's stack frame layout. This makes the adjustment of the stack frame a more or less tedious task.

The first step is to identify the very first instruction(s) inside function `f` that allocate the current stack frame by decreasing or increasing the stack pointer by a given number of bytes. In their current implementations for RISC-V and TriCore, `adjustStack` simply matches a few typical, hard-coded patterns for stack allocating code and extracts the current stack frame size from them.

Next, the additional stack capacity from `mAdditionalStackSize` is added to the current stack frame size, where processor-specific alignment requirements might apply. The identified stack allocating code is then modified such that the stack pointer is increased/decreased by the new byte size of the stack frame.

The rest of `adjustStack` deals with potential adjustments of existing accesses to the stack frame within the `WIR` code of `f`. Here, issues can arise depending on the strategic decision where in a function's stack frame the spill area shall be. The generic register allocator uses stack pointer-relative offsets for spills beginning with 0. These offsets are passed as argument `stackPos` to methods `insertSpillLoad` and `insertSpillStore` (cf. Section 18.1.6). Assuming a stack growing downwards to the lower addresses, such a scheme places spills at the bottom of a stack frame, very close to the stack pointer. This is beneficial for architectures supporting only a rather narrow range of offsets that can be used by stack pointer-relative load or store operations.

However, many ABIs [25, 30] mandate that this region at the very bottom of a stack frame is exclusively reserved for so-called overflow arguments. Thus, the current implementations of the RISC-V and TriCore register allocators keep the overflow region at the bottom of the stack frame, followed by the region for spills residing above it, and finally followed by the region containing the local data of function `f` at the top of the stack frame.

Placing the spill region “in the middle” of the stack frame leads to the following issues:

- The offsets of spill instructions in general do not start with 0, but must be adjusted such that they start from the actual size of the overflow region.
- The stack region containing the function's local data is shifted upwards in the stack frame by the size of the spilling region, this way invalidating the offsets of all existing stack accesses.

The first issue is solved within the RISC-V- and TriCore-specific implementations of methods `insertSpillLoad` and `insertSpillStore`. These implementations assume that the size of the overflow region has been stored somewhere before register allocation using method `setFrameSize` (cf. Listing 8.6 on Page 44). By using `getFrameSize`, the methods `insertSpillLoad` and `insertSpillStore` are able to add the size of this overflow region to the stack offset from `stackPos` and this way generate appropriate spill code accessing the region above the overflow region.

The second issue is resolved within `adjustStack` by iterating over the entire code of function `f`, identifying all accesses to the local data region, and adjusting their stack offsets by the size of the spill region. During this process, however, any spill operations or accesses to the overflow region must not be adjusted, since their stack offsets are already correct. The current RISC-V and TriCore implementations of `adjustStack` thus assume that operations accessing the overflow region are marked by locking at least one of its parameters with `setDontOptimize` (cf. Listing 6.4 on Page 33).

Future Work

The authors of the `WIR` software and of this document are fully aware of the ugliness of this part of the register allocator.

One solution to getting rid of the need to adjust existing accesses to the local data stack region would be to allocate the spill region at the top of the stack frame. This could, however, result in rather large offsets of spill code which go beyond what typical addressing modes could support. Supporting such large stack offsets could potentially result in lots of additional code having a severe performance and code size impact.

The best solution is to add a well-designed model of stack frames and their internal organization to `WIR` functions. This is planned for the near future.

18.1.8.3 Miscellaneous

In addition to all the virtual methods of the `WIR` register allocator discussed up to this point, class `WIR_GraphColoring` provides many more opportunities to tweak the allocator's behavior for a specific processor architecture. This can be achieved by overloading the following virtual methods so that processor-specific behavior can be

realized instead of the allocator's default implementations:

```

1 virtual bool isPriorityRegister( const WIR_VirtualRegister &r ) const;
2 virtual WIR_Operation &selectCoalescingCandidate( const WIR_InterferenceGraph &igraph,
3                                                  const WIR_OperationSet &workListMoves ) const;
4 virtual WIR_VirtualRegister &selectSpillCandidate( const WIR_InterferenceGraph &igraph,
5                                                  const WIR_VirtualRegisterSet &workListSpill ) const;
6 virtual unsigned int getStackSize( const WIR_BaseRegister &r ) const;
7 virtual void postRACleanup( WIR_Function &f );

```

Method `isPriorityRegister` (Line 1) serves to identify virtual registers `r` that shall get a high priority during graph simplification (Step 2 from Page 257). A high priority means that such a virtual register is put at the very front of the allocator's worklists so that it is likely (though not guaranteed) that a free color can be found. In its default implementation, `isPriorityRegister` always returns false. As an example, it is overloaded by the TriCore register allocator such that registers serving as counters in loop operations get high priority and thus most likely are not spilled.

During Step 3 from Page 257, one move operation out of possibly many ones needs to be selected for coalescing. For this purpose, `selectCoalescingCandidate` (Line 2) is called with the current `IG` and the set of move operations being coalescing candidates as arguments. In its default implementation, this method chooses the move with the highest loop nesting depth. Like this, the allocator strives to keep the performance-critical bodies of innermost loops free of unnecessary moves whenever possible.

Method `selectSpillCandidate` (Line 4) is of similar nature. During spilling (Step 4 from Page 257), one virtual register among all `IG` nodes with significant degree has to be selected, which will afterwards become a potential spill. For this purpose, `selectSpillCandidate` gets called with the current `IG` and the set of registers being spill candidates as arguments. Using Chaitin's or Bernstein's spilling heuristics [5, 3], the method in its default implementation determines the virtual register with minimal spill costs and returns it.

The spill code generation from Step 7 on Page 257 needs the information how many bytes a spilled register `r` occupies on the stack. This information is provided by method `getStackSize` from Line 6. By default, this method returns the register's bit width divided by 8. Again, this method can be overloaded if other implementations are required.

At the very end of the register allocation algorithm, after Step 12 from Page 257, virtual method `postRACleanup` can be overloaded for processor-specific final cleanups of the allocated `WIR` code. By default, this method does nothing. In the derived allocators for the RISC-V and TriCore architectures, it detects and eliminates redundant move operations where source and target are the same register due to coalescing. Furthermore, any locks of parameters set using `setDontOptimize` (cf. Listing 6.4) are reset.

Finally, the `WIR` register allocator provides dedicated places – so-called hooks – where processor-specific code can be added explicitly after particular steps of the overall algorithm from Page 257. These hooks are virtual methods of class `WIR_GraphColoring` that do absolutely nothing by default. They are listed below:

```

1 virtual void initializationHook( WIR_Function &f );
2 virtual void saveBestSolutionHook( void );
3 virtual void restoreBestSolutionHook( void );
4 virtual void rewriteProgramHook( WIR_Function &f );
5 virtual void postProcessingHook( WIR_Function &f );

```

The `initializationHook` (Line 1) is executed before creation of the `IG` in Step 1 on Page 257. If a processor-specific register allocator uses additional internal data structures, they can be initialized there.

`saveBestSolutionHook` and `restoreBestSolutionHook` (Lines 2 and 3) come into play when Bernstein's best-of-three spilling is activated using `setBestOfThreeSpilling` (cf. Listing 14.5 on Page 192). In this case,

Steps 1 to 6 are executed in a loop that applies all available spill heuristics and cost estimations to the current IG. At the end of this loop, the allocation leading to overall minimal spill costs is kept and the allocation algorithm continues with Step 7.

When iterating all spill heuristics in this way, the internal state of the allocation produced in the current iteration of this loop has to be saved if it leads to smaller costs compared to all previously iterated spill heuristics. At the end of the loop, the internal state of the allocator needs to be restored to that one producing overall minimal costs. While saving and restoring the allocator's internal state, these two hooks are called so that processor-dependent data structures can also be handled.

Method `rewriteProgramHook` (Line 4) gets executed after Step 8 from Page 257. Once one allocation round of the overall algorithm is done, this hook can be used to fine-tune the spill code created just before in a processor-specific manner.

Finally, after the last Step 12 of the entire algorithm is done, the `postProcessingHook` (Line 5) can be used to tidy up the resulting WIR code if needed.

18.2 Dead Code Elimination

The WIR dead code elimination does not require any specific adaptations for new processor architectures. The only thing that needs to be provided is a bit-true data flow analysis that has been retargeted to the new architecture according to Section 17.1. If this data flow analysis is available, the steps to set up the infrastructure for dead code elimination are rather straightforward:

1. Create a new directory `WIR/arch/archDir/optimizations/deadcode`.
2. Add a new line
`include arch/archDir/optimizations/deadcode/Makefile.src`
to file `WIR/arch/archDir/optimizations/Makefile.src`.
3. Create a new file `WIR/arch/archDir/optimizations/deadcode/Makefile.src` with the following contents:

```
1 libwir_la_SOURCES      += arch/archDir/optimizations/deadcode/newModeldeadcode.cc
   arch/archDir/optimizations/deadcode/newModeldeadcode.h
```

4. Create a file `WIR/arch/archDir/optimizations/deadcode/newModeldeadcode.h` according to the general structure depicted in Listing 18.3.
5. Create a file `WIR/arch/archDir/optimizations/deadcode/newModeldeadcode.cc` according to the general structure depicted in Listing 18.4.

As can be seen from Listings 18.3 and 18.4, the code structure for dead code elimination is minimal, and the constructors simply call the constructors of base classes `WIR_DeadCode` and `WIR_Optimization`.

What is missing in Listing 18.4 is the implementation of the overridden method `runOptimization` from Line 31 of Listing 18.3. The task of this method is to create an instance of the new processor's bit-true data flow analysis and to have it used during the actual dead code elimination. This boils down to the following few lines of code:

Listing 18.3: Header File for a New Processor Model's Dead Code Elimination

```

1  #ifndef _newModel_DEADCODE_H
2  #define _newModel_DEADCODE_H
3
4  // Include WIR headers
5  #include <optimizations/deadcode/wirdeadcode.h>
6
7  namespace WIR {
8
9  class WIR_CompilationUnit;
10 class WIR_Function;
11 class WIR_System;
12
13 class newModel_DeadCode : public WIR_DeadCode
14 {
15     public:
16
17         // Default constructor for system-level optimization.
18         explicit newModel_DeadCode( WIR_System & );
19
20         // Default constructor for compilation unit-level optimization.
21         explicit newModel_DeadCode( WIR_CompilationUnit & );
22
23         // Default constructor for function-level optimization.
24         explicit newModel_DeadCode( WIR_Function & );
25
26         // Destructor.
27         virtual ~newModel_DeadCode( void );
28
29     protected:
30
31         virtual void runOptimization( WIR_Function & ) override;
32 };
33
34 } // namespace WIR
35
36 #endif

```

```

1 void newModel_DeadCode::runOptimization( WIR_Function &f )
2 {
3     newModel_BitDFA analyzer { f };
4     setDFA( analyzer );
5
6     WIR_DeadCode::runOptimization( f );
7 };

```

18.3 Constant Folding

WIR constant folding identifies operations that, under all circumstances, always produce the very same and constant values in registers. It also relies on the bit-true data flow analysis so that the basic steps to retarget constant folding to a new architecture are identical with Section 18.2.¹

In addition to that, the purely virtual method `doConstFolding` has to be provided so that the constant folding's

¹Constant folding uses directory name `constfold`, file names `newModelconstfold.h` and `newModelconstfold.cc`, and class name `newModel_ConstFold`.

Listing 18.4: Rudimentary Implementation of a New Processor Model's Dead Code Elimination

```

1  #ifndef HAVE_CONFIG_H
2  #include <config_wir.h>
3  #endif
4
5  // Include WIR headers
6  #include <wir/wir.h>
7  #include <arch/archDir/newModel.h>
8  #include <arch/archDir/analyses/bit/newModelbitdfa.h>
9
10 // Include local headers
11 #include "newModeldeadcode.h"
12
13 namespace WIR {
14
15 // Default constructor for system-level optimization.
16 newModel_DeadCode::newModel_DeadCode( WIR_System &s ) :
17     WIR_Optimization { s },
18     WIR_DeadCode { s }
19 {};
20
21 // Default constructor for compilation unit-level optimization.
22 newModel_DeadCode::newModel_DeadCode( WIR_CompilationUnit &c ) :
23     WIR_Optimization { c },
24     WIR_DeadCode { c }
25 {};
26
27 // Default constructor for function-level optimization.
28 newModel_DeadCode::newModel_DeadCode( WIR_Function &f ) :
29     WIR_Optimization { f },
30     WIR_DeadCode { f }
31 {};
32
33 // Destructor.
34 newModel_DeadCode::~newModel_DeadCode( void )
35 {};
36
37 } // namespace WIR

```

header file needs to include:

```

1  virtual bool doConstFolding( const WIR_Operation &o,
2                             const std::map<WIR_id_t, WIR_UpDownValue> &outValue,
3                             const std::map<WIR_id_t, WIR_UpDownValue> &inValue ) override;

```

This method receives an operation `o` that has already been identified as constant as first argument. Argument `outValue` is a map that provides the constant up/down values for each register parameter of `o` that is defined or def-used. The keys of this map are the unique IDs of each such parameter. Similarly, argument `inValue` provides the up/down values for all input parameters of `o`, i.e., which are used or def-used.

The purpose of this method is to generate new operations that simply load the known constant values that `o` produces into the respective register. Depending on the size of the constant, this frequently is achieved with one single machine operation (e.g., `mov <reg>, #const` for TriCore or `addi <reg>, x0, const` for RISC-V RV32I). For larger constants, two new operations might result from constant folding (e.g., `movh` and `addi` for TriCore or `lui` and `addi` for RISC-V). These newly created operations are not directly inserted into the [WIR](#) code. Instead, method `doConstFolding` inserts them into map `mNewInstructions` by using operation `o` to be folded

as key.

Constant folding is performed repetitively until no more possible foldings are found. For this reason, `doConstFolding` needs to pay attention that operations that are actually created during folding are not folded once again. If `doConstFolding` has generated some new operations folding `o`, it returns `true`, and `false` otherwise.

It is important that `doConstFolding` keeps the bit-true [DFG](#) correct when it generates new operations. This means on the one hand that if more than one operation is created, the register parameter defined by the first one must get a `WIR_BitValues` container (cf. [Page 182](#)) attached that stores the outgoing [DFG](#) edge from the first to the second operation together with its up/down value. Likewise, the register parameter used by the second new operation needs to obtain a container representing its incoming edge from the first operation with its up/down value.

On the other hand, the fact the operation `o` gets replaced by the newly created operations also needs to be reflected in the [DFG](#). Thus, all register parameters receiving input from `o` along incoming [DFG](#) edges must be updated such that these edges start at the parameter defined by the last new operation.

If branch folding shall also be supported, the virtual method `doBranchFolding` needs to be overloaded as follows:

```
1 virtual bool doBranchFolding( const WIR_Operation &o,
2                             const std::map<WIR_id_t, WIR_UpDownValue> &inValue ) override;
```

As before, an operation `o` and a map storing all incoming up/down values are provided as arguments. This method needs to check whether `o` is a conditional branch and if the incoming up/down values always lead to a satisfied branch condition or always to an unsatisfied condition. In the former case, the conditional branch can be replaced by an unconditional jump. In the latter case, no new operation needs to be generated and `o` can simply be erased since the branch never is taken. Again, `doBranchFolding` stores all newly created operations (or an empty list in the latter case above) in map `mNewInstructions` and returns whether a folding was done or not.

Finally, the virtual method `doMOVFolding` can be provided if move folding shall be realized. The signature of this method is identical to `doBranchFolding` so that we omit its discussion here. Move folding checks whether operation `o` is an arithmetical operation that provably has no effect so that one of its used input registers is simply copied to its defined output register. Examples of such unnecessary arithmetical operations are additions/subtractions with 0, the bitwise and with 1, multiplications or divisions by 1, etc. `doMOVFolding` thus consists of a sequence of if-statements, each of them checking a certain code pattern (e.g., an add where one input operand is 0). Then, map `mNewInstructions` is again extended by a new entry replacing `o` by some register-register move operation.

18.4 Constant Propagation

The constant propagation pass of [WIR](#) aims to replace operations that get constants in register parameters as input by operations using immediate parameters instead. Similar to constant folding from [Section 18.3](#), this optimization makes use of the bit-true data flow analysis, and the steps to retarget constant propagation to a new architecture are the same as described in [Section 18.2](#).²

A retargeted constant propagation has to override the purely virtual method `doConstProp` as follows:

```
1 virtual bool doConstProp( const WIR_Operation &o,
2                           const std::map<WIR_id_t, WIR_UpDownValue> &inValue ) override;
```

²Constant propagation uses directory name `constprop`, file names `newModelconstprop.h` and `newModelconstprop.cc`, and class name `newModel_ConstProp`.

The meaning of this method's arguments and its return value is identical to that of methods `doBranchFolding` or `doMOV Folding` from Section 18.3. Inside this method, operations should be identified that use or def-use a register parameter for which `map inValue` contains a constant incoming up/down value. Such an operation should then be replaced by a variant where the identified constant shows up as immediate parameter instead of a register parameter. This transformation should consider the commutativity of many operations. For example, `add x8, x19, x9` as well as `add x8, x9, x19` should be replaced by `addi x8, x9, 42` if register `x19` is known to have the constant value 42.

Thus, retargeting constant propagation relies on a careful analysis of a processor's ISA and on the identification of pairs of operations once featuring register parameters and once using immediates. Based on this analysis' results, the realization of `doConstProp` is the straightforward implementation of all these identified operation pairs. In analogy to constant folding, care is required for the proper modelling of the bit-true data flow for an operation created during constant propagation. This means that all register and immediate parameters of a newly created operation must come with `WIR_ BitValue` containers, and they have to store the correct up/down values for all DFG edges starting or ending in the new operation.

18.5 Redundant Code Elimination

The last WIR optimization based on the bit-true data flow analysis is redundant code elimination. For a defined register parameter `p1` and a used parameter `p2`, it checks whether the bit-true up/down values leaving `p1` are equal to the incoming up/down values for `p2`, and that the definition in `p1` always reaches the use in `p2`. If this is the case, `p2` gets replaced within its WIR operation by `p1` (cf. Section 14.12). Once more, the actual steps to regartet this optimization pass are identical with those from Section 18.3.

Most parts of this optimization are fully generic and processor-independent. However, there might be processor-specific situations where the replacement of some use `p2` by some definition `p1` has to be avoided. This can be realized by overriding the virtual method `areRedundant` as follows:

```

1 virtual bool areRedundant( const WIR_RegisterParameter &def, const WIR_UpDownValue &outValue,
2                           const WIR_RegisterParameter &use,
3                           const WIR_UpDownValue &inValue ) const override;

```

This method gets the defined register parameter and its associated outgoing up/down value as first arguments, followed by the use and its incoming up/down value. Based on this information, `areRedundant` can check whether the replacement of use by def shall be performed or not by returning `true` and `false`, resp.

Frequently, `areRedundant` suppresses the optimization when the considered register parameters are somehow involved in function calls. As already discussed earlier (cf. Section 18.1.8.1), the handling of registers during function calls is critical and requires the adherence of calling conventions. One typical situation that should not be optimized by redundant code elimination is when `def` is an implicit parameter of a function call operation. This means that `def` serves to pass the called function's return value to the callee. As such return value passing registers regularly are not callee-saved, the physical register associated with `def` could be overwritten by other function calls between `def` and `use`. In such a situation, `areRedundant` might return `false` in order to prevent the optimization.

Another case could arise if `def` writes a register that, according to some calling convention, is used for argument passing during a function call. Here, the operation of `def` is providing a value to be passed to some callee. Again, such argument passing registers regularly are not callee-saved so that it cannot be guaranteed that the value of `def` really reaches `use`. Thus, the optimization might be prevented by `areRedundant` in this situation, too.

Listing 18.5: Header File for a New Processor Model's Jump Correction

```

1  #ifndef _newModel_JUMPCORRECTION_H
2  #define _newModel_JUMPCORRECTION_H
3
4  // Include WIR headers
5  #include <optimizations/jumpcorrection/wirjumpcorrection.h>
6
7  namespace WIR {
8
9  class WIR_BasicBlock;
10 class WIR_System;
11
12 class newModel_JumpCorrection : public WIR_JumpCorrection
13 {
14     public:
15
16         // Default constructor for system-level optimization.
17         explicit newModel_JumpCorrection( WIR_System &s );
18
19         // Destructor.
20         virtual ~newModel_JumpCorrection( void );
21
22         // Determine the costs in terms of byte size of a jump correction.
23         virtual int getJumpCostsSize( const WIR_BasicBlock &s, const WIR_BasicBlock &t,
24                                     bool addToSrc ) override;
25
26         // Determine the costs in terms of clock cycles of a jump correction.
27         virtual int getJumpCostsCycles( const WIR_BasicBlock &s, const WIR_BasicBlock &t,
28                                     bool addToSrc ) override;
29
30     protected:
31
32         virtual void runOpt( WIR_BasicBlock &b ) override;
33 };
34
35 } // namespace WIR
36
37 #endif

```

Finally, the [WIR](#) operation containing use might be of a specific operation format that prevents the application of redundant code elimination. As discussed earlier (cf. Listing 9.23 on Page 81), operation formats can be registered such that only specific physical registers are allowed as register parameters. If, e.g., the format of the operation featuring use mandates that use is the stack pointer, but if def refers some different register, then `areRedundant` should also return `false`.

18.6 Jump Correction

Retargeting the [WIR](#) jump correction to a new processor architecture can be quite laborious. The general steps, however, are very simple and similar to those already elaborated in the previous sections of this chapter:

1. Create a new directory `WIR/arch/archDir/optimizations/jumpcorrection`.
2. Add a new line
`include arch/archDir/optimizations/jumpcorrection/Makefile.src`

Listing 18.6: Rudimentary Implementation of a New Processor Model's Jump Correction

```

1  ifndef HAVE_CONFIG_H
2  #include <config_wir.h>
3  #endif
4
5  // Include WIR headers
6  #include <wir/wir.h>
7  #include <analyses/liveness/wirlivenessanalysis.h>
8  #include <arch/archDir/newModel.h>
9
10 // Include local headers
11 #include "newModeljumpcorrection.h"
12
13 namespace WIR {
14
15 // Default constructor for system-level optimization.
16 newModel_JumpCorrection::newModel_JumpCorrection( WIR_System &s ) :
17     WIR_JumpCorrection { s }
18 {};
19
20 // Destructor.
21 newModel_JumpCorrection::~newModel_JumpCorrection( void )
22 {};
23
24 } // namespace WIR

```

to file WIR/arch/archDir/optimizations/Makefile.src.

3. Create a new file WIR/arch/archDir/optimizations/jumpcorrection/Makefile.src with the following contents:

```

1  libwir_la_SOURCES      += arch/archDir/optimizations/jumpcorrection/newModeljumpcorrection.cc
   arch/archDir/optimizations/jumpcorrection/newModeljumpcorrection.h

```

4. Create a file WIR/arch/archDir/optimizations/jumpcorrection/newModeljumpcorrection.h according to the general structure depicted in Listing 18.5.
5. Create a file WIR/arch/archDir/optimizations/jumpcorrection/newModeljumpcorrection.cc according to the general structure depicted in Listing 18.6.

As can be seen from Line 17 of Listing 18.5, jump correction features only one single constructor taking a whole WIR system as argument. Applying the optimization only locally to, e.g., a single function, is not supported. This is because jump correction heavily uses the WIR memory layout from Section 10.4 that bases on the global, system-wide addresses of basic blocks, regions, and sections.

The jump correction pass inspects all jumps inside the given WIR system, determines the distance that the jump bridges from its source to its jump target, and modifies a jump operation if this jump distance is too large for the jump's actual operation format. Modifying a jump operation can have the effect that memory addresses of some subsequent basic blocks change, rendering other jumps invalid. For this reason, jump correction iterates over a system's jumps until no more corrections were applied.

Such invalid jumps occur especially in the context of optimizations that allocate WIR code to different memory regions that are far apart from each other in a processor's address space [23]. Invalid jumps that are not able to bridge the large distance between such memory regions need to be replaced by long jumps during jump correction. Such memory allocating optimizations might wish to take the overheads due to jump correction into

account. This is supported by methods `getJumpCostsSize` and `getJumpCostsCycles` from Lines 23 and 27 of Listing 18.5.

Both of them are purely virtual in base class `WIR_JumpCorrection` and thus must be overridden in derived classes. They both receive references to the basic blocks `s` and `t` as first arguments, which represent source and target of some jump. `getJumpCostsSize` returns the byte size of the code required for a long jump from `s` to `t`, while `getJumpCostsCycles` returns the number of clock cycles typically required for such a long jump. Depending on the Boolean flag `addToSrc`, both methods return the costs of a long jump attributed either to the jump's start or target basic block.

The main work of jump correction is done by method `runOpt` (cf. Line 32 in Listing 18.5) that has to be overridden, too. This method gets a reference to a basic block `b` as only argument and determines whether its last operation is a jump requiring correction. For this purpose, this method should distinguish between all the different scenarios of a jump (e.g., conditional jump, unconditional jump, function call, return, implicit jump) and call appropriate methods to handle these scenarios. Thus, it is very common to add dedicated methods to a retargeted jump correction class as follows:

```

1 private:
2
3 // Correct an invalid unconditional jump.
4 unsigned int adjustUnconditionalJump( WIR_Operation &o );
5
6 // Correct an invalid implicit jump.
7 unsigned int adjustImplicitJump( WIR_Operation &o );
8
9 // Correct an invalid conditional jump.
10 unsigned int adjustConditionalJump( WIR_Operation &o );

```

All these methods are called with a jump operation that potentially is invalid. They return the number of effectively corrected operations, i.e., 0 if a jump did not need to be corrected. The implementation of these methods heavily depends on the underlying processor architecture – a processor's ISA on the one hand defines which branch distances can be bridged by different branch operations and which distances thus result in an invalid branch. On the other hand, the code patterns required to replace an invalid jump by a long jump are obviously also architecture-dependent. For these reasons, no general rules can be given here how the above methods need to be realized. However, a few typical cases can be exemplified in the following.

1. Correction of unconditional jumps:

- a) *Replacing an unconditional jump with reduced bit width by a regular-width jump:* ISAs with operations of reduced bit width (e.g., 16-bits wide RISC-V RV32IC operations) feature branches that support only a very short distance. Such a branch could be corrected by simply replacing it by a regular branch with native bit width (e.g., a 32-bits wide RV32I branch operation).
- b) *Replacing an unconditional jump by a long indirect jump:* Here, the address of the branch target is loaded into a register using the target block's symbol. Hereafter, an indirect branch to this pre-computed pointer is generated, replacing the original invalid jump.
- c) *Erasing the unconditional jump:* If it turns out that the target of an unconditional jump is the basic block is the direct successor of the branching block (which can happen in the context of memory allocation optimizations), the unconditional jump is not required and can simply be eliminated.

2. Correction of implicit jumps:

- a) *Source basic block coming with no branch:* An implicit jump occurs if the source basic block *s* does not contain any jump operation at its end. Then, the program's execution continues with the very next basic block *t* that follows *s* in memory. If some optimization moves target block *t* away from *s*, an explicit unconditional jump to *t* must be added to the tail of *s*.
 - b) *Source block ending with function call:* A call operation at the end of source block *s* always comes with an implicit jump. This is because the program's execution continues with the direct successor *t* of *s* in memory once the called function returns. If *t* gets allocated elsewhere in memory, correcting the invalid call requires adding a so-called trampoline. This trampoline is a new basic block immediately after *s* holding only the long jump to *t*.
3. *Correction of conditional jumps:* Correcting conditional jumps is the most complicated case, since such jumps regularly have two targets – either the target explicitly given in the jump that is executed when the branch is taken, or the implicit target being the next operation in memory whenever the branch is not taken.
- a) *Same explicit and implicit jump targets:* In such situations, the branch can simply be eliminated.
 - b) *Moved implicit jump target:* If the basic block being a jump's implicit target is allocated elsewhere in memory, the invalid conditional jump has to be corrected by adding a trampoline to this target block immediately after the jump, in analogy to Step 2b) above.
 - c) *Replacing a conditional jump with reduced bit width by a regular-width jump:* If the explicit target of a conditional jump is out of reach, an attempt for correction could be to replace the jump by a regular jump with full bit width in analogy to Step 1a) above.
 - d) *Conditionally reached trampoline:* If the previous case does not work out, an invalid conditional jump to its explicit target has to be corrected by generating a trampoline. In contrast to Steps 2b) and 3b), the challenge here is to find a good position within the CFG where to place this trampoline so that it is reached only from the conditional branch to be corrected.

Ideally, some unconditional jump or a function return can be found that is within reach for the conditional jump to be corrected. In this case, the trampoline can be placed immediately after such an unconditional jump or return, keeping the control flow inherently intact. The original conditional jump then simply needs to be updated to explicitly branch to the trampoline.

If such an ideal position does not exist, the trampoline is placed directly after the invalid conditional jump, before its original implicit successor. In order to keep the control flow correct, the condition of the jump to be corrected needs to be inverted, and the basic block originally being implicit successor becomes the corrected jump's explicit target.

Depending on the nature of the code required to realize a long jump in a given ISA, it is very likely that some free register is required that is used to compute the pointer of the long jump's target. If jump correction is applied before register allocation, a fresh virtual register can simply be created and used. Otherwise, jump correction needs to apply a liveness analysis to determine whether there is some free physical register. If this is the case, the long jump's code can be generated in a straightforward fashion using this physical register.

Otherwise, a currently occupied physical register needs to be chosen for the long jump. This physical register first needs to be spill-stored onto the stack. After that, it can be used by the operations of the long jump. And finally, the original contents of the physical register needs to be restored via a spill-load. Since this spill-load usually comes after the long jump, care needs to be taken when determining the position in the CFG where to put this spill-load. Obviously, the creation of spill code during jump correction involves allocating some more space in the current WIR function's stack frame, plus the potential adjustment of stack offsets (cf. Section 18.1.8.2).

Listing 18.7: Header File for a New Processor Model's Operation Bit Width Reduction

```

1  #ifndef _newModel_OPERATIONBITWIDTHS_H
2  #define _newModel_OPERATIONBITWIDTHS_H
3
4  // Include WIR headers
5  #include <optimizations/operationbitwidths/wioperationbitwidths.h>
6
7  namespace WIR {
8
9  class WIR_CompilationUnit;
10 class WIR_Function;
11 class WIR_Instruction;
12 class WIR_Operation;
13 class WIR_System;
14
15 class newModel_OperationBitWidths : public WIR_OperationBitWidths
16 {
17     public:
18
19         // Default constructor for system-level optimization.
20         explicit newModel_OperationBitWidths( WIR_System &s );
21
22         // Default constructor for compilation unit-level optimization.
23         explicit newModel_OperationBitWidths( WIR_CompilationUnit &c );
24
25         // Default constructor for function-level optimization.
26         explicit newModel_OperationBitWidths( WIR_Function &f );
27
28         // Destructor.
29         virtual ~newModel_OperationBitWidths( void );
30
31     protected:
32
33         // Reduce bit widths of a given operation.
34         virtual std::list<
35             std::reference_wrapper<
36                 WIR_Operation>>::iterator reduce( WIR_Instruction &i,
37                                                     std::list<
38                 std::reference_wrapper<
39                     WIR_Operation>>::const_iterator pos ) const override;
40 };
41
42 } // namespace WIR
43
44 #endif

```

18.7 Operation Bit Width Reduction

The optimization reducing the bit widths of operations for mixed-width ISAs is rather straightforward to retarget to a new processor architecture. As usual, some boilerplate code needs to be set up, as described in the following steps:

1. Create a new directory `WIR/arch/archDir/optimizations/operationbitwidths`.
2. Add a new line
`include arch/archDir/optimizations/operationbitwidths/Makefile.src`

Listing 18.8: Rudimentary Implementation of a New Processor Model's Operation Bit Width Reduction

```

1  #ifdef HAVE_CONFIG_H
2  #include <config_wir.h>
3  #endif
4
5  // Include WIR headers
6  #include <wir/wir.h>
7  #include <arch/archDir/newModel.h>
8
9  // Include local headers
10 #include "newModeloperationbitwidths.h"
11
12 namespace WIR {
13
14 // Default constructor for system-level optimization.
15 newModel_OperationBitWidths::newModel_OperationBitWidths( WIR_System &s ) :
16     WIR_OperationBitWidths { s }
17 {};
18
19 // Default constructor for compilation unit-level optimization.
20 newModel_OperationBitWidths::newModel_OperationBitWidths( WIR_CompilationUnit &c ) :
21     WIR_OperationBitWidths { c }
22 {};
23
24 // Default constructor for function-level optimization.
25 newModel_OperationBitWidths::newModel_OperationBitWidths( WIR_Function &f ) :
26     WIR_OperationBitWidths { f }
27 {};
28
29 // Destructor.
30 newModel_OperationBitWidths::~newModel_OperationBitWidths( void )
31 {};
32
33 } // namespace WIR

```

to file `WIR/arch/archDir/optimizations/Makefile.src`.

3. Create a new file `WIR/arch/archDir/optimizations/operationbitwidths/Makefile.src` with the following contents:

```

1  libwir_la_SOURCES      +=
    arch/archDir/optimizations/operationbitwidths/newModeloperationbitwidths.cc
    arch/archDir/optimizations/operationbitwidths/newModeloperationbitwidths.h

```

4. Create a file `WIR/arch/archDir/optimizations/operationbitwidths/newModeloperationbitwidths.h` according to the general structure depicted in Listing 18.7.
5. Create a file `WIR/arch/archDir/optimizations/operationbitwidths/newModeloperationbitwidths.cc` according to the general structure depicted in Listing 18.8.

The actual task of reducing an operation's bit width is performed by method `reduce` from Line 36 of Listing 18.7. The operation to be examined and reduced is given by a reference to its enclosing `WIR` instruction `i` first, followed by an iterator position `pos` that refers to the position of the operation within `i`'s list of operations. If `reduce` is able to replace this operation by another one with smaller bit width, it returns an iterator to the newly created operation within `i`. Otherwise, it returns the `end()` iterator.

Listing 18.9: Header File for a New Processor Model's Predicated Execution Pass

```

1  #ifndef _newModel_PREDICATEDEXECUTION_H
2  #define _newModel_PREDICATEDEXECUTION_H
3
4  // Include WIR headers
5  #include <optimizations/generic/wiropimization.h>
6
7  namespace WIR {
8
9  class WIR_CompilationUnit;
10 class WIR_Function;
11 class WIR_System;
12
13 class newModel_PredicatedExecution : public WIR_Optimization
14 {
15     public:
16
17         // Default constructor for system-level optimization.
18         explicit newModel_PredicatedExecution( WIR_System &s );
19
20         // Default constructor for compilation unit-level optimization.
21         explicit newModel_PredicatedExecution( WIR_CompilationUnit &c );
22
23         // Default constructor for function-level optimization.
24         explicit newModel_PredicatedExecution( WIR_Function &f );
25
26         // Destructor.
27         virtual ~newModel_PredicatedExecution( void );
28
29     protected:
30
31         // Apply predicated execution to a given function.
32         virtual void runOptimization( WIR_Function &f ) override;
33 };
34
35 } // namespace WIR
36
37 #endif

```

The implementation of `reduce` requires a careful analysis of an architecture's [ISA](#). This analysis should identify pairs of regular operations with full bit width and functionally equivalent operations of reduced width. `reduce` can then be implemented as a sequence of if-statements that match the given operation with the identified patterns of regular operations. If such an operation pattern has been found, the operation gets replaced by an according operation of reduced width.

18.8 Predicated Execution

Exploiting predicated execution in some [WIR](#) optimization pass is a highly processor-specific task for which no general guidelines or algorithmical approaches exist. Modern [ISAs](#) can provide fundamentally different styles of predication, rendering a generic optimization within [WIR](#) difficult. For this reason, there exists no dedicated infrastructure for predicated execution within [WIR](#). An optimization pass thus simply inherits from the generic base class `WIR_Optimization`, overrides the desired constructors, and provides a virtual destructor as well as a

Listing 18.10: Header File for a New Processor Model's Peephole Optimization

```

1  #ifndef _newModel_PEEPHOLE_H
2  #define _newModel_PEEPHOLE_H
3
4  // Include WIR headers
5  #include <optimizations/peephole/wirpeephole.h>
6
7  namespace WIR {
8
9  class WIR_Function;
10
11  class newModel_Peepphole : public WIR_Peepphole
12  {
13  public:
14
15      // Default constructor for function-level optimization.
16      explicit newModel_Peepphole( WIR_Function &f );
17
18      // Destructor.
19      virtual ~newModel_Peepphole( void );
20
21  protected:
22
23      // Determine whether the given peephole is suitable for optimization.
24      virtual bool matchPeepphole( const WIR_Peepphole::peephole &p ) override;
25
26      // Optimize the current peephole.
27      virtual void transformPeepphole( const WIR_Peepphole::peephole &p ) override;
28  };
29
30 } // namespace WIR
31
32 #endif

```

dedicated `runOptimization` method (cf. Listing 18.9). `runOptimization` then detects processor-specific code patterns and replaces them by predicated execution code.

For the sake of brevity, we omit the discussion of the generic steps to set up the infrastructure for predicated execution within `WIR` – they are identical to those from Section 18.2.³

18.9 Peephole Optimization

Retargeting a peephole optimizer to a new processor architecture is a quite simple procedure. The required basic steps to set up its infrastructure are omitted here for the sake of brevity, see Section 18.2 again. The new optimizer inherits from base class `WIR_Peepphole` as shown in Listing 18.10.

The implementation of the constructor from Line 16 has to set the basic properties of the peepholes to be formed. This most importantly includes the specification of the peephole's size(s) by calling method `addPeeppholeSize` from Line 11 of Listing 14.9. The `WIR` peephole optimizer supports peepholes with different sizes so that `addPeeppholeSize` can be called several times by the constructor. Furthermore, the new optimizer's constructor

³Predicated execution uses directory name `predicatedexecution`, file names `newModelpredicatedexecution.h` and `newModel-predicatedexecution.cc`, and class name `newModel_PredicatedExecution`.

Listing 18.11: Header File for a New Processor Model's Local Instruction Scheduler

```

1  #ifndef _newModel_LOCALSCHEDULING_H
2  #define _newModel_LOCALSCHEDULING_H
3
4  // Include WIR headers
5  #include <optimizations/scheduling/wirlocalscheduling.h>
6
7  namespace WIR {
8
9  class WIR_BasicBlock;
10 class WIR_BlockSchedulingRegion;
11 class WIR_CompilationUnit;
12 class WIR_Function;
13 class WIR_SchedulingPriority;
14 class WIR_System;
15
16 class newModel_LocalScheduling : public WIR_LocalScheduling
17 {
18     public:
19
20         // Default constructor for system-level optimization.
21         newModel_LocalScheduling( WIR_System &s, const WIR_SchedulingPriority &p, bool v = false,
22                                   bool t = false );
23
24         // Default constructor for compilation unit-level optimization.
25         newModel_LocalScheduling( WIR_CompilationUnit &c, const WIR_SchedulingPriority &p, bool v = false,
26                                   bool t = false );
27
28         // Default constructor for function-level optimization.
29         newModel_LocalScheduling( WIR_Function &f, const WIR_SchedulingPriority &p, bool v = false,
30                                   bool t = false );
31
32         // Destructor.
33         virtual ~newModel_LocalScheduling( void );
34
35     protected:
36
37         // Generate a new processor-specific block region for scheduling.
38         virtual WIR_BlockSchedulingRegion *generateBlockRegion( WIR_BasicBlock &b ) const override;
39 };
40
41 } // namespace WIR
42 #endif

```

can specify whether the formed peepholes shall span over several [WIR](#) basic blocks or not by calling method `setCrossBasicBlocks` from Line 17 of Listing 14.9.

Detecting specific code pattern(s) in a given peephole is done by method `matchPeephole` (cf. Line 24). As discussed in Section 14.16, a sliding window is moved over the [WIR](#) instructions inside basic blocks. Thus, the instructions within a peephole are represented by iterators referring to their positions within their enclosing basic blocks. A `std::vector` of such iterators thus represents a peephole, and a reference to such a peephole is handed over to `matchPeephole`. It then inspects the peephole's instructions and operations, checks whether they match some particular code pattern, and returns whether a pattern was identified successfully or not.

In the former case, method `transformPeephole` (cf. Line 27) is invoked next. It gets a successfully matched

peephole as argument and optimizes it by transforming the peephole's code locally.

18.10 Local Instruction Scheduling

Retargeting the local instruction scheduler of [WIR](#) requires to specialize various abstract base classes, i.e., `WIR_LocalScheduling`, `WIR_BlockSchedulingRegion` and a couple of scheduling priorities. For this purpose, the following steps need to be done:

1. Create a new directory `WIR/arch/archDir/optimizations/scheduling`.
2. Add a new line
`include arch/archDir/optimizations/scheduling/Makefile.src`
to file `WIR/arch/archDir/optimizations/Makefile.src`.
3. Create a new file `WIR/arch/archDir/optimizations/scheduling/Makefile.src` as follows:

```

1 libwir_la_SOURCES      += arch/archDir/optimizations/scheduling/newModelblockschedulingregion.cc \
2                          arch/archDir/optimizations/scheduling/newModelblockschedulingregion.h \
3                          arch/archDir/optimizations/scheduling/newModellocalscheduling.cc \
4                          arch/archDir/optimizations/scheduling/newModellocalscheduling.h \
5                          arch/archDir/optimizations/scheduling/newModelmaxdelaypriority.cc \
6                          arch/archDir/optimizations/scheduling/newModelmaxdelaypriority.h \
7                          arch/archDir/optimizations/scheduling/newModelmobilitypriority.cc \
8                          arch/archDir/optimizations/scheduling/newModelmobilitypriority.h \
9                          arch/archDir/optimizations/scheduling/newModelnoofsuccspriority.cc \
10                         arch/archDir/optimizations/scheduling/newModelnoofsuccspriority.h

```

4. Create a file `WIR/arch/archDir/optimizations/scheduling/newModellocalscheduling.h` following the general structure depicted in Listing 18.11.

The constructors from lines Lines 21 to 27 are straightforward, as they only need to delegate their arguments to the constructors of base class `WIR_LocalScheduling` from Listing 14.11. `generateBlockRegion` (cf. Line 35 of Listing 18.11) creates a new scheduling region for a given basic block `b` and builds its dependence graph:

```

1 WIR_BlockSchedulingRegion *newModel_LocalScheduling::generateBlockRegion( WIR_BasicBlock &b ) const
2 {
3     auto *res = new newModel_BlockSchedulingRegion( b, mVerbosity, mKeepTmpFiles );
4     res->buildDG();
5
6     return( res );
7 };

```

5. Create a file `WIR/arch/archDir/optimizations/scheduling/newModelblockschedulingregion.h` according to the general structure shown in Listing 18.12. Depending on a new architecture's timing properties, some virtual methods from Listing 18.12 might need to be overridden, some others maybe not.

Method `moveOperations` (cf. Line 23) receives a bundle `b` of operations, i.e., a list, as first argument. All operations within `b` are assumed to be scheduled in parallel, and they shall be moved after the instruction pointed to by `pred`. Thus, `moveOperations` creates a new `WIR` instruction after `pred` and puts all operations of `b` therein. Processor-specific rules how such bundles can be scheduled need to be ensured by `moveOperations`.

The `postProcessingHook` (Line 26) can perform processor-specific cleanups after a block region has been scheduled. If, e.g., a silicon bug optimizer introduces NOP operations (cf. Section 14.16 on Page 200), they might become obsolete after local scheduling and should be removed by `postProcessingHook`.

Listing 18.12: Header File for a New Processor Model's Block Scheduling Region

```

1  #ifndef _newModel_BLOCKSCHEDULINGREGION_H
2  #define _newModel_BLOCKSCHEDULINGREGION_H
3
4  // Include standard headers
5  #include <list>
6
7  // Include WIR headers
8  #include <optimizations/scheduling/wirblockschedulingregion.h>
9
10 namespace WIR {
11
12 class newModel_BlockSchedulingRegion : public WIR_BlockSchedulingRegion
13 {
14     public:
15
16         // Default constructor.
17         newModel_BlockSchedulingRegion( WIR_BasicBlock &b, bool v = false, bool t = false );
18
19         // Destructor.
20         virtual ~newModel_BlockSchedulingRegion( void );
21
22         // Move a bundle of operations after a given instruction in the scheduling region.
23         virtual WIR_Instruction *moveOperations( std::list<WIR_Operation *> &b,
24                                                 const WIR_Instruction *pred ) override;
25
26         // Perform processor-specific actions after scheduling of a block region.
27         virtual void postProcessingHook( void ) override;
28
29     protected:
30
31         // Check for a processor-specific uncertain dependence between an operation and a register parameter.
32         virtual bool checkUCert( const WIR_Operation &o, const WIR_RegisterParameter &rp ) const override;
33
34         // Check for a processor-specific control dependence between an operation and a register parameter.
35         virtual bool checkCtrl( const WIR_Operation &o, const WIR_RegisterParameter &rp ) const override;
36
37         // Get the processor-specific latency between two dependent operations.
38         virtual long long computeLatency( const WIR_Operation &o1, const WIR_Operation &o2,
39                                           WIR_DGEdgeType t ) override;
40
41         // Get the number of clock cycles between issuing an operation until availability of its results.
42         virtual long long getLatency( const WIR_Operation &o ) override;
43
44         // Get an operation's earliest execution cycle if it were scheduled as first operation in the region.
45         virtual long long getStartCycle( const WIR_Operation &o ) const override;
46 };
47
48 // namespace WIR
49 #endif

```

Methods `checkUCert` and `checkCtrl` (Lines 31 and 34) need to be overridden in order to capture processor-specific uncertainties and control dependencies while creating the dependence graph for a scheduling region. Both methods get a reference to an operation `o` as first argument and a reference to some register parameter `rp` as second one, and they return true if there is a dependence between `o` and `rp`.

Listing 18.13: Header File for a New Processor Model's Maximum Delay Scheduling Priority

```

1  #ifndef _newModel_MAXDELAYPRIORITY_H
2  #define _newModel_MAXDELAYPRIORITY_H
3
4  // Include WIR headers
5  #include <optimizations/scheduling/wirmaxdelaypriority.h>
6
7  namespace WIR {
8
9  class newModel_MaxDelayPriority : public WIR_MaxDelayPriority
10 {
11     public:
12
13         // Default constructor.
14         newModel_MaxDelayPriority( void );
15
16         // Copy constructor.
17         newModel_MaxDelayPriority( const newModel_MaxDelayPriority &p );
18
19         // Destructor.
20         virtual ~newModel_MaxDelayPriority( void );
21
22     protected:
23
24         // Compute the maximum delay priority for a given operation.
25         virtual std::pair<long long,
26                     std::list<WIR_Operation *>> computePriority( const WIR_Operation &o,
27                                                                 const WIR_Operation *pred,
28                                                                 long long c ) override;
29
30         // Create a deep copy of a scheduling priority.
31         virtual WIR_SchedulingPriority *clone( void ) const override;
32 };
33
34 // namespace WIR
35
36 #endif

```

computeLatency (Line 37) aims to determine the latency in clock cycles between two operations o1 and o2 that depend on each other. The actual type of dependence (e.g., RAW or WAR) is provided by enumeration type WIR_DGEdgeType specified in file WIR/optimizations/scheduling/wir_dgedgeproperty.h. To compute this latency, this method needs to take processor-specific pipeline behaviors such as stalls or parallel execution of operations into account.

In contrast, method getLatency (Line 40) returns the processor-specific response time of an operation, i.e., the number of clock cycles it takes from issuing the operation until its results are available. The purpose of this method is to distinguish between, e.g., fast integer operations, slower integer multiplications or divisions, or costly floating-point operations.

Method getStartCycle considers the special scenario that a given operation shall be scheduled as first operation inside a basic block. Then, potential dependencies between the last operations of preceding basic blocks need to be taken into account and possible stall cycles have to be determined.

As discussed in Section 14.17, processor-specific realizations of heuristics to determine scheduling priorities are required next. For the maximum delays priority, this involves:

6. Create file `WIR/arch/archDir/optimizations/scheduling/newModelmaxdelaypriority.h` following the general structure shown in Listing 18.13.

The constructors from Lines 14 and 17 are straightforward, they simply have to invoke the respective constructors of base class `WIR_MaxDelayPriority`. In analogy to Line 51 of Listing 16.2, the `clone` method simply needs to call the copy constructor of the new class:

```

1 WIR_SchedulingPriority *newModel_MaxDelayPriority::clone( void ) const
2 {
3     return( new newModel_MaxDelayPriority( *this ) );
4 };

```

The implementation of method `computePriority` (Line 25) is the essential part of a scheduling priority. It receives a reference to a `WIR` operation `o` as first argument, for which the scheduling priority shall be computed. The second argument `pred` points to another operation that has already been scheduled in the current execution cycle and that is predecessor of `o`. If no such preceding operation exists, `pred` can be the `nullptr`. Finally, argument `cycle` represents the current execution cycle to be considered. The result returned by `computePriority` is a tuple whose first element stores the determined scheduling priority for operation `o`. The tuple's second element is a list of operation pointers denoting all `WIR` operations to be scheduled together with `o` in the current execution cycle. Regularly, a processor-specific implementation of `computePriority` starts with determining `o`'s initial priority using method `computePriority` of the scheduling priority's base class. Hereafter, processor-specific checks can be carried out to refine this initial priority. Such checks can take into account the particular type of `o`, on which hardware units it gets executed, how it interacts within a processor's pipeline with the preceding operation `pred`, etc.

Depending on which internal data structures are required inside a new priority's class and method `computePriority`, the destructor from Line 20 finally requires some specific implementation.

7. Repeat Step 6 for the remaining scheduling priorities mobility and number of successors.

Part IV

Developer's Guide

-
- **TODO:** *Registry? Eher nicht in User's Guide da registry nicht public ist.*

TODO

Bibliography

- [1] Andrew W. Appel and Maia Ginsburg. *Modern Compiler Implementation in C*. Rev. and expanded ed. Cambridge University Press, 2004. ISBN: 0-521-60765-5.
- [2] ARM Limited. *ARM Architecture Reference Manual*. ARM DDI 0100I. 2005.
- [3] David Bernstein et al. “Spill code minimization techniques for optimizing compilers”. In: *Proceedings of “Conference on Programming Language Design and Implementation” (PLDI)*. July 1989, pp. 258–263.
- [4] *Boost C++ Libraries*. <https://www.boost.org>. Aug. 2026.
- [5] Preston Briggs. “Register Allocation via Graph Coloring”. PhD thesis. Houston: Rice University, Apr. 1992.
- [6] Gregory J. Chaitin. “Register Allocation and Spilling via Graph Coloring”. In: *SIGPLAN Notices* 17.6 (June 1982), pp. 89–105. DOI: [10.1145/872726.806984](https://doi.org/10.1145/872726.806984).
- [7] *DOT Language, Abstract grammar for defining Graphviz nodes, edges, graphs, subgraphs, and clusters*. <https://graphviz.org/doc/info/lang.html>. Aug. 2026.
- [8] *Doxygen homepage*. <https://www.doxygen.nl>. Aug. 2026.
- [9] *Executable and Linking Format (ELF) Specification*. Version 1.2. Tool Interface Standard (TIS) Committee, May 1995.
- [10] Heiko Falk and Paul Lokuciejewski. “A compiler framework for the reduction of worst-case execution times”. In: *The International Journal of Time-Critical Computing Systems (Real-Time Systems)* 46.2 (Oct. 2010), pp. 251–300. DOI: [10.1007/s11241-019-09337-9](https://doi.org/10.1007/s11241-019-09337-9).
- [11] *GCC, the GNU Compiler Collection*. <https://gcc.gnu.org>. Aug. 2026.
- [12] *GMP, the GNU Multiple Precision Arithmetic Library*. <https://gmplib.org/>. Aug. 2026.
- [13] *GNU Bison*. <https://www.gnu.org/software/bison>. Aug. 2026.
- [14] *Introducing the GNU Build System*. https://www.gnu.org/software/automake/manual/html_node/GNU-Build-System.html#GNU-Build-System. Aug. 2026.
- [15] *GNU Flex*. <https://github.com/westes/flex>. Aug. 2026.
- [16] *GNU Make*. <https://www.gnu.org/software/make>. Aug. 2026.
- [17] *Graphviz*. <https://graphviz.org>. Aug. 2026.
- [18] Raimund Kirner. “Extending Optimising Compilation to Support Worst-Case Execution Time Analysis”. PhD thesis. Vienna: Universität Wien, May 2003.
- [19] Heiko Falk. *LIBUSEFUL – A Collection of Useful C++ Routines and Classes for Compiler Development and Debugging*. <https://github.com/TUHH-IES/LIBUSEFUL>. Aug. 2026. DOI: [10.5281/zenodo.22101586](https://doi.org/10.5281/zenodo.22101586).
- [20] *The LLVM Compiler Infrastructure Project*. <https://llvm.org>. Aug. 2026.
- [21] *MIPS/SPIM Reference Card*. <https://pageperso.lis-lab.fr/~peter.niebert/archi/mipsref.pdf>. 2007.
- [22] Steven S. Muchnick. *Advanced Compiler Design & Implementation*. Morgan Kaufmann Publishers, 1997. ISBN: 1-55860-320-4.

- [23] Dominic Oehlert, Arno Luppold, and Heiko Falk. “Practical Challenges of ILP-based SPM Allocation Optimizations”. In: *Proceedings of “Software & Compilers for Embedded Systems” (SCOPES)*. May 2016, pp. 86–89.
- [24] Andrew Waterman and Krste Asanović, eds. *The RISC-V Instruction Set Manual, Volume I: User-Level ISA*. Document Version 2.2. RISC-V Foundation, May 2017.
- [25] Kito Cheng and Jessica Clarke, eds. *RISC-V ABIs Specification*. Document Version 1.0. RISC-V International, Nov. 2022.
- [26] Infineon Technologies AG. *TC1796 EES-BC, ES-BC, BC Errata Sheet*. Rel. 2.0. June 2008.
- [27] Infineon Technologies AG. *TC1797 EES-AC, ES-AC, AC Errata Sheet*. Rel. 1.3. Dec. 2009.
- [28] Infineon Technologies AG. *TriCore 32-bit Unified Processor DSP Optimization Guide, Part 1: Instruction Set*. V1.6.4. Jan. 2003.
- [29] Infineon Technologies AG. *TriCore CompilerWriter’s Guide*. V1.4. Dec. 2003.
- [30] Infineon Technologies AG. *TriCore 32-bit Unified Processor Core, Embedded Applications Binary Interface (EABI)*. V2.3. Feb. 2007.
- [31] Infineon Technologies AG. *TriCore 1 32-bit Unified Processor Core, Volume 2 Instruction Set V1.3 & V1.3.1 Architecture*. V1.3.8. Jan. 2008.
- [32] Jens Wagner. “Retargetable Exploitation of Special-Purpose Operations for Embedded Systems with a Bit-True Value Flow Analysis (in German)”. PhD thesis. Dortmund: University of Dortmund, Jan. 2006.
- [33] *WCET-aware Compilation*. <https://www.tuhh.de/es/esd/research/wcc>. Aug. 2026.